GAWorld 教学手册

从一个人到一座城：生成式社会仿真平台的实操指南

GAWorld 项目组

2026 年 10 月

目录

GAWorld 教学手册
怎样读这本手册
延伸阅读
第 1 章　导读：GAWorld 是什么，以及怎么用这本书
1.1 一句话定位：可控制的社会实验场
1.2 核心循环：一个居民是怎么过一天的
1.3 三类人，三个入口
1.4 五级学习台阶
1.5 两条效度轴：不混成一个总分
1.6 本手册怎么用
1.7 两条真实的使用边界
本章小结
动手练习
第 2 章　安装与工作区：把平台跑起来的前十分钟
2.1 这一章解决什么问题
2.2 两条安装路线
2.3 工作区：pip 装完之后东西落在哪
2.4 目录地图
2.5 启动与验证
2.6 常见安装故障
2.7 磁盘与成本预期
本章小结
动手练习
第 3 章　接入模型：让居民真的能思考
3.1 没配模型，仿真连第一步都走不完
3.2 三类后端，先选一个
3.3 配置路径 A：改配置 + 设环境变量
3.4 配置路径 B：控制台的「配置 → 模型」
3.5 路由：同一个居民，哪些事找谁
3.6 成本控制：先量，再省
3.7 常见故障与排查
3.8 隐私与合规：开课前先讲这一句
本章小结
动手练习
第 4 章　居民与种子数据：平台吃什么、吐什么
4.1 一条居民 = 一行 CSV + 一节 Markdown

4.2 种子数据放在哪
4.3 状态 CSV：九个状态变量
4.4 profile Markdown：人格化叙事档案
4.5 大五人格（OCEAN）
4.6 加一个人、改一个人
4.7 常见坑
本章小结
动手练习
第 5 章　从地名造一座城：城市、地图与人口
5.1 造城流水线：六步，每一步都能降级
5.2 两种地图模式
5.3 完整命令序列
5.4 人口预设：六种社会结构
5.5 城市知识库：让城市影响居民
5.6 居民的地方特征：`locale.json`
5.7 切换当前城市
5.8 城市包的目录结构
5.9 失败与降级
5.10 面板：城市实验台
本章小结
动手练习
第 6 章　跑第一次仿真：命令、步长与产物
6.1 最小可运行：一条命令
6.2 时间粒度：这一章的骨架
6.3 运行参数逐条解释
6.4 两个「随机性」旋钮
6.5 产物地图：跑完去哪儿看
6.6 重置：改配置后为什么必须先 reset
6.7 复现性：种子、清单与版本
6.8 成本与时间预期
本章小结
动手练习
第 7 章　仿真控制台图解：界面就是一份实验配置
7.1 工具栏：四组字段就是四个实验条件
7.2 工作区：地图与三块「这个人」的画像
7.3 选中一位居民：用一屏做质性观察
7.4 其它入口：别名路由与轨迹回放
7.5 面板显示空？先查这三条
7.6 什么时候该放下控制台
本章小结
动手练习
第 8 章　Agent Studio：把一个居民拆开看
8.1 一个面板，两种身份
8.2 第 1 步 · 身份
8.3 第 2 步 · 状态 · 性格
8.4 第 3 步 · 能力 · 技能
8.5 第 4 步 · 记忆
8.6 第 5 步 · 社交 · 关系
8.7 第 6 步 · 行为 · 目标

8.8 第 7 步 · 复核 · 部署
8.9 改动如何生效
8.10 城市切换与 Studio 的关系
8.11 一条能用的工作流
8.12 与 Population Studio 的分工
本章小结
动手练习
第 9 章　群体采访：一次问一群人
群体采访是什么
完整流程：面板与 CLI 两条路
选受访者：个体居民与群体智能体
出题：三种题型
追问一轮
读结果：统计与分组分布
导出的 Markdown 文档
成本与并发
常见故障
边界：采访不能影响之后的仿真
本章小结
动手练习
第 10 章　游戏场：拿真实居民当对手
10.1 游戏场是什么
10.2 四个家族：按问题形状选游戏
10.3 三个游戏跑通
10.4 判定的两种模式
10.5 通用操作流程
10.6 教学建议：游戏场是概念直觉的入口
10.7 常见困惑
10.8 HTTP API
本章小结
动手练习
第 11 章　研究工作台：把研究想法变成可执行的方案
研究工作台是什么
五个页签的分工
实施方案：从想法到一份能判定的报告
为什么「先读论文再设计」的顺序不能反
预注册的价值：把判定写死在跑之前
产物落盘在哪里
四种判定的读法
严肃游戏：一句话指引
一条方法论提醒：产物是研究材料，不是结论
本章小结
动手练习
第 12 章　平行世界：把同一段历史分叉成 N 条
12.1 一个思想：除了「发生了什么」，其它一切都不变
12.2 两个层级：`compare-event` 与 `parallel-worlds`
12.3 世界规格：一份 spec 的真实结构
12.4 怎么跑：命令行与面板
12.5 逐步测量：偏离度、分叉点、逐人影响

12.6 估计量：`gaworld/parallel/causal.py` 里的确定性代码
12.7 剂量—反应与参数扫描
12.8 怎么读结果
12.9 解读接口：一次模型调用，每条发现都被绑回一行估计
12.10 产物落盘与复现要求
12.11 与第 11 章研究工作台的关系
本章小结
动手练习
第 13 章　三种研究页签：政策仿真、复制拟合与严肃游戏
13.1 三个页签在哪、产出到哪
13.2 政策仿真与优化：让代码挑出更好的那一版
13.3 社会调查/实验拟合度：这群居民像不像真人
13.4 严肃游戏：把一段描述变成一张桌子
13.5 三个页签怎么选
本章小结
动手练习
第 14 章　大规模人群：参数化人口与群体模拟
为什么需要这一层
参数化人口合成
群体（cohort）模拟
验证门：这个近似能回答哪类问题
什么时候能用、什么时候不能
成本对比与零成本预演
实践建议：三步走
本章小结
动手练习
第 15 章　插件开发：写一个新子系统，不改主循环
15.1 架构约定：微内核，以及”主循环零改动”为什么是硬规矩
15.2 最小插件：一份能直接读懂的代码
15.3 三种钩子语义：选错了会发生什么
15.4 装配方式三选一（可叠加）
15.5 当前可订阅的事件目录
15.6 认知管线 K2：替换、插入、消融
15.7 状态与数据所有权
15.8 动作校验与运行时干预
15.9 从零到一：写、装、跑、验
15.10 给工程师的一条建议：别从零写
本章小结
动手练习
第 16 章　多用户部署：把平台交给一班学生
16.1 三档部署形态，中间那档最难
16.2 开启账号库：五个子命令
16.3 权限模型：分三层看就够
16.4 每人一个世界
16.5 归属与可见性：谁做的东西归谁
16.6 配额与限额
16.7 教师控制台
16.8 手机入口：给学生的短链接
16.9 上线清单：照着做一遍

16.10 安全：哪些事必须由部署方负责
本章小结
动手练习
第 17 章　分布式世界与多人共玩
17.1 分布式世界：一座城可以分家
17.2 多人共玩：人走进城里
17.3 协作面板
17.4 部署形态全景
17.5 本章小结
17.6 动手练习
第 18 章　测试与质量门
18.1 两套测试：冒烟与全量
18.2 行数预算：主循环不许变胖
18.3 Lint 棘轮：只许变好
18.4 Epoch：让「结果不可比」变成一件可声明的事
18.5 测试环境是隔离的
18.6 新代码的交付清单
18.7 写测试的取向
18.8 提交与协作
18.9 本章小结
18.10 动手练习
第 19 章　排错手册
19.1 先分诊：没跑起来 / 跑不动 / 跑完了但结果不对
19.2 没跑起来
19.3 跑不动
19.4 跑完了但结果不对
19.5 面板显示空
19.6 读数看不出差别
19.7 多人部署的问题
19.8 怎么问出一个能被回答的问题
19.9 一张症状速查表
19.10 本章小结
19.11 动手练习
第 20 章　速查附录
20.1 命令速查
20.2 配置速查
20.3 产物速查
20.4 术语表
20.5 文档地图
20.6 本章小结
20.7 动手练习

GAWorld 教学手册

从一个人到一座城：生成式社会仿真平台的实操指南

GAWorld 是一���生成式多智能体社会仿真平台。它让人物档案、人格、家庭责任和过往经历构成居民的决策情境；让城市、经济、天气和社会网络构成仿真环境，并影响行动结果；让研究者观察轨迹、访谈居民，并在平行世界里比较不同事件与政策的影响。

这本手册面向第一次使用 GAWorld 的人，也面向要把它讲给学生的老师。全部界面截图取自本仓库真实运行的控制台，全部插图由本仓库结构生成；每条命令都来自当前代码，可以直接复制运行。

> **使用边界**：仿真输出是模型与配置共同产生的研究材料，不应直接当作真实人群的预测。文化情境是建模的起点，仿真行为是否真实需要实验验证。

怎样读这本手册

手册共 20 章，按「跑起来 → 看懂人 → 造世界 → 做实验 → 写出来」五级台阶排列。

你是谁	建议路线	大约需要
第一次接触的学生	第 1–2 章 → 第 6–8 章 → 第 10 章	3–4 小时
准备开课的老师	第 1–7 章 → 第 12 章（平行世界）→ 第 19 章（排错）	1 天
要做研究的研究者	第 1–6 章 → 第 9 章 → 第 11–13 章 → 第 18 章	2–3 天
要改代码的工程师	第 2 章 → 第 6–8 章 → 第 15 章（插件）→ 第 18 章	2 天

每章末尾都有「本章小结」与「动手练习」。练习都带验收标准，做完能立刻看出结果对不对。

延伸阅读

- `textbook/` —— 22 章 + 4 附录的《多智能体社会仿真：原理、技术与案例》，讲思想、方法论与经典案例，另附 md / pdf / epub 三种格式
- `docs/PROJECT_GUIDE.zh-CN.md` —— 中文完整使用指南，本手册的细节出处
- `docs/FEATURES.md` —— 能力目录，按子系统罗列所有可调项
- GAWorld Bench —— 功能效度与人类效度的评测方法

ewpage

ewpage

第 1 章　导读：GAWorld 是什么，以及怎么用这本书

第一次打开 GAWorld 的人通常卡在同一个地方：控制台能跑起来，居民会动、会说话、会有情绪变化——然后就没有然后了。「所以这算是什么？」本章先给定位，再给路线图，最后说明这本书怎么用，以及两条不能越过的边界。

1.1 一句话定位：可控制的社会实验场

GAWorld 是一个生成式多智能体社会仿真平台。项目 README 的定位是「在城市环境中模拟居民行为，比较不同事件与政策的影响」；中文完整指南说得更直接——目标不是简单地「跑一群 Agent」，而是提供一个**可控制的社会实验场**。

这个定位里有三个词值得拆开。

可控制。仿真不是录好的动画。每个居民有可编辑的档案、人格、家庭责任和过往经历；城市、经济、天气和社会网络构成环境并影响行动结果；研究者在运行过程中可以注入事件、修改配置、移除居民、直接改写某个状态变量。这些干预不是偷偷进行的——`Controller` 的每一次调用都会自动审计。

社会。人不是孤立的。居民的日程受家庭约束（接送孩子、照料老人、回家吃晚饭）；钱跑在一条货币守恒的闭环里（企业 / 政府 / 银行部门池、个税与五险一金代扣、现金约束消费加信贷）；情绪会在同一户内传染。单个 agent 的行为只是这个系统的一角，把它单独拿出来分析，等于把实验条件扔掉了。

实验场。实验场的意思是能摆对照组。这三个字是 GAWorld 全部能力的来路：平行世界（同一批居民、同一个随机种子、同一个天数与模型，唯一的区别是发生了什么）、事件对照实验、政策仿真、复制拟合度、预注册协议——它们都在回答「在可比条件下，只有自变量不同，结果差多少」。

于是它**不是**两样东西。

第一，它不是「跑一群 Agent」的玩具。玩具的产出是「看起来很有意思」，实验场的产出是「在可比条件下只有自变量不同」。两者都能出漂亮的轨迹和截图，但只有后者能支撑一个结论。

第二，它不是对真实人群的预测器。仿真输出是模型与配置共同产生的研究材料，不应直接当作真实人群的预测。README 的原话是：文化情境是建模的起点，仿真行为是否真实需要实验验证。这个平台能做的是「如果我改了这一条规则，系统会怎样反应」，不是「杭州人明年会怎样」。



图 1–1　GAWorld 系统分层架构 最上面是你在浏览器里点的东西，下面才是仿真本体。关键的一层是中间的微内核（`gaworld/kernel/`）：时钟、事件总线、插件注册表、动作校验与可审计干预、统一事件流。往下的认知管线把每个 agent 的一步展开成 12 个可配置阶段，再往下的插件层挂载全部领域子系统——干预、技能、兴趣成长、人生事件、经济、物理感知、真实工作、动态行为、空间偏好等。你写新功能时，只需要在这一层挂一个插件，主循环一行都不用改。

1.2 核心循环：一个居民是怎么过一天的

每个居民每天循环经历五步。这是理解整个平台的最小单位，也是后面所有章节的骨架。

环节	发生什么	留下的痕迹
感知	读环境、关系、消息、需求与状态	一组当前情境
计划	在日程与约束下决定今天要做什么	当日意图
日程 / 动作生成	把意图变成具体动作	动作与目标
动作执行	真的去做：移动、交谈、工作、消费	轨迹、账目、关系变动
反思与记忆更新	回看今天，写下经历与结论	情景记忆、长期总结

实现层面，这五步被展开成 12 个可配置阶段，写在 `gaworld/sim/pipeline.py` 的默认顺序里：

```
CONFIG["pipeline"]["agent_step"] = [
    "prepare", "perceive", "interrupts", "plan", "adjust_activity",
    "move", "select_action", "reflect", "update_state",
```

```
    "broadcast", "memorize", "record",
]
```

阶段可以消融、替换或插入，改的只是这份配置，不需要改代码。`prepare` 和 `record` 是结构性的（前置钩子与落盘），真正的消融目标是中间那些认知阶段——这一条对做实验的人很重要：你可以在不重新实现任何东西的前提下，问「如果模型不反思会怎样」。

随着天数推进，系统持续累积六类东西：episode 记忆、长期总结、基于上下文的习惯、日级意图、关系变化、收支与资产变化。

这里有一句话值得单独说：这些累积物**不是日志附件，而是下一轮感知与计划的输入**。记忆会改变判断，习惯是「基于上下文的」（所以同一件事在不同人身上会触发不同反应），关系变化会改变下一次社交的可能性。这条「积累 → 反馈 → 再积累」的闭环是长期仿真和单日问答的分水岭——也是为什么跑 5 天和跑 60 天得到的不只是更长的日志，而是不一样的东西。

12 个阶段里有几个值得现在就记住，因为它们解释了日常观察到的那些「不按计划走」的现象：

- `interrupts`——日程可以被需求、环境事件或别人的请求打断。动态行为系统在这一层提供情绪驱动的即兴行为、社交偶遇链、需求中断和承诺度感知的日程中断。
- `adjust_activity`——计划好的活动不是死的。物理环境感知会在这一层根据节点拥挤度、营业时间、天气和异常检测重排当天受影响的时段，并让居民习得「避开某些地点」这类偏好。
- `broadcast`——一个人身上的变化要变成别人的输入。户内情绪传染、社交关系网的共变都发生在这里。
- `memorize` 与 `reflect` 是两回事：`reflect` 是这一步的回顾，`memorize` 是把回顾写进分层记忆。分开是为了让「反思」这个研究对象可以被单独消融。

1.3 三类人，三个入口

GAWorld 的功能面很宽，但不同角色真正每天会打开的东西差别很大。下面这张控制台实拍，左侧导航基本就是这个分界。

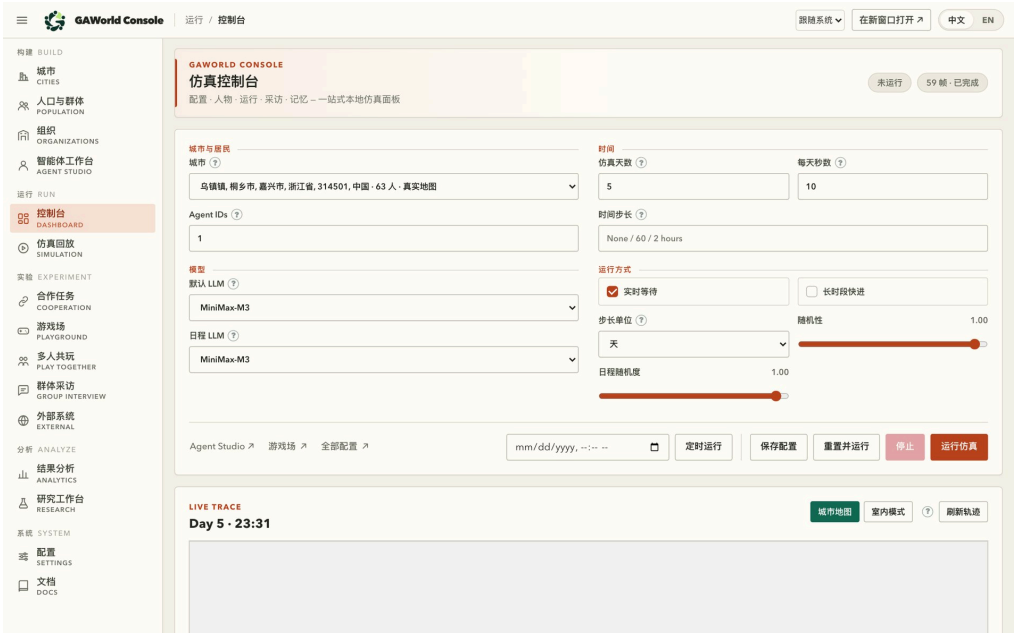


图 1–2　GAWorld 仿真控制台 左边一列是全部功能入口，按「构建 → 运行 → 实验 → 分析 → 系统」分组；右上是城市、天数、每天秒数、时间步长与随机性，中部是模型选择（默认 LLM 与日程 LLM 可以分开指定），底部是运行控制与

实时轨迹。想清楚「该点哪个按钮」之前，先记住这个分栏结构。

教师与学生

入口是控制台与命令行中最轻的部分：装好依赖、配好模型、跑一个城市几天，然后看人。

- 展示用：`gaworld dashboard` 打开控制台；「仿真控制台」页配好城市与模型，点「运行仿真」，下方实时轨迹随仿真推进刷新。
- 造场景：`python -m gaworld.city create "绍兴柯桥" --size 200` 从一个地名生成整座城市（真实 OSM 道路网络，取不到时用程序化地图兜底）；`python -m gaworld.city list` / `show` / `use` 管理多座城市。
- 玩起来：控制台「游戏场」有一批游戏，分四组（对一个居民 / 对一群人 / 任务协作与创意 / 长期同居与多人参与）。其中「说服游戏」的判定规则值得在课堂上讲：结束时复问同一个问题，裁判模型比较首尾两个答案，**核心结论真的变了**才算说服成功，措辞变化或让步但结论不变都不算。
- 问居民：控制台「群体采访」一次问一群人同一套问题；单个居民用 `python generative_city_sim.py interview --agent-id 31 --question "..."`。

这一层的目标是让人相信「这些居民的行为不是随机数」。做到这一点，你就已经准备好了进入下一层。

研究者

入口是平行世界与研究工作台，也是这个平台真正的价值所在。

- 最小对照：`python generative_city_sim.py compare-event --event-name "..." --sim-days 3 --seed 42`，在「有事件 / 无事件」两条分支上并行跑并出对比报告。
- 完整对照：`python generative_city_sim.py parallel-worlds --spec worlds.json`，一次最多 8 个世界，共用同一批居民、种子、天数与模型，各带自己的事件表或 config 补丁。报告不只给终局差多少，还逐步测量每个世界与基准的距离，回答**从哪一步开始分叉**，并指出这件事落在哪些居民身上。
- 研究工作台（控制台「研究工作台」页签）：贴一篇论文进去，先单独解读出理论 / 方法 / 变量 / 发现 / 假设供研究者审阅修改，再据此设计；也可以把方案编译成**预注册协议**——条件变成平行世界，假设变成指标加两个条件加方向加最小效应，指标只能选自可测量目录——然后跑、判、报告。判定由代码按固定规则做（supported / contradicted / inconclusive / unmeasured），模型只负责解读给定数字。
- 政策仿真与优化（研究工作台的另一个页签）：选一座城市，用一句白话描述一项政策（也可以给一个候选方案），一批按种子抽样的居民对每个版本按固定量表的 JSON 作答，代码聚合打分；然后一次模型调用提出修改建议、给出修订版政策，同一批居民再跑一遍，综合分最高的版本作为推荐。接口是 `POST /api/research/policy/run`。
- 复制拟合度（研究工作台「社会调查 / 实验拟合度」页签）：上传一篇报告了真人社会调查或实验结果的文献，一次模型调用抽成可编辑的研究规格——真人受访者与年龄范围、实验条件与材料、题目及其量程、文献报告的真人结果（逐条注明出处，**没报告的不估算**）；研究者核对后，选城市、居民规模（5–500）、分配方式（组间随机 / 配对）与种子，让居民按同样的设计作答，再由代码比较：逐题逐条件的误差、真人值是否落在居民 95% 区间内、实验的处理效应方向与显著性，合成一个拟合度分。
- 效度自查：GAWorld Bench，见 1.5。

工程师

入口是内核与插件接口。

- 读 `docs/PLUGIN_AUTHORING.md`，然后写一个 `gaworld.kernel.Plugin` 子类。插件可以订阅模拟生命周期、向 agent 的感知注入内容、改写被选中的动作、注册动作校验器与运行时干预，并拥有自己的 per-agent 状态和输出目录。
- 装配点有两处：内置插件走 `gaworld/plugins/` 的 `builtin_plugins()`，第三方走 `CONFIG["plugins"]` 或 pip 包的 `gaworld.plugins` entry point。两种扩展主循环都不用改。
- 运行时干预有 4 个标准动作：`set_agent_state`（写一个状态变量）、`update_config`（改配置的点分路径）、`remove_agent`（排队，日边界生效）、`inject_life_event`（由人生事件插件注册）。每次调用自动审计。
- 钩子有三种语义：observe（返回值被忽略）、collect（返回 0..n 条贡献）、filter（改写载荷）。选错语义是插件最常见的 bug 来源。

一张表把三个人分开

角色	主要入口	一次成功的标志
教师 / 学生	控制台、城市 CLI、游戏场	能讲清一个居民「为什么会那么做」
研究者	平行世界、事件对照、研究工作台	能说清「我的对照组是什么、种子是什么、分母是什么」
工程师	`gaworld/kernel/`、插件作者指南	能加一个不改主循环的子系统

1.4 五级学习台阶

上手路径不是一条直线，是五级台阶。每一级都有可交付的产出物；上一级没做完就做下一级，会卡在「跑不动」或「看不懂结果」。



图 1-3　教学路线：五级学习台阶 ① 跑起来（装依赖、配模型、跑 5 天、看 `output/`）→ ② 看懂人（Agent Studio、记忆、人格、家庭）→ ③ 造世界（建城市、加人口、切城市）→ ④ 做实验（平行世界、政策仿真、复制拟合）→ ⑤ 写出来（预注册、复现包、论文与同行评议）。

五级的实际含义：

- ① **跑起来**：`pip install` 装好，`gaworld dashboard` 起来，在「配置 → 模型」里选服务商、填 API Key、测连通性，然后跑 5 天。产出物是 `output/` 下面真实长出来的目录：`memory/`（记忆与成长）、`state/`（状态历史长表）、`logs/`（运行日志）、`records/`（Recorder 事件流）、`environment/`（环境事件与 `timeline.jsonl`）。看到这五个目录里有东西，①就算过了。
- ② **看懂人**：打开 Agent Studio 看一个居民的身份、九个状态变量、记忆分层、Dunbar 社交圈和家庭。这一级解决的是「我读不懂他为什么那么做」。
- ③ **造世界**：`python -m gaworld.city create` 建城，`python -m gaworld.population --size 500 --seed 42 --out data/town` 合成人口，或者用 `python -m gaworld.city add-agents` 往现有城里加人。多座城市用 `python -m gaworld.city use` 切换。
- ④ **做实验**：平行世界、事件对照、政策仿真、复制拟合。这一级才是本平台的真正价值所在。

- ⑤ **写出来**：预注册协议、复现包、论文与同行评议。

第 ④ 级是分水岭。 没有对照组，前面三级都只是「演示」——不管居民演得多像真的，只要没有「在同样条件下、只改一个变量」的那一半，你手上就只有故事，没有证据。反过来说，④ 的门槛也不高：一条 `compare-event` 命令就是一个最简陋的对照组，难的是把对照设计得让人信服（同一批居民、同一随机种子、同一天数、同一个模型、明确分母、多个种子重复），这部分后面几章会展开。

1.5 两条效度轴：不混成一个总分

做过仿真的人最常见的一个坏习惯，是把所有指标压成一个数字。GAWorld Bench 明确拒绝这么做——两条轴分开报告。



图 1–4　GAWorld Bench 的两条效度轴 上排是功能效度——「平台自己做到位了吗」：任务是否完成（记忆是否真的被检索、工具是否真的被调用、干预是否真的改变了轨迹），机制是否生效（关掉某条规则行为是否真的改变，改一个参数效应量是否按预期移动）。下排是人类效度——「像不像真人」：与真实参照比对（社会调查复现页把真人数据作为对照，逐格比误差），以及人类基线在谁身上（当前原生问答每项使用一个 agent，其成绩不能直接证明完整多 agent 协作或长期城市仿真的效度）。两条轴不合成总分，因为「机制跑通了」和「像真人」是完全不同的问题。

在 GAWorld Bench 里，这两条轴进一步拆成五条 track：Track A 宏观经验拟合（聚合统计量是否贴近真实分布）、Track B Stylized-facts 复现（是否复现已知社会规律，而非静态快照对齐）、Track C 因果 / 反事实有效性（对照实验引擎本身可信吗）、Track D 可信度与人设一致性、Track E 可复现性与成本（地基门槛）。每条 track 产出一个连续分 `score ∈ [0,1]` 和一个 pass/fail 门槛。

评测主页与评分方法：https://s2014628.github.io/gaworld-bench/。

读到这个链接时请注意 README 里的那句限定：两条轴分别报告，不混合成一个总分；历史 FOS 机制与多角色实验保留独立协议和分母。换句话说，看到任何一个单独的分数，都要先问三件事：分母是什么、协议是哪个协议、和哪些历史结果不可直接比较。

1.6 本手册怎么用

每章的结构是固定的：**正文 + 图 + 本章小结 + 动手练习**。正文讲机制，图讲结构或流程，图注用白话解释它在讲什么；小结是本章真正要记住的 3–6 条；练习是真能跑的，验收标准都可观察。读完一章不要只读小结，练习那部分才是把「看懂了」变成「会用了」的地方。

全书 20 章，不必从头读到尾。按角色挑一条线：

你的位置	建议路径
第一次接触，想先看到东西	本章 → 装环境与跑通一次仿真 → 控制台与 Agent Studio
要做研究	本章 → 平行世界与对照实验 → 效度与 Bench → 写作与复现

你的位置	建议路径
要写插件或改平台	本章 → 微内核与插件机制 → 认知管线 → 运行与产物

遇到具体功能、具体命令、具体文件路径时，往这四处找延伸阅读：

- `textbook/`——《多智能体社会仿真：原理、技术与案例》，7 编 22 章正文 + 4 附录，约 35 万中文字符，交付 md / pdf / epub / zip 四种格式。它讲的是「为什么这么做」与方法论，第一编讲社会科学为什么要做仿真，第三编讲仿真作为研究方法的完整流程，第四编里第 13–16 章是 GAWorld 自己的四个现代案例。
- `docs/PROJECT_GUIDE.zh-CN.md`——中文完整使用指南。功能怎么装、怎么配、每个模块的详细口径都在这里。
- `docs/FEATURES.md`——能力目录。它的组织方式是五节：一、CLI 命令（直接可用）；二、核心仿真特性（配置开关）；三、新特性（v2）；四、微内核插件体系；五、运维与调试。每一行都是「功能特性 → 作用 → 访问方法」三列，命令均在项目根目录执行。配套还有一张 `FEATURE_MAP.html`，按「使用者 × 能力子轴」重组成三级功能地图。
- 本手册附录——术语、命令与常用文件路径集中在一处，忘了某个命令或某个目录先翻它。

另有 `docs/proposals/` 下一批成型的实验方案（记忆一致性、误信息传播、极化、宏观经济、网络演化、出行行为等），可以直接当实验设计参考。

1.7 两条真实的使用边界

平台自己写在文档里的边界有两条，这两条不是免责套话，是会直接影响你怎么写结论的硬约束。

第一，仿真输出是模型与配置共同产生的研究材料，不应直接当作真实人群的预测。 这句话的完整意思是：你看到的所有行为，来源是「模型的行为倾向」和「你写的配置」共同作用的结果，它是一个研究材料——可以被分析、被比较、被反驳，但它的外推边界就是「在这套模型和这套配置下」。要谈真实人群，需要另外的证据（人类效度那一轴的工作）。

第二，当前原生问答每项只用一个 agent，其成绩不能直接证明完整多 agent 协作或长期城市仿真的效度。 一个 agent 答得好，只说明「单体的问答行为在这个参照下不离谱」；它不能推出「多 agent 协作会涌现出某种规律」，更不能推出「跑 600 天之后这个城市的行为可信」。分母不同、协议不同，历史 FOS 机制与多角色实验还各自保留独立协议——这些数字不能直接加总或互相比较。

把这两条翻译成一句写结论时能用的话：说「在 GAWorld 的这次运行里，事件 X 让指标 Y 发生了 Z 的变化」，不要说「事件 X 会让真实居民……」。

本章小结

- GAWorld 是**可控制的社会实验场**：居民有档案与记忆、环境有经济与网络、干预可审计、运行可对照；它不是「跑一群 Agent」的玩具，也不是对真实人群的预测器。
- 核心循环是感知 → 计划 → 日程 / 动作 → 执行 → 反思与记忆更新，实现层展开成 `CONFIG["pipeline"]["agent_step"]` 里的 12 个可配置阶段；天数推进累积 episode 记忆、长期总结、基于上下文的习惯、日级意图、关系变化与收支资产，而这些累积物是下一轮决策的输入，不是日志附件。
- 教师 / 学生走控制台与城市 CLI，研究者走平行世界与研究工作台，工程师走 `gaworld/kernel/` 与插件接口；三个入口的标志分别是「讲得清一个人」「说得清对照组」「加得进不改主循环的子系统」。
- 五级台阶里，**第 ④ 级「做实验」才是本平台的真正价值**：没有对照组，前三级都只是演示。

- 效度分**功能效度**与**人类效度**两条轴，分开报告、不混成总分；GAWorld Bench 见 https://s2014628.github.io/gaworld-bench/。
- 两条边界必须照抄：仿真输出不应直接当作真实人群的预测；原生问答每项只用一个 agent，其成绩不能证明多 agent 协作或长期城市仿真的效度。

动手练习

1. **把控制台跑起来并截图首页控制台**（约 20 分钟）
   - 装依赖（Python ⩾ 3.11）：`python -m pip install -e ".[dev]"`，或按 README 用 `python -m pip install "git+https://github.com/wuchaozju/GAWorld.git@main"`。
   - 起控制台：`python generative_city_sim.py dashboard --port 8766`（pip 安装后可用 `gaworld dashboard`），浏览器打开 http://127.0.0.1:8766/dashboard。
   - 在「配置 → 模型」里选服务商、填自己的 API Key、点连通性测试（这一步不测也行，先看界面）。
   - 截图整页控制台。
   - **验收标准**：截图里能同时看到左侧导航的五个分组（构建 / 运行 / 实验 / 分析 / 系统）、「仿真控制台」标题、右上的仿真天数与每天秒数输入框，以及底部的「运行仿真」按钮；把这张截图存进你自己的笔记目录（例如 `notes/ch01-console.png`）。
2. **在 `docs/FEATURES.md` 里找 3 个你没用过的能力，各写一句用途**（约 15 分钟）
   - 打开 `docs/FEATURES.md`，按五节（CLI 命令 / 核心仿真特性 / 新特性 v2 / 微内核插件体系 / 运维与调试）各扫一遍。
   - 挑 3 个你此前没用过、名字也不太懂的条目，把「作用」列改写成一句你自己的话：它能帮你回答什么问题。
   - **验收标准**：三条都写成了「这个能力能帮我……」的句子（不是抄「作用」列），并且你能在文件里指出每一条属于哪一节；如果你顺手跑了其中任意一条命令，把命令和它的输出贴在自己的笔记里。

ewpage

第 2 章　安装与工作区：把平台跑起来的前十分钟

2.1 这一章解决什么问题

上一章把 GAWorld 讲成了「生成式社会仿真平台」。这一章要把它变成一件可以上手操作的东西：从零装好、起一个本地服务、在浏览器里看见控制台，并且清楚每一步装到了磁盘的哪里、以后升级会不会动你的数据。

目标是十分钟能走完这条链路：安装 → 初始化工作区 → 起服务 → 浏览器打开控制台 → 用一条 `curl` 命令确认服务真的活着。

有两件事先说清楚，能省掉很多困惑：

第一，**安装和浏览控制台不需要任何模型 API Key**。Key 是仿真真正跑起来、居民要开口说话的时候才需要的。装完就能点开面板、翻配置、看城市地图。

第二，GAWorld 当前**没有发布到 PyPI**，只通过 GitHub 分发。所以 pip 安装命令里必须带一个 GitHub 地址，不是普通的 `pip install gaworld`。

安装、初始化、起服务、验证服务，这四步全部不碰模型。真到「让居民开口说话」那一步才需要 Key，那是第 3 章的事。把两件事分开，你今天就能先把平台跑起来，而不用担心课程预算。

2.2 两条安装路线

先做选择。两条路线的产物不一样：一条给你一个能用的工作区，一条给你一整个可编辑的仓库。

路线	面向谁	得到的东西	适合什么时候
pip 安装	使用者、课堂部署	一个独立的可写工作区 + `gaworld` 命令	只想跑实验，不想碰代码
源码安装	开发者、贡献者	完整仓库 + 全部测试与工具链	要改代码、要提 PR、要写插件

包名是 `gaworld`，当前版本 `0.2.1`，元数据在 `pyproject.toml` 里。两条路线都要求 **Python 3.11+**，pip 方式还需要 **Git**（pip 要从 GitHub 拉源码）。

2.2.1 pip 安装：面向使用者与课堂部署

```
python -m pip install "git+https://github.com/wuchaozju/GAWorld.git
        @main"
```

装完系统里多了两个命令，入口都写在 `pyproject.toml` 的 `[project.scripts]` 里：

命令	指向	用途
`gaworld`	`gaworld.launcher:main`	工作区初始化与本地控制台
`gaworld-deploy`	`gaworld.apps.deploy_services:main`	服务器上的服务部署

课堂部署建议把安装放进虚拟环境里，避免污染系统 Python：

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activat
        e
python -m pip install --upgrade pip
python -m pip install "git+https://github.com/wuchaozju/GAWorld.git
        @main"
```

激活之后 `gaworld` 就在 `PATH` 里了。如果后面遇到「命令找不到」，问题基本都在这里（见 2.6）。

命令一律写成 `python -m pip` 而不是裸 `pip`。裸 `pip` 装到哪个解释器取决于当时的 PATH 状态，`python -m pip` 一定装进你正在用的那个解释器。少踩一类坑。

装完可以确认版本：

```
python -m pip show gaworld
```

会打印 `Name: gaworld`、`Version: 0.2.1` 和 `Location:`。`Location` 指向你的虚拟环境 `site-packages`，说明装对了地方。

要升级就再加一个 `--upgrade`：

```
python -m pip install --upgrade "git+https://github.com/wuchaozju/G
        AWorld.git@main"
```

注意升级**只换代码，不动工作区**——已有的工作区保留原版本，你可以初始化一个新版本的工作区，再单独迁移自己的数据。

2.2.2 源码安装：面向开发者与贡献者

```
git clone https://github.com/wuchaozju/GAWorld.git
cd GAWorld
python -m pip install -e ".[dev]"
python -m pytest --suite core
```

`-e` 是可编辑安装：装进环境的是指向仓库的链接，你在 `gaworld/` 里改一行代码，Python 下次导入就看到新代码，不用重装。

`[dev]` 这个 extra 装五样东西：`pytest`、`pytest-cov`、`ruff`、`black`、`mypy`。而 `requirements-dev.txt` 比它多两项：`openapi-spec-validator` 和 `playwright`。所以如果你要跑完整的测试集，再补一句：

```
python -m pip install -r requirements-dev.txt
```

运行时依赖在 `requirements.txt` 和 `pyproject.toml` 的 `dependencies` 里是同一份，共六个包，按用途分三类：

类别	包	干什么
数据处理	`pandas`、`numpy`	状态长表、统计、绘图数据
网络与解析	`requests`、`pypdf`	抓取外部信息、读研究工作台上传的论文 PDF
图与可视化	`networkx`、`matplotlib`	社交图、轨迹回放与图表

`--suite core` 是冒烟集：只跑 `tests/suites/core.txt` 里列出的那批用例，覆盖主仿真路径。它同时是防漂移闸门——清单里多一条已经改名或删掉的测试，整轮直接失败。我在这台机器上实测：**489 passed, 1 skipped，约 68 秒**。那 1 个 skip 是 `openapi-spec-validator` 没装导致的（装上就会跑）。

2.3 工作区：pip 装完之后东西落在哪

pip 装完，磁盘上并没有一个「GAWorld 文件夹」等着你去改。这是有意的设计。

`gaworld.launcher` 在启动前会按顺序找一个「工作区」——一个自包含、可写、目录结构与仓库一致的运行时副本：

1. 环境变量 `GAWORLD_WORKSPACE` 指定的目录；
2. 当前目录下有 `.gaworld-workspace.json` 标记文件，就用当前目录；
3. 当前是源码检出（有 `pyproject.toml` 且有 `site/dashboard/index.html`），就直接用仓库；
4. 都不是，就在 `~/.local/share/gaworld/workspaces/default` 建一个默认工作区。

规则 3 解释了为什么在仓库里敲 `gaworld dashboard` 不会多出一份拷贝——它跑的就是仓库本身。

规则 1 给你一个手动指定的办法。假设你有两套实验，一套在 `~/research-a`，一套在 `~/research-b`：

```
export GAWORLD_WORKSPACE=~/research-a
gaworld dashboard
```

这个环境变量优先于其它一切规则。指错了会得到一句明确的拒绝：

```
GAWORLD_WORKSPACE must point to an initialized workspace.
```

同样退出码 2。它不会「猜」你指的是哪个目录——工作区必须带 `.gaworld-workspace.json` 标记且有 `generative_city_sim.py`，才算数。

2.3.1 显式初始化：gaworld init

想自己指定目录，就用 `init`：

```
gaworld init ./my-world        # 建在 ./my-world
cd my-world
gaworld dashboard
```

不带目录参数时，`init` 建在当前目录下的 `gaworld-workspace`。成功后会打印三行提示：

```
GAWorld workspace ready: /绝对路径/my-world
Run: cd /绝对路径/my-world
     gaworld dashboard
```

`init` 是幂等的：目标目录里已经有 `.gaworld-workspace.json` 且 `generative_city_sim.py` 还在，直接返回，不重复解压。

但如果目标目录**非空**（里面没有工作区标记文件，却已经有你自己的文件），它会拒绝并保留你原有的文件：

```
Choose an empty directory; existing files will be preserved: /路径/
my-world
```

退出码是 2。这不是 bug，是保护：工作区里会有你的居民数据和运行产物，覆盖等于丢数据。换个空目录再来即可。

2.3.2 工作区里到底装了什么

pip 装出来的 wheel 里带着一份压缩包 `gaworld/_assets/runtime.zip`。`init` 做的事就是把它解开到目标目录。解包前会逐条检查归档：路径不能是绝对路径、不能含 `..`、不能是符号链接；同时必须包含这四个文件，缺一个就报「Runtime archive is incomplete」并中止：

必需文件	作用
`generative_city_sim.py`	CLI 入口，仿真主程序
`site/dashboard/index.html`	控制台前端首页
`data/hangzhou_agents_state_init.csv`	种子居民状态表
`data/hangzhou_profiles_with_names.md`	种子居民档案

解完之后写一个标记文件 `.gaworld-workspace.json`，内容是包名和版本：

```
{
  "package": "gaworld",
  "version": "0.2.1"
}
```

工作区里还有 `gaworld/`（核心包）、`site/`（前端）、`data/`（种子资产）、`scripts/`、`docs/`。注意 **`tests/` 不在工作区里**——那是给开发者用的，pip 用户不需要。

前端是静态文件，不编译，直接由本地服务读出来。`site/` 下按用途分了几块：`dashboard/` 是控制台及各个专项面板（Agent Studio、城市、研究工作台、游戏场、群体采访…），`console/` 是运行/控制台页面，`simviz/` 是轨迹回放，`citymap/` 是城市地图页，`home/` 是站点首页，`documentation/` 是站内文档。

工作区建好之后，可以用一条命令确认它确实完整：

```
ls my-world/generative_city_sim.py my-world/site/dashboard/index.ht
        ml
ls my-world/data/hangzhou_agents_state_init.csv my-world/data/hangz
        hou_profiles_with_names.md
```

四个文件都在，才算一个能跑的工作区。

2.3.3 为什么换目录，而不是就地改仓库

一句话：仓库是代码，工作区是你的实验台。

仓库里的 `data/`、`docs/`、`tests/` 属于项目本身，升级时应当跟着代码一起换；工作区里的居民数据、记忆、日志、运行产物属于你，升级时必须原地保留。

所以 pip 路线坚持「装到另一个目录」：装包 → 建工作区 → 在工作区里跑。已初始化的工作区不会被覆盖；升级安装包之后，可以初始化一个新版本的工作区，再把旧工作区里自己攒的数据单独迁移过去。

这套分离还解决了一个很现实的问题：**一台机器上跑多套互不干扰的实验**。每个工作区有自己的居民数据、自己的记忆库、自己的 `output/`。想做「同一个人群，不同政策」的两组对照，就建两个工作区，各自跑，互不污染；想复现旧结果，就把旧工作区整个留着不动。

反过来，源码安装没有这层隔离——仓库里只有一份 `output/`。开发者要对比两次运行的产物，得自己先归档。这也是为什么 `output/` 不进版本控制：它随时会被清空，代码才是要留下的东西。

2.4 目录地图

先看平台本身的分层。理解了这张图，目录地图就是它的磁盘版。



图 2–1　系统分层架构

从上到下依次是交互层（`site/`）、API 层（`gaworld/apps/`）、微内核（`gaworld/kernel/`）、12 阶段认知管线、插件层（内置插件装配点 `builtin_plugins()`），最后是记忆、社交、家庭、经济、人格、城市等领域子系统。这张图是后面几章共同的骨架，本章只需要记住一件事：**界面层和 API 层只依赖内核，领域子系统全部以插件形式挂在事件总线上。**

目录地图就是这张图落到磁盘上的样子：

路径	是什么	你什么时候动它
`gaworld/`	核心包，所有功能的正式实现	要改行为、写插件时
`site/`	前端：`dashboard/`（各面板）、`console/`、`simviz/`、`citymap/`、`home/`	改界面时
`data/`	种子资产：居民 CSV、档案 MD、城市包 `cities/<slug>/`	造自己的城市时
`docs/`	设计文档、教程、部署说明	找答案时
`output/`	运行产物：记忆、状态长表、日志、事件流	只看，不提交

路径	是什么	你什么时候动它
`tests/`	测试套件	加功能时同步加测试
`legacy/`	旧版 flat 模块，已弃用、不参与构建	不要动
`scripts/`	辅助脚本（生成城市地图等）	按需
`generative_city_sim.py`	CLI 入口	几乎不动

`output/` **不纳入版本控制**——`.gitignore` 里有一条 `/output/` 把它整个排除掉。仿真跑得越久、实验开得越多，这个目录越大（详见 2.7）。它的定位是「研究材料」，不是「代码」；想留档就自己归档，别指望 git 会帮你记着。

`output/` 内部按用途分目录，值得认几个：

子目录	放什么
`memory/`	居民的记忆与成长记录
`state/`	状态历史长表（CSV），分析的原始依据
`logs/`	运行日志
`records/`	Recorder 事件流
`environment/`	环境事件与 `timeline.jsonl`
`parallel_worlds/`	平行世界实验的各世界产物
`research/`、`interviews/`	研究工作台与群体采访的产物

`data/` 里的城市包是另一回事：`data/cities/<slug>/` 放城市清单、地图、环境与人口，是**种子资产**，进版本控制；当前用的是哪座城市，记在仓库根的 `dashboard_config.json` 里，形如 `{"city": slug}`。

`legacy/` 里的旧版 flat 模块已经弃用、不参与构建（`pyproject.toml` 的 `packages.find` 明确排除了它），`legacy/README.md` 有一张「旧模块 → 新位置」的对照表。新代码不要往那儿写，旧的导入路径也只作为兼容层存在。

还有一条规矩值得记住：**新代码只写进 `gaworld/` 包，不在仓库根目录加新的模块**。根目录只保留 `generative_city_sim.py` 和 `environment.py` 两个 Python 模块，它们也是打包时分发的全部根模块（`pyproject.toml` 的 `py-modules`）。

2.5 启动与验证

2.5.1 起服务

源码安装下有两条等价命令，参数一样：

```
python generative_city_sim.py dashboard --port 8766
```

```
python dashboard_server.py --port 8766
```

第二条是根目录的兼容入口，它内部转调 `gaworld.apps.dashboard_server.run_server`，供老部署脚本继续用。pip 安装则用 `gaworld dashboard`（等价于第一条）。

默认只监听 `127.0.0.1`，也就是本机。想让同局域网的同学访问，需要显式加 `--host 0.0.0.0`，同时想清楚鉴权问题（见 `docs/SERVER_DEPLOYMENT.md`）。

启动后终端不会返回提示符，服务在前台跑着，请求日志一行行打出来（`127.0.0.1 - - [...] "GET /api/health HTTP/1.1" 200 -`）。这既是心跳，也是排查依据：出现 404 说明路由没接上，500 说明处理器抛了异常。停掉服务按 `Ctrl+C`。

浏览器打开：

```
http://127.0.0.1:8766/dashboard
```

`/dashboard`、`/console`、`/`、`/board` 都会打开控制台首页。

2.5.2 用 curl 验证服务真的活着

「浏览器打开了」不等于「服务正常」。命令行上有一条更硬的检查——健康检查路由 `/api/health`。我在这台机器上实测过：

```
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8766/api/
        health
```

返回：

```
200
```

再看一眼响应体：

```
curl -s http://127.0.0.1:8766/api/health
```

返回：

```
{"ok": true, "service": "gaworld-dashboard", "accounts": false, "ac
        counts_required": false}
```

四个字段各有含义：`ok` 是就绪标志，`service` 是服务名，`accounts` 表示账号库是否可用，`accounts_required` 表示是否强制登录。单用户模式下后两个都是 `false`——这也是**多用户部署**时判断自己有没有切到登录模式的依据。

这条路由有特殊待遇：它不返回任何用户数据，且**在登录鉴权之前就被应答**。所以就算你开了多用户账号系统，`curl` 也不会收到 401，可以放心拿它当探针。

2.5.3 控制台上该看到什么

服务起来之后，浏览器里是这个界面：

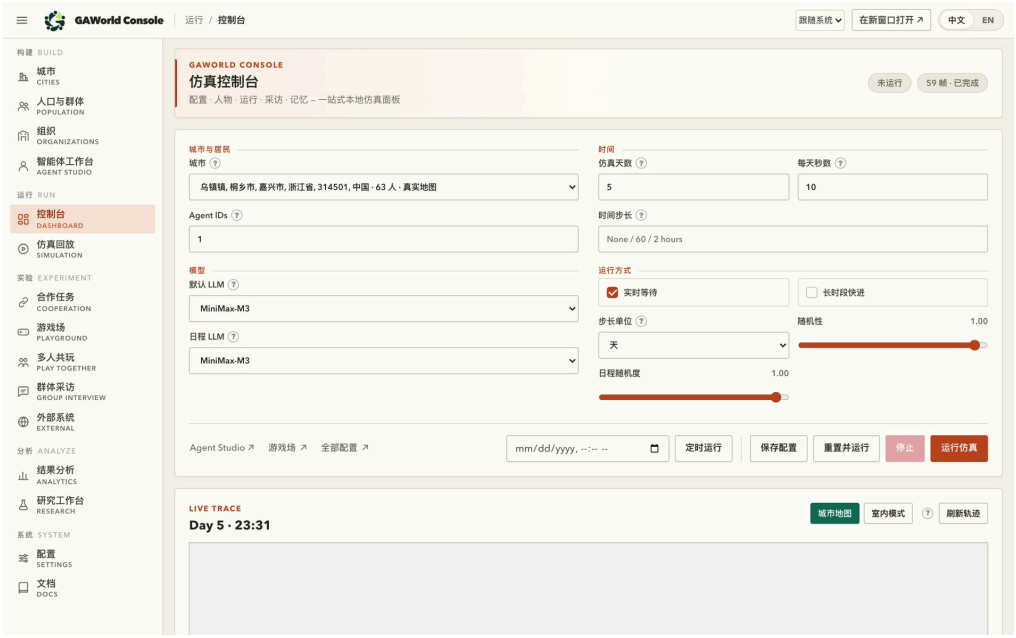


图 2–2　控制台入口

左边竖着五组导航——构建（城市 / 人口与群体 / 组织）、运行（控制台 / 仿真回放）、实验（合作任务 / 游戏场 / 多人共玩 / 群体采访 / 外部系统）、分析（结果分析 / 研究工作台）、系统（配置 / 文档）。中间是「仿真控制台」，四组配置摆在一屏上：城市与居民、时间、模型、运行方式。右上角的「未运行 / 59 帧 · 已完成」是状态指示。

这四组配置对应后面几章的内容：城市怎么选见第 5 章，模型怎么接见第 3 章，居民档案和种子数据见第 4 章；时间步长、长时段快进和运行时干预属于仿真运行那一部分的内容。

再往下一层是模型配置：

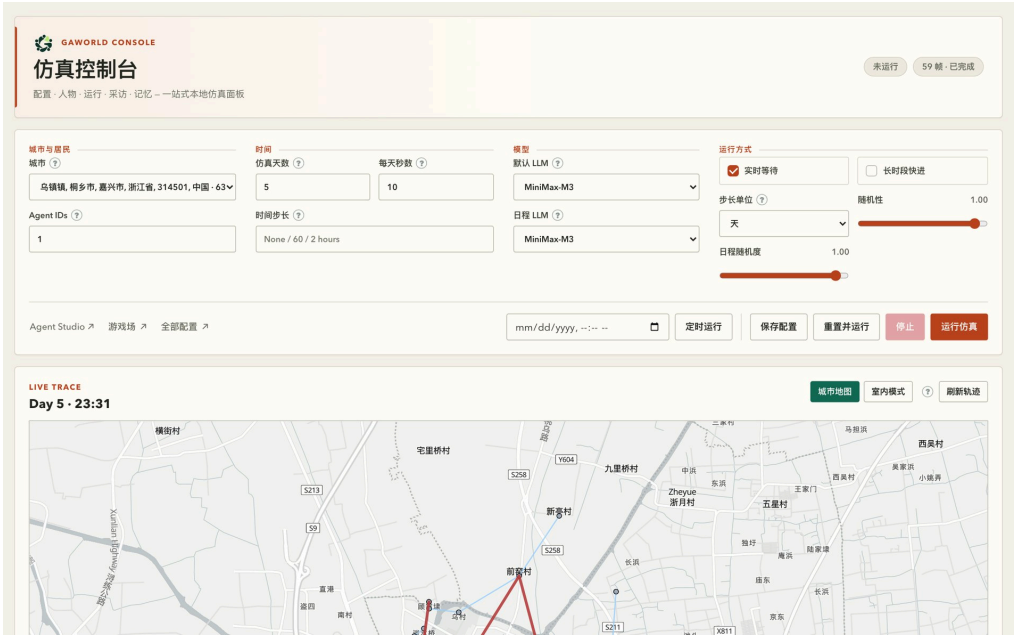


图 2–3　控制台配置面板

这是控制台的上半屏细节：左边「城市与居民」和「Agent IDs」决定跑谁，中间「时间」决定跑多久（仿真天数、每天秒数、时间步长），右边「模型」选默认 LLM 与日程 LLM，最右「运行方式」选实时等待还是长时段快进、随机性滑块。

底部的「运行仿真」是启动按钮。API Key 本身填在左下角「配置 → 模型」里，选定服务商、填 Key、点「测试连通性」，确认通了再回这里启动。**模型调用可能产生服务商费用**；但安装、开面板、看地图、翻配置都一分钱不花。

「测试连通性」这一步本身会产生少量模型调用，别在课上反复点；点通了就把结果记住，配置面板会保存下来。

控制台不是唯一入口，`gaworld/client.py` 提供了一个按 OpenAPI `operationId` 调用任意接口的 Python 客户端，写自动化实验脚本时用得上——每条路由都在 `gaworld/apps/openapi.py` 登记过，客户端按 `operationId` 找它，不用手拼 URL。

2.6 常见安装故障

按遇到的频率排。每一个都给了可以直接抄的排查命令。

2.6.1 Python 版本过低

症状是装完一 import 就报语法错误，或者 pip 直接拒绝。GAWorld 要求 Python ≥ 3.11。

```
python -V
```

打出 3.10 或更低就是它了。macOS 上系统自带的 `python3` 常常是 3.9，装一个新版解释器，或者用 `pyenv` / `conda` 切一个 3.11+ 的环境。

2.6.2 依赖冲突

```
python -m pip list | grep -Ei "pandas|numpy|matplotlib|networkx|pypdf|requests"
```

平台对这几个包都给了上界（例如 `pandas>=2.0,<3.0`），和 `numpy>=1.24,<3.0` 搭配时如果混进了 numpy 2.x 以外的老版本，或者系统里残留了别的项目留下的旧 pandas，就容易报 `ImportError` 或 `ValueError: numpy.dtype size changed`。干净的虚拟环境最省事：

```
python -m venv .venv && source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

2.6.3 端口被占用

这是最容易和「服务坏了」混淆的一种。真实报错长这样：

```
OSError: [Errno 48] Address already in use
```

查是谁占着：

```
lsof -i :8766
```

输出里会有进程号和命令名（我这次遇到的是一个已经在跑 `python3 ... dashboard` 的进程）。两种处理：杀掉那个进程，或者换个端口。我实测过换端口的写法：

```
python generative_city_sim.py dashboard --port 8799
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8799/api/health
```

返回 `200`，服务正常。注意换端口之后浏览器地址也要跟着换。

2.6.4 gaworld 命令找不到

```
which gaworld
python -m pip show gaworld
```

`pip show` 能查到、但 `which` 查不到，就是 PATH 问题——命令装到了某个没进 PATH 的目录。macOS/Linux 上最常见的是没激活虚拟环境（`source .venv/bin/activate`），或者装到了用户目录 `~/.local/bin` 而这个目录不在 PATH 里。临时补上：

```
export PATH="$HOME/.local/bin:$PATH"
```

确认一下装到哪了：

```
python -c "import sysconfig; print(sysconfig.get_path('scripts'))"
```

如果 `pip show gaworld` 直接报 `Package not found`，那是根本没装成功，回去重跑 2.2.1 的安装命令。

2.6.5 工作区目录非空

```
Choose an empty directory; existing files will be preserved: /路径/my-world
```

换一个空目录就行：

```
gaworld init ./my-world-2
```

如果提示的是 `Incomplete GAWorld workspace`，说明目录里有标记文件但 `generative_city_sim.py` 丢了——多半是手动删过文件。换个干净目录重建，再把需要的数据拷回去。

2.6.6 冒烟测试里 openapi 那条失败

`pytest --suite core` 里有一条 `test_every_server_route_is_documented`，它会静态扫描 `gaworld/apps/` 里的路由字面量，跟 `gaworld/apps/openapi.py` 的文档逐条比对。如果这个测试报「add these routes to gaworld/apps/openapi.py」，而你并没有加过路由，那多半是工作区里 `gaworld/apps/*.py` 正处于半改状态（比如同事正在改、还没保存完）。把改动收干净再跑一次。更深一层的排错见第 19 章。

2.6.7 装成功了却 import 不到

```
python -c "import gaworld; print(gaworld.__file__)"
```

报 `ModuleNotFoundError: No module named 'gaworld'`，但 `python -m pip show gaworld` 又查得到——说明装它的解释器和你现在用的不是同一个。最常见的原因是漏了 `source .venv/bin/activate`，或者在别的虚拟环境里执行的安装命令。对照着看：

```
python -V
which python
python -m pip show gaworld | grep Location
```

三行的路径要指向同一个环境。

2.6.8 pip 下载阶段就失败

pip 命令里的地址是 `git+https://github.com/...`，pip 会先调 git 拉源码。所以 git 不在 PATH，或者网络到 GitHub 不通，pip 会直接报错而不是进入依赖解析。检查：

```
git --version
git ls-remote https://github.com/wuchaozju/GAWorld.git HEAD
```

第二条能打印出一个提交号，说明网络和 git 都正常。挂代理时，pip 和 git 各走各的配置——`pip` 认 `HTTPS_PROXY`，`git` 认 `https_proxy` 配置项，两边都要配。

2.7 磁盘与成本预期

2.7.1 磁盘

`output/` 会长，而且长得不慢。每次运行会写出记忆库、状态历史长表、运行日志、Recorder 事件流、环境时间线；做实验还会多出 `parallel_worlds/`、`research/`、`interviews/` 这些目录。

我在这台机器上量到的实际占用是 **646 MB**，其中 `parallel_worlds/` 310 MB、`memory/` 119 MB、`video/` 89 MB、`cities/` 24 MB。单个 `output/logs/run.log` 也能到 MB 级。

这不是问题，是研究材料的正常体量。几条实用建议：

- 长期实验把 `output/` 里的单个实验目录单独归档，别一直堆在同一个盘上；
- 删旧产物前先确认它不是你要引用的研究材料（记忆和状态表是分析依据）；
- `output/` 不会进 git，也不用管它和 `.gitignore` 的关系——它本来就被忽略了。

2.7.2 成本

模型调用按服务商的标准计费，一次仿真里每个居民每个时段都可能触发调用，所以**居民数 × 仿真天数 × 每天时段数**是费用的量纲。

省钱的做法按顺序是：先用 1 个居民、1 天跑通全流程，确认流程和输出都对；再把天数拉长；最后才加大居民数。控制台上的「仿真天数」「每天秒数」两个框就是调节阀。控制成本的具体手段（哪些阶段走确定性规则、哪些才进模型）见第 3 章。

课堂场景还有两个提醒：

- **给学生的练习题**。让全班每人跑 5 个居民 × 10 天，比让每人跑 50 个居民 × 10 天便宜一个数量级，而且大部分教学目标（看轨迹、读记忆、比较个体差异）在这个规模上都成立。
- **先算再跑**。正式实验开跑前，用控制台上的「配置」面板把服务商、模型和规模填好，心里先有一份预算，再点「运行仿真」。跑完了看服务商后台的实际账单，再回头校准估算——比事后发现超支好得多。

还有一点值得知道：平台并不是每个环节都调模型。大五人格有三条通道，其中 `rules` 那条是确定性计算、**零 token**；群体（cohort）模式下每群每天只花 1 次调用。知道这些，你才知道「把规模调大」这件事的边际成本在哪。模型路由的细节见第 3 章。

本章小结

- 两条路线：pip 装给使用者（`gaworld dashboard` 即可），源码装给开发者（`pip install -e ".[dev]"` + `pytest --suite core`）。GAWorld 未发布 PyPI，pip 命令必须带 GitHub 地址。
- pip 安装产出一个**独立可写的工作区**，默认在 `~/.local/share/gaworld/workspaces/default`；`gaworld init [目录]`

可显式指定，但拒绝写入非空目录——这是保护你的数据。

- 工作区是「你的实验台」，仓库是「项目代码」；升级包不动你的数据，跨版本迁移要自己搬。
- 仓库目录地图：`gaworld/` 是核心包，`site/` 是前端，`data/` 是种子资产，`output/` 是产物且不纳入版本控制，`legacy/` 已弃用。
- 验证服务用 `curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1:8766/api/health`，返回 200 即为正常；`/api/health` 在登录鉴权之前应答，可以当探针用。
- 端口占用、`gaworld` 不在 PATH、工作区非空，是最常见的三类安装故障，各自都有明确排查命令。

动手练习

1. **源码安装并让冒烟测试全绿**（约 20 分钟） 在一台干净机器或全新虚拟环境上执行：

```
git clone https://github.com/wuchaozju/GAWorld.git
cd GAWorld
python -m venv .venv && source .venv/bin/activate
python -m pip install -e ".[dev]"
python -m pytest --suite core
```

验收标准：最后一行输出形如 `489 passed, 1 skipped`（数字随版本变化，判据是 **0 failed**），且 `python -c "import gaworld; print(gaworld.__file__)"` 指向你克隆的仓库路径——说明可编辑安装生效。

2. **在空目录里初始化工作区并确认标记文件**（约 10 分钟） 先用 pip 装一次（见 2.2.1），然后：

```
mkdir -p ~/gaworld-lab && cd ~/gaworld-lab
gaworld init ./my-world
ls -a ./my-world
cat ./my-world/.gaworld-workspace.json
```

验收标准：`./my-world` 目录里同时出现 `generative_city_sim.py`、`site/`、`data/` 和 `.gaworld-workspace.json`；`cat` 出来的 JSON 里 `"package"` 是 `gaworld`。

再验证一次保护机制。**注意**：对一个已经初始化成功的工作区重跑 `gaworld init` 是幂等的，不会报错——要触发「目录非空」的拒绝，得用一个**没有标记文件**的目录：

```
mkdir -p ./not-empty && touch ./not-empty/my_note.txt
gaworld init ./not-empty
ls ./not-empty
```

验收标准：`init` 打印 `Choose an empty directory; existing files will be preserved`，退出码非 0，且 `ls ./not-empty` 里 `my_note.txt` 还在——你的文件没有被删。

附加一步：进到 `my-world` 里跑 `gaworld dashboard`，用 `curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8766/api/health` 确认返回 `200`，然后浏览器打开 `http://127.0.0.1:8766/dashboard` 看见控制台首页。

ewpage

第 3 章　接入模型：让居民真的能思考

前两章把平台的面貌和城市的样子讲清楚了。现在要解决一个更硬的问题：**让居民动起来**。GAWorld 的居民不是规则脚本，而是靠大模型在感知、计划、动作、反思里一步步想出来的。所以这一章只做一件事——把模型接上，并且接得能复现、能省钱、能查错。

3.1 没配模型，仿真连第一步都走不完

`gaworld/llm/providers.py` 的最后一行是 `LLM_ROUTER = LLMRouter(CONFIG)`，它在模块导入时就把所有后端实例化一遍。`LLMRouter._init_providers` 里只有一句硬话：

```
if not providers:
    raise ValueError("No LLM providers configured.")
```

这句话的意思是：一个可用后端都没有，进程在第一个居民迈步之前就退出了。你在控制台里点「运行仿真」，看到的不是一份空日志，而是这一行异常。

配好之后，居民的一天就被拆成若干次模型调用。下表是当前代码里最常用的任务名（`call_llm(prompt, task=...)` 的第二个参数），它们是后面做路由和省钱时唯一认得的名字：

环节	任务名	说明
感知	`perception`	描述此刻的环境、他人与制度
日程	`schedule`	生成一天 6–10 项日程；解析失败时退回启发式模板
计划	`planning`	在日程与约束下决定这一步做什么
动作	`actions` / `location_actions`	为每个活动生成具体动作与地点偏好
反思	`reflection`	回顾结果、写下感受与教训
意图	`daily_intentions`	每天开头生成优先级与回避项
日记与总结	`daily_diary` / `summary`	写日记、压缩长期记忆
记忆	`memory_consolidation` / `memory_review`	整合经历、提取教训
采访	`interview` / `interview_summary`	回答问题并汇总
游戏与实验	`games.*` / `experiment` / `research`	裁判、评审、实验臂回答

反过来，有一批环节**根本不发请求**，它们是纯代码算出来的。搞清这一条很重要：它决定了你的账单能压到多低。

- **大五人格的 `rules` 通道**：`gaworld/personality/traits.py` 写得很明确——它做的是「确定性调制」，「Zero tokens, fully reproducible」。人格对行为的影响是加权，不是提示词。
- **出行决策**：`gaworld/travel/trigger.py` 的 `decide()` 是纯函数加 seeded 随机数，「the same seed puts the same agent on the same trip」。出门去哪、待几天都不花 token。
- **基准日程的兜底**：模型没给出可解析的日程时，`generate_schedule()` 会退回 `_heuristic_schedule(agent)`——按人物原型拼出来的模板。
- **经济与公共部门**：货币守恒、税率、选举与政策效果都是确定性推进。
- **外部环境事件**：这一块分模式。`external_environment.generator.mode` 默认让模型每天编事件（任务名 `external_environment`，多天合成一段时是 `external_environment_period`）；改成从事件池按规则抽就免费，但会重复。配置说明写得很直白：「Have an LLM invent events (more varied, costs money) or draw them by rule from the event pools (free, repetitive)」。

顺便看一眼模型到底收到了什么。以动作生成为例（`gaworld/sim/_action.py` 的 `_llm_generate_actions_batch()`），一次提示词由四块拼成：角色资料（姓名、年龄、职业、人格行、日常生活与习惯、价值观与公共事务态度）、检索到的近期记忆、当天要覆盖的活动列表，以及「输出 JSON、每个活动 5–10 个中文动作」这样的格式契约。反思那一环（`generative_city_sim.py` 的 `reflection()`）也是同样的结构：结果 + 回忆片段 + 输出格式。所有这些文本都会发给模型服务商——第 3.8 节会再说这件事。

还有一层例外要注意：**群体（cohort）模式默认是走模型的**。`gaworld/group/cohort_day.py` 每个 cohort 每天发一次 `group_cohort_day` 调用，失败时退回确定性摘要（`fallback_cohort_digest`）。想彻底不花钱，用 `python -m gaworld.group --no-llm`，帮助里写的是「Deterministic run, zero LLM calls」；或者把 `long_run.brief_llm` 置为 `false`。

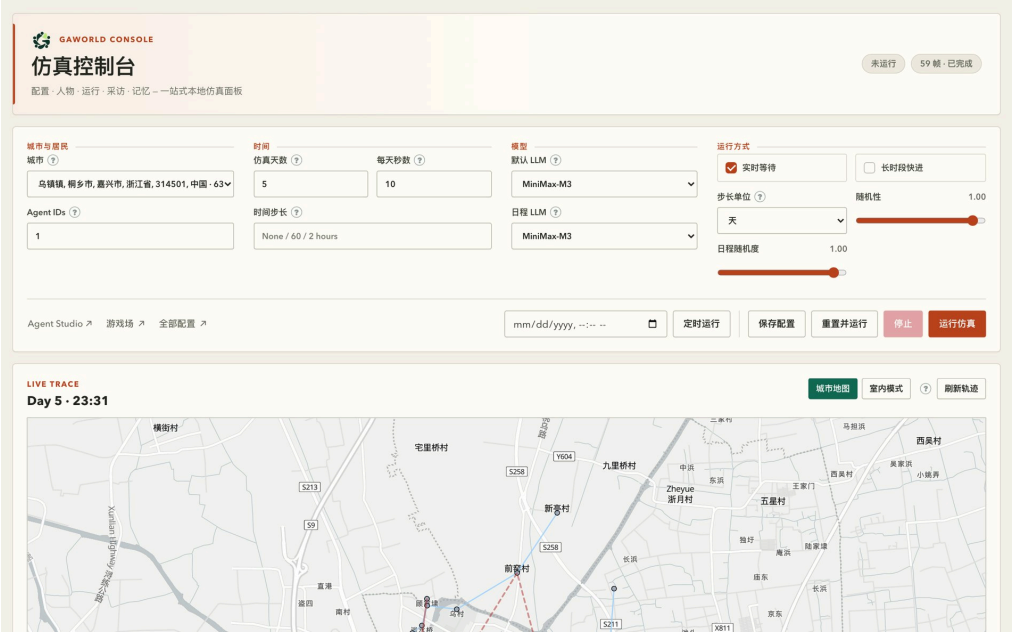


图 3–1　仿真控制台的配置区 这一屏是每次开跑前要看的全部内容。左边「城市与居民」和「时间」决定跑谁、跑多久，中间「模型」决定谁替居民思考，右边「运行方式」决定跑多细。注意它和下方实时轨迹是同一页：配置区改了，下一次运行才生效，仿真跑起来以后这几个控件会被禁用，想换配置得先停。

3.2 三类后端，先选一个

GAWorld 只认三种协议，由 `build_provider()` 里的三个分支决定（`gaworld/llm/providers.py`）：

类型	协议与默认地址	密钥	适
`ollama`	Ollama `/api/generate`，默认 `http://localhost:11434/api/generate`	不需要	离网为
`openai`	OpenAI Chat Completions，默认 `https://api.openai.com/v1`	`api_key_env` 指名的环境变量	国（智 vL St
`anthropic`	Anthropic Messages，代码自动拼 `/v1/messages`	`api_key_env` / `api_key_envs`	M 区 其

「OpenAI 兼容」指的是**接口格式**，不要求买 OpenAI 的 Key。`docs/MODEL_PROVIDERS.md` 里的原话是：「这里的 `openai` 表示接口格式，不要求购买 OpenAI 的 Key。」反过来也别填错——「不要把 GLM 或 DeepSeek 的 Key 填入 MiniMax 的输入框。密钥按账号、协议和完整 Base URL 绑定。」

三类的取舍很直白：

- **本地 Ollama**：不花钱、断网可用，但慢。`build_provider()` 给它的默认超时是 600 秒，注释里写着「a 12B answer takes ~75s on this machine」。教学场景先 `--sim-days 1` 试水。
- **云端接口**：快、模型强，但按量计费。超时默认 120 秒。
- **Anthropic 兼容**：除了 Claude，还覆盖 MiniMax 中国区这类网关。`gaworld/settings/llm.py` 里的 `minimax` 预设把 `ANTHROPIC_AUTH_TOKEN` 之类的通用 Anthropic 变量排除在外，只认 `MINIMAX_API_KEY`——注释说得很清楚：fallback 到通用 Anthropic 变量「would send an Anthropic token to Minimax and 401」。

代码里已经预置了几个可以直接用的后端：`deepseek_flash`（`https://api.deepseek.com`，模型 `deepseek-flash`）、`deepseek_pro`（同端点，模型 `deepseek-v4-pro`）、`glm_5`（`https://open.bigmodel.cn/api/paas/v4`，模型 `glm-5`）。它们**不含任何内置密钥**，各自读 `DEEPSEEK_API_KEY` / `ZHIPU_API_KEY`。这三个预设都显式写了 `"thinking": "disabled"`，原因写在 `docs/MODEL_PROVIDERS.md` 里：「新增的云端预设显式关闭思考模式，避免仿真任务的短输出预算被推理耗尽。」

3.3 配置路径 A：改配置 + 设环境变量

平台把配置分成几层，后面的层覆盖前面的层（`gaworld/settings/overrides.py` 的 `apply_runtime_overrides()`）：

1. **代码默认值**：`gaworld/settings/` 下的各片段（`llm.py`、`runtime.py`…）由 `build_default_config()` 汇总成 `CONFIG`。
2. **控制台覆盖文件**：`dashboard_config.json`（仓库根目录）。
3. **`GAWORLD_CONFIG_OVERRIDES`**：一段 JSON 环境变量，在导入时合并。
4. **环境配置文件**：`data/environment_config.json`。它只认 `environment` / `external_environment` / `external_environment_service` / `environment_server` 这四个块，**碰不到 LLM 设置**。
5. **城市包配置**：选了城市之后，城市自己的配置再叠上去。

6. **`GAWORLD_CONFIG_OVERRIDES` 再合并一次**——所以它在最后落地，**对 LLM 路由有最终决定权**。

配置页面对每一项都会标出「这个值是从哪来的」（默认 / 覆盖文件 / 环境变量 / 配置文件），页面上还有一个「只看被改过的」勾选框。这不是装饰。`site/dashboard/settings.js` 的设计注释里写得很直白：「来源比数值更重要。改了一项、保存成功、却什么都没变，是这里最贵的一种困惑。」排查「我改了怎么没生效」时，第一步永远是看这一行来源标签。

先看默认配置长什么样（`gaworld/settings/llm.py` 的真实结构）：

```
"llm": {
    "providers": {
        "openai_gpt": {
            "type": "openai",
            "base_url": "https://api.openai.com/v1",
            "model": "gpt-5.4",
            "api_key_env": "OPENAI_API_KEY",
            "timeout": 120,
        },
        "ollama_qwen": {
            "type": "ollama",
            "url": "http://localhost:11434/api/generate",
            "model": "qwen3.5:9b",
            "timeout": 600,
        },
    },
    "routing": {
        "default": "ollama_gemma4",
        "tasks": {"schedule": "ollama_gemma4"},
        "fallback": ["ollama_gemma4"],
    },
},
```

要用源码方式接一个新后端，就是往 `providers` 里加一个块，再把 `routing.default` 指过去。字段名必须和 `build_provider()` 读的一致：OpenAI 兼容用 `base_url` / `model` / `api_key_env` / `timeout` / `stream` / `max_tokens` / `temperature` / `thinking`；Anthropic 兼容用 `base_url` / `model` / `api_key_env` / `api_key_envs` / `anthropic_version` / `max_tokens` / `authorization_scheme` / `include_x_api_key` / `authorization_retry_schemes`；Ollama 用 `url` / `model` / `timeout` / `think` / `num_ctx`。

密钥走环境变量，**不要写进配置**。仓库根目录的 `.env` 会在导入时被 `gaworld/__init__.py` 读进来（`.env.example` 里有完整清单）：

```
# OpenAI 兼容（DeepSeek / 智谱 GLM / vLLM 都走这一类）
export OPENAI_API_KEY="sk-..."
# Anthropic 兼容；中国区 MiniMax 的例子来自 docs/TUTORIAL.v2.md
export ANTHROPIC_BASE_URL="https://api.minimaxi.com/anthropic"
export MINIMAX_API_KEY="your_key_here"   # 或 ANTHROPIC_AUTH_TOKEN
```

改完确认一下生效的到底是什么：

```
python - <<'PY'
from gaworld.settings import build_config
cfg = build_config()
print(cfg["llm"]["routing"]["default"])
print(cfg["llm"]["routing"]["tasks"].get("schedule"))
```

```
print(sorted(cfg["llm"]["providers"]))
PY
```

输出的第一个名字必须在 `providers` 里存在，否则 `LLMRouter.call` 会抛 `Provider '...' not found in config.`。`routing` 一共四个键，用途各不相同：

键	作用
`default`	兜底后端：所有没有单独指定任务的调用都走它
`tasks`	按任务名指定，如 `{"schedule": "便宜后端"}`
`agents`	按居民 id 指定，给个别人开小灶
`fallback`	主后端失败后依次尝试的后端列表

`fallback` 是一张安全网，不是一张省钱表：一次请求在主后端失败后落到 fallback，答案就不是同一个模型给的了。实验场景要的是可比性，所以 `call_llm` 提供了 `allow_fallback=False`——「an experiment would rather lose a cell than have it answered by a different model than its neighbours」。

3.4 配置路径 B：控制台的「配置 → 模型」

多人部署时不该让学生去改仓库里的 Python 文件。控制台的配置页里有一个「语言模型」分区（地址 `settings.html?tab=llm`），官方文档和面板提示里都简称它为「配置 → 模型」。

打开后有三件事要做：

一、登记后端。「添加本地或云端模型」卡片里的六个字段：类型、名称、模型名、接口地址、密钥环境变量、超时（秒）。地址的填法有讲究——OpenAI 兼容「地址填到 `/v1` 为止，不含 `/chat/completions`」，Anthropic 兼容「地址不含 `/v1/messages`，代码会自己拼」，Ollama「地址填到 `/api/generate` 为止，模型名要和 `ollama list` 里的完全一致」。

二、测连通性。这个按钮不是 TCP 探活，它会真的发一次生成请求，提示词是 `Reply with the single word: OK`。`probe_provider()` 的注释解释了为什么非做不可：「the ways an LLM backend is 'reachable but useless' (model name not pulled, key rejected, reasoning budget consumed before any text) all return a healthy connection and would pass any cheaper test」。想先测草稿再保存也行——草稿测试只有管理员能用（`test_provider` 里对非管理员的草稿路径抛 `PermissionError`）。

三、选默认模型和日程模型。回到仿真控制台，把「默认 LLM」和「日程 LLM」都指到刚测通的后端，点「保存配置」。这两个下拉分别写的是 `llm.routing.default` 和 `llm.routing.tasks.schedule`（见 `site/dashboard/app.js` 的 `configPayloadFromForm()`）。保存后再打开配置确认一遍，别只信保存动作。

「已配置的后端」列表里的每一行会显示类型、模型名、端点、超时，以及密钥状态（「密钥已就绪」或「缺密钥：某某变量」）。缺密钥的那一行说明后端本身没问题，只是环境变量没读到。配置页还有两个附带的页签：「环境变量」只显示有没有配、配的是哪一个（密钥打码），「配置文件」直接显示各层覆盖文件的原文——排查冲突时比翻目录快。

这里有两条安全设计要理解：

- **面板不收明文密钥。** `dashboard_config.json` 是会被版本库收录的文件，所以「添加后端」表单里填的是环境变量的**名字**（`api_key_env`），不是密钥本身；服务端也会直接拒绝带 `api_key` 的请求。个人密钥走单独的「我的 API Key」入口（`POST /api/settings/llm/credential`），或者单人部署直接写进 `.env`。

- **多人部署的密钥按账号隔离。** `gaworld/llm/credentials.py` 把密钥绑在「账号 + 协议 + 完整 Base URL」三元组上，同一账号的两个 DeepSeek 预设因为端点相同可以复用，其它账号拿不到。存放位置由 `GAWORLD_LLM_SECRETS_PATH` 指定，必须是 Git 仓库之外的绝对路径，目录权限 0700、文件 0600。

同一套能力也有 HTTP 接口，可以在脚本里用：

```
POST /api/settings/llm/provider   {"name": "...", "type": "...", "config": {...}}
POST /api/settings/llm/test        {"name": "..."}   或 {"config": {...}}（草稿）
POST /api/settings/llm/credential {"name": "...", "action": "save", "api_key": "..."}
```

管理员登记一个本地后端的现成例子在仓库里：`deployment/multi-user/provider-qwen27b-server13.json`，内容是 `{"name": "qwen_27b_server13", "config": {"type": "ollama", "url": "http://127.0.0.1:11436/api/generate", "model": "qwen3.5:27b-q4_K_M", "timeout": 600, "think": false, "num_ctx": 65536}}`，提交给上面第一个接口即可。



图 3–2　外部系统观测台 这一页是「看住世界本身」的四个页签，图中停在「货币系统」。真正和本章相关的是第四个页签「对外服务」：里面有一张 LLM 路由卡片，列出当前已登记的后端、默认后端，以及一张按任务分派的路由表。配好模型后回来看一眼这张卡，比回配置页更快确认「跑起来的那一轮到底用的谁」。

3.5 路由：同一个居民，哪些事找谁

`call_llm()` 的完整签名是：

```
call_llm(prompt, task=None, agent_id=None, provider=None, system=None,
         temperature=None, allow_fallback=True, images=None, max_tokens=None)
```

`LLMRouter._select_provider()` 的解析顺序是固定的：显式传的 `provider` → `routing.tasks[task]` → `routing.agents[agent_id]` → `routing.default` → 第一个已登记的后端。之后 `_resolve_chain()` 把 `routing.fallback` 里的名字追加在后面，去重、跳过不存在的。

省钱的第一个杠杆就是让 `schedule` 走便宜后端。 日程是典型的「量大、不需要聪明」：只要输出一个 6–10 项的 JSON 数组。配置面板自己也是这么写的——`llm.routing.tasks` 的说明是「给个别任务单独指定后端，比如把排日程这种量大又不需要聪明的活派给便宜的本地模型」。做法是在配置里加一行：

```
{
  "llm": {
    "routing": {
      "default": "deepseek_pro",
      "tasks": {
        "schedule": "ollama_qwen",
        "actions": "ollama_qwen",
        "perception": "ollama_qwen"
      }
    }
  }
}
```

日程、动作、感知是三个调用密度最高的环节，把它们派给本地小模型，思考类的 `planning` / `reflection` 留给强模型。注意控制台目前只暴露了「日程 LLM」一个任务键，其余任务名要自己写进配置。



图 3–3 「模型」组的两个下拉 左边「默认 LLM」决定绝大部分调用的去向，右边「日程 LLM」单独指定生成每日日程的后端。两者可以是同一个，也可以不同——把日程派给便宜模型、其余留给强模型，是最省事的省钱设置。下拉里的选项来自当前已登记的后端，登记新的后端后要刷新页面才会出现。

还要记住两条边界：

- **改了模型或思考模式就改了实验语义。** `docs/MODEL_PROVIDERS.md` 的原话是「更改模型或思考模式会改变实验语义，应在实验记录中保留配置」。所以新增的云端预设都显式写 `"thinking": "disabled"`，避免推理块把短输出预算吃光。
- **fallback 不是一个模型。** 如果一轮实验里有些格子是被 fallback 后端回答的，比较就废了。`run_manifest` 会把这种请求单独记成 `requests_fell_back`，看到它不为零就要警惕。

还有三个参数是**每次调用**可以临时覆盖的，源码里给出了它们的用途：`system` 和 `temperature` 默认是 `None`，意思是「用后端配置的默认值」，它们存在是为了做提示词实验——「the system message *is* the treatment and the sampling temperature is part of the protocol rather than a deployment setting」；`max_tokens` 可以临时抬高或压低单次调用的输出上限，因为这个仿真的一轮通常只要「一个事件、一个决定、一段短档案」，而需要整篇结构化文档的调用方必须自己说清楚，否则服务商会在半路截断 JSON，下游解析器看到的是畸形输入而不是一个预算问题。

3.6 成本控制：先量，再省

怎么量。 每次运行结束，`output/run_manifests/` 下会写一份 manifest（JSON 与 HTML 各一份），其中 `llm` 段直接给出这次运行的账单：

```
"llm": {"call_count": 512, "by_task": {...}, "by_provider": {...},
        "failure_count": 3, "requests": 500,
        "requests_failed": 1, "requests_fell_back": 11}
```

`by_task` 告诉你钱花在哪个环节上——如果 `perception` 占了八成，那把感知派给本地模型就是最直接的省钱动作。`by_provider` 告诉你钱花在哪个后端上：两个后端各占一半，说明 fallback 链被触发得很频繁，值得回头看一眼主后端是不是在超时。

量级直觉。 差别来自 `gaworld/settings/runtime.py` 的注释：同样 10 年、同样 50 个居民，快进模式下按年粒度是 500 次调用，按天粒度是 182,500 次。

跑法	调用次数	说明
快进 + 按年步长	500	10 年 × 50 人，每步一条简报
快进 + 按月步长	6,000	120 个月 × 50 人
快进 + 按天步长	182,500	3,650 天 × 50 人
普通日内循环	取决于步长与人数	每个 tick 每人数次（感知、计划、动作、反思）

按年粒度是 500 次，按天是 182,500 次——**差 365 倍**。跑数年以上的实验必须切到月或年粒度，这句话不是建议，是算术。

还有一个容易踩的坑：`time_grid_snap` 默认关闭，而它关着的时候，「the master timeline is the *union* of the grid and every agent's LLM-authored times, so tick count — and therefore total LLM cost — grows super-linearly with the agent count」。也就是说，**人越多，调用量的增长不是线性的**。打开它（需要同时设 `time_step_minutes`），tick 数被钉死在 `1440 / time_step_minutes`，与人数无关。注意它默认关着是有理由的：它会改变日内时刻，开了之后同种子轨迹就不再逐位可比了。

其它可查的量级：一次采访轮次（interview round）大约 200 次模型调用（`gaworld/accounts/usage.py` 的注释里给的数，「a call is what a teacher can reason about」）；控制台里「测试连通性」那一次探测也算一次调用。

多用户部署的软配额。 多人模式下每次 `call_llm`（包括失败的）都会往账号库旁边的 `usage.jsonl` 追加一行（时间、用户、世界、任务）。管理员可以给每人设每日上限：

```
curl -X POST -H "Authorization: Bearer $GAWORLD_DASHBOARD_TOKEN" -H
    "Content-Type: application/json" \
  -d '{"daily_llm_calls_per_user": 2000}' http://127.0.0.1:8766/
    api/worlds/settings
```

`daily_llm_calls_per_user` 默认 0，也就是不限。超了之后**只拒绝会启动模型调用的请求**（返回 429），已经在跑的任务不中断，改配置、停止、删除都不受影响，管理员也不受限。管理员用 `GET /api/auth/usage` 看每人今日与累计调用次数。

具体的成本模型、每档模型的单价、怎么在预算内做实验设计，第 6 章展开。

3.7 常见故障与排查

出问题的顺序基本固定：先看配置对不对，再看密钥，再看服务商。逐条对照下表。

现象	原因	怎么确认	怎么绕开
面板显示「缺密钥」	后端要的 `api_key_env` 与实际环境变量名对不上；或进程启动时环境变量还没设	「配置 → 环境变量」页签只显示有没有配、配的是哪一个	变量名写进 `.env` 后**重启**进程；单人部署最省事
HTTP 401 / 403	密钥无效、没权限，或把 A 家的密钥填进了 B 家的端点	服务商控制台看密钥状态与额度；错误信息里 `api_key_source` 会告诉你是从哪个变量读到的	换正确端点的密钥；`authorization_scheme` 与 `include_x_api_key` 按服务商要求调
HTTP 400「余额不足 / 额度」	账号没充值，或套餐不含这个模型	服务商控制台看余额与模型权限	换后端或换 `routing.default`；GLM 编码套餐不等于标准 API 额度
HTTP 429 速率限制	并发或配额超了	响应头 `Retry-After` 会被读进等待时间	平台会自动重试（3 次，指数退避）；`retry_delay()` 会尊重 `Retry-After`；降低并发或改用更贵的档位
返回内容被内容过滤拒绝	提示词或上下文触发了服务商审核	错误响应体会被截断并把密钥替换成 `[REDACTED]` 后打出来	该次调用会走 fallback 链；换后端；注意这类请求失败不重试但仍会尝试下一级后端
模型「看不见」图片	文本模型收到图片块	配置里显式写 `"vision": true/false`；否则按模型名猜测	群体采访会先问 `provider_supports_images()` 再决定是否附图，必要时用文字描述代替；建城按草图走则直接报错要求换多模态模型
回复是空的或被截断	`max_tokens` 预算被推理块吃光	提示里写「检查思考模式和输出 token 预算」；日志里 `LLM output was truncated by max_tokens=…`	调大 `max_tokens`（Anthropic 兼容默认 512，MiniMax 预设已调到 16384）；关掉思考模式
网页测试成功，命令行失败	两者读的不是同一份环境	面板的「环境变量」页签看有没有读到；命令行直接看自己那个终端	环境变量属于启动它的那个终端及其子进程。已经在跑的控制台不会因为你后来 export 了变量就重新读；要么重启进程，要么把变量写进仓库根目录的 `.env`

关于重试，代码里的规则是确定的：状态码 `{408, 425, 429, 500, 502, 503, 504}` 算可重试，最多 3 次、指数退避；其它错误不重试，但**仍然会尝试 fallback 链上的下一个后端**。Anthropic 兼容还有个细节：401 时会按 `authorization_retry_schemes` 换一种鉴权头再试一次（`x-api-key` → `Bearer` → 裸 token），这是为兼容各家网关的怪癖。

最后一条经验值得单独记：**「有回复」不等于「回复完整」**。`providers.py` 的注释里有一组实测数字——在 848 次真实的 `actions` 调用中，有 318 次（37.5%）被 `max_tokens` 截断而当时无人察觉。截断的答案往往还能用（动作解析器会抢救出已闭合的部分），所以它不报错，但下游看到的是残缺数据。看到日志里那条截断警告时，别当噪音划过去。

3.8 隐私与合规：开课前先讲这一句

居民的档案（姓名、年龄、职业、家庭、价值观）、记忆（情景记忆、日记、长期总结）、状态与社交关系，都会作为提示词的一部分发给模型服务商。这意味着：

- 用云端后端 = 你的教学数据离开了本机。涉及真实人物（真人蒸馏功能会把一个真人的公开资料写成居民）时，这一点要格外小心。
- 多人部署时，密钥按账号隔离（`credentials.py` 绑「账号 + 协议 + 完整 Base URL」），但**提示词内容不隔离**——每个用户跑的是自己的世界，费用记在各自账上，内容也各自承担。
- 存档与复现包（`output/` 下面的一切）里包含这些提示词和模型回答。给学生发复现材料之前，先想一遍要不要连 `output/` 一起发。

开课前用一句话讲清楚就够了：「你写的居民会把你给它的档案发到模型服务商那里去，所以档案里别写真实的人名和身份证号。」

本章小结

- **没有可用后端，进程在第一个居民迈步前就退出**：`LLM_ROUTER = LLMRouter(CONFIG)` 在导入时实例化，没有 provider 就抛 `No LLM providers configured.`。
- 三类后端只认三种协议：`ollama`（`url`，默认 `http://localhost:11434/api/generate`）、`openai`（`base_url`，默认 `https://api.openai.com/v1`）、`anthropic`（自动拼 `/v1/messages`）。「OpenAI 兼容」说的是接口格式，不是 OpenAI 的 Key。
- 配置分几层：代码默认值 → `dashboard_config.json` → `GAWORLD_CONFIG_OVERRIDES` → `data/environment_config.json`（只管环境那四块）→ 城市包 → `GAWORLD_CONFIG_OVERRIDES` 最后再合并一次（有最终决定权）。密钥永远走环境变量或「我的 API Key」，不进配置文件。
- 路由有四个键：`default` / `tasks` / `agents` / `fallback`。省钱的第一个动作是把 `schedule`（以及 `actions`、`perception`）派给便宜后端；`fallback` 是安全网，不是省钱表——实验要可比就该用 `allow_fallback=False`。
- **省 token 的地方也很多**：大五 `rules` 通道零 token，出行决策是纯 seeded 随机数，日程解析失败退回启发式模板，经济与公共部门全确定性，外部环境改成事件池模式就不花钱，群体模式用 `--no-llm` 可以做到零调用。
- 账单先量后省：`output/run_manifests/*.json` 的 `llm` 段给出 `call_count` / `by_task` / `by_provider` / `requests_fell_back`；同样 10 年 × 50 人，按年粒度约 500 次调用，按天粒度 182,500 次，差 365 倍。

动手练习

1. **在控制台里配好一个可用后端，并让连通性测试通过**（约 25 分钟）
   - 准备一个可用的服务：本地 Ollama（`ollama serve` + `ollama pull qwen3.5:9b`）或任意一个 OpenAI 兼容的云端服务。

- 在仓库根目录的 `.env` 里写好密钥变量（照 `.env.example` 的格式），或在多人部署下用登录账号的「我的 API Key」。
- 打开控制台的「配置 → 模型」→「添加本地或云端模型」，填类型、名称、模型名、接口地址、密钥环境变量、超时；点「先测一下」或保存后点「测试连通性」。
- 回到仿真控制台，把「默认 LLM」和「日程 LLM」都指到这个后端，点「保存配置」，再重新打开配置确认两个下拉都还在。
- **验收标准**：连通性测试出现「✓」并给出耗时与模型回声（内容应当是 `OK` 之类的短回复）；重新打开「配置 → 模型」，该后端在「已配置的后端」列表里带「密钥已就绪」标记；仿真控制台的两个模型下拉都显示这个后端的名称。做完顺手把配置文件里那一行 `api_key` 是否为空确认一下——面板不应该、也确实不会替你把明文密钥写进 `dashboard_config.json`。

2. **把日程 LLM 换成便宜后端，跑一次仿真，比较调用量与产物差异**（约 40 分钟）
   - 记录基线：用当前（较强）模型跑一次 `GAWORLD_CONFIG_OVERRIDES='{"agent_ids":[1],"sim_days":1,"random_seed":42}' python generative_city_sim.py run`（或在控制台里把「Agent IDs」填成 1、仿真天数填 1）。跑完打开 `output/run_manifests/` 下最新的 manifest，记下 `llm.call_count` 与 `llm.by_task`。
   - 改配置：把 `llm.routing.tasks.schedule` 指向一个便宜后端（本地小模型最省事），`llm.routing.default` 保持不变，存进 `dashboard_config.json` 或 `GAWORLD_CONFIG_OVERRIDES`。
   - 用**同样的居民、同样的一天、同一个种子**再跑一次（只换 `tasks.schedule` 这一项），记下新的 `call_count` 与 `by_task`。
   - 对比产物：打开 `output/memory/agent_1.json` 与 `output/state/`，看这一天的日程条目数量、活动名称、以及状态历史里哪些变量变了。
   - **验收标准**：两次 manifest 都存在且能读出 `llm.call_count` 与 `llm.by_task`；两次的日程条目数都在 6–10 项之间（说明日程照常生成，没有退回空日程）；你能指出哪些内容变了（活动的措辞通常会变，因为是不同模型写的）、哪些没变（居民 id、日期、状态变量名与结构），并用一句话说清「换便宜模型省下的钱换来了什么可解释的代价」。

ewpage

第 4 章　居民与种子数据：平台吃什么、吐什么

这一章回答一个很具体的问题：仿真里那些居民是从哪来的，怎么被捏出来的，你改了哪一行他们就会变。答案不在数据库里，在磁盘上三个文件里。

4.1 一条居民 = 一行 CSV + 一节 Markdown

GAWorld 没有"居民数据库"。一位居民是在仿真启动的那一刻现造出来的，造它的材料是两个文件：**状态 CSV 的一行**（结构化初始状态）和 **profile Markdown 的一节**（人格化叙事档案）。

合成发生在 `generative_city_sim.py` 的 `build_agent()`（第 733 行）里：

```
def build_agent(agent_id, df, city_map=None):
    row = df[df["id"] == agent_id].iloc[0]
    text = parse_profile(load_profile_from_md(agent_id))
    agent = {
        "id": agent_id,
        **text,                          # profile 散文抽出来的字段
        "gender": row.get("gender", ""),
        "hukou": row.get("hukou", ""),
        "residence": row.get("residence", ""),
        "employment": _safe_text(row.get("employment", "")),
        "industry": _safe_text(row.get("industry", "")),
        "state": { ... },                # 九个状态变量 + 八个运行期变量
        "memory": [],
        "social_neighbors": []
    }
    ...
    init_agent_locations(agent, city_map)
    return agent
```

它做四件事：取 CSV 行；用正则从 Markdown 里切出 `## Profile NN｜姓名` 那一节并抽成字典（`gaworld/sim/agents_loader.py` 的 `parse_profile`）；合并成一个 dict；最后调 `init_agent_locations()` 把人放到城市地图上，给出 `locations`。

这个 dict 就是仿真内部到处传递的"居民"。`gaworld/core/agent.py` 里的 `Agent` dataclass 是它的一层薄适配器：底层仍是一个 dict，`Agent.from_dict()` / `view_as_agent()` 包一层，新代码可以写 `agent.state`、`agent.name`、`agent.need("energy")`，老的 `agent["state"]["emotion"]` 也照样有效——迁移不用一次做完。

谁覆盖谁。 CSV 赢。`name` / `age` / `job` / `personality` 这些来自 profile 散文；`gender` / `hukou` / `residence` / `employment` / `industry` 来自 CSV；`monthly_income` 优先用 CSV 列，没有才从散文里正则抓 `月收入约 12,000 元`。代码注释里写了理由：CSV 里的数字是人口合成器自己算的，profile 散文是它的渲染，两者曾经长期互相矛盾（326 位居民上相关系数只有 0.305，账本按 1.39 倍于档案所述的数字发薪，把收入基尼系数从 0.468 拉平到 0.348）。

种子文件一共三个，各管一段：

文件	内容	谁在读
`data/hangzhou_agents_state_init.csv`	53 行 × 15 列：身份 + 九个状态变量	`build_agent`、Agent Studio、大五面板

文件	内容	谁在读
`data/hangzhou_profiles_with_names.md`	53 节 `## Profile NN | 姓名` 叙事档案	`parse_profile`、Agent Studio
`data/agents_big5.csv`	54 行 × 10 列：O/C/E/A/N 的 z 分数	`BigFivePlugin`，构建时读一次

4.2 种子数据放在哪

`ls data/` 实际是这些：

```
agents_big5.csv                          大五 z 分数
cities/                                  城市包（每座城一个子目录）
citymap.md / .citymap.json / .geojson / _visualization.json    城市
地图
environment_config.json                  环境与事件
hangzhou_agents_state_init.csv           状态 CSV
hangzhou_profiles_with_names.md          profile Markdown
hangzhou_profiles_with_names.v1.md       改写前的语料备份
info_sources.json / news_cache.json / news_source.md
skills/  experiments/
```

跑哪座城由 `dashboard_config.json` 决定。现在它是 `{"city": "wuzhen"}`，于是 `gaworld/city/config.py` 的 `apply_city()` 把 `CONFIG["csv_path"]` 改写成 `data/cities/wuzhen/agents.csv`、`md_path` 改写成 `data/cities/wuzhen/profiles.md`。只有不选城市、用默认世界的时候，才回到根目录的 `data/hangzhou_*`。

一座城市的包长这样：

```
data/cities/wuzhen/
├── agents.csv                63 行状态 CSV
├── profiles.md               63 节档案
├── city.json  citymap.md  citymap.citymap.json  map.geojson
├── environment.json  news.json
└── population_manifest.json
```

“造一座城”这件事本身有向导：Dashboard 的「人口与群体」页从人口模板出发，设定人口结构、心理状态，再跑模拟、检查结果。这一步产出的就是上面那两个文件。



图 4–1 「人口与群体」向导的入口 第 1 步只问四件事：人口模板、居民人数、随机种子、小镇名称。右边的「参数体检」当场告诉你这组参数能不能生成（户均人数只能落在 1.8–4.8 之间、中位年龄只能落在 26–60 岁之间，是参数互相牵制的结果，不是 bug）。**随机种子**是关键：参数一样、种子一样，生成出来的居民完全相同，所以同一组人口结构换一批人只需要改种子。

4.3 状态 CSV：九个状态变量

`head -3 data/hangzhou_agents_state_init.csv` 读出来的真实表头是 15 列（文件以 UTF-8 BOM 开头，`EF BB BF`）：

```
id,name,gender,age,hukou,residence,emotion,stress,econ_security,city_identity,policy_sensitivity,platform_dependence,risk_preference,voice_propensity,mobility_intent
```

城市包的 `agents.csv` 多三列（`employment`、`industry`、`monthly_income`），共 19 列——那三列是人口合成器拟合出来的，老语料本来没有。列可以按用途分四组：

分组	列名	缺列会怎样
身份	`id` `name` `gender` `age` `hukou` `residence`	`id` 缺了整行不可用；`name` / `gender` 缺失留空
就业	`employment` `industry` `monthly_income`	老语料没有；收入退回从档案散文正则提取
核心状态（4）	`emotion` `stress` `econ_security` `city_identity`	缺任一列 → 构造时 `KeyError`，整场 run 起不来
增强状态（5）	`policy_sensitivity` `platform_dependence` `risk_preference` `voice_propensity` `mobility_intent`	缺任一列 → 取默认 `0.5`（中点），不崩

九个状态变量在代码里有一份权威清单：`gaworld/population/schema.py` 的 `STATE_VAR_KEYS`（第 64–74 行），注释写得很清楚——“The nine [0,1] state variables written to the state CSV”。它们分成两批，语义在 profile Markdown 开头的「统一状态变量规范表」里有定义：

变量名	0 的语义	1 的语义	谁的代码在用
`emotion`	极度消极	高度积极	情绪传染、情绪基准点、提示词里的状态简报
`stress`	无压力	极高压力	压力相关目标的优先级排序
`econ_security`	极不安全	极度安全	经济模块；目标优先级里”提高收入稳定性”
`city_identity`	强烈疏离	强烈认同	现实感模块：低于 0.45 时把”保持社区连接”列为目标之一
`policy_sensitivity`	很少感知政策	对政策高度敏感	环境系统打政策类事件标签、人生事件系统
`platform_dependence`	线下/多元收入	高度平台绑定	信息源模块：决定信息食谱里社交源的占比
`risk_preference`	强烈风险厌恶	强烈风险偏好	经济模块的遗产继承概率
`voice_propensity`	沉默/回避	主动表达/投诉	群体模式（集体表达）、环境系统的事件标签
`mobility_intent`	强烈定居	强烈流动	环境系统事件标签、行为模块的优先级加成

九个都在 [0,1]，前四个是”核心内部状态”（档案里那行 `**核心状态变量**:`），后五个是”研究增强变量”（档案里那五条 `**研究增强变量初始化**:` bullet）。这套划分不是装饰：`gaworld/research/measures.py` 把九个都列为研究测量的核心指标，`gaworld/parallel/analysis.py` 给每一个配了中文名。

这些数字进提示词时会先分档。`gaworld/sim/_prompt.py` 的 `_state_brief_for_prompt()` 把 `emotion`、`stress`、`energy`、`hunger`、`fatigue_debt`、`time_pressure`、`self_control`、`social_need` 读出来，各自归到”低 / 中 / 高”三档，拼成四行状态简报，每行后面再附上原始数值（`stress=0.62`）。分档的理由写在注释里：提示词的语言要对小幅数值抖动不敏感。所以你在 CSV 里把 `stress` 从 0.62 改成 0.64，档位还在”中等”，措辞不会变；压到 0.40 以下才掉到”较低”那一档。

降级规则写死在 `build_agent` 里，值得背下来：

- 四个核心变量用 `float(row["emotion"])` 这种直接下标取——缺列是 `KeyError`；而 `agents = [build_agent(i, df, city_map=city_map) for i in AGENT_IDS]` 没有 try，整场 run 直接死。

- 五个增强变量用 `row.get(key, 0.5)`——缺列取 0.5，不崩，但这个人在这五维上就是"平均"。
- `employment` / `industry` 缺列 → 空串。
- `monthly_income` 缺列 → 从 profile 散文正则提取；档案里写"无固定收入"会正确地得到 `None`。

4.4 profile Markdown：人格化叙事档案

档案文件里每位居民一节，标题格式是：

```
## Profile 01｜李泽宇
```

两位补零、全角竖线 `｜`、竖线后一个空格——这不是排版偏好，是解析契约。`load_profile_from_md()`（`generative_city_sim.py:554`）用正则 `## Profile {agent_id:02d}｜.*?(?=\n## Profile |\Z)` 切块，补零位数和全角竖线都在正则里。手写成 `## Profile 1｜李泽宇`，仿真就找不到这个人。

`parse_profile` 从一节里抽走这些字段：

档案里的段落	落到 agent 的键	抽取方式
`## Profile NN｜姓名`	`name`	标题里 `｜` 之后
`**基础信息**:`	`age` `living`	正则 `(\d+)岁`、`居住(?:于)?(.+?)[，。]`
`**职业与工作节奏**:`	`job`（同时复制为 `work_style`）	整行
`**性格与情绪特征**:`	`personality`	整行
`**人格与行为倾向**:`	`behavior_tendencies`	整行；老语料没有这行，默认空串
`**日常生活与生活习惯**:`	`daily_life`	整行
`**价值观与公共事务态度**:`	`values`	整行
全文里的 `月收入约 X 元`	`monthly_income`	正则 `月收入[约为是：:\s]*([0-9][0-9,，]*)\s*元`

`**教育与收入背景**:` 和 `**社交网络情况**:` 两行不在这张表里——`parse_profile` 不读它们，是给人看的展示字段。文件开头那几节「研究增强层」「统一状态变量规范表」「最小参数接口」也是文档，解析器只认 `## Profile` 开头的块。

还有一处值得知道：`性格与情绪特征` 和 `人格与行为倾向` 不会同时出现在提示词里。`gaworld/personality/anchors.py` 的 `personality_line()` 优先渲染 `behavior_tendencies`（也就是「人格与行为倾向」），只有在老语料里没有这一段时才退回渲染「性格与情绪特征」。原因写在注释里：这两段的来源不同——老的那段是手写的、和状态变量写在一起，新的那段是从独立采样的 OCEAN 分数反推出来的，在文档记录的那份 51 人语料上有 9 个人的两段互相矛盾；两句都印出去，就是每条提示词里连续两行自相矛盾。而且采样分数驱动的是 `rules` 通道，如果提示词里印的是老标签，提示词通道和规则通道描述的就是两个人。

档案不只进提示词。这是最容易被忽略的一点。`gaworld/behavior/dynamic.py` 把 `personality` 和 `daily_life` 拼成一个 blob 做关键词匹配：

- 命中 `外向` / `活泼` / `社交` / `热情` → 心情好或孤独时，自发行为的强度 ×1.2；
- 命中 `内向` / `安静` / `独处` / `沉稳` → 焦虑时强度 ×1.15；遇到「约朋友出去 / 找人聊天 / 去人多的地方」强度 ×0.6；
- `is_extrovert` 的判据是 blob 命中 `外向` / `活泼` / `社交` / `热情` / `开朗`。

`gaworld/economy/finance.py` 也会拿 `hukou` + `residence` + `living` + `values` + `daily_life` 去匹配遗产继承的加成关键词。档案里的词是有牙齿的。

怎么写才会让行为更真

原则一句话：**写可观察的行为，不写抽象的性格标签。**

差的写法（档案里的真实片段）：

> **性格与情绪特征**：性格偏内向理性，低冲突倾向，习惯用技术问题掩盖情绪问题。

模型读到”偏内向”能做的推断很有限。内向可以是”话少但交心”、“不参加聚会但主动约饭”、“在群里从不主动发言”——三种截然不同的人，行为完全不同。模型只能挑一种，挑错了整条人格线就歪了。

好的写法（同一个人，档案里的原文）：

> **性格与情绪特征**：性格偏内向理性，低冲突倾向，习惯用技术问题掩盖情绪问题。整体情绪稳定，但在绩效评估、裁员传闻或行业波动时期会出现明显焦虑。
>
> **人格与行为倾向**：他入职第一年养成了习惯：每天早上刷一次内网邮件，看leader有没有回他那篇PR。有次等到晚上九点半，盯着未读红点预演了两轮最坏结果，leader第二天上午才回了个”lgtm”。他主导的方案第一次上线那天，隔壁组同事凑过来想帮他调参数，他头也没抬说”我自己来”。公司年会报名链接群里发了一周，他等到截止前最后一刻才填，理由是”那天大概率加班，去的话也待不久”。

同一个人的两段，差别在四个地方：

1. **有时间、有地点、有别人。**“等到晚上九点半”不是性格，是一次具体经历；模型能据此生成”今晚要不要再看一眼邮件”这种可执行的行动。
2. **有可复现的阈值。**人格锚句模板也是这个路子：`gaworld/personality/anchors.py` 里每个维度都写成第二人称、带一个具体门槛的话——外向性高的人写”独处超过两天就会觉得不对劲”，而不是”你很外向”。
3. **有行为，没有形容词。**“头也没抬说’我自己来’”比”低冲突倾向”好演得多。
4. **和状态变量方向一致。**CSV 里 `voice_propensity` 0.20 的人，档案里最好也看不到主动表达的迹象。两边打架时模型会挑一边，而你就自己埋了雷。

写完拿 `性格与情绪特征` 和 `日常生活与生活习惯` 两段拼起来看：里面有没有 `外向/活泼/社交/热情` 或 `内向/安静/独处/沉稳` 这几个关键词？你想要哪一边，就让那一边赢。

4.5 大五人格（OCEAN）

每位居民带一组五个 z 分数：开放性 O、尽责性 C、外向性 E、宜人性 A、神经质 N（`gaworld/personality/traits.py` 的 `DIMENSIONS = ("o","c","e","a","n")`）。注意 N 是神经质，不是情绪稳定性——N 高意味着更容易受影响。z 分数裁剪在 ±2.5。

分数从哪来。 `BigFivePlugin`（id `big_five`）只挂一个钩子 `agents.built`，两条取数路径：

1. 有 `data/agents_big5.csv` 就用离线标定的值（`load_profiles()`，`utf-8-sig` 读）。这份文件的真实表头是 10 列：`id,name,o,c,e,a,n,source,unstated,redundant`。当前 54 行：id 1–51 是 `sampled_authored`，id 61–63 是 `persona_distilled`。

2. 文件缺失、或某人不在里面 → 从人口先验采样（`prior_traits()`）：用配置里的 OCEAN 相关矩阵做 Cholesky 分解抽样，再按样本重标定到均值 0 / 标准差 1。种子固定 20260820，每个人的随机流由 `seed * 1000003 + agent_id` 派生，所以抽签与名册顺序无关——调换居民顺序不会换掉任何人的分数。

当前语料里有个现成的例子：`agents_big5.csv` 有 1–51，没有 52（王思颖）和 53（郭林峰），所以这两人跑起来是 `source=prior_sampled`，其余 51 人是 `sampled_authored`。

生成方向很重要：先独立采样五维分数，再据此改写 profile 里的行为描述（`scripts/author_personality.py`），而不是从 profile 文本反推分数。反过来做过，不成立：老语料里「性格与情绪特征」中位数只有 20 字（是标签不是描述），而五个增强状态变量就写在同一份 profile 里紧挨着它——散文和数字是同一个作者对同一个人的两种写法，反推出的开放性和 `mobility_intent` 相关达 0.90，共线性闸门在 O/C/E 三维上判不合格。改写之后每份 profile 多出一个「人格与行为倾向」段（原来的「性格与情绪特征」原样保留），老语料备份在 `data/hangzhou_profiles_with_names.v1.md`。

三条通道

通道	改什么	token 成本
`rules`	`choose_action` 里的加性 `trait_style_fit` 分量；中断阈值、自发行为、社交偶遇、决策噪声、冲动绕过、财富驱动上的有界乘子；个人情绪基准点	零
`prompt`	把人格锚句注入日程生成、活动调整、目标、新闻等**决策类**提示词	有
`voice`	同样的锚句，只进**日记**提示词	有

分成三条而不是一个总开关，是为了让实验把「**决策**变了」和「**文风**变了」分开归因：只留 `rules` 得到的是零 token 的确定性调制；只留 `prompt` / `voice`，变的只是措辞和句式，决策逻辑一个字没动。

默认开关在 `gaworld/settings/personality.py`：`{"rules": True, "prompt": False, "voice": True}`——`prompt` 通道默认是关的，`rules` 和 `voice` 默认开。想开 `prompt` 改一行配置。

特质不漂移，中性即恒等

插件里没有 `on_day_end`，这是故意的：成年人 OCEAN 每十年只变 0.1–0.2 个标准差，一天一变是噪声不是心理学。随时间变的是状态变量，不是人格。特质在构建智能体时一次性种入，之后只读。

没有特质、或关掉某条通道，行为与加人格之前逐位一致。 这条契约落在 `traits.py` 里：`traits_of()` 在没有记录时返回全零；`style_fit()` 对中性人格返回 `0.0`（加性分量的恒等元）；`trait_modifier()` 返回 `1.0`（乘子的恒等元）。也就是说，关掉任何一条通道，那条路径上的调制正好是恒等变换，不会留下任何痕迹。

审计。 每次跑完，`output/traits/agent_traits.csv` 记下真正生效的那份：表头 `id,name,o,c,e,a,n,source,channels,unstated,redundant`。`channels` 列写明这个人当时开着哪几条通道；`source` 列写明分数是离线标定的

（`sampled_authored` / `persona_distilled`）还是先验采样的（`prior_sampled`）。

最后两列是溯源用的，值得看一眼再解释结果。`unstated` 记的是「人物设定里根本没提这个维度」——那一维按 0 处理，既不贡献任何东西也不渲染锚句，但如果不记下来，你分不清"测出来是平均值"和"压根没测"。`redundant` 记的是被共线性闸门刷掉的维度：`scripts/big5_collinearity.py` 把每个维度对已有的五个个体级状态变量做回归，某一维如果基本能被线性预测出来，就判它不独立；这些维度的效应在这次运行里不能当成人格效应来解释。启动时终端也会打一行 `🧬 大五人格已就绪：N 人（标定 X / 先验采样 Y），启用通道 …`，并把这两个警告直接印出来——因为报告是没人会打开的。

4.6 加一个人、改一个人

有三条路。

Agent Studio。打开 `http://localhost:<port>/site/dashboard/studio.html`。左栏选城市和居民，右边七步：身份 → 状态·性格 → 能力·技能 → 记忆 → 社交·关系 → 行为·目标 → 复核·部署。



图 4–2　Agent Studio：一位居民的全部可编辑部分 上面那一排是流程，下面分成两块：左边「基础信息」和「叙事 Profile」直接对应状态 CSV 的一行和档案 Markdown 的一节；右边「状态速览」的雷达图就是那九个状态变量，轴上的字是缩写（"经济感"而不是"经济安全感"，因为图幅放不下）。顶栏四个按钮分别对应新建、批量导入、从真人蒸馏、保存。

保存的落点有两个：档案正文 → profile Markdown 里那一节整体替换（`gaworld/apps/residents.py` 的 `save_agent_profile`）；状态滑杆 → 状态 CSV 那一行（`save_agent_state`，先写 `.tmp` 再 `os.replace`，原子替换，不会写坏半个文件）。

写回时谁说了算：CSV 是权威。改完状态之后，`_sync_profile_state_lines()` 会把档案里那两行结构化数字（`**核心状态变量**:` 和 `**研究增强变量初始化**:` 的五条 bullet）改写成 CSV 里的值，但**不碰任何叙事散文**；某一节缺这两行，就什么都不改。

命令行。

```
python -m gaworld.city add-agent wuzhen --name 张伟 --age 32 --gend
        er 男 --job 程序员 --income 18000
python -m gaworld.city add-agents wuzhen --size 100
```

```
python -m gaworld.population --preset cn_county_town --size 500 --s
        eed 42 --name my_town --out output/town
```

`add-agent` 的新 id 是现有最大 id + 1，档案用和 `parse_profile` 配套的模板渲染（`gaworld/city/agents.py` 的 `_render_single_profile`），九个状态变量没给就按 `uniform(0.35, 0.70)` 随机。

批量导入。 Agent Studio 顶栏「+ 批量导入」把一份外部用户表（CSV / xlsx / jsonl）变成居民。列名中英文都认（`姓名`→`name`、`月薪`→`monthly_income`、九个状态变量同名直认），默认开匿名化（确定性假名池 + 直接丢弃手机 / 邮箱 / 身份证），还能"扩容到 N 人"——剩下的用 IPF 按上传库的边际分布抽样补齐，得到的虚拟居民与上传库同分布但身份独立。写入 `data/cities/<slug>/agents.csv` 和 `profiles.md`，新 id 同样从最大 id + 1 起。完整流程见 `docs/BULK_IMPORT_TUTORIAL.md`。

大五分数也能在 Studio 里改（状态·性格那一步），但它写的是 `data/agents_big5.csv`，**下一次 run 才生效**——插件在 `agents.built` 时读一次文件，之后那份记录只读。其它种子文件同理。

4.7 常见坑

1. **编号每座城市各自从 1 开始。** `gaworld/city/config.py` 的注释写得很直白："Agent ids restart at 1 in every city"。杭州的 7 号和乌镇的 7 号是两个人。记忆、大五、社交、财务都是按编号扁平存放的，所以换一座城，ID 范围就变，同一个 id 背后的数据全换了。跨城市比较前先看 `population_manifest.json`。
2. **CSV 有 BOM。** 仓库里两个种子 CSV 都以 `EF BB BF` 开头。pandas 读时会自动剥掉（`df.columns[0]` 就是 `'id'`），但 `csv.DictReader` 之类不带 `utf-8-sig` 的读法会拿到 `'\ufeffid'`，于是 `row.get("id")` 返回 None。平台自己的读法都带 `utf-8-sig`（`personality/plugin.py`、`apps/residents.py`、`city/agents.py`），写的时候也用 `utf-8-sig`（`city/agents.py` 的 `_write_rows`）——手改文件请保持这个约定。
3. **profile 和 CSV 不一致时，没有"以谁为准"的仲裁。** Agent Studio 的写回顺序是"先写 CSV，再回填 profile 的数字行"，这是工具的约定，不是平台的规则。如果你自己改了两个文件且数值不一致，那么仿真读到的是**各自的值**：CSV 里的九个状态变量走 `state`，profile 里的散文走提示词和关键词规则，两套值同时生效，谁也覆盖不了谁。
4. **缺字段的降级不对称。** 四个核心状态变量缺列 → `KeyError`，整场 run 起不来；五个增强变量缺列 → 静默取 0.5。`## Profile NN` 小节不存在 → `ValueError: Profile N not found`，同样直接崩。`**基础信息**:` 那行如果丢了「NN 岁」或「居住」→ `parse_profile` 里的 `re.search(...).group(1)` 抛 `AttributeError`。所以改档案时，`**基础信息**:` 里的逗号和句号是解析契约，别动。
5. **人数写在文件头上，可能是旧数字。** `data/hangzhou_profiles_with_names.md` 第一行写着「(51人)」，但文件里实际有 53 个 `## Profile` 小节、CSV 里也有 53 行（id 1..53）。以 `grep -c '^## Profile'` 为准。
6. **`## Profile` 的编号必须两位补零。** `## Profile 1 |` 不被 `load_profile_from_md()` 认出来（正则要 `01`），但 Agent Studio 的读法是宽松的。结果就是 Studio 里看得到、仿真里读不到——排查时先确认这一条。
7. **城市包不带 `agents_big5.csv`。** `gaworld/city/config.py` 的 `AGENT_FILES` 里写着 `"personality.profile_path": "agents_big5.csv"`，意思是城市包会到自己的目录里找这份文件——但实际的城市包里没有它（`ls data/cities/wuzhen/` 就能看到）。结果就是：默认世界的杭州居民有 51 份离线标定的分数，而乌镇的 63 位居民全部走人口

先验采样，跑完之后 `output/traits/agent_traits.csv` 里这些人的 `source` 会是 `prior_sampled`。这不是 bug，是"没有标定数据就抽样"的正确表现，但比较两座城的人格分布时必须先看这一列。

8. **`agents_big5.csv` 里可能有孤儿行。** 当前这份文件有 54 行：id 1–51 是杭州居民（`sampled_authored`），id 61–63 是 `persona_distilled`（鲁迅、孙宇晨、郁建兴）——但状态 CSV 的 id 只到 53，这三个人在这份语料里没有对应的 CSV 行和档案节。读这份文件时别假设每一行都对应一个能跑起来的居民。

本章小结

- 一条居民 = 状态 CSV 的一行 + profile Markdown 的一节，在 `build_agent()` 里合成；档案散文提供身份与叙事，CSV 提供性别、户籍、住址和九个状态变量，CSV 优先。
- 三个种子文件各管一段：状态 CSV 管结构化初始状态（九个 [0,1] 状态变量，权威清单在 `gaworld/population/schema.py` 的 `STATE_VAR_KEYS`）、profile Markdown 管人格化叙事、Big Five CSV 管可选的离线 z 分数。
- 档案的标题格式 `## Profile NN｜姓名` 和 `**基础信息**：` 的标点都是解析契约；档案散文还会被 `behavior/dynamic.py`、`economy/finance.py` 做关键词匹配，所以它不只影响文风。
- 写档案要写可观察的行为（有时间、有地点、有他人、有阈值），不写抽象性格标签；抽象标签给模型留下的推断空间太大。
- 大五人格三条通道分开是为了把"决策变了"和"文风变了"分开归因；特质在构建时一次性种入、运行期不漂移；没有特质或关掉通道时行为与加人格之前逐位一致。
- 改种子文件后必须重跑才生效；换城市后 ID 范围会变，CSV 与 profile 的数值不一致时两套值会同时生效。

动手练习

1. **把一位居民的档案改成可观察的行为，然后跑一次仿真**（约 30 分钟） 打开 `data/hangzhou_profiles_with_names.md`，找到 `## Profile 01｜李泽宇`，把「人格与行为倾向」那一段改写成至少三句带时间、地点、他人的具体行为（照 4.4 节的"好的写法"改；注意保留段落标签 `**人格与行为倾向**：` 不动，删掉这一段整行会让 `behavior_tendencies` 变成空串）。先把原文复制到别处存档。然后把 `dashboard_config.json` 的 `agent_ids` 改成 `[1]`，跑 `python generative_city_sim.py run --sim-days 2`。跑完看两个地方：`output/diaries/agent_1/day_001.md`（日记的「今天主要发生的事情」和「今天的感想」两节，是从当天 episode 的 `action` 与 `reflection` 拼出来的），以及 `output/memory/agent_1.json`（这位居民的长期记忆条目，一个字符串一个条目的列表）。 **验收标准**：日记或记忆条目里出现了你新写的具体动作中的至少一个关键词（是"又刷了一遍邮件"这样带动作的句子，而不是"今天有点焦虑"这种抽象描述）。如果一次没出现，就把那句话改得再有动词、时间和地点，再跑一次——这正是"可观察行为 vs 抽象标签"这一课要教的东西。最后把原文改回去再跑一次，确认那句话消失了。

2. **算出某位居民的大五 z 分数对应的人群百分位**（约 15 分钟） 跑这段脚本，它读 `data/agents_big5.csv`（用 `utf-8-sig` 绕开 BOM），把 id=1 那一行的五个 z 分数换算成正态分布百分位：

```
python3 -c "
import csv, math
rows = list(csv.DictReader(open('data/agents_big5.csv', encodin
        g='utf-8-sig')))
r = [x for x in rows if x['id'] == '1'][0]
print(r['name'], '| source =', r['source'])
```

```
for d, zh in zip('ocean', '开放性 尽责性 外向性 宜人性 神经质'.split
        ()):
     z = float(r[d]); pct = 0.5 * (1 + math.erf(z / math.sqrt
        (2)))
    print(f'{zh} {d}: z={z:+.4f}  百分位 {pct*100:.1f}%')
"
```

然后跑一次仿真，打开 `output/traits/agent_traits.csv`，找到 id=1 那一行，对比 `source` 列（是 `sampled_authored` 还是 `prior_sampled`）和五个分数。 **验收标准**：脚本输出五个维度的 z 分数与百分位，z=0 对应 50.0%；`output/traits/agent_traits.csv` 里同一 id 那一行的 `source` 与种子文件一致（如果 `agents_big5.csv` 里查不到这个 id，它一定是 `prior_sampled`）；能指出全体居民里 z 最高的是哪一位、属于 OCEAN 哪一维，并把”这一维在规则通道里会改什么”写一句话（例如 C 高 → `choose_action` 的 style-fit 加性分量偏向”progress”标签）。

ewpage

第 5 章　从地名造一座城：城市、地图与人口

前四章把跑起来需要的东西备齐了：模型接好，居民有了，种子数据也读懂了。但到这一步你手上仍然只有「杭州」这一座城——地名写在 `background` 提示词里，地图是 `data/citymap.md` 一张文件，居民是固定的 63 个人。想换个地方做研究，唯一办法是手改配置里的三条路径，改完就回不去。

本章讲 `gaworld.city` 这套工具：给一个地名，它给你一座能跑仿真的城——真实的路网、按当地纬度和规模推出来的气候与事件、联网调研出来的产业画像、调研出来的姓名池，以及一批住在真实行政区里的居民。多座城市可以同时存在，各跑各的，互不覆盖。

5.1 造城流水线：六步，每一步都能降级

`python -m gaworld.city create` 背后是六个步骤。之所以要一步步讲，是因为这六步**每一步都有降级路径**：没有网络、这个村子 OSM 上没有、甚至你给的地名根本不存在，最后都得给你一座能跑的城。



图 5–1　造城流水线六步 左边三步决定这座城「长什么样」，右边三步决定这座城「是谁在过日子」。每一步都带一条降级支线：地理编码失败退成合成地点，真实路网抓不到退成程序化地图，程序化地图则**永远**生成——它既是 `map.geojson` 被删掉之后的兜底，也是喂给 LLM 的提示词上下文。断网时用 `--offline` 直接从第 3 步开始，第 4 步之后的一切照常工作。

第 1 步：地理编码——决定「这是多大的地方」

`gaworld/city/geocode.py` 拿地名去问 Nominatim（`nominatim.openstreetmap.org`），拿回坐标、行政区 bbox 和一个**规模档位**。规模档位有五档：`tiny` / `small` / `medium` / `large` / `metro`。

档位是怎么定的？先看 Nominatim 返回的 `addresstype`：`village`、`hamlet` 算 `tiny`，`town`、`suburb` 是 `small`，`borough`、`municipality` 是 `medium`，`city`、`county` 是 `large`，`state`、`province` 是 `metro`。认不出来就退回按人口猜（2000 / 2 万 / 20 万 / 100 万四道门槛），还认不出来默认 `medium`。`--scale` 可直接覆盖。

规模不是个标签，它同时决定三件事：程序化地图生成几个枢纽、**有没有地铁**（`tiny`/`small` 没有）；环境事件池用哪一档措辞（村里是「集市开集」，大城市才有「地铁运营延误」）；以及 bbox 允许多大。

最后这条最容易被忽略。行政区的官方 bbox 常常远大于你想要的「一座城」——柯桥区的原始 bbox 跨 66 km，照抓会把隔壁诸暨的镇一并拉进来，居民通勤 70 km。所以 bbox 会按规模收敛到 `2 × SCALE_BBOX_HALF_DEG`（`tiny` 0.015、`small` 0.035、`medium` 0.07、`large` 0.13、`metro` 0.22 度纬度），柯桥区最终收到 28.9 km，抓到的全是兰亭街道、漓渚镇这些真正属于柯桥的地方。

地理编码失败（查不到、网络不通）不报错，而是退成一条**合成地点记录**：坐标落在杭州的默认锚点（30.2741, 120.1551）附近，`source` 标成 `offline`，规模按 `--scale` 或 `medium` 算。这条记录在 `city.json` 里看得见——所以「这座城到底是不是真的查到了」永远不是玄学。

第 2 步：真实 OSM 路网

`gaworld/city/osm.py` 用 Overpass API 抓这块 bbox 里的真实地物，产出 `map.geojson`。抓的类别是刻意选过的粗粒度集合，跟仿真的抽象层级对齐：住宅区（上限 55 个）、交通枢纽（20）、学校（22）、医院（22）、商业（30）、休

闲景点（26）、政府（12），另外**不设上限**地抓全部地铁站，再单独取地铁线路和一条主要河流。几百个节点，不是一栋一栋楼。

这里有三个抗脆弱的设计值得知道：

- **粘住可用镜像**。Overpass 的公共镜像经常一个个被墙或过载。代码按实测可靠性排了三个镜像（`overpass-api.de`、`overpass.kumi.systems`、`maps.mail.ru`），某个镜像一旦答过一次就排到最前，不再逐个重试死镜像。
- **冷却窗口**。刚失败的镜像 60 秒内不重试。一个 bundle 大约要发 9 次查询，每次都从一个已经 504 过的镜像开头，代价是 45 秒 × 9。
- **整体 wall-clock 预算 240 秒**。`urlopen` 的 timeout 是**每次 socket 操作**的，一个慢慢吐数据的镜像能远远跑过它。所以请求跑在独立线程上，线程池 `shutdown(wait=False)`，预算到了就走人。超时就降级到程序化地图——「造城」是交互操作，等 20 分钟比拿一张生成地图更糟。

还有两个阈值：抓回来的节点少于 **8 个**就判定「太稀疏，撑不起一张真实地图」，直接降级（小村庄这是正常的）；全部镜像三轮都失败（轮次退避 5 秒、10 秒、20 秒）也降级。

降级不是失败。日志里会打 `overpass error via <url>`，`city.json` 的 history 里记一条 `map.real.skipped`，原因写在里面。

第 3 步：程序化地图始终生成

`citymap.md` 一定会写，哪怕真实地图抓成功了。两个原因：它是兜底（`map.geojson` 删掉城市仍然能跑），也是给 LLM 看的提示词上下文。

它按地名**确定性**生成：种子是地名的 MD5 前 8 位（`seed_from_name`），所以同一个地名永远得到同一座城——这正是 `--offline` 适合写测试和复现实验的原因。规模决定布局：枢纽与住宅区数量 `tiny` 6/3、`small` 9/4、`medium` 13/5、`large` 18/6、`metro` 24/7，地铁只有 `medium` 以上才有。枢纽名保持英文结构名（`Central Block`、`Riverside Park`），因为地图层的 `infer_category` 是按英文关键词给未声明分类的节点归类的；只有城市本身和河流带真实地名。

第 4 步：环境按纬度和规模推导

`gaworld/city/environment.py` 从 `place.lat` 和 `place.scale` 推出 `environment.json`。气候分五档，按绝对纬度：23.5 以内热带、35 以内亚热带、50 以内温带、66.5 以内大陆性、其余极地。同一套代码，三亚出「台风外围环流」，诺里尔斯克出「暴风雪预警」。社会事件另按规模分三档（`tiny`/`small` 用「集市开集」那一档，`medium` 单独一档，`large`/`metro` 用有地铁延误的那一档）。

这一步最要紧的其实是 `background` 提示词。默认的 background 写着「中国·杭州」，不换掉的话，新城市里的居民会一边住在柯桥、一边以为自己在杭州。`environment.json` 里的 `background` 会被改写成「中国·柯桥区。大城市，亚热带气候。……」。

第 5 步：人口合成

这一步的细节在 5.4 单独讲。简要说：复用 `gaworld.population` 的完整管线（IPF 采样、家庭、工作单位、社交图），但把 `geography.district_weights` 换成这座城**真实存在的行政区**——所以居民的住所写出来是「兰亭街道·自住房」而不是「余杭·商品房」。产出三个文件：`agents.csv`（状态，utf-8-sig）、`profiles.md`（人物小传）、`population_manifest.json`（可复现记录）。

第 6 步：城市包落盘

最后把上面所有东西写进 `data/cities/<slug>/`，并生成 `city.json` 清单：地名、坐标、bbox、规模、地图模式、人口数，外加一条 **history**——每一步操作都往里追加一条带时间戳的记录（`map.virtual`、`map.real`、`environment`、

`knowledge`、`population.add`……）。造城过程中出了什么状况、走了哪条降级路径，翻 history 就知道。

目录名由地名生成。非 ASCII 地名没有不引依赖就能做的音译，退回哈希会让 `data/cities/` 变成一堆乱码目录，所以 `绍兴柯桥` 直接当目录名，人能读。

5.2 两种地图模式

城市包里可能有两张图。`city.json` 的 `map.mode` 告诉你现在用的是哪张：

- `real`——有 `map.geojson`，来自真实 OSM。路网、地铁线、河流、场所节点都是真的。
- `virtual`——只有 `citymap.md`，程序化生成。

`map_mode` 是个动态判断：只有 `map.geojson` **还在**才算 `real`。文件被删了，它自动退回 `virtual`，城市照跑。

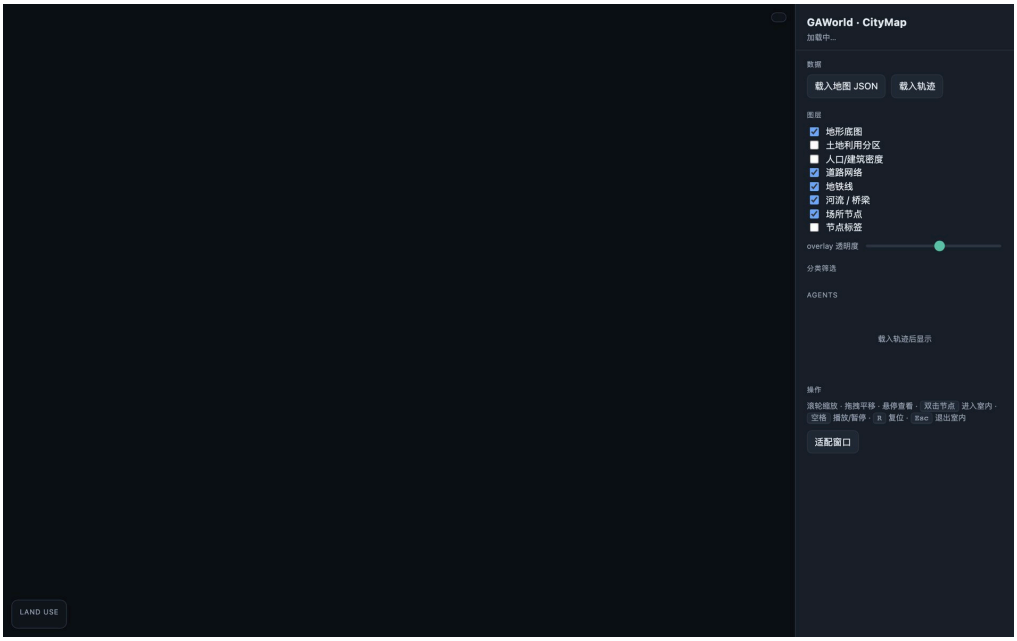


图 5–2　城市地图渲染器（CityMapView）的图层面板 这一屏是城市地图的渲染器本身。勾选哪些图层，决定屏幕上画什么：道路网络、地铁线、河流与桥梁、场所节点来自地图包，人口/建筑密度和土地利用分区是叠在上面的分析图层，「overlay 透明度」调的是叠加层盖住底图的多少。城市面板的预览、仿真回放、Agent Studio 用的是同一个渲染器（`site/dashboard/citymap-view.js`），所以**这里看到的和跑起来时是同一张图**，不会出现「预览一张、跑另一张」。

还有一条值得记住的规矩：**真实地图里没找到河，就是没有河**。原先找不到河流时会退回示例的钱塘江，于是绍兴（OSM 包里 0 条河）被画上 2794 个水面 tile 和 15 座凭空的桥。地铁同理：用了 `@` 指令的地图不再继承默认地铁，没写 `@metro` 就是真没有。

5.3 完整命令序列

下面这一串是可以直接复制运行的。子命令共 11 个：`create` / `list` / `show` / `add-agents` / `add-agent` / `migrate` / `knowledge` / `locale` / `news` / `use` / `delete`。

```
# 1. 联网造城：地理编码 + 真实 OSM 路网，顺便生成 200 名居民
python -m gaworld.city create "绍兴柯桥" --size 200

# 2. 断网造城：完全程序化，同一地名永远得到同一座城
python -m gaworld.city create "柳溪村" --offline --scale tiny

# 3. 造一座本来不存在的城：让模型照描述设计
```

```
python -m gaworld.city create "翡翠屿" --description "热带火山岛，靠渔
        业和潜水旅游为生"

# 4. 看一眼
python -m gaworld.city list
python -m gaworld.city show 绍兴柯桥

# 5. 往城里加智能体：批量、单个、从别处迁入
python -m gaworld.city add-agents 绍兴柯桥 --size 200 --preset cn_co
        unty_town
python -m gaworld.city add-agent  绍兴柯桥 --name 林素 --age 34 --job
        "社区医生"
python -m gaworld.city migrate    绍兴柯桥 --agent-id 31

# 6. 让仿真用这座城（下次运行生效）
python -m gaworld.city use 绍兴柯桥

# 7. 删掉（--yes 是必须的）
python -m gaworld.city delete 绍兴柯桥 --yes
```

`list` 打出来是一张表，列是 `SLUG / SCALE / MAP / AGENTS / NAME`。`show` 打的是完整的 `city.json`（格式化 JSON）。

`create` 的参数（`python -m gaworld.city create --help` 的真实输出）：

参数	取值	作用
`name`	位置参数	地名，如 `绍兴柯桥` 或 `Kyoto`
`--slug`	字符串	目录名，默认按地名生成
`--scale`	`tiny` / `small` / `medium` / `large` / `metro`	覆盖推断出的规模
`--offline`	开关	跳过地理编码与 OSM，纯程序化
`--description`	字符串	照描述虚构这座城，不查地名
`--image`	PNG/JPEG 路径	照草图虚构这座城，需要多模态模型
`--force`	开关	覆盖同 slug 的已有城市
`--seed`	整数	程序化布局的种子
`--size`	整数	顺便生成多少居民
`--preset`	见 5.4	人口预设；不给就跟随城市调研结果

`add-agents`：`--size`（默认 100）、`--preset`、`--seed`、`--replace`（默认是**追加**）。`add-agent`：`--name` 和 `--age` 必填，其余有默认值——`--gender`（女）、`--job`（自由职业）、`--hukou`（本地）、`--education`（本科）、`--income`（0.0）、`--residence`（不给就从该城行政区里随机挑一个）。`migrate`：`--agent-id` 必填，`--from-csv` / `--from-md` 默认指向 `data/` 下的默认语料，`--from-city` 覆盖它们，`--keep-residence` 关掉重新分配住所。`use` 只有 `--clear`（恢复默认世界）。`delete` 只有 `--yes`，不给就拒绝执行。

几个容易踩的点：

- **`size` 会被收敛到 20–5000**（IPF 需要足够样本才稳定）。要 5 个人会得到 20 个，CLI 和面板都会把这件事说出来，不会闷声改掉。
- **默认追加，`--replace` 才是替换**。追加的一批有自己的家庭和社交图，**不会**和先前那批织在一起——要整座城互相连通，就一次生成完。

5.4 人口预设：六种社会结构

`--preset` 有六个合法值。下面是它们在 `gaworld/population/schema.py` 里的真实数值——注意 `cn_county_town` 和 `custom` 是**空覆盖**，走的是数据类的默认基线（`cn_county_town` 就是这个基线本身，`custom` 则是「什么都由你自己给」的起点）。

预设	代表什么社会结构
`cn_county_town`	中国县城/乡镇基线：中位年龄 36、流动人口占比 0.38、大专以上 0.35、就业率 0.68、服务业占 0.38、月收入中位 6500、基尼 0.42、户均 2.6 人
`cn_tier1_district`	一线城区：中位年龄 34、外来人口 0.55、大专以上 0.52、就业率 0.74、科技业占 0.30、月收入中位 11000、基尼 0.46
`aging_community`	老龄社区：中位年龄 52、65 岁以上占 0.34、外来人口仅 0.12、大专以上 0.18、就业率 0.42、服务业 0.46、医疗 0.18、月收入中位 4200、三代同堂户占 0.24
`college_town`	大学城：中位年龄 27、大专以上 0.80、就业率仅 0.35、教育业占 0.34、合租占 0.45、流动意愿 0.62、平台依赖 0.66
`us_suburb`	美国郊区：中位年龄 39、18 岁以下占 0.23、外来人口 0.14、大专以上 0.45、就业率 0.63、月收入中位 5800、基尼 0.48、户均 2.5 人、单人户 0.28、合租仅 0.05
`custom`	空模板，所有字段由你自己给

不给 `--preset` 也可以。不给就用这座城调研出来的人口结构（东莞的外来人口占比 0.62，美国郊区 0.14），结果来自 `locale.json` 的 `suggested_overrides`；给了就以你给的为准——显式选了 `aging_community` 就是要老龄社区，不该被调研结果覆盖。

5.5 城市知识库：让城市影响居民

地图决定居民**在哪**，知识库决定这里**是个什么样的地方**——什么产业、在发展什么、缺什么人。没有它，柯桥和鹤岗的居民除了地名不同，活得一模一样。

知识库是两个文件的组合：`knowledge.json`（产业与就业画像）和 `news.json`（滚动本地新闻缓存）。前者是静态的、后者是活的。

来源分三层，越往下越保守

```
联网搜索 + LLM 结构化 ──失败──▶ OSM 地图统计 ──太稀疏──▶ 空画像（什么都不声称）
```

LLM 被明确禁止用常识补全。提示词里原话是「不要凭常识补充或推测；搜索结果没提到的就不要写」。对真实城市，一句自信的「柯桥以旅游业为主」会被下游四个通道全部当成事实，比空画像坏得多。

地图兜底也有门槛。OSM 节点数量衡量的是**标注密度**，不是就业——绍兴的地图包里只有 55 个住宅区 + 22 所学校，直接按比例算会得出「教育业 100%」。所以要求至少 **2 个不同产业类别、6 个节点**，不够就返回空画像，四个通道保持静默，仿真回到没有这个功能时的行为。地图兜底出来的画像趋势一律是 `stable`：静态快照不知道方向。

四条影响通道

通道	作用点	效果
提示词	`background` → 认知/日程/目标	居民知道自己住在什么样的城市
技能成长	`derive_growth_profile` + `evolve_growth_profile`	旅游城市的居民更可能去学导游、酒店管理
就业收入	`economy.macro.industry_conditions`	本地主导且增长的行业收入系数更高
城市级 RAG	每位居民的 `external_info`	城市事实可被检索，与个人外部信息一起参与排序

这四条在 `gaworld/city/context.py` 里各自对应一个读出口，面板上的 `GET /api/city/knowledge` 会把它们的实际产出原样返回（`channels.prompt` / `channels.growth` / `channels.economy` / `channels.rag`），所以你能直接看到「这座城的知识库到底在往下游说什么」。

产业名是自由文本，不绑定六分类。 经济模块的 `JOB_INDUSTRY_MAP` 只有 tech/finance/medical/education/service/trade 六类，**没有旅游业**。知识库里存的是当地真实叫法（「纺织业」「跨境电商」「全域旅游」），只有需要落到经济模块时才由 `industry_mix()` 映射到最近的一类。这是刻意的：一座城的性格恰恰活在六分类抹掉的那些区别里——把旅游业和物流、零售一起压成 `service`，就把特意查来的东西丢掉了。

一个具体例子：城市发展旅游业

假设这座城正在发展旅游业。`knowledge.json` 里会写着：主要产业「全域旅游」趋势 `growing`，发展重点「文旅综合体」，本地紧缺「导游、民宿管家、景区讲解」。

1. **提示词通道**：`prompt_block()` 把这些压成一句「主要产业：全域旅游↑；发展重点：文旅综合体；本地紧缺：导游、民宿管家」，追加到 `background` 后

面。于是每一次认知提示词里都带着「这座城正在往旅游上走」。

2. **技能成长通道**：`growth_hint()` 把「正在发展：全域旅游」交给兴趣/技能画像的推导，让居民更可能把「学导游」列为一项成长。这里有个**必须记住的细节**：`profile_signature` 是成长画像的缓存键，城市签名**必须并进这个键**，否则同一个居民在纺织城和旅游城会共用一份缓存，第二座城的影响永远不会出现。
3. **就业收入通道**：`industry_conditions()` 把产业占比和趋势换算成六类的收入系数（落在 `[0.7, 1.4]` 区间内），写进 `economy.macro.industry_conditions`。旅游占主导且趋势向上，`service` 这一类的系数就偏高——**这一行真的更赚钱**。这条通道没有改经济模块一行代码：`industry_conditions` 本来就从配置读，城市包只是提供了一个覆盖。
4. **RAG 通道**：`rag_chunks()` 生成「[本地产业]……的旅游业正在增长」「[本地就业]……目前紧缺：导游……」这样的独立事实，写进每位居民的 `external_info` 记忆库，之后检索时与个人外部信息一起参与排序。

还有一层区分值得注意：**职业动机和社交传染是分开的两条路径**。因为政府推旅游而去学导游，是一次**职业**决策（`kind=skill`、`category=职业`、`career_link=True`），不是从朋友那里染上的爱好。默认采纳概率是社交传染的一半（`adopt_chance` 0.35 × `opportunity_factor` 0.5）——城市经济是按月推着人走的，不会一夜换岗。

运行时新闻：两个时钟

仿真 365 天可能一下午就跑完。「每个仿真日拉一次今天的新闻」等于在新闻根本没变的同一个真实下午里发 365 次搜索。所以：

- **抓取**按**真实时间**门控（`city_news.ttl_hours`，默认 6 小时）——跑多快都只抓一次
- **投喂**按**仿真时间**轮转——每个仿真日取缓存里不同的一小片（默认每天 2 条，缓存上限 60 条），长跑不会天天同一条头条

网络断了也不会每个仿真日重试——时间戳照常更新，窗口照常推进。

```
python -m gaworld.city knowledge 绍兴柯桥                  # 看
python -m gaworld.city knowledge 绍兴柯桥 --rebuild        # 联网重建
python -m gaworld.city knowledge 绍兴柯桥 --rebuild --offline  # 只用
    地图估算
python -m gaworld.city news 绍兴柯桥 --refresh             # 过期才抓
python -m gaworld.city news 绍兴柯桥 --force               # 强制抓
```

5.6 居民的地方特征：`locale.json`

知识库回答「这地方是干什么的」，locale 回答「**住在这儿的人叫什么**」。

在这之前，人口合成器把中国的一套写死了：全国通用的百家姓、户籍四档（本地/省内/外省/外国）、商品房/老小区这套居住形态。于是拿 `us_suburb` 模板建的美国城市，统计上像美国，人却是中国的——「王伟，28，本地户籍，住西古山·商品房」。

建城时会额外发一次 LLM 调研，产出 `locale.json`。以英国大伦敦为例（仓库里 `data/cities/伦敦/locale.json` 的真实内容）：

字段	管什么	英国大伦敦实例
`code` / `label`	地区代码与名称	`en-GB-GLA` / 英国大伦敦
`surnames`	姓氏池（76 个）	Smith、Jones、Brown…

字段	管什么	英国大伦敦实例
`given_male` / `given_female` / `given_neutral`	名池（43/43/15 个）	Oliver、Olivia、Alex…
`given_mode` / `joiner` / `given_first`	名字怎么拼	`whole`（完整的名）、空格分隔、名在前
`residency`	户籍/居住身份四档	本地 / 省内 / 外省 / 外国
`residence_suffixes`	居住形态（9 种）	自住公寓、出租公寓、议会公屋、联排住宅、独栋住宅…
`suggested_overrides`	人口结构建议	中位年龄 35.6、外来占比 0.37、大专以上 0.58、月收入中位 28500

姓氏是按城市查的，不是按国家。 广东该以陈黄梁林罗为主，而不是照搬全国百家姓；东莞这种制造业城市的外来人口占比要到 0.6 以上。

只本地化名字，不本地化档案

人物小传、职衔仍然是中文，这是刻意的：`gaworld/sim/agents_loader.py` 的 `parse_profile` 用 `**基础信息**：…NN岁，…户籍，居住…` 这个形状的正则读档案；`gaworld/economy/finance.py` 靠**中文关键词**（运营 / 销售 / 工程师）反推行业和收入区间。把职衔换成英文，这两条契约当场就断，而且是**静默地断**——行业会全部落到兜底分类。所以 `residency` 哪怕在美国城市也还是中文词（本市/本州/外州），只是**语义**对了。

调研失败就退回内地默认值，并且说出来

locale 调研和知识库一样是 best-effort，从不阻断建城。**失败、离线建城、虚构城市**这三种情况都退回内地默认池——也就是这个功能存在之前的行为——并在 `city.json` 的 history 里记一条 `locale.default`，`reason` 写明是 `research_failed` 还是 `offline`。所以一座美国城市如果满屏 `王伟`，是查得到原因的，不是玄学。

```
python -m gaworld.city locale 东莞长安                # 看这座城现在用的是什么
python -m gaworld.city locale 东莞长安 --rebuild      # 重新调研
```

`locale` 的输出里，如果这座城还在用默认池，会明写「(尚未调研，用的是内地默认值 — 加 —rebuild 重新研究)」。

5.7 切换当前城市

三个入口写的是同一个开关——仓库根目录 `dashboard_config.json` 里的 `"city"` 字段：

```
python -m gaworld.city use 绍兴柯桥          # CLI
python -m gaworld.city use --clear 绍兴柯桥  # 恢复默认世界
```

- **控制台「城市」页签** → 选中城市后点「用这座城市运行」
- **控制台主面板的「城市」下拉**（运行控件第一项）→ 直接选好再点「运行仿真」

下拉里每项显示「城市名 · N 人 · 真实地图/程序化」。**没有居民的城市是禁选的**——它跑不起来，与其让你选完再被服务端拒绝，不如一开始就不给选。

选中之后，这些配置项整体指向该城市包：

配置项	来自
`map_mode` / `map_path` / `real_map_path`	城市包（有 `map.geojson` 才是 `real`）
`csv_path` / `md_path`	城市包（有人口才写入）
`environment` / `external_environment`	`environment.json`
`background`	`environment.json`
`memory_dir` / `log_dir` / 另外十来个运行期目录	该城市的**运行根目录**（见下）

优先级：`dashboard_config.json` → 全局 `data/environment_config.json` → **城市包** → `GAWORLD_CONFIG_OVERRIDES`。

城市排在全局环境文件**之后**是刻意的：那个文件是默认值，不是针对某座城的意图。否则一座大陆性气候的城市会继承全局文件里的亚热带事件。而 `GAWORLD_CONFIG_OVERRIDES` 仍然最后生效，保留了逃生舱。配置里写了一座已被删除的城市也不会让仿真起不来——解析失败就退回默认世界。

Agent IDs 是按城市各自编号的

这是最容易踩的一脚：**编号在每座城市都从 1 开始。**

配置里留着 `agent_ids: [37]` 再切到一座只有 30 人的城，`build_agent` 会对空结果取 `.iloc[0]`，几分钟后在运行日志里抛一个光秃秃的 pandas `IndexError`——那时你早就不看了。

所以运行前会**先拦下来**：

```
城市「梅湾村」只有 30 位居民，但 Agent IDs 里有 [37]。请改成 1–30 之间的编号。
```

下拉旁边的 Agent IDs 输入框也会跟着把 placeholder 改成当前城市的实际范围。

每座城市有自己的运行根目录

选中城市后，**整棵运行树**从 `output/` 挪到 `output/cities/<slug>/`：

```
output/cities/<slug>/
├── memory/            记忆、sim_state.json（世界时钟）、向量库、目标、关系、经济状态
├── logs/              每个 agent 的运行日志
├── diaries/           日记
├── state/             状态曲线 CSV        ├── economy/     经济账本
├── visualization/     轨迹 trace          ├── network/     社交网络图
├── environment/       环境事件时间线      ├── life_events/ 生活事件
├── intervention/ ├── work/ ├── collaboration/ ├── twin/ ├── imported_agents/
```

这件事是必须的，不是整理癖：记忆和日志只按 **agent id** 命名，`sim_state.json` 里也只有**一个**世界时钟。共用一棵树意味着 A 城的 1 号和 B 城的 1 号是同一份 `agent_1.json`、同一本日历——换城市不是换世界，是让新居民继承上一座城的人生。

几点需要知道的：

- **没选城市时一切照旧**，还是 `output/memory`、`output/logs`。旧数据不动。

- 目录名与默认 `output/` 树**逐项对齐**：Analytics 是把 `state/`、`economy/` 拼到运行根目录上读的，回放页是靠 glob `output/*/visualization` 找 trace 的。
- 运行根目录放在 `output/` 而**不是**城市包里，因为城市包是这座城的*定义*，应当能直接拷给别人，不该捎带某个人二十年的记忆。
- **删城市会连带删掉它的运行目录**。否则重建一座同名的城，新居民会直接继承被删那座城的记忆和世界时钟。

顺带说一句容易混的：多人共玩部署里，每个用户有一个自己的 world（`output/worlds/<id>/`），那是账号层面的隔离，和城市层面的 `output/cities/<slug>/` 是两套东西——world 的运行产物用的还是同一套 `RUN_PATHS` 布局。

任何城市的居民都能读，但 Studio 只能编辑当前城市

这是两个不同的能力，别混：

- **读，是不设限的**。`GET /api/city/agents?city=<slug>`、`GET /api/city/agent?city=<slug>&id=<n>` 直接读城市包，**不必先切成当前城市**。城市面板的「城市居民」列表就支持按姓名/职业/住所/编号搜索。
- **写，只限当前城市**。Agent Studio 始终只编辑当前城市，因为编号在每座城市各自从 1 开始，而记忆、社交与财务、大五人格表都按编号存放，跟着当前城市走。同时编辑两座城，就会把 A 城 3 号的改动写到 B 城 3 号名下。

从城市面板「在工作台查看」跳进 Studio（带 `?city=<slug>&agent=<id>`）时只是**只读浏览**；面板上的「设为当前城市」按钮一按才切过去并进入编辑。

5.8 城市包的目录结构

`data/cities/<slug>/` 里有什么：

```
data/cities/<slug>/
├── city.json                  清单：地名、坐标、bbox、规模、地图模式、人口数、操作 history
├── citymap.md                 程序化地图（始终存在：兜底 + 提示词上下文）
├── map.geojson                真实 OSM 地图（抓取成功才有）
├── environment.json           环境事件 + background
├── knowledge.json             城市画像（产业、发展重点、紧缺岗位、来源链接）
├── locale.json                居民的姓名池、居住形态、户籍分档、人口结构建议
├── news.json                  本地新闻缓存（按真实时间刷新）
├── agents.csv                 人口状态（utf-8-sig，仿真器契约）
├── profiles.md                人物小传
└── population_manifest.json   人口生成的可复现记录
```

这些文件**不是每座城都有**：离线建的城市没有 `map.geojson`，没做 locale 调研的没有 `locale.json`，知识库调研没成功的 `knowledge.json` 是空画像。文件出现与否本身就是能力是否成功的标记。

注册表是**目录扫描**，不是索引文件——索引会在有人手工拷贝或删除文件夹时悄悄和现实脱节。清单坏掉的目录会被跳过，不会让整个注册表不可用。

`city.json` 里的投影锚点

真实地图包的 `meta.origin` 记着这座城自己的投影锚点：

```
{"lat": 30.084796, "lng": 120.490807, "lat_per_km": 0.009009009, "lng_per_km": 0.010381686}
```

地图层的内置常量是按杭州（约 30°N）标定的，一个经度约 96 km。换到别的纬度，经度度数会随 `cos(lat)` 收缩——巴黎（48.86°N）用杭州常量算东西向距离会**偏大 31%**。带锚点后误差归零。没有 `meta.origin` 的包（比如已提交的 `data/hangzhou_real.geojson`）行为完全不变。

5.9 失败与降级

四种常见情况，以及代码实际会做什么：

情况	表现	降级到	怎么绕开
地名查不到	`geocoding failed (...)` 警告	合成地点记录（坐标落在杭州锚点附近，`source=offline`），真实路网直接跳过	换个更完整的名字，或 `--slug` 手工指定
OSM 抓取失败	`overpass error via <url>`，节点少于 8 个判定太稀疏	只有程序化地图，`city.json` 记 `map.real.skipped`	等镜像恢复；急的话直接 `--offline`
完全断网	地理编码、OSM、知识库调研、locale 调研全部失败	合成地点 + 程序化地图 + 空画像 + 内地默认姓名池	`python -m gaworld.city create "柳溪村" --offline --scale tiny`
虚构城市	`--description` / `--image`	走另一条流水线：模型出 JSON 设计	见下

虚构城市和 `--offline` 是两回事。`--offline` 只是不联网，城的样子仍然由地名的哈希决定，描述里的火山和渔港不会出现在图上。`--description` / `--image` 走的是另一条：Nominatim 和 Overpass 为真实地方做的那份工作，这里由一次大模型调用来做。**模型输出的是 JSON，不是 `citymap.md`**——出 JSON 则每个字段都能校验、能兜底；直接让它写指令格式，换来的失败模式是「解析出半座城，而且没人发现」。

校验层刻意写得很宽容——模型编了个不存在的分类、把坐标写到 0–1 之外、或者忘了连某个城区的路，代价应该是城市朴素一点，而不是创建失败：分类不在词表里落到 `mixed`，坐标越界夹到 0–1，城区重名丢掉后一个（否则会在地图层合并成一个节点），某个城区没连路就挂一条 collector 上去（不可达的城区意味着在那上班的人回不了家），地铁只有两站就当作没有地铁。一个城区都没有才报错——这时候静默退回程序化，会给你一座忽略了描述、却看起来像照做了的城。

坐标约定写死在提示词里：**x=0 最西、x=1 最东；y=0 最南、y=1 最北**，所以描述里「北边是火山」，火山就在图的上方。**中文地名要显式声明分类**：`infer_category` 认的是英文关键词，所以每个小地点都额外写一行 `@node: 鱼市 | kind=place | category=commerce`——只带分类、不带坐标，位置仍然交给地图层的街区排布算法。

气候和产业来自**同一次**调用：描述里的「热带火山岛」变成 `climate=tropical`，再换算成一个代表纬度喂给环境层，于是这座城真的会下台风；「靠渔业和潜水旅游为生」变成 `knowledge.json` 里的产业，`source` 标成 `imagined` 而不是 `web`——下游任何一处都能分清哪些是查来的、哪些是编的。虚构城市**不会**去联网搜产业：搜一个不存在的地方，最好的结果是什么都搜不到，最坏的结果是搜到一个同名的真实地方。

> **草图需要多模态模型。** 路由到的模型不支持图片输入时会**直接报错**，不会假装看过图。在「配置」面板给 provider 加 `"vision": true`，或把 `city_design` 路由到多模态后端。另外城市设计的输出有几千 token，这一处调用会自己申请 `max_tokens=8000`——provider 默认的 512 会把 JSON 从中间截断。

5.10 面板：城市实验台

命令行的功能在控制台上都有对应，而且比命令行更顺手。



图 5–3　控制台的「城市」页签 上面是新建城市表单：地名、规模（默认「自动判断」，即不给 `--scale`）、初始人口、人口模板、随机种子，还有「离线模式」开关；「真实地名 / 虚拟城市」两个入口对应 `--description` 那条路线。下面是已有城市列表——每项标着地图来源（真实地图 / 程序化）、规模档位和智能体数，运行中的那座带「运行中」标记，右上角显示当前使用的世界。

选中一座城市后，左栏中部出现**城市地图**预览（滚轮缩放、拖拽平移、双击复位），左栏下部是这座城的居民列表，右栏是详情和「用这座城市运行」。联网创建要 20–60 秒（取决于 Overpass 镜像状态），表单期间会禁用并显示进度文案。

地图预览下方是「城市知识库」卡片，卡片上那行说明正是 5.5 那四条通道：它「会进入认知提示词、影响居民学什么技能、影响本地各行业的收入，并写入每位居民的检索记忆」。三个按钮「联网重建」「按地图估算」「抓取最新新闻」对应 5.5 里的命令。

本章小结

- 造城是六步流水线：地理编码 → 真实 OSM 路网 → 程序化兜底 → 环境推导 → 人口合成 → 城市包落盘。**每一步都有降级路径**，断网时用 `--offline` 从第 3 步起步。
- 规模（`tiny`–`metro`）不只影响布局：它同时决定有没有地铁、环境事件的措辞、以及 bbox 允许多大——柯桥区的 bbox 收到 28.9 km 才不会把隔壁诸暨拉进来。
- 人口预设六选一，也可以不给；不给就跟随这座城调研出来的人口结构，给了就以你给的为准。`size` 会被收敛到 20–5000，`add-agents` 默认追加、`--replace` 才替换。
- 城市知识库通过**四条通道**影响居民：提示词、技能成长、就业收入系数、城市级 RAG。产业名用当地真实叫法，只在落到经济模块时才映射到六分类。
- `locale.json` 只本地化名字，**不**本地化档案——档案解析和经济分类都靠中文正则与关键词契约；调研失败退回内地默认值，并在 `city.json` 里记一笔。
- 每座城市的 Agent ID 都从 1 开始，各有各的运行根目录 `output/cities/<slug>/`。任何城市的居民都能读，但只有当前城市的居民能编辑。

动手练习

1. **造一座虚构城市，并让它出现在列表里**（约 15 分钟）

   用离线模式造一座不存在的城，并跑通「建城 → 加人 → 查看 → 切换」这条链：

   ```
   python -m gaworld.city create "柳溪村" --offline --scale tiny --
           size 30
   python -m gaworld.city add-agent 柳溪村 --name 林素 --age 34 --jo
           b "社区医生"
   python -m gaworld.city list
   python -m gaworld.city use 柳溪村
   python -m gaworld.city locale 柳溪村
   ```

   验收标准：`list` 的输出里出现 `柳溪村` 一行，`MAP` 列是 `virtual`（没联网抓 OSM），`SCALE` 列是 `tiny`（`--scale tiny` 覆盖了推断），`AGENTS` 列是 **31**（`--size 30` 在 20–5000 区间内不变，`add-agent` 再加一位）；`use` 之后仓库根目录的 `dashboard_config.json` 里出现 `"city": "柳溪村"`；`locale` 的输出最后一行提示这座城还在用内地默认值。

   想验证确定性，可以把 `create` 的 `--seed` 固定成同一个数再跑一遍（换个 slug 名），对比两次的 `citymap.md`——应当逐字节相同。

2. **造完城切过去，跑一次仿真，找到这次的产物**（约 30 分钟）

   接着上面（或用你已有的任何一座城），让仿真在这座城里跑起来，并确认运行产物落在了**这座城自己的**运行根目录下：

   ```
   python -m gaworld.city use 柳溪村
   python generative_city_sim.py run --sim-days 3
   ls output/cities/柳溪村/
   ```

   验收标准：`output/cities/柳溪村/` 下面出现 `memory/`、`logs/`、`environment/` 等目录，并且**不是**空的；而 `output/memory/agent_1.json`（若存在）的修改时间**没有**随这次运行更新——先记下运行前的时间戳，跑完再比一次，这是「记忆没有串城」的直接证据。

   再切回默认世界（`python -m gaworld.city use --clear 柳溪村`）跑一次，`output/memory/` 会被更新，而 `output/cities/柳溪村/` 不动——这就验证了「每座城市有自己的运行根目录」不是文档里的一句话。

   > 如果运行时报「城市『柳溪村』只有 30 位居民，但 Agent IDs 里有 [37]」——那正是 5.7 说的编号按城市各自从 1 开始。把配置里的 `agent_ids` 改成 1–30 之间的编号，或直接清空。

ewpage

第 6 章　跑第一次仿真：命令、步长与产物

前五章把「跑得起来」需要的东西备齐了：模型接好、居民造好、城建好、种子数据读懂。现在该真正跑一次。

这一章只解决一个问题：让你能自己判断一次运行**跑了什么、花了多少、结果在哪**，并且知道下一次该把它放大到多大。核心是一个工程决策——**时间粒度**。同一套仿真内核有四种跑法，50 个居民跑十年，调用量从 500 次到几十万次不等；选错了，要么烧掉预算，要么根本看不到你问的那个问题。

6.1 最小可运行：一条命令

前提是模型已经配好（见第 3 章）：`CONFIG["llm"]["routing"]["default"]` 指向一个可用的 provider，环境变量里有对应的 key。除此之外不需要任何准备——城市包、居民 CSV、日程模板都是仓库自带或你在第 5 章造出来的。

```
python generative_city_sim.py run
```

终端会开始刷日志。默认是 `simple` 模式，每个 tick 大约四行：

```
── [李泽宇 @ 09:30] 上午工作 ──
Loc: 互联网公司
Act: 推进最重要的一项任务
Refl: 感受：情绪有一点波动；教训：下次要更早判断状态和代价；后续倾向：更偏向省力或稳妥
```

想看完整字段（感知、计划、记忆召回、需求状态）切详细模式：

```
GAWORLD_LOG_MODE=verbose python generative_city_sim.py run
GAWORLD_LOG_LEVEL=DEBUG  python generative_city_sim.py run   # 含 token 计数与延迟
```

跑之前的那份配置是从哪来的，决定了你跑出来的到底是不是你以为的那个仿真。实际生效的配置是四层叠起来的，后者覆盖前者：

1. `gaworld/settings/` 里的代码默认值——`sim_days`、`agent_ids`、`long_run`、`routine_change` 这些都在这儿；
2. 仓库根目录的 `dashboard_config.json`——**控制台面板的草稿文件**，改面板参数就是改它，CLI 也读它；
3. 城市包——如果你用 `python -m gaworld.city use <slug>` 选过城市，地图、环境、背景提示词、以及整棵产物树的落盘位置都换成那座城的；
4. `GAWORLD_CONFIG_OVERRIDES` 环境变量（一个 JSON 字符串）——**它有最终决定权**。

想临时回到「纯净的代码默认值」，把第 2、3 层都屏蔽掉：

```
GAWORLD_IGNORE_LOCAL_CONFIG=1 python generative_city_sim.py run
```

跑完去哪里看结果？三个地方，按粒度从粗到细：

- **终端**：实时的人话摘要（上面那四行）。
- **`output/logs/run.log`**：完整结构化日志，事后考古用。
- **控制台面板**：`python generative_city_sim.py dashboard --port 8766`，然后开 http://127.0.0.1:8766/dashboard。下面是配好参数、点「运行仿真」的那一屏——四组参数分别是城市与居民、时间、模型、运行方式，底下是「定时运行 / 保存配置 / 重置并运行 / 停止 / 运行仿真」。

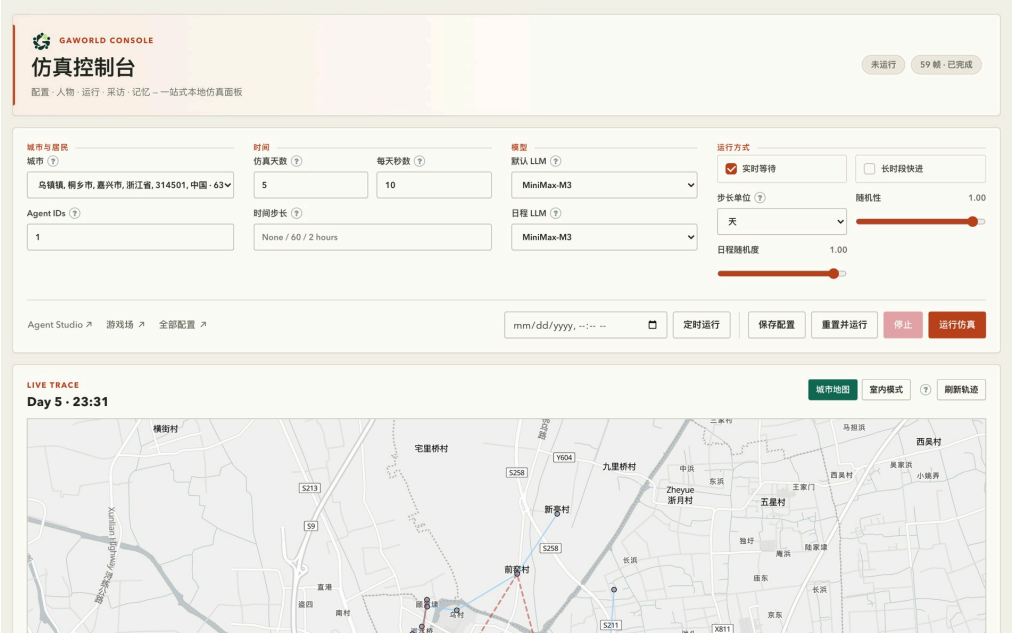


图 6–1　仿真控制台：把「跑什么」变成四组参数 最右边「运行方式」那一栏是本章的重点：`长时段快进` 勾上以后，步长单位、随机性、日程随机度三个控件才生效——它们正是 6.4 节要讲的两个旋钮。「仿真天数」在勾选快进、切换单位之后会跟着变成「仿真月数」或「仿真年数」，浏览器只提交 `{unit, count}`，天数由服务端按真实日历换算，和 CLI 的 `--sim-years 10` 得到完全相同的 `sim_days`。左下角「重置并运行」等价于先跑一次 `reset` 再跑 `run`，6.6 节会讲为什么改了配置后这一步几乎是必须的。

6.2 时间粒度：这一章的骨架

GAWorld 的仿真内核跑的是一条**日内时刻循环**：每天有一批时刻点，每个时刻点每个居民要走一遍完整的认知管线（`gaworld/sim/pipeline.py` 里的 12 个阶段）。每走一遍就是一次 LLM 调用。这是最细的分辨率，也是最贵的。

想跑长一点，有三条路把它们压扁。压扁的方式都是同一个：把**一步**压缩成**每个居民一条简报**，一步 = 一次 LLM 调用。

模式	命令	每居民每天的调用	50 人 × 10 年
日内逐步（默认）	`run`	数十次（每个 tick 一次）	数百万次级
按天快进	`run --sim-days 600 --fast-forward`	1 次	约 18.25 万次
按月	`run --sim-months 120`	1/30 次	6,000 次
按年	`run --sim-years 10`	1/365 次	500 次

时间粒度怎么选：调用量与保真度的取舍

以 50 位居民、10 年为例，按天与按年的调用量差三个数量级——大跨度模拟必须换步长。

步长	每人的简报频率	10 年总调用量	保留下来的细节	适合回答的问题
日内逐步（默认）	每天多条时刻级行动	数十万次	全天作息、偶遇、即时反应	一天内的人际互动与节律
按天快进	每天 1 条日简报	约 18.25 万次	状态、目标、关系的日级推进	一周到一个月的习惯养成
按月（`--sim-months`）	每月 1 条阶段简报	约 6,000 次	月度结算、兴趣衰减、家庭开销	季度到年度的季节性现象
按年（`--sim-years`）	每年 1 条阶段简报	约 500 次	生涯轨迹、资产的多年后效	政策的长期效应与代际变化

> 底层日历仍逐日推进（大小月、闰年都算对），日边界钩子按 ≤30 天区块补跑——跑满一年会扣满一年房租、走满 12 次月度结算，而不是只扣一天。

图 6-2　四种步长的调用量与保真度 从第一行到最后一行，总调用量差三个数量级，保留下来的细节也换了一批：日内逐步留下的是「全天作息、偶遇、即时反应」，按天留下的是「状态、目标、关系的日级推进」，按月留下「月度结算、兴趣衰减、家庭开销」，按年留下「生涯轨迹、资产的多年后效」。**一行都换一种问题**——不是精度分级，是问题分级。

> 图里「按天快进」那一行的调用量以正文表格为准：50 人 × 3653 天 = 182,650 次，约 18.25 万次。

6.2.1 一步里到底发生了什么

快进不是「把日内循环加速播放」，而是**换了一条认知路径**。按天快进时，每个居民每天收到一条「日简报」，模型在这条简报里同时给出：状态增量、目标推进、社交信号、意图。按月/按年时是「阶段简报」，额外带 2–4 条 `highlights` 里程碑，逐条写进记忆——所以跑完十年，居民的长期记忆里确实有那十年的事，只是粒度是「这个月」而不是「今天下午」。

容易误解的一点：**底层日历仍然逐日推进**。一步跨过的是真实的日历月或年，大小月、闰年都算得对——`--sim-months 24` 实际覆盖 731 天（取决于起始日期，默认 `calendar.start_date` 是 `"today"`），`--sim-years 10` 覆盖 3653 天。日边界钩子（经济结算、兴趣衰减、家庭开销、房租）按 `long_run.hook_chunk_days`（默认 30 天）的区块**补跑**，所以跑满一年会扣满一年房租、走满 12 次月度结算，而不是只扣一天。关系衰减也按「距上次联系的天数」算，天然吃满整段跨度。

工资是唯一需要特别处理的：日内模式下工资是逐时刻结算的，粗粒度下没有时刻可结算，经济模块会按「宏观调整后的时薪 × 目标工时 × 工作日数」补记一笔近似工资（从资金池支出，货币守恒，计入当月计税基数）。否则跑满一年会得到「只付房租不挣钱」的全员破产。

6.2.2 代价

代价是彻底的，而且不可逆：**日内与逐日细节全部消失**。一步之内只剩一条叙事简报和一组累计增量。状态增量还有夹逼上限——`long_run.max_state_delta` 默认 0.15，按月 ×2、按年 ×3——意思是「一个月能明显改变一个人，但不会一步跳到极值」。

所以判断标准很清楚：想看「一天内的人际互动与节律」，只能用日内逐步；想看「一周到一个月的习惯养成」，按天；想看「季度到年度的季节性现象」，按月；想看「政策的长期效应与代际变化」，按年。**用错粒度的代价不是「精度低一点」，是「这个问题在错误的尺度上被问」。**

6.3 运行参数逐条解释

`run` 子命令的参数很少。先把 `--help` 的真实输出抄在这里：

```
usage: generative_city_sim.py run [-h] [--sim-days SIM_DAYS]
                                  [--sim-months SIM_MONTHS]
                                  [--sim-years SIM_YEARS]
                                  [--time-unit {day,month,year}]
                                  [--fast-forward]
```

就这五个，加上 `-h`。其余旋钮（跑哪些居民 `agent_ids`、每天模拟多少秒 `seconds_per_day`、日内时间步长 `time_step_minutes`、用哪个模型）都不在 CLI 上，在配置里。

参数	含义
`--sim-days N`	覆盖仿真天数。默认取配置里的 `sim_days`。
`--fast-forward`	长时段快进：把每一步压成每个居民一条简报（一步一次 LLM 调用），跳过日内时刻循环。状态、目标、关系仍会近似推进。
`--sim-months N`	跑 N 个日历月。**自动打开快进**，步长单位为月（除非 `--time-unit` 另说）。
`--sim-years N`	跑 N 个日历年。**自动打开快进**，步长单位为年（除非 `--time-unit` 另说）。
`--time-unit {day,month,year}`	快进的步长单位：每个居民每天 / 每月 / 每年一条简报。**自动打开快进**。

三条必须记住的规则：

1. **`--sim-months` / `--sim-years` / `--time-unit` 任意一个都会自动打开快进。** 一个月长的日内时刻循环不是一个东西，所以没有「不开快进」这个选项。
2. **跨度优先于天数。** `--sim-days` 先算出天数，`--sim-months` / `--sim-years` 随后按真实日历把天数覆盖掉。两者都给时，**跨度赢**。
3. **`--sim-years` 和 `--time-unit` 是两件事。** `--sim-years 10` 说的是「跑十年」，`--time-unit` 说的是「一步多大」。`--sim-years 10 --time-unit day` 的意思是「跑十年，但一天一步」——3653 步、每居民 3653 次调用；而 `--sim-years 10 --time-unit month` 是 120 步。

快进模式打开时，终端会先打一行确认：

```
⏩ 长时段快进模式已启用：3653 天 = 10 个「年」步，每步每个智能体生成一条简报（近似推进状态/目标/关系）。
```

另外两个你可能马上想用的子命令（真实 `--help`）：

```
usage: generative_city_sim.py reset [-h]

usage: generative_city_sim.py interview [-h] --agent-id AGENT_ID
                                        [--question QUESTIONS]
                                        [--questions-file QUESTIONS
_FILE]
                                        [--context CONTEXT]
```

`reset` 没有任何选项（6.6 节）。`interview` 必须给 `--agent-id`，`--question` 可以重复给多次，`--questions-file` 从 UTF-8 文本文件里读（一行一个问题），`--context` 是可选背景。它的输出**打印到标准输出**（一段 JSON），不像群体采访那样落盘到 `output/interviews/`——那件事要另跑 `python -m gaworld.interview`，是另一条命令线。

```
python generative_city_sim.py interview --agent-id 31 --question
        "你最近在担心什么？"
```

6.3.1 顶层还有哪些命令

`generative_city_sim.py --help` 的完整子命令表（真实输出）：

```
{run,reset,interview,create-agent-from-social,rag-add,rag-import,co
mpare-event,
```

```
  parallel-worlds,serve-viz,dashboard,serve-distributed}
```

其中 `compare-event`（跑两个平行仿真，一个带事件一个不带，然后对比）和 `parallel-worlds`（按一份 JSON 规格跑 N 个世界并报告各自偏离基线多远）是 6.5 节那棵产物树里 `output/comparisons/` 和 `output/parallel_worlds/` 的来源——**普通 `run` 不会生成它们**。`serve-viz` 单独起一个轨迹回放服务（默认 http://127.0.0.1:8000/site/simviz/index.html），`dashboard` 起控制台，`serve-distributed` 起分布式中继。

6.4 两个「随机性」旋钮

配置里有两个都叫 randomness 的东西，作用完全不同，混起来会得到莫名其妙的实验结果。

`long_run.randomness`（默认 0.3）——世界里会发生什么。 它是快进期间的随机性：越高，**突发事件**越频繁（意外开销、机会、冲突、健康、人际变故），智能体**状态波动**也越大；`0` = 完全确定，无突发、无抖动。机制上，每个居民每天按约 `0.3 × randomness` 的概率掷出一次突发事件，命中时简报里会自然带出这件事、状态随之明显波动（日志里以 `⚡` 标记）；同时每天给情绪、压力等状态叠一层零均值抖动，幅度随 `randomness` 增大。按月/按年步长时，期望突发事件个数按这一步覆盖的天数缩放，上限 4 件/步。**它只在快进模式下有意义。**

`routine_change.randomness`（默认 0.0）——居民多常不按自己的日程过。 这是日程偏离的总旋钮：越高，智能体越不按既定的每日 routine 行事——临时改活动、偏离计划更频繁，高承诺活动更容易被放弃，躁动更多；`0` = 严格按调好的默认值走。睡眠时段不受影响。它同一块配置里还有 `enabled`（默认开）、`base_chance` 0.08、`event_boost` 0.08、`policy_boost` 0.05、`max_chance` 0.45 和 `severity_pivot`（事件严重度超过它才推动改计划）。**精细模式和快进模式都起作用。**

一句话对照：

	`long_run.randomness`	`routine_change.randomness`
决定什么	外部世界发生多少意外、状态抖多厉害	居民多常临时改计划
生效范围	只在快进模式	两种模式都生效
调高的后果	剧情更动荡，状态曲线更毛躁	作息更不稳定，行为更难预测
默认值	0.3	0.0

两个滑杆都在控制台工具栏上：图 6-1 里最右边那栏，上面的「随机性」是 `long_run.randomness`，下面的「日程随机度」是 `routine_change.randomness`。它们的提示文字写得很准——随机性那个是「仅在长时段快进时生效」。

配置位置：`gaworld/settings/runtime.py`，`long_run` 块在第 36 行起，`routine_change` 块在第 329 行起。改 `dashboard_config.json` 或用 `GAWORLD_CONFIG_OVERRIDES` 都行，后者优先级更高。

6.5 产物地图：跑完去哪儿看

所有运行产物都在 `output/` 下。



图 6-3　output/ 目录地图 上半部分是「单个居民」的产物（`memory/` 存记忆与成长、`state/` 存状态长表、`traits/` 存大五分数），中间是「整座城与整次运行」（`logs/` 日志、`environment/` 环境时间线、`records/` 插件事件流），下半部分是「专项产物」（平行世界、研究、群体采访）。**一次普通仿真主要落在上半部分和中段**；专项产物要跑对应的实验才会出现。

按目录展开（括号里是实际存在的关键文件）：

目录	放什么	关键文件
`memory/`	每个居民的记忆条目、目标、关系、日程、成长进度、向量库	`agent_<id>.json`（记忆条目列表）、`agent_<id>_goals.json`、`agent_<id>_relationships.json`、`agent_<id>_schedule.json`、`agent_<id>_episodes.jsonl`、`agent_<id>_growth.json`、`growth_profiles.json`、`vector_db.sqlite`
`state/`	状态历史长表（每行一个 agent × 指标 × 值）	`agent_state_history.csv`（表头 `agent_id,step,metric,value`）、`agent_state_over_time.png`
`logs/`	运行日志	`run.log`（完整结构化）、`agent_<id>.log`
`records/`	Recorder 的插件事件流，每类插件一个 jsonl	`agent.step.jsonl`、`family.*.jsonl`、`home.observation.jsonl`、`big_five.seeded.jsonl`
`environment/`	环境事件时间线	`timeline.jsonl`
`traits/`	本次运行实际生效的大五人格分与来源	`agent_traits.csv`、`calibration_audit.csv`
`economy/`	账本、财富快照、宏观状态、部门池与守恒审计	`daily_ledger.csv`、`wealth_snapshot.csv`、`macro_state.json`、`sectors.json`、`conservation_audit.csv`、`agents/agent_<id>_ledger.csv`
`work/`	真实工作产物（默认开启）	`capabilities.json`、`queue.jsonl`、`market.jsonl`、

目录	放什么	关键文件
		`agent_<id>/<task_id>/`
`visualization/`	轨迹回放数据	`simulation_trace.json`、`latest_frame.json`、`runs/`
`network/`	社交网络图	`social_network.png`
`intervention/`	干预指标	`intervention_metrics.csv`
`diaries/`	每个居民的日记	`agent_<id>/day_005.md`
`run_manifests/`	每次运行的清单与可离线打开的 HTML 报告	`<时间戳>.json`、`<时间戳>.html`

专项产物：`research/`（研究方案与 `studies/` 实验结果）、`interviews/`（群体采访会话）、`parallel_worlds/`（平行世界，每世界一棵完整产物树）、`comparisons/`（事件对比实验，`compare-event` 的 `--output-root` 默认值；每次实验一个 `<时间戳>_<事件名>/` 目录，下面是 `without_event/` 与 `with_event/` 两棵树）。这几类要跑对应命令或面板操作才会出现，普通 `run` 不会生成。

如果当前选过城市，产物会落在城市自己的运行根目录下。 `python -m gaworld.city use 柳溪村` 之后，整棵树挪到 `output/cities/柳溪村/`，里面的目录名和上面**一一对应**（`memory/`、`logs/`、`state/`、`records/`、`environment/`、`traits/`、`economy/`、`work/`、`visualization/`、`network/`、`intervention/`、`diaries/`、`run_manifests/`）——这是有意设计的，不是巧合：Analytics 把 `state/` 和 `economy/` 接到运行根上，轨迹回放靠 glob `output/*/visualization` 找 trace，都依赖名字不变。所以查产物前先确认当前城市是哪一个。

查产物的常用命令：

```
# 某个居民的记忆（agent_<id> 换成实际存在的编号）
cat output/memory/agent_31.json | python -m json.tool

# 实时盯运行日志
tail -f output/logs/run.log

# 状态长表的前几行
head -3 output/state/agent_state_history.csv

# 环境时间线的最后几行
tail -5 output/environment/timeline.jsonl

# 这次运行到底花了多少次模型调用
python -c "import json,glob,os; f=max(glob.glob('output/run_manifes
        ts/*.json'), key=os.path.getmtime); d=json.load(open(f));
        print(d['llm'])"
```

同一份信息在控制台面板里也能看，而且是加工过的：下面这张是记忆面板，把一个居民的**日程**、**近期 episode**（带状态增量数值）、**Agent 日志**（Loc / Env / Act / Refl / EconomyStep / DailyDiary / EconomyDayEnd）并排放在一起，家庭结构、居家模式、采访入口都在左边。

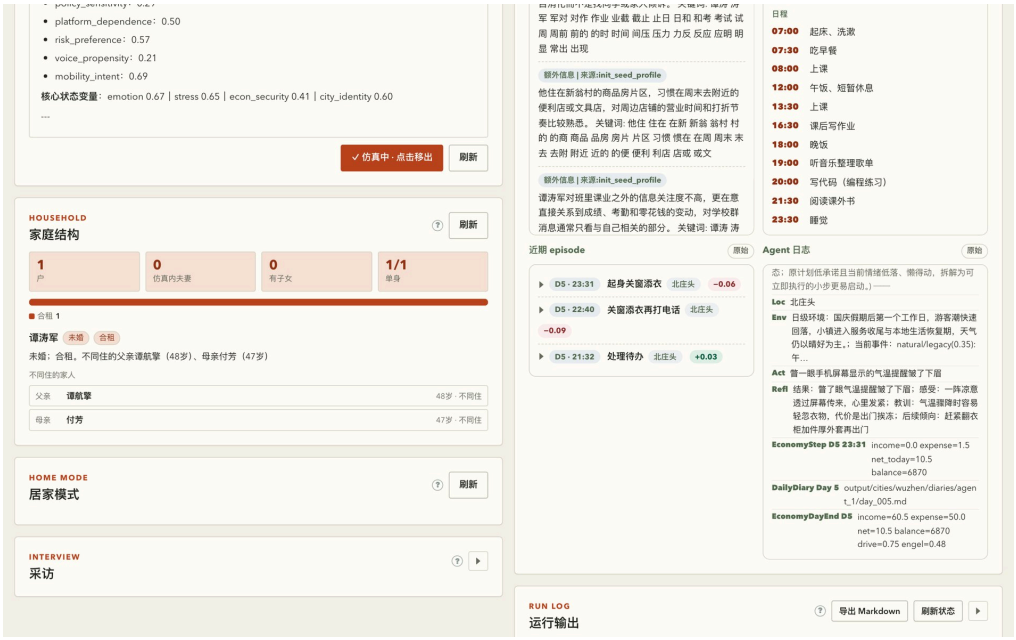


图 6-4　控制台记忆面板：把「一个居民的一天」拼在一起 左边是结构性信息（他住几口之家、和父母同住但不同籍），右边是过程性信息——日程表给出当天每一个时刻要做什么，近期 episode 给出带数值的状态增量，Agent 日志给出每一次行动的定位、周边环境、动作、反思和当天的收支。**这四条线是同一段经历的不同投影**：日程是计划，episode 是结果，日志是过程，家庭结构是约束。分析一个人的行为时，缺哪条线都可能得出相反的结论。

6.6 重置：改配置后为什么必须先 reset

```
python generative_city_sim.py reset
```

它**清**这些目录的内容（目录本身保留）：`memory/`、`logs/`、`state/`、`network/`、`environment/`、`diaries/`、`visualization/`、`intervention/`、`life_events/`、`records/`、`organizations/`，以及记忆向量库 `memory/vector_db.sqlite`。同时把仿真状态归零：`last_day = 0`、每个居民的日游标清空。

它**不清**这些：`economy/`、`work/`、`traits/`、`run_manifests/`、`research/`、`interviews/`、`parallel_worlds/`、`comparisons/`、以及 `output/cities/` 下的其它城市。

注意 `economy/` 不在清理列表里——重跑之后账本会接着上一轮长，不是从零开始。要一个干净的账本，得自己 `rm` 那个目录。

还有一点：`reset` 清的是**当前配置指向的那棵产物树**。切过城市之后，`reset` 清的是 `output/cities/<slug>/` 下面那棵，默认世界那棵原封不动。所以「换城市重跑」和「同一座城重跑」要用不同的 `reset` 次数。

那为什么改了配置要先 reset？三个理由：

1. **仿真会续跑，不会重跑。** 仿真状态记着 `last_day`，第二次 `run` 会从上次的天数继续，而不是从 Day 1 开始。想比较「改配置前」和「改配置后」，必须先归零。
2. **记忆 schema 变了会被直接拦住。** `require_clean_reset_on_memory_model_change` 默认开启，一旦记忆模型的版本号和上次跑的不一致，`run` 会报错停下：

   ```
   Memory model version changed. Please run `python generative_city_sim.py reset` once, then rerun simulation.
   ```

3. **旧记忆会污染新起点。** 居民带着上一轮的日记、关系、成长进度进入新运行，你看到的「Day 1」已经不是种子状态了。

控制台上的「重置并运行」按钮做的是同一件事。

6.7 复现性：种子、清单与版本

种子在哪配。 `CONFIG["random_seed"]`，代码默认值是 `None`（即不固定）。实际有两种配法，优先级从低到高：在 `dashboard_config.json` 里写一个顶层 `"random_seed": 42`；或者用环境变量 `GAWORLD_CONFIG_OVERRIDES='{"random_seed": 42}'`——后者有最终决定权，不受 `dashboard_config.json` 和城市包影响。

没配会怎样。 仿真器会自己从系统熵里抽一个，打印出来并写进运行清单：

```
🎲 random_seed 未设置，本次自动取 1748392051（已写入运行清单，可用它复现）
```

种子管什么。 它同时喂给 `random.seed()`、`np.random.seed()`，以及内核给每个插件派生的独立随机流（`gaworld/kernel/context.py` 用 `random.Random(seed)` 建流）。所以它固定的是**随机流本身**——哪天爆发、抖动多少、抽签结果是什么。

但要诚实地说清楚：**种子不控制 LLM 的返回文本。**同一个 prompt 调两次，模型可能给出不同措辞。所以「同样的种子 + 同样的配置」得到的是**同样的随机结构**，不是逐字相同的对话记录。做统计比较没问题，做文本 diff 不行。另外 `gaworld/core/runner.py` 明确提醒：并发度 `max_workers > 1` 时全局 `random` 状态会被以非确定顺序消耗，默认 `max_workers = 1` 才保持确定性。

每次运行都会留下一份清单：`output/run_manifests/<时间戳>.json`（旁边还有一份可离线打开的 `.html`）。里面记着这次运行的 `sim_days`、参与运行的 `agent_ids`、`random_seed` 及其来源（`seed_source` 是 `"config"` 还是 `"auto"`）、`stateful` 标志、LLM 调用统计（`llm.call_count`、`failure_count`、`avg_latency_ms`、按任务和按供应商的细分），以及一份配置快照。**清单是复现的起点**：想知道上次到底跑了什么，先看清单，不要靠记忆。

改了默认配置，就该追加一个 Epoch。 `gaworld/core/comparability.py` 维护了一条版本线：每一条 `Epoch` 记一个编号、落地日期、改了什么、影响了哪些量（当前是 Epoch 11）。判据写得很清楚——**当一次改动改变了「默认配置下」一次运行会产生的结果，并且让更早的运行不再可比时**，就追加一条；只影响默认关闭的可选开关（出行、内生拥堵之类）则不用，因为运行清单里的配置快照已经能区分它们。规则是**只增不重编号**，永远不要改旧条目。

这条机制有牙齿：平行世界的复现汇总会跳过别的事件本的种子；研究评估器不会用跨版本的种子宣布一个假设「被支持」或「被推翻」；GAWorld-Bench 的 Track C 也不会给跨版本比较发信任门。**换句话说，跨版本的数据不会被悄悄合并。** 作为实践者，你的责任是：改完默认配置，要么别和旧运行做比较，要么把基线在新版本上重跑一遍。

6.8 成本与时间预期

先把账算清楚。记 `N` = 参与的居民数，`D` = 仿真天数，`T` = 一天里的时刻点数。总调用量：

- **日内逐步**：`N × T × D`。`T` 是**所有居民日程时刻的并集**（`build_master_timeline` 取的是全体日程写出的时刻，不是单个居民的），所以人口越多越贵。默认 `time_step_minutes` 留空，一天十几个到几十个时刻很常见。
- **按天快进**：`N × D`。
- **按月 / 按年**：`N × 步数`，步数由 `plan_horizon` 按真实日历切出来。

以 50 个居民为例（下表的「按月」「按年」步数是按真实日历切出来的实数）：

场景	日内逐步	按天快进	按月	按年
5 天	`50 × T × 5`	250	50	50
60 天	`50 × T × 60`	3,000	100	50
600 天	`50 × T × 600`	30,000	1,000	100
3653 天（10 年）	`50 × T × 3653`	182,650	6,000	500

「日内逐步」那一列之所以写不出具体数字，是因为 `T` 取决于人口和各自的日程，不是常数。**要知道自己真实花多少次，用运行清单量**：跑完看一眼 `output/run_manifests/<时间戳>.json` 的 `llm` 段。三个数字要分清——`calls_in_run` 是**本次运行**发出的调用，`call_count` 和 `process_call_count` 是**整个进程**的累计（从控制台或测试套件里起的运行，进程早先可能已经调过模型）。命令行直接跑时两者相等。

这里有一个很容易踩的坑：**`--sim-years 10 --time-unit day` 不是「按年跑十年」，是「跑十年、一天一步」。** 它的跨度是十年（3653 天），步长是天，于是每个居民要发 3653 次简报——比 `--time-unit year` 贵 365 倍。`--sim-years` 管跨度，`--time-unit` 管步长，两个参数各管各的，别指望它们联动。

时间也用同一把尺子量：总时长 ≈ 总调用数 × 单次调用耗时，而单次调用耗时在不同供应商之间差一个数量级，仓库里也不该写死一个数。跑完看一眼清单里的 `llm.avg_latency_ms`，用它乘以调用数，比拍脑袋靠谱。

推荐的放大路径：先用 5 天验证配置，再放大到 600 天。

1. **`--sim-days 3` 跑一次**，检查：终端有没有报错、四组参数是不是你以为的、产物目录是不是落对了地方（默认世界还是 `output/cities/<slug>/`）、`output/memory/agent_<id>.json` 里有没有东西。这一步花掉的调用量可以忽略，但它能挡掉绝大多数「跑了两小时才发现模型 key 错了」的浪费。
2. **把粒度切到你要问的问题对应的档位**。想看长期演化就切到 `--fast-forward`；想看十年尺度就直接 `--sim-years 10`（500 次调用，比按天少 365 倍）。**不要用日内模式硬扛长跨度**。
3. **放大时先确认 `agent_ids`**。50 人和 200 人差 4 倍调用量，而默认只跑配置里那几个居民。想省钱的第一个杠杆是人数，不是天数。
4. **放大前想清楚可复现性**：种子配好没有？跑完的清单存下来了没有？改过默认配置的话，基线是不是也要重跑？

本章小结

- 最小可运行是 `python generative_city_sim.py run`；跑完先看 `output/logs/run.log`，再上控制台面板看加工过的结果。
- 生效配置是四层叠的：代码默认值 → `dashboard_config.json` → 城市包 → `GAWORLD_CONFIG_OVERRIDES`；要纯净默认值就设 `GAWORLD_IGNORE_LOCAL_CONFIG=1`。
- 时间粒度是全书最重要的工程决策：日内逐步 / 按天 / 按月 / 按年，50 人跑十年从 500 次到 182,650 次调用；**换粒度是换问题，不是换精度**。
- 粗粒度下底层日历仍逐日推进，日边界钩子按 ≤30 天区块补跑，所以跑满一年会扣满一年房租、走满 12 次月度结算。
- `long_run.randomness` 管「世界发生多少意外」，`routine_change.randomness` 管「居民多常不按日程过」；两个滑杆都在控制台工具栏上。
- 每次运行都在 `output/run_manifests/` 留一份清单，记着种子、配置快照、LLM 调用数和可比性 Epoch——复现从清单开始。

动手练习

1. **跑一次三天仿真，把产物认全**（约 20 分钟）

```
python generative_city_sim.py reset
python generative_city_sim.py run --sim-days 3
ls output/
ls output/memory/ | head
ls output/state/ output/records/ output/environment/
```

验收标准：在 `output/` 下面找到**至少三类**产物并能说清它们分别是什么——例如 `state/agent_state_history.csv` 是状态长表（每行一个 agent × 指标 × 值，表头是 `agent_id,step,metric,value`）、`records/*.jsonl` 是插件事件流、`environment/timeline.jsonl` 是环境时间线、`memory/agent_<id>.json` 是单个居民的记忆。再打开 `output/run_manifests/` 里最新那个 `.json`，读到 `run.sim_days` 是 3、`llm.calls_in_run` 是这次实际的调用次数、`run.random_seed` 和 `run.seed_source` 是什么。

> 如果你之前用 `python -m gaworld.city use <slug>` 切过城市，上面这些路径要换成 `output/cities/<slug>/` 下的同名目录。

2. **比较默认模式与快进模式各跑三天**（约 30 分钟）

```
python generative_city_sim.py reset
python generative_city_sim.py run --sim-days 3
cp "output/run_manifests/$(ls -t output/run_manifests | grep -v
        partial | head -1)" /tmp/ff_default.json

python generative_city_sim.py reset
python generative_city_sim.py run --sim-days 3 --fast-forward
cp "output/run_manifests/$(ls -t output/run_manifests | grep -v
        partial | head -1)" /tmp/ff_fast.json

python - <<'PY'
import json
for name in ("/tmp/ff_default.json", "/tmp/ff_fast.json"):
    d = json.load(open(name))
    print(name, "→", d["llm"].get("calls_in_run"), "次调用")
PY
```

验收标准：快进那次清单里的 `llm.calls_in_run` **明显小于**默认那次（默认模式每个居民每个时刻都要调一次模型，快进模式每居民每天只有一次），你能说出倍数大致是多少；并且能指出两处产物差异——快进的 `output/state/agent_state_history.csv` 行数更少（状态推进更粗），`output/records/` 里逐时刻的 `agent.step.jsonl` 记录密度更低。

ewpage

第 7 章　仿真控制台图解：界面就是一份实验配置

前六章你已经能用命令行把一套仿真跑起来了。但研究一旦进入「改一个旋钮、立刻看看差别」的阶段，命令行就开始碍事：改一次配置要动文件、重启进程、重新等它跑完，来回一趟十几分钟。

GAWorld 自带一个本地控制台：

```
python generative_city_sim.py dashboard --port 8766
# 浏览器打开 http://127.0.0.1:8766/dashboard
```

本章不按「按钮说明书」的方式讲它，而是先建立一个判断力：

> 控制台不是一个「网页版点选器」，而是一份实验配置的表单。

工具栏上那四组字段——城市与居民、时间、模型、运行方式——合起来就是一次运行的**完整条件**。你在表单里改的每一个值，最后都变成运行时配置里的一个键。把这句当成使用手册的总纲，后面所有细节就都有地方放了。

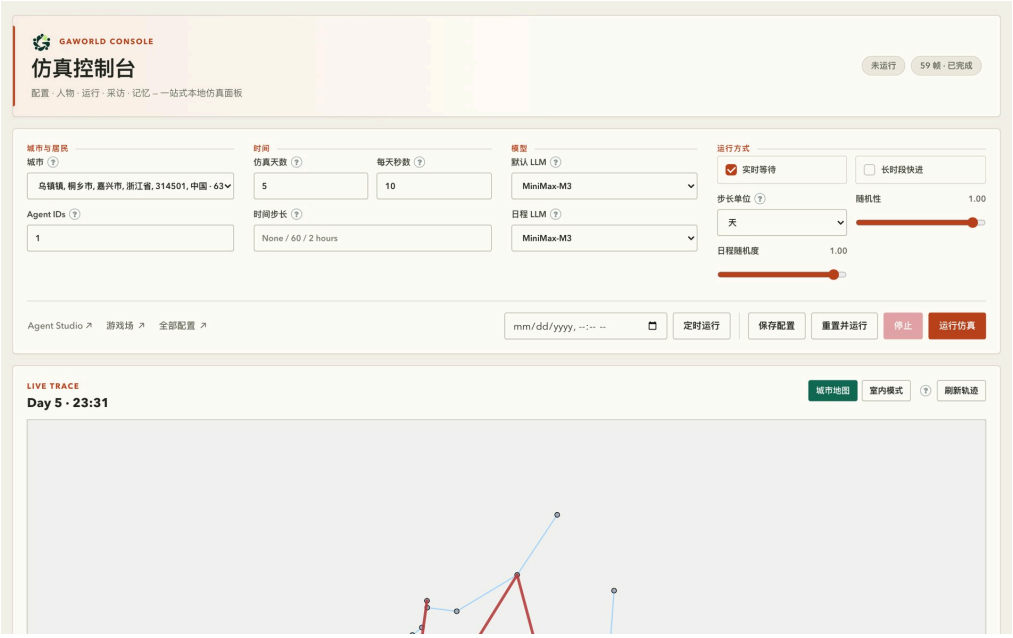


图 7–1　控制台全貌 一屏从上到下分三层：最上面是标题与状态徽标（未运行 / 59 帧 · 已完成），紧跟着是横向的工具栏——它就是本章要拆的实验配置表单；再下面是工作区，左边是占了大半屏的实时轨迹地图，右边一列是人物、家庭、居家、记忆等面板。工具栏是唯一改变运行条件的地方，下面的面板只负责「看」。

7.1 工具栏：四组字段就是四个实验条件



图 7–2　工具栏的四组字段 四个带边框的分组从左到右是「城市与居民」「时间」「模型」「运行方式」，每个字段名后面跟一个问号圆点，鼠标悬停会给出该字段的完整解释。底部一行是执行按钮：定时运行的时刻输入框与「定时运行」，以及

「保存配置」「重置并运行」「停止」「运行仿真」。左下角还有三个跳转链接——Agent Studio、游戏场、全部配置。

这一组字段不是「显示当前设置」用的，它是一个**可编辑的表单**：你改值 → 点「运行仿真」→ 表单内容整体变成一次运行的配置。

7.1.1 城市与居民：决定「在哪儿、跟谁」

`#citySelect` 城市下拉列出所有可用的城市，其中「默认世界」是 `data/` 目录下的原始杭州数据。选中哪座城市，就决定了这次仿真里的地图、环境事件、背景设定和居民名册——这四件事是一起来的，不是四个独立的开关。

选好城市，「Agent IDs」输入框（`#agentIdsInput`）的占位符会立刻变成 `1-<这座城的人数>`，提示你能填的编号范围。填法有两条规则：

- 填逗号分隔的列表，如 `1,2,3`，表示精确指定这几位居民；
- 只填一个数字 N，表示取**前 N 位**居民。

编号是**每座城市各自从 1 开始**的。切到一座人更少的城市后，原来那串编号可能指向不存在的居民。服务端在启动前会挡下来，提示「城市「X」只有 N 位居民，但 Agent IDs 里有 […]」。这个检查故意放在启动这一步：放晚了，pandas 会在几分钟后抛一个没人看得懂的 IndexError，而那时你已经不再盯着日志了。

这个输入框是「本次谁参与」的唯一真相来源。人物面板里那份下拉列表会实时跟着它走：在名单里的居民前面带一个 ▶ 标记，名字后面缀上「(仿真中)」。你在下拉里换人，或者在名单里点头像增删成员，改的都是这一个输入框。

7.1.2 时间：决定「跑多久、跑多细」

`#simDaysInput` 仿真天数——但这个字段的含义会跟着「步长单位」变：单位是天时它叫「仿真天数」，切成月时变成「仿真月数」，切成年时变成「仿真年数」。换单位时数字会按 1 天 = 1 天、1 月 ≈ 30 天、1 年 ≈ 365 天粗略换算（30 天切成「月」会变成 1），换算只是一个起点，你随后改成什么就是什么；精确的折算在服务端做，按真实日历算闰年和 28/31 天的月份。

`#secondsPerDayInput` 每天秒数：一个仿真日折算成多少真实秒。数值越大，日内推进越细、回放越慢。这里有个容易误会的地方——**只有勾上「实时等待」时它才真的让你等**；不勾的时候，仿真会跑到 CPU 和模型能跑多快就跑多快，这个值只影响回放动画的节奏。

`#timeStepInput` 时间步长：日内推进的时间粒度。留空表示「只在日程写明的时刻推进」；填 `60` 就是每小时一步，填 `2 hours` 也认。步长越小，一天要跑的次数越多，LLM 调用量跟着涨。

7.1.3 模型：两个下拉，两份账单

`#defaultProviderSelect` 默认 LLM 是智能体思考、行动、采访时默认调用的提供方。`#scheduleProviderSelect` 日程 LLM 单独指定**生成每日日程**用的模型。

分成两个下拉不是多余的。日程是每天每人一份，量大但内容模式化；思考和行动是真正需要判断力的部分，量小但每次都贵。把日程指向一个便宜模型、默认 LLM 指向强模型，是控制台能给你的最直接的一笔省钱。日程下拉留空时会自动回落到默认 LLM。

7.1.4 运行方式：决定「以什么节奏、多粗的粒度」

这一组最容易被误解，因为它管的是仿真的**时间粒度**，而不只是快慢。

`#realtimeInput` 实时等待：勾上后仿真按真实时间走，像看直播。做实验一般不勾。

`#fastForwardInput` 长时段快进：勾上后，一个时间步被压缩成一条简报——每个智能体每步一次 LLM 调用——日内那圈时刻循环被整个跳过，状态、目标、关系仍然近似推进。代价是日内细节全部消失。按天这个单位适合 60 天、600 天这种尺度。

`#stepUnitSelect` 步长单位：快进时「一步」代表多长时间，三个选项是天 / 月 / 年。这里有两条联动规则，界面上会替你做掉：

- 选「月」或「年」会自动勾上「长时段快进」——没有「按月的日内时刻循环」这种东西，不自动打开就等于闷头按天跑完整个跨度；
- 反过来取消「长时段快进」，步长单位会被拉回「天」。

跑数年以上必须用月或年，否则调用量按天数线性增长——10 年 × 50 人，按年是 500 次调用，按天是 18 万次。

`#randomnessInput` 随机性（0–1 滑杆）：快进期间的随机程度。越高，突发事件越频繁、智能体状态波动越大；0 表示完全确定，同种子必然复现。只在长时段快进时生效。

`#routineRandomnessInput` 日程随机度（0–1 滑杆）：居民有多大概率临时偏离既定日程。越高，高承诺的活动越容易被放弃、无端的躁动更多；0 是严格照日程执行（睡眠时段不受影响）。这个旋钮和上一个不是一回事：随机性改的是「世界发生什么」，日程随机度改的是「人对既定安排的反应」。

7.1.5 底部五个按钮：五个不同的语义

保存配置：把当前表单写进 `dashboard_config.json`（仓库根目录；多用户部署下写的是那个世界自己的配置文件），不启动任何东西。改完参数先存一份、关掉页面明天再跑，用的是它。

运行仿真：按当前表单启动一次运行。注意它**同时也会把表单写进配置文件**——服务端在启动前会把随请求带来的配置落盘。所以「保存配置」和「运行仿真」的区别不是「一个存一个不存」，而是「一个只存、一个存完就跑」。

重置并运行：先清状态再跑。服务端会先执行一次 `generative_city_sim.py reset`（清缓存与日志、天数归 1），确认成功后再跑 `run`。想两次运行从同一起点开始对比，用它，不要用「运行仿真」——后者会接着上次的记忆和天数往下跑。

停止：中断当前运行。仿真进程先收到 SIGTERM，8 秒不退再被 SIGKILL；已经写到磁盘的产物不动。如果有个不是这个控制台启动的仿真进程在写同一个世界，「停止」也会去终止它。

定时运行：在 `#scheduleAtInput` 里选一个时刻，到点自动启动。时间按**运行面板所在机器的本地时间**计算，定时器活在服务器进程里，页面关掉也不影响。设完会出现一个「取消定时」按钮。适合挂一晚长时段快进，第二天早上回来看结果。

运行期间工具栏上的输入框会全部变灰（城市下拉除外），避免跑到一半改了参数。运行状态有两个徽标：一个报仿真进程状态，一个报轨迹帧数。

7.2 工作区：地图与三块「这个人」的画像

7.2.1 城市地图（Live Trace）

地图面板是工作区的主体，标题栏写着当前帧（第几天几点），右上角两个模式按钮在「城市地图」和「室内模式」之间切换，「刷新轨迹」强制重读一次轨迹文件。

下面那条时间轴可以拖动回看任意一帧，勾上「跟随最新」就自动追到最新帧。地图下方还有一个默认收起的「当前帧数据」折叠块，展开是当前时刻的原始 JSON：这一天、第几点、每个人的位置、状态和当前活动都在里面。写代码的人靠它对字段，纯粹好奇的人展开看一眼就够。

7.2.2 人物面板

面板顶上是头像和那个居民下拉框，下面是「参与本次运行」的名单（来自 Agent IDs 输入框，点头像即可切换选中的人），再下面是只读的人物设定渲染——身份、状态变量、叙事档案都在这里。要改人物设定，控制台不提供入口，去工具栏的「Agent Studio ↗」。

面板底部两个按钮：「加入仿真」把当前选中的居民加进/移出本次名单（改的还是 Agent IDs 输入框），「刷新」重新拉一次。

7.2.3 家庭面板与居家面板

「家庭结构」面板给的是本次运行里的住户关系：户数、仿真内夫妻数、有子女数、单身数，以及每个居民的婚姻状态和同住/不同住的家人。婚姻状态是按年龄段抽样生成的，同住的家人共享一个住处、共担开销、互相影响情绪。这个面板要完整跑过一轮仿真才有内容。

「居家模式」面板给的是每位居民的居家环境：户型、家具、整体气质，以及他每次在家时所在的房间和当时的氛围（光线、声音、气味、体感、整洁度）。点地图上头像旁的 🏠 标记，或者在人物面板里选中某个人，这里会定位到对应记录。

7.2.4 可折叠的面板

页面下方几块面板默认是收起的——「采访」「智能体互动」「人生事件」「FOS AI Scientist Export」「运行输出」都是点标题展开。只有「记忆与习惯」是默认展开的，因为它最常用。

「记忆与习惯」展开后是四张卡片：长期记忆、日程/习惯/意图、近期 episode、Agent 日志。每张卡片右上角有一个「原始」按钮，点开是未经加工的原文，可以复制或下载：

- episode 是 JSONL，取尾部若干行，文件名叫 `agent_<id>_episodes.jsonl`；
- Agent 日志是纯文本，文件名叫 `agent_<id>.log`。

「原始数据」按钮则把上面这些合在一个弹窗里，用标签页切换。

「运行输出」面板显示后台仿真进程的实时日志，这是排查问题最直接的窗口；「导出 Markdown」能把这段日志存成一份文件，直接贴进实验记录或发给同学。

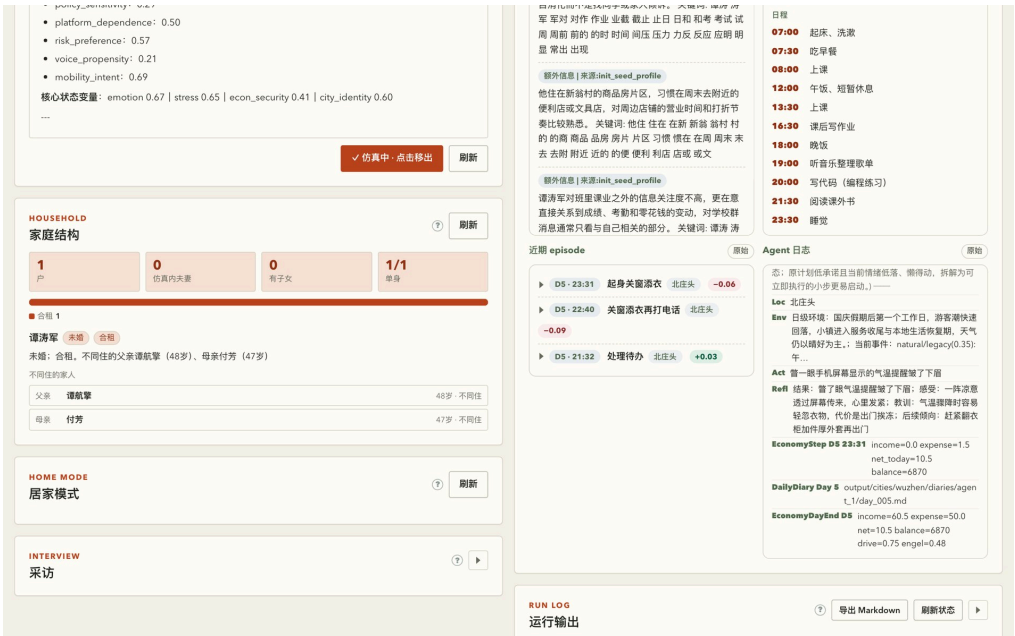


图 7–3　展开后的工作区面板 左列是人物的档案与家庭结构（户数、婚姻状态、不同住的父母），再往下是居家模式；右列是记忆面板的几张卡片——日程表按时间列出 07:00 起床、洗漱一直到 23:30 睡觉，episode 卡片按 D5 的时刻倒序列出最近发生的事，Agent 日志卡片则是一行行原始记录（状态、位置、环境、行动、反思、经济结算、每日期末）。同一屏里既有「结构化摘要」，也有「原始证据」——做质性分析时这两个都要看。

7.3 选中一位居民：用一屏做质性观察

控制台最被低估的用法，是把它当成一个**单人的观察台**。

在人物面板的下拉里换一个人，右侧的人物设定、家庭结构、居家模式三块会同步刷新，记忆面板的四张卡片也换成这个人的，地图上的镜头跟着聚焦到他——他在哪儿、他的动线、他在这一帧正在干什么。地图上还会标出他的家。

这一屏能回答的问题是：这个人今天在哪儿待得最久？他有没有按自己的日程走（对照日程卡片和地图动线）？这一天里发生了什么值得记住的事（episode 卡片）？他的心情和钱怎么变的（人物设定的状态变量，加上 Agent 日志里的 `EconomyStep` / `EconomyDayEnd` 行）？家里的关系有没有影响他的行为（家庭结构面板配上相关的 episode）？

做法很朴素：选人 → 拖时间轴看动线 → 展开记忆面板读日程和 episode → 必要时用「采访」面板问他一个问题（模型以他的人物设定与记忆为背景作答）。这一套是定性分析的入口。

7.4 其它入口：别名路由与轨迹回放

控制台不是唯一的一扇门。服务端把几组路径指向了不同的页面（`STATIC_ROUTES`）：

路径	指向	说明
`/`	`site/index.html`	项目落地页，所有视图的入口
`/console`	`site/console/index.html`	控制台外壳：左侧导航把控制台、仿真回放、城市、分析、Agent Studio 等视图串成一套
`/dashboard`	`site/dashboard/index.html`	本章讲的仿真控制台
`/board`、`/todo`	`docs/todo_board.html`	项目的待办看板，**不是**仿真面板

换句话说，`/dashboard` 是「仿真控制台」这一页，`/console` 是装了一堆页面的外壳，`/board` 和 `/todo` 是完全不相干的看板。别把 `/board` 当成「另一个控制台」。

轨迹回放页是另一件事，地址是 `/site/simviz/index.html`，从 `/console` 外壳左侧的「仿真回放」标签进去，或从落地页的「查看仿真回放 ↗」链接进去。

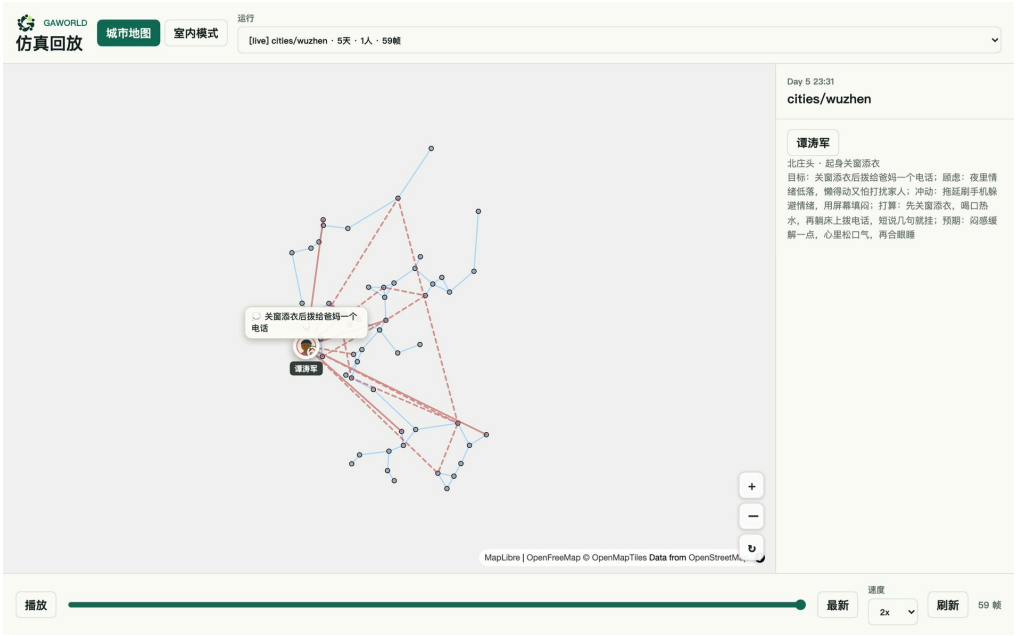


图 7–4　轨迹回放页 顶部是「运行」下拉，选的是哪一次运行：当前还在写的那次标成 `[live]`，历史运行被自动归档，对比实验的运行来自各自的输出目录。底部是播放、拖动时间轴、「最新」、速度和刷新。右侧检视面板给出当前帧里选中的那个人在做什么、想要什么、顾虑什么、打算怎么应对——心理解释和地图上的位置是同步的。

回放页擅长的是**逐帧重看**：跑完的仿真、想看清「那个转折点到底发生在哪一刻」的时候，用它比控制台的地图更顺手。轨迹数据本身在磁盘上是 `output/visualization/simulation_trace.json`，控制台服务器会静态托管这个目录；控制台里的「当前帧数据」折叠块则给你当前那一帧的 JSON。

7.5 面板显示空？先查这三条

面板空白几乎总是下面三件事之一（更细的排查见第 19 章）：

1. **还没跑过仿真。** 家庭、居家、记忆、运行输出这些面板读的是运行产物（`output/` 下的记忆、日志、居家记录）。一次都没跑过，它们就是空的——这不是 bug。
2. **城市没切。** 切城市会同时换掉居民名册。控制台还显示着上一座城的居民，而你正在看新城市的编号，或者反过来。选中的那座城市是准绳。
3. **产物目录是空的。** 城市自己的运行根目录（`output/cities/<slug>/`）里没有对应文件。跑一次就都会有。

7.6 什么时候该放下控制台

一条实用建议：面板适合调参与观察，批量与可复现的实验走命令行。

原因很实际。控制台的表单状态存在 `dashboard_config.json` 里，那是一份「当前设置」，不是一份「这次运行的参数记录」；跑完一次，你很难从文件里看出当时到底传了什么。而命令行天然把参数留在历史里：

```
python generative_city_sim.py run --sim-days 600 --fast-forward
python generative_city_sim.py run --sim-years 10
python generative_city_sim.py run --sim-months 24 --time-unit month
```

这几条命令就是完整的研究条件——跨度、粒度、模式，全在一条可复制的字符串里。要批量跑多个种子、多个参数组合，要让别人复现你的实验，这种可读性比界面方便重要得多。

参与名单是唯一的例外：CLI 的 `run` 子命令没有 `--agent-ids` 参数，名单走配置。要在命令行里临时换名单，用环境变量 `GAWORLD_CONFIG_OVERRIDES`——它是一段 JSON 覆盖块，在配置叠加里**最后生效**，优先级高于 `dashboard_config.json`：

```
GAWORLD_CONFIG_OVERRIDES='{"agent_ids":[1,2,3],"sim_days":10}' \
  python generative_city_sim.py run
```

配置的实际叠加顺序是：代码里的默认值 → `dashboard_config.json` → `GAWORLD_CONFIG_OVERRIDES` → 环境配置文件 → 城市包 → `GAWORLD_CONFIG_OVERRIDES`（最后一遍，最后的话）。所以「在控制台里改了参数，命令行一跑又变回老样子」这种困惑，通常就是被 `GAWORLD_CONFIG_OVERRIDES` 覆盖掉了。

本章小结

- 控制台不是网页版点选器，是一份实验配置的表单：工具栏四组字段就是一次运行的完整条件。
- 「城市与居民」定范围（地图、环境、名册，编号每座城市各自从 1 开始），「时间」定跨度与粒度，「模型」把日程和思考分给两个可能不同的提供方，「运行方式」决定快进与步长单位。
- 选月/年会自动打开快进，取消快进会把单位拉回天；这两个下拉是联动的。
- 五个按钮语义不同：保存配置只落盘，运行仿真落盘并启动，重置并运行先清状态再跑，停止中断，定时运行到点自动启动且不依赖页面开着。
- 人物 / 家庭 / 居家三块面板加上记忆卡片，构成一个单人的质性观察台；选中一位居民后它们会同步刷新。
- 面板空白最常见的三条原因是：没跑过仿真、城市没切、产物目录为空。
- 面板用来调参与观察；批量与可复现的实验走 CLI，因为命令会把研究条件留在命令历史里。

动手练习

1. **只动一个旋钮，跑两次对比行为**（约 30 分钟）

   在控制台里选一座你用过的城市、选 2–3 位居民，把「日程随机度」先设成 `0.00`，跑 3 天（其余参数保持不动），记下这几位的行为；再把「日程随机度」调到 `0.80`，**只改这一个旋钮**，其它一律不动，再跑 3 天。

   验收标准：

   - 两次运行用的城市、居民名单、仿真天数、每天秒数、模型选择完全一致——可以对照两次的 `dashboard_config.json`，除 `routine_change.randomness` 外没有别的差异；
   - 第二次运行里，被选中的居民在「记忆与习惯」的**近期 episode** 卡片中出现了「临时改计划」类的条目（不再严格照 07:00 起床 → 23:30 睡觉的日程走），第一次没有；
   - 两份 episode 卡片各导出或截图一份，能指着具体条目说清差别出在哪儿。

2. **把日程 LLM 指向另一个模型，算清楚省在哪**（约 20 分钟）

   打开工具栏的「模型」分组，把「默认 LLM」和「日程 LLM」指向两个不同的提供方（例如一个贵、一个便宜），其余参数不动，跑一次 3 天的仿真。

   验收标准：

   - 「模型」分组里两个下拉的值确实不同（截图留证）；
   - 运行结束后打开「运行输出」面板，找到本次运行的 LLM 调用统计，说明**日程那部分调用走的是便宜模型**——用一句话讲清「便宜模型承担了哪

一部分工作」（提示：日程是每人每天一份、量大且内容模式化；思考与行动是量小但每次都贵）；
    - 明确回答两件事：省下来的是哪一部分成本、省下多少取决于什么（居民人数 × 仿真天数），而**没有**省下的是哪一部分。

ewpage

第 8 章　Agent Studio：把一个居民拆开看

前面几章讲的都是一座城市怎么跑起来：几百个居民、十几个插件、每天几十次模型调用。但你真正想搞清楚的往往不是这座城市，而是**这一个人**——他今天为什么没去上班、为什么在群里不说话、为什么跟邻居越走越远。Agent Studio 就是干这件事的面板：它把一个居民拆成七块，让你一块一块地看，一块一块地改。

8.1 一个面板，两种身份

Agent Studio 有两副面孔，这两副面孔是同一套代码、同一批接口。

当查看器，它把 GAWorld 内部那些平时看不见的东西摊开：九个归一化状态变量的真实数值、大五人格的 z 分、这个人记得什么（情景记忆、长期总结、习惯、日程）、他和谁熟（Dunbar 分层）、脑子里装着哪些技能、算出来的认知指数是多少。这些数字平时散落在 CSV、Markdown 和 `output/` 里的 JSON 中，Studio 把它们聚到一屏。

当编辑器，同一批字段大多可以直接改，改完写回真实的种子文件，**下一次运行**就按新的设定开始。改完不用重写配置文件，也不用去编辑 CSV——但改的东西真的落盘、真的生效。

打开地址是 `http://127.0.0.1:8766/site/dashboard/studio.html`（从控制台工具栏的「Agent Studio ↗」进也一样）。页面结构由三块组成：

- **顶部动作条**：返回控制台、＋新建、＋批量导入、＋从真人蒸馏、保存、运行仿真。
- **七步导航条**（`site/dashboard/studio.html` 的 `step-rail`）：① 身份 → ② 状态·性格 → ③ 能力·技能 → ④ 记忆 → ⑤ 社交·关系 → ⑥ 行为·目标 → ⑦ 复核·部署。
- **左栏「被观察者」**：城市选择器、居民选择器、头像、姓名与摘要、一张小雷达图、存档状态提示。右栏是当前步骤的正文。

页脚那一行小字其实是面板的自我说明：`Model GAWorld Sim` / `State 9 vars · [0,1]` / `Seed data/hangzhou_agents_state_init.csv`。最后一项就是当前的种子来源——面板不是对着一个抽象的模型对象编辑，而是对着种子文件编辑。



图 8–1　Agent Studio 面板实拍（身份步）

左边竖着七步导航、当前亮在第 1 步；中间上方是姓名、性别、年龄、户籍、居住地五个输入框，下方是这一位居民的「叙事 Profile」Markdown 原文；右侧的九边形雷达是九个状态变量，右上角那条绿色多边形就是「谭涛军」此刻的初始状态形状。左栏的居民下拉里目前只有 ID 1——每座城市的智能体编号各自从 1 开始，这是后面 8.10 节要展开的坑。

8.2 第 1 步 · 身份

身份步有两张卡。

基础信息五个字段：姓名、性别、年龄（输入框限定 16–90）、户籍、居住地。它们和九个状态变量在**同一行 CSV** 里（`data/hangzhou_agents_state_init.csv`），表头长这样：

```
id,name,gender,age,hukou,residence,emotion,stress,econ_security,cit
y_identity,policy_sensitivity,platform_dependence,risk_preference,v
oice_propensity,mobility_intent
```

叙事 Profile 是一段 Markdown，默认是渲染后的样子；点「编辑」切到源码，点「确认修改」才写回 profile 文件的 `## Profile NN｜姓名` 那一块。这段文字是模型每次生成日程、决策、对话时的背景输入——它比状态变量更"软"，也更接近"这个人是个什么样的人"。

新建居民时，身份步还会多出两个字段：职业和人格描述，作为生成 profile 与初始状态的种子。

8.3 第 2 步 · 状态 · 性格

这一步是整个面板的核心：左边九根滑块，右边一张可编辑的雷达图，雷达图随滑块实时变形。底部还有一张独立的大五人格卡片（下面单独讲）。

九个状态变量

九个变量都是归一化到 `[0,1]` 的浮点数，名字就是 CSV 的列名。每一根滑块两头各有一个文字标签，那是这个变量的**两极**，不是"越多越好"：

变量（CSV 列名）	中文名	低极	高极
`emotion`	情绪	消极	积极
`stress`	压力	无压力	极高
`econ_security`	经济安全感	不安全	安全
`city_identity`	城市认同	疏离	认同
`policy_sensitivity`	政策敏感度	迟钝	敏感
`platform_dependence`	平台依赖	多元	绑定
`risk_preference`	风险偏好	厌恶	偏好
`voice_propensity`	公共表达	沉默	主动
`mobility_intent`	流动意愿	定居	流动

有两件事必须想清楚，否则很容易用错这个面板。

第一，这是初始条件，不是运行结果。 状态 CSV 在仿真启动时被一次性读进来（`generative_city_sim.py` 里的 `pd.read_csv(CSV_PATH)` 之后 `build_agent` 逐个构造 Agent），从那一刻起，Agent 就在内存里带着这份起点演化，`output/state/` 里的状态历史长表记录的是它一天天怎么变。所以你在 Studio 里把 `stress` 从 0.3 拉到 0.8，**不会**让正在跑的那次运行里的他焦虑起来——它改的是**下一次**运行的起点。这两件事请务必分开，8.9 节会再讲一遍。

第二，数值本身是"倾向"，不是"程度"。 `voice_propensity` = 0.9 说的是"这个人在公共表达上偏主动"，`emotion` = 0.2 说的是"他这段时间情绪偏消极"。它们被读成什么，取决于下游插件怎么用；你在雷达上看到的形状，是这个人的**性格底色**，不是他的**人生进度条**。

雷达图本身也是可编辑的——拖滑块，多边形实时跟着变。拖完之后底部提示条会从「与已保存一致」变成「未确认」，必须再点一次「确认修改」才会落盘。这个两步确认是刻意的：滑块一动就写盘的话，手滑一下就毁掉一条种子数据。

大五人格卡片

状态步下半部分是一张独立的「大五人格」卡片，画的是 Openness / Conscientiousness / Extraversion / Agreeableness / Neuroticism 五个维度的 **z 分**（不是 `[0,1]`）。滑块以 0 为中心，范围 ±2.5，步长 0.05；两端写着这一维的两极描述。`0` 表示人群平均，`|z| ≥ 0.5` 才算这个人身上真正突出的一维。

分数从哪来？两条路：

1. **优先读 `data/agents_big5.csv`**（`gaworld/personality/plugin.py` 的 `load_profiles`）。这张表有 `id,name,o,c,e,a,n,source,unstated,redundant` 若干列，`source` 记着这行分数的来历（`calibrated` 离线标定 / `sampled_authored` 采样后又据此写了段落）。
2. **这个人不在表里，就按人口先验采样**（`residents.py` 的 `_prior_big5_row` 调 `gaworld.personality.plugin.prior_traits`）。先验带相关矩阵（用 Cholesky 分解开相关），每个居民一条独立的随机流（同一颗种子 + 同一个 id 永远得到同一次抽样），最后整批居民按维度重新缩放到均值 0、标准差 1——不然 51 个人的抽样均值偏一点，整座镇子的人格就偏了。城市包里不带 `agents_big5.csv`，所以新建城市的居民走的就是这条先验路。

Studio 面板把这两条路的结果合并显示：你看到的五根滑块，永远是"这次运行真正会用的那组分数"。

人格影响行为有**三条通道**（`gaworld/personality/traits.py` 开头的注释写得很清楚）：

- `rules` —— 确定性调制。`choose_action` 里的 `trait_style_fit` 分量、`behavior/dynamic.py` 的小基础率、经济模块的财富驱动、情绪基线。零 token，完全可复现。
- `prompt` —— 把行为锚句塞进**决策类**提示词（日常、目标、新闻、活动调整）。
- `voice` —— 同样的锚句只进**表达类**提示词，也就是日记。

这三条通道分开的意义，是让"决策变了"和"文风变了"能被分别观测：如果只开 `prompt` 而关掉 `voice`，这个人做的选择会带人格色彩，但他写下来的日记口气不变。反过来，`voice` 只影响表达，不影响选什么。

卡片底部还有一段「人格与行为倾向」段落，以及一组一致性标记（`rewrite.drift` / `rewrite.flip` / `now_missing` / `now_moot`）。这些标记算的是"profile 里那段话还准不准"：生成器当初只在 `|z| ≥ 0.5` 的维度上写句子，你把分数改了，段落可能就跟不上了。**与段落相反**（flip）是最严重的一种——规则通道会按新分行动，提示词却还在描述旧的那个人。

面板刻意把大五卡片和九根滑块分成两块，不混在一起。源码里的理由是：当初人格分数是**先采样、再据此写出段落**，而不是从段落反推的（如果反推，开放性和 `mobility_intent` 的相关会高到 0.90，等于把状态变量读了一遍）。把它们摆成一家人会诱使人手工把这个独立性抹掉。

8.4 第 3 步 · 能力 · 技能

这一步全是**读**的——能力画像、成长档案、认知指数这些都由仿真产出，Studio 只是把散在 `output/` 里的东西摆出来。

- **能力画像**：职业标签、这个人的技能列表、可交付物、备注。没跑过仿真时是空的（数据来自 `output/work/capabilities.json`）。
- **兴趣与成长**：每项兴趣一条进度条，下面写着练习总时长、连续天数、最后一次练习是第几天。
- **认知指数**：一个总分（60–140 的区间），由五个分项派生——技能广度、产出能力、成长水平、记忆积累、外部知识。面板特意注明"非测量智商"，它是一个透明的派生量，不是心理学量表。
- **私有技能**：仿真过程中从经历里蒸馏出来的、带文件名的个人技能，和下面的全局技能库分开列。

8.5 第 4 步 · 记忆

记忆步把一个人的记忆拆成五组，用一张辐射状的图呈现：中心是这个人，每个"枢纽"是一类记忆，叶子是一条条具体内容。

- **长期记忆**：情景经验与固化出的长期总结（带 RAG 标记的另算一格「外部知识」）。
- **习惯**：什么场景下我通常会怎么做——每条有时段（早晨/下午/傍晚/夜里）、偏好动作、强度条、最后更新到第几天。
- **意图**：优先事项、回避的事、想交往的人、想恢复的事、成长重点。
- **日程**：时间点 + 活动。

这五组的划分不是随便切的，它对应记忆本身的生长路径：



图 8–2　记忆分层图

一次具体经历（事件层）会经历反思，固化成长期总结（巩固层），再沉淀成习惯和日级意图（习惯与意图层），最后进入衰减与检索：旧记忆自然淡出，检索时按相关性取而不是全取。Studio 第 4 步看到的就是这条链的当前快照——所以那里显示的是"长期记忆 / 习惯 / 意图 / 日程"四类计数加上外部知识，而不是一份日志。

数据读自 `output/memory/`：每位居民一个 `agent_<id>.json`（长期记忆条目列表），关系另存在 `agent_<id>_relationships.json`。这一层可以**手动补写**——`POST /api/agents/<id>/memory` 接受 `kind` 为 `memory` 或 `rag` 的一条文本，追加进他的记忆并同步进索引。这是"外部知识"那格的正规来路：把研究材料喂给一个人，比让模型自己撞上更可控。

8.6 第 5 步 · 社交 · 关系

左边的同心圆是 Dunbar 分层，四层，容量写在标签上：

层	容量
亲密 `inner`	5
挚友 `close`	15
熟人 `acquaintance`	50
弱连接 `weak`	150

圆上的每个点是一条真实的关系边，鼠标悬停能看到姓名和角色（母亲、挚友、同事、邻居……）。这些边不是配置出来的，是仿真跑出来的：关系在 `output/memory/agent_<id>_relationships.json` 里，带亲密度 `closeness`、信任、义务、摩擦、上次联系是第几天。右边的关系编辑器可以改这些边——改名字、改角色、改所属层、改亲密度。

同一屏还有一张「影响倾向」卡，只显示 `voice_propensity` 和 `platform_dependence` 两条，它们是社交行为的主要驱动；以及一张家庭卡（婚姻状态、伴侣、子女、同住长辈），走的是家庭覆写文件。

这一层和第 4 步的边界：记忆是"发生过什么"，社交是"和谁的关系"；两者都会在仿真中演化，所以刚建好一个人时这一屏基本是空的——必须先跑一次。

8.7 第 6 步 · 行为 · 目标

左卡是「行为倾向」，五根滑块：`mobility_intent`、`voice_propensity`、`risk_preference`、`platform_dependence`、`policy_sensitivity`。

注意这里有个容易误会的地方：**这五根滑块不是另一套参数**，它们就是第 2 步那九个状态变量中的五个，只是在这里按"它驱动哪一类决策"重新分组了一遍。改这里等于改那里——第 2 步的雷达图会同步变形。所以这一屏的用处是**聚焦**：不用在九个轴里找，直接调那个真正影响你关心的行为的选择倾向。

右卡是「三层目标」，分人生目标、长期目标、短期目标三层，可以就地编辑标题、状态（进行中 / 已完成 / 已放弃 / 暂停）、领域和进度条。三层各自有活动目标数量上限（2 / 3 / 4），达到上限时「+」按钮会禁用并说明原因。最下面一行是最近一次目标回顾（哪一天、总结了什么）。

8.8 第 7 步 · 复核 · 部署

最后一步是确认页，不是欢迎页。上面依次是 Agent Card（名字、编号、职业、技能/兴趣/交付物标签、接口地址）、身份摘要、九个状态变量的最终数值，然后是三块功能卡：

- **财务**：活期、存款、投资、公积金四个账户，加上工资、租金、债务和两个比率（恩格尔系数、储蓄率）。余额只算活期 + 存款 + 投资，不含公积金。改完写进 `agent_<id>_economy.json`，**仿真下次以有状态模式启动时**读取。
- **Moltbook**：把这位居民注册到居民社交网络，状态机是 未注册 → 待认领 → 已认领；卡片还列出他在这儿发过的每条行动。API key 留在服务端，前端只看得到打码的提示。
- **采访预览**：就地问一句，`POST /api/interview` 调一次模型，把回答打在面板里。想在动笔改人之前先听听他怎么讲自己，这一步最快。
- **生成自传**：约 2 万字的长篇第一人称自传，依据身份、大五、记忆、目标、生命事件、关系、财务写成。这是**异步后台任务**，通常 1–10 分钟，面板每 3 秒刷新一次进度；生成完存在盘上，可以下载。

最下面两个按钮：「保存改动」和「用此居民运行仿真」——后者会发起一次只跑这一个居民（`agent_ids: [当前 id]`）的仿真。

8.9 改动如何生效

面板上所有的"改"，最后落成三种完全不同的东西。搞混这三种，是使用 Studio 最容易犯的错。

第一种：改档案。 动的是 profile Markdown（`data/hangzhou_profiles_with_names.md`）和状态 CSV 里的身份字段（姓名、性别、年龄、户籍、居住地）。它改的是**这个人的设定**——他是谁、住在哪、多大年纪、干什么工作、性格描述怎么写的。这类改动**落盘**，写进种子文件，下次运行模型读到的背景就变了。写回方式有两种：叙事编辑整块替换 profile 里 `## Profile NN｜姓名` 的内容；状态保存则会把 CSV 的九个值**同步**到 profile 的 `**核心状态变量**` 和 `**研究增强变量初始化**` 两行——CSV 是权威源，避免两处漂移。

第二种：改初始状态。 动的是九个状态变量在状态 CSV 里的值。它改的是**这一次（或下一次）运行的起点**——这个人从什么数值开始往下演化。落盘，同样持久。`stress` 拉到 0.8 意思是"下次开跑时他的压力基线是 0.8"，不是"他现在很焦虑"。

第三种：只在这一轮有效。 运行中的仿真可以被干预（`POST /api/interventions/<name>`），标准干预有 `set_agent_state`、`update_config`、`remove_agent` 和 `inject_life_event`（由事件插件注册）。`set_agent_state` 直接改**内存里**那个 Agent 的状态字典，立即生效，每次调用都写一条审计记录。这种改动**活不过这次运行**——进程一停就没了，也不会回到种子文件。Studio 里没有这个按钮；干预走的是运行/观察面板（见第 6 章）。要在运行中途推一个人一把，用干预；在运行前重新定义这个人，用 Studio。

还有两处例外值得记住：

- 大五分数写在**另一个文件** `data/agents_big5.csv`（世界模式下是各世界自己的一份副本），不在状态 CSV 里。人格插件在 `agents.built` 阶段一次性读入，之后记录只读——所以改分数也是**下次运行生效**。改完还会把 `source` 标成 `hand_edited`（不再归功于当初的采样器），并把"段落当初是按哪组分写的"快照进 `authored` 列，好让一致性比对在刷新后依然成立。
- 财务写在 `agent_<id>_economy.json`，**有状态模式**下一次的启动才读；单次跑不读。

8.10 城市切换与 Studio 的关系

左栏那个「选择城市」下拉是本章第二个容易踩的坑。

规则只有一条：**Studio 编辑的永远是当前运行的那座城市。** 别的城市的居民可以读，但在这个面板里改不了。原因很实在：记忆、大五、社交、财务**都按智能体编号存放**，而每座城市的编号各自从 1 开始。A 城的 1 号和 B 城的 1 号是完全不同的两个人，读错城市就会张冠李戴。

于是面板有两种模式：

- **浏览态**（选中的城市不是当前城市）：右侧只渲染身份表、profile 文本、状态雷达和九个种子值，顶部挂一条醒目提示"正在查看「×」，只读"，下面解释为什么不能改，并给一个「设为当前城市」按钮。点它，那座城市成为当前城市，面板随即切回可编辑模式。
- **编辑态**（选中的城市就是当前城市）：七步全开，滑块、编辑器、保存按钮都在。

`GET /api/city/catalogue` 给出全部城市和当前城市是谁；`GET /api/city/agents?city=<slug>` 可以只读地列出任意城市的居民。城市包在 `data/cities/<slug>/`，当前城市写在 `dashboard_config.json` 的 `{"city": slug}`。

8.11 一条能用的工作流

面板功能很多，但别一上来就改十处。推荐这个节奏：

先读懂一个人。 打开第 2 步看雷达的形状——是"压力高、表达低"还是"认同强、流动意愿强"？翻到第 4 步看他的记忆里反复出现什么主题，翻到第 5 步看他是跟家人热络还是跟同事热络。第 3 步的认知指数和成长档案也扫一眼。这些读完之后，你手上的假设才有依据。

然后只改一处。 挑一个你真正想验证的因果关系，改一个地方。最干净的选择是 profile 里的一句话——它直接进提示词，改动最容易被在记忆里看出来。九个状态变量其次，因为它们影响的是基础倾向，容易和别的因素纠缠。

重跑三天，比对。 一次短跑就够：跑三天，去 `output/memory/agent_<id>.json` 看他新记下了什么，去 `output/state/` 的状态长表看九个变量怎么动。改对了，差异应该出现在**你能指出来的地方**；没出现，说明你改的那一处并不是你以为的那条因果链——这本身就是有用信息。

记下你改了什么。 种子文件是共享的，下一个人打开 Studio 看到的是你改过的版本。把改动写在实验记录里，否则一周后你无法解释这一批数据为什么长这样。

8.12 与 Population Studio 的分工

一句话：**Agent Studio 改一个人，Population Studio 造一批人。**

	Agent Studio	Population Studio
产出	一个居民（CSV 一行 + profile 一块）	一座小镇（几百个居民 + 关系 + 网络）
入口	控制台工具栏 / `studio.html`	控制台「人口与群体」页 / `population.html`
参数	九个状态变量 + 身份 + profile	人口结构旋钮：年龄、家庭、就业、收入
步骤	7 步	5 步：选模板 → 人口结构 → 心理状态 → 跑模拟 → 检查结果



图 8–3　Population Studio 实拍（选模板步）

左边选一个人口模板、居民人数、随机种子和小镇名，右边常驻一块「参数体检」：不生成任何人、瞬时返回，直接告诉你当前设定下哪些旋钮互相矛盾、每个旋钮的可达区间是多少。种子决定的是"从哪股子开始抽"——参数一样、种子一样，生成的五百个人完全相同。

Population Studio 背后的分工是清楚的：`gaworld/population/schema.py` 是旋钮的唯一契约（`normalize_spec` 归一化 + 夹取，`check_feasibility` 做纯数学预检）；`synth.py` 负责把边缘分布变成一个个具体的人（用 IPF 拟合年龄 × 性别 × 学历 × 就业 × 行业的列联表，收入用秩变换保证中位数和基尼精确命

中）；`report.py` 分两道闸——`validate_population` 是硬闸（有没有生成出不可能的东西），`build_report` 是软闸（和要求的差多少，每个边缘分布都报”目标 vs 实际”）；`writer.py` 最后把结果序列化成**仿真器已经会读的那个格式**：一行 CSV 加一个 `## Profile NN｜姓名` 的 Markdown 块，BOM 和标点都得一模一样。

所以两者的关系是：Population Studio 造好一座镇子之后，你可以切到 Agent Studio，挑其中一个人细看、细改。反过来不行——Agent Studio 改一个人的档案，不会自动重造整批人。

本章小结

- Agent Studio 是同一个面板的两种身份：**查看器**把模型内部摊开，**编辑器**把改动写回种子文件；七步导航是身份、状态·性格、能力·技能、记忆、社交·关系、行为·目标、复核·部署。
- 九个状态变量（`emotion`、`stress`、`econ_security`、`city_identity`、`policy_sensitivity`、`platform_dependence`、`risk_preference`、`voice_propensity`、`mobility_intent`）是**初始条件**，不是运行结果；改它们改的是下一次运行的起点。
- 大五人格是五维 z 分，优先读 `data/agents_big5.csv`，缺失时按人口先验采样；它经 `rules` / `prompt` / `voice` 三条通道起作用，三条通道分开才能把”决策变了”和”文风变了”分别观测。
- 三种”改”要分清：改档案和改初始状态都会**落盘**、下次运行生效；运行时干预只活过当前这一次运行，并留有审计记录。
- Studio 只能编辑**当前运行的那座城市**；别的城市可读不可改，因为每座城市的智能体编号各自从 1 开始，而记忆、人格、社交、财务都按编号存放。
- 干活的节奏是：先读懂一个人 → 只改一处 → 重跑三天 → 去 `output/memory/` 和 `output/state/` 里找差异 → 记下改了什么。

动手练习

1. **读懂一个居民**（约 20 分钟） 打开 `http://127.0.0.1:8766/site/dashboard/studio.html`，在左栏选中一位居民（记下他的 ID 和姓名）。走一遍第 1 步到第 7 步，抄下：九个状态变量的具体数值（用第 2 步的滑块或第 7 步的「状态变量」摘要卡）、五个大五维度的 z 分、以及第 3 步的认知指数总分。回到第 4 步和第 5 步，用两三句话写下：他反复在惦记什么、他和谁最亲近。再看第 2 步的雷达形状，说一句”这九个数字在一次运行里会怎么影响他”——至少举出两个变量的具体后果（例如 `stress` 高会在日程和决策里体现为什么，`voice_propensity` 低会让记忆或日记里少出现什么）。 **验收标准**：能拿出九个状态变量和五个 z 分的具体数值（不是”大概偏高”），并且能指名道姓地说出至少两个变量与后续行为之间的联系。数值对不上就回第 2 步的滑块和第 2 步底部的滑块各读一遍。
2. **改一处，重跑三天**（约 40 分钟） 挑一位居民，在第 1 步的「叙事 Profile」里点「编辑」，把某一句能体现身份的话改掉（比如把职业那句、或”性格与情绪特征”那句改一个字），点「确认修改」写回。回到第 7 步点「用此居民运行仿真」跑 3 天；或者在控制台跑一次只含他的 3 天仿真。然后去 `output/memory/agent_<id>.json` 里翻他这三天新增的条目，找出**至少一条**能看出这次改动的记忆（例如提到了新身份、新工作场景、新地点的条目）。 **验收标准**：在 `output/memory/agent_<id>.json` 里能指出一条具体记忆条目，并说明它和你改的那句话之间的对应关系。如果三天后找不到任何差异，先确认改的是 profile 而不是状态变量——只有 profile 会直接进提示词，状态变量只改行为倾向的基线，痕迹更淡。

ewpage

第 9 章　群体采访：一次问一群人

GAWorld 大部分时候是「让城市自己跑起来」：给居民一份档案、给世界一套规则，然后看它演化。这一章讲一个方向相反的用法——**你问城市，城市回答**。一次问一群人，问的是同一套问题，拿到的是能按城市、年龄、户籍切开的统计结果，而不是一段闲聊。

群体采访（group interview）和单人采访不是同一个东西加了循环。仓库里本来就有单人采访（`python generative_city_sim.py interview --agent-id 31 --question "…"`），它回答「31 号居民怎么想」；群体采访回答的是「这群人怎么想，答案会不会按城市 / 年龄 / 户籍分化」。这个差别不是包装，是结构性的，下面第一节就说清楚。

群体采访是什么

一句话：**在一次会话里，把同一套问题问给一群受访者**。受访者可以是**个体居民**（一位一个声音），也可以是**cohort 群体智能体**（一格人一个声音，代表整格人），两种可以混在同一场会话里，而**一场会话可以横跨多座城市**。

和单人采访的三个关键差别

第一，连续性只存在于会话记录里，不写回智能体记忆。 每位受访者是逐题被问的，他答第 4 题时，提问里回放着他自己前 3 题的问答。这份「记忆」是会话里的转录，不是智能体记忆库里的一条 episodic 记录——`gaworld/interview/local.py` 里构建居民时调用的记忆召回只会轻轻推动内存里那个对象的情绪状态，子进程一退出那个对象就没了，不写记忆、不写日记、不写状态文件。理由很硬：一次「居民记得自己被调查过」的运行和一次没有的运行是两个不同的实验，这不该作为提问的副作用发生。后面「边界」那一节会回到这一点。

第二，选择题和是非题的回答会被解析回选项标签并统计。 不是留成一堆散文等你读。单人采访的产出是 stdout 上的一段 JSON，你得自己看；群体采访里，模型被要求从给定选项里**原样**挑出答案，解析器再把「B. 不支持」「我选不支持」这类写法映射回你声明的选项标签，于是「支持 / 不支持 / 说不清」各有多少人可以直接数。

第三，结果给出每个选项的两种计数。 这是本章最容易被忽略但最有用的一点：

计数	含义	代码里的字段
受访者	多少个**受访者**（个体居民或群体智能体）选了它	`respondents`
代表人数	这些受访者**代表多少居民**，一个 8 人的 cohort 算 8 人	`people`
占比	代表人数 ÷ 有效回答代表的总人数	`share`

两个数都报，是因为混了个体和群体智能体之后，不加权的数字讲的是**抽样设计**——你挑了 5 个个体加 1 个代表 40 人的 cohort，那 6 个「受访者」的表决权其实差了近十倍。不加权的计数讲的是「我挑了谁」，加权的计数讲的是「人口怎么分布」。哪一套是你要的，得由你决定；系统不替你选。

一次会话里能放什么

	单人采访	群体采访
受访者	一位居民	多位居民 + cohort，**可跨城市**
题型	自由问答	开放题 / 选择题 / 是非题（后两者被统计）
材料	纯文本	可附**图片**与**网址**
连续性	一次一批问题	逐题作答，答后面时看得见自己前面的回答
产出	stdout 上的 JSON	会话落盘 + 统计 + 分组分布 + 完整 Markdown
执行	一次同步调用	后台任务，按城市分进程，带进度

完整流程：面板与 CLI 两条路

群体采访的六步长这样：

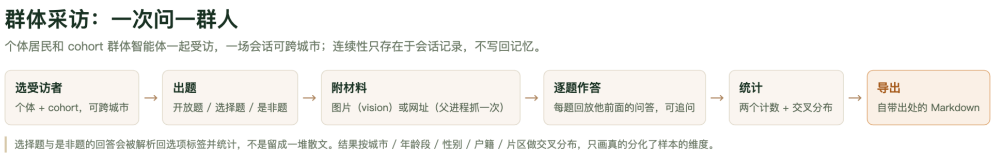


图 9–1　群体采访的六步流程 从左边选人开始，到右边导出一份自带出处的文档结束。关键的一步是「逐题作答」：每题的提问都带着受访者自己前面的问答，所以连续性是流程里的一个环节，不是附件。底部那行是这一章反复要说的两件事——选择题和是非题会被解析回选项标签并统计，结果按城市 / 年龄段 / 性别 / 户籍 / 片区做交叉分布，而且只画真的分化了样本的维度。

面板：控制台里的「群体采访」

面板在 `site/dashboard/survey.html`，起服务后打开：

```
python -m gaworld.apps.dashboard_server --port 8766
```

然后访问 `http://127.0.0.1:8766/site/dashboard/survey.html`。面板分三步：

1. **选受访者**——城市下拉框默认「全部城市」；勾个体居民，或勾群体智能体（每张卡片写着「代表 N 人」）；右栏「这次要问多少」实时显示受访者数、代表人数、问题数和**模型调用次数**。这个数字在开跑前就能看到，因为面板在开跑前先调一次 `POST /api/interview/plan`，那个接口只校验不花钱。
2. **出题**——写标题、访谈背景，添加问题，设题型（开放题 / 选择题 / 是非题），选择题填选项，题上可以挂图片或网址。
3. **看结果**——摘要、选项统计、按城市 / 年龄段 / 户籍的分布，展开某个人看原话，「下载完整 Markdown」导出文档。



图 9-2　群体采访面板的第 1 步 左边是受访者选择区：上面是群体智能体卡片（每张标着「代表 N 人」），下面是该城市的个体居民列表。右边「这次要问多少」这一栏是成本闸门——受访者 0、代表人数 0、问题 0、模型调用 0，每勾一个人、每加一道题就往上跳。面板顶部还显示当前可选的城市数和可采访居民总数。

面板背后的 HTTP 接口全在 `/api/interview/` 下面：

方法	路径	说明
GET	`/api/interview/roster`	城市、个体、群体智能体、可用模型、可用维度
POST	`/api/interview/plan`	校验并回报「要问多少人、多少次调用」，不花钱
POST	`/api/interview/run`	起一轮，返回 `job_id`（202）
GET	`/api/interview/jobs/<id>`	进度 / 结果 / 错误
GET	`/api/interview/sessions`	历史会话列表
GET	`/api/interview/sessions/<id>`	一次会话的全部内容
GET	`/api/interview/sessions/<id>/export`	Markdown 文档 + 建议文件名
POST	`/api/interview/delete`	删一次会话

这些路径都带尾段。原来的单人采访端点 `POST /api/interview` 没有变，Agent Studio 的采访框仍然走它。同一时间只允许一轮采访在跑：两轮同时往一个会话文件追加会写坏记录，两轮打同一个模型后端只会让两边都更慢。

CLI：一个城市一份子进程

面板背后真正干活的命令是：

```
python3 -m gaworld.interview --spec round.json --out answers.json
```

它只有两个参数，都必填。`--spec` 是一份会话规格 JSON，`--out` 是子进程写回转录的位置。进度以 JSON 行打到 stdout，最终结果写进 `--out` 而不是 stdout——某个 import 顺手打印一行警告就能把结果流毁掉。

规格文件长这样（这是真实会话里落盘的 `spec-default.json` 的形状）：

```
{
  "title": "地铁 5 号线延伸线意见调查",
  "context": "…访谈背景…",
  "provider": "",
  "concurrency": 6,
  "questions": [
    {"id": "q1", "text": "你支持吗", "kind": "choice",
     "options": ["支持", "不支持", "说不清"], "multi": false, "attach
        ments": []},
    {"id": "q2", "text": "如果房租因此上涨，你会搬走吗", "kind": "boole
        an"},
    {"id": "q3", "text": "还有什么想说的", "kind": "open"}
  ],
  "respondents": [
    {"kind": "agent", "city": "", "ref": "1", "label": "闫然（18岁·
        男）",
     "demographics": {"city": "default", "age_band": "18-34", "gend
        er": "男",
                      "hukou": "本地", "district": "新亭村"},
     "size": 1, "members": [], "uid": "agent::1"}
  ],
  "history": {}, "history_questions": [], "material": {},
  "city": "", "round": 1
}
```

要注意的是：**CLI 跑的是一个进程能看见的那一座城市。** `gaworld/interview/local.py` 里的居民是从当前进程配置指向的城市构建的，所以跨城市的编排不在这条命令里，而在它的上一级——`gaworld/interview/session.py` 里的 `run_round` 按城市分组，**每座城市起一个子进程**，环境变量是 `GAWORLD_CONFIG_OVERRIDES={"city": "<slug>"}`。这样做的原因是 `build_agent`、记忆库、向量库读的都是配置加载时定好的模块级路径，一个进程里换城市等于在线程池底下改全局配置，还会去动记忆库那个进程级单例 SQLite 连接。面板走的就是这条编排路径。

选受访者：个体居民与群体智能体

名单直接读自**城市 bundle**（`data/cities/<slug>/agents.csv` + `profiles.md`），不经过运行中的配置——否则你只能看见当前选中那一座城市的居民，「跨城市」就无从谈起。没选城市时的默认人口（`data/hangzhou_*`）以**「默认世界」**的名字出现，在统计里它的城市键是 `default`。

群体智能体（cohort） 按属性把人口切成若干格，每格带着均值和离散度两套统计量。它一次回答代表整群人，所以 500 人的小镇问得起；代价是拿不到个人原话。群体划分维度只有能从磁盘上还原的那几个：

维度	说明
`age_band`	年龄段（默认开）

维度	说明
`hukou`	户籍（默认开）
`gender`	性别
`district`	片区，由 `residence` 的「·」前半段得到

没有 `industry`：行业只存在于人口合成时的内存对象上，既没写进 `agents.csv` 也没写进 profile。把它列成维度只会把所有人塞进 `industry=none`，那是一个看起来能用、其实骗人的选项。默认的两维是 `age_band` + `hukou`——年龄决定生活阶段，户籍决定归属与福利，这两样是城市调查最常要切的轴，而且够粗，cohort 数量（也就是成本）可控。

小于 4 人的格子会被并进最近的邻居（`MIN_COHORT_SIZE = 4`）：一个人的「群体」没有离散度可言，只是白白付了群体开销。维度一改，cohort 的编号（`c001`、`c002`……）就指向另一群人，所以面板会清掉已选的群体智能体。受访者记录里带着**成员 id**，子进程照着它重建那一群人，不重新划分——否则 `c003` 在子进程里可能已经是另一群人了。

跨城市怎么组织：面板的城市选择器可以同时勾多座城，`run_round` 按城市分组后一座城一个子进程。**一座城市失败不影响其他城市**——那座城市的受访者会拿到一条带错误的转录，文档里写明谁没问到。

出题：三种题型

题型	模型要交什么	怎么统计
`open` 开放题	2–4 句话	不计数，只生成摘要，但照样报有效回答数
`choice` 选择题	从给定选项里**原样**挑（可设允许多选）+ 一句理由	按选项计数
`boolean` 是非题	先明确「是」或「否」+ 一句理由	是 / 否两桶

选择题至少 2 个选项，最多 12 个（`MAX_OPTIONS`）——再往上模型就不分得清了，计数也就没了意义。

解析对形式宽容，对内容严格。 模型把 JSON 包在 ```` ```json ```` 里、前面加一段客套话、把选项写成「B. 不支持」，都能读出来：解析器会先剥掉枚举前缀（`A.`、`1)`、`选项二:`），再归一化大小写和标点做匹配，匹配不上时做双向包含判断，**长选项优先**（这样「支持」不会吞掉「不支持」）。但**答了选项之外的东西**会被标成 `unparsed`：原话照样进文档，只是不进统计。

这条界线是刻意的。把它塞进某个桶等于替受访者编了一个意见；悄悄丢掉则会在不说明的情况下缩小分母——那正是一份调查用三份有效回答宣称「100% 支持」的方式。文档和面板都会写明有多少份没解析出来。

附材料：图片与网址

网址在**父进程**里抓一次，用现有的正文抽取器压成 ≤1800 字，注入提问。在父进程里抓一次给所有人——否则每个子进程各抓一遍，既是对别人服务器的无谓压力，更糟的是第 1 位和第 40 位受访者可能读到了不同版本的页面。抓不到正文时不会丢掉这道题，改用你填的说明文字，并记下「网址抓取失败或无正文」。

图片真的作为图像发给模型，三种线格式都支持：OpenAI 的 `image_url`（data URL）、Anthropic 的 `image` block（图片排在提问文字之前）、Ollama 的 `images` 数组（裸 base64）。图片能不能真的被「看见」取决于模型。provider 配置里可以显式声明：

```
{"llm": {"providers": {"gpt": {"type": "openai", "model": "gpt-5.
        4", "vision": true}}}}
```

没声明就按模型名启发式判断（`gaworld/llm/providers.py` 里的 `_VISION_MODEL_RE`，匹配 `gpt-4o` / `claude` / `gemini` / `llava` 这类片段）。**判断为不支持时，图片会降级成你填的那句「说明」，并在回答上留下「材料降级」标记**——「这张图说明了什么」由一句 caption 答出来，和由图片答出来是两种不同的测量，只有一种是你以为你问的那个。

如果模型当场拒绝图片（`vision` 设错了、endpoint 不支持 image part），会自动去掉图片重问一次，并记下「模型拒绝了图片输入，已改用文字说明重问」。丢掉一张图并说明，好过因为一处配置错误丢掉整场采访。

追问一轮

同一位受访者的问题是**逐题**问的，每一题的提问里都回放着**他自己**前面的问答，块标题是「【这次访谈里你已经说过的话】」，下面逐条列「问：… 我答：…」。结果页点「继续追问」，写新问题再跑一轮，新一轮的提问里会带上前面所有轮的问答。

问题编号跨轮连续：第二轮从 `q3` 开始（第一轮问了两题的话），不复用旧编号——复用会把两个不同问题的统计合并成一份。

追问时不能换人。 追问复用会话自己的受访者名单，面板在发请求时干脆不把名单发过去，只发 `session_id`：

```
// 一条追问请求里，受访者名单根本不存在
if (state.sessionId) payload.session_id = state.sessionId;
else payload.respondents = picked;
```

理由是：否则改一下已选名单，就能在一次系列的中途悄悄重定义样本——第一轮问的是 20 个人，第二轮问的是另外 20 个人，统计会混在一起，而文档上看不出来。要换人，开一次新会话。

读结果：统计与分组分布

每个问题给三样东西。

一、摘要。 一次模型调用，读所有回答，写主流看法 + 值得注意的少数看法 + 分歧和什么相关。最多取 60 份回答进摘要（`SUMMARY_SAMPLE`）——这个上限是关于上下文窗口的，不是关于成本的：再多的回答，最早的回答会掉出小模型的窗口，「摘要」就悄悄变成了对样本尾部的描述。

二、选项统计。 每个选项三个数：受访者、代表人数、占比。占比的分母是**有效回答代表的总人数**，所以你拿到一个百分比时，也知道它的分母是谁。

三、分组分布。 按城市 / 年龄段 / 性别 / 户籍 / 片区交叉，每组按**自己的**有效回答归一化到 100%。真实产出长这样：

城市	支持	不支持	说不清	有效回答
默认世界	30 / 73%	9 / 22%	2 / 5%	8
绍兴柯桥	3 / 75%	0 / 0%	1 / 25%	4

单元格是「代表人数 / 该组占比」，最后一列是该组的有效回答份数。归一化到组内是因为问题问的是「年长的人答得和年轻人不一样吗」——那是比例问题；用绝对计数会让一个大组的形状彻底盖掉一个小组的分歧。

只分化了样本的维度才画。 一座城市的调查画一张只有一根柱子的「按城市」图，等于请读者去看一个根本没测过的分布。实现上很直接：一个维度如果只落进一个组（`len(groups) < 2`），这一维就不出现在结果里。所以一城一题的调查里

「分组统计」整节是空的——这不是坏了，是没有可报告的分化。

导出的 Markdown 文档

「下载完整 Markdown」给的是一份能独立存在的文档，顺序是 摘要 → 统计 → **每个人的原话**：

```
# <标题>
| 采访时间 / 受访者 / 代表人数 / 涉及城市 / 问题数 / 回答模型 |

## 访谈背景
## 一、汇总摘要          每题：摘要 + 统计表 + 有效回答说明
## 二、分组统计          每题每个维度一张交叉表
## 三、全部回答          每位受访者的全部问答，含降级与解析失败标记
## 附录 A：未能作答的受访者
## 附录 B：问题清单      题型、选项、附带的材料
```

「一、汇总摘要」里每道题是这样的（下面是格式示意，数字是编出来的示例值）：

```
### Q1. 你支持吗

*（选择题，第 1 轮）*

群体访谈中……（模型写的 3–5 句摘要）

| 选项 | 受访者 | 代表人数 | 占比 |
| --- | ---: | ---: | ---: |
| 支持 | 8 | 30 | 73.2% |
| 不支持 | 3 | 9 | 22.0% |
| 说不清 | 1 | 2 | 4.9% |

有效回答 12 份；代表 41 人；**1 份未按题型作答，未计入统计**。
```

注意「受访者」列加起来是 12，「代表人数」列加起来是 41——差就差在 cohort 上：一位 15 人的群体智能体，在「受访者」里算 1，在「代表人数」里算 15。

「三、全部回答」里每位受访者一个小节，标题后面跟一行标签：`*个体｜代表 1 人｜city=默认世界，age_band=18–34，gender=男，hukou=本地，district=新亭村*`。回答上带三种标记：解析失败写「(未能按题型解析，未计入统计)」，材料降级写「(材料降级：当前模型不支持图片输入，已改用图片说明文字)」，出错写「(出错：…)」。

编号是自适应的：没有可报告的分组分布时，「全部回答」就直接是「二、全部回答」；有人没答上才有「附录 A」；「附录 B：问题清单」每轮都有，记着题型、选项和附带的材料（图片只记说明文字，不内嵌——一题一个 base64 PNG 会把可读文档变成几兆噪声）。

自带出处是这份文档的设计要求：哪个模型答的、什么时候、问了谁、问题原文，以及**什么没成功**。一份悄悄省掉六位解析失败居民的文档，是一份夸大了自己样本的文档。

落盘位置

```
output/interviews/<session_id>/
    session.json          问题、受访者、全部问答、统计
    report.md             渲染好的文档，每轮重新生成
    round-<n>/
        spec-<城市>.json      发给那座城市子进程的任务
        answers-<城市>.json   子进程写回的原始回答
        images/               这一轮问题里附的图片
```

刻意不放在城市的运行目录下。 别的运行产物都被重指到 `output/cities/<slug>/`，因为它们属于某一座城市的历史；一次跨城市的会话不属于其中任何一座——归到某一座，换城市后它就消失了；每座存一份，记录就分叉了。

图片存成文件，不留在 `session.json` 里。 base64 比原文件大约三分之一，而 `session.json` 每次追问、每次面板打开会话都要**整份**读出来——留在里面的话，一张 3MB 的照片会变成这些读取里每一次的 4MB 字符串，面板还要把它再传回浏览器（而浏览器本来就有这张图）。附件改为记一个仍然可解析的文件路径，所以问题记录依旧忠实地记着当时问的是什么。

成本与并发

一次调用 = 一位受访者 × 一个问题。 80 人 × 5 题 = 400 次。这正是连续记忆的代价：他答第 4 题时看得见自己第 1 题的回答，就不能把一轮压成一次调用。所以面板在开跑前先 `POST /api/interview/plan`，把这个数显示出来——`plan` 接口返回的 `calls` 就是 `问题数 × 受访者数`，一分钱不花。

执行结构分三层：

- **按城市一个子进程**（`python -m gaworld.interview`，环境变量 `GAWORLD_CONFIG_OVERRIDES={"city": "<slug>"}`）。子进程的超时是算出来的：180 秒启动额度 +（总提问数 ÷ 并发数）× 90 秒——本地 9B 模型答一问可能要 60 秒，80 人 × 5 题 4 路并发跑下来是实打实的一段时间。超了就杀掉，并在失败记录里写明。
- **子进程内：受访者档案串行构建**，因为记忆库持有的是**进程级单个** SQLite 连接，从工作线程里用会直接报「SQLite objects created in a thread can only be used in that same thread」。
- **提问循环按受访者并发**（`concurrency`，默认 4，上限 16）。同一位受访者内部严格串行——第 4 题必须由记得第 1 题的人来答。

一位受访者失败不影响其他人；一座城市失败不影响其他城市——失败的人拿到一条带错误的记录，文档里写明谁没问到。

硬上限：一次最多 500 位受访者、每轮最多 20 个问题。

什么时候该减人数。 成本是线性增长的，而 20 题 × 80 人 = 1600 次调用已经足够贵。更实际的做法是：先用 cohort 覆盖大盘（一个 40 人的群体只要 1 次调用就代表 40 人），再用少量个体居民去校准群体的说法；或者先问 10 个人找问题问得对不对，把题型和选项改好再铺开。选择题的统计要的是**分布**，而分布靠的是有区分度的样本，不是样本量。

常见故障

问：选择题的统计比受访者数少。 有回答没按题型作答，标成了 `unparsed`。面板和文档都会写明有几份（「N 份未按题型作答，未计入统计」）。小模型上把选项写短一些、数量少一些会好很多——选项越长，模型越容易跑偏到选项之外。

问：某座城市整体没问到。 那座城市的子进程失败了（超时、配置错、人口文件缺失）。会话的 `rounds[].failures` 里有原因，是一串「城市名：原因」；结果页顶部和文档附录 A 也会写。其他城市的回答不受影响。真实会话里的 `rounds` 长这样：

```
{"round": 1, "questions": ["q1"], "started_at": 1789664295.17,
 "finished_at": 1789664300.26, "failures": []}
```

问：图片好像没起作用。 先看回答上有没有「材料降级」。有就是路由到的模型被判定为不支持图片——在 provider 配置里加 `"vision": true`，或换一个多模态模型。

问：追问时能不能换人？ 不能。追问复用会话自己的受访者名单，否则改一下已选名单就能在一个系列中途悄悄重定义样本。要换人，开一次新会话。

边界：采访不能影响之后的仿真

目前采访**不能**改变被采访的人，这是刻意设计，不是没做完。理由在第一节已经说过：采访是对人口的一次**观测**，不是干预。一次「居民记得自己被调查过」的运行和一次没有的运行是两个不同的实验，这不该作为提问的副作用发生。

想做「把采访当干预」的实验，应该走显式通道：

- **事件注入**——`gaworld/events/plugin.py` 注册的 `inject_life_event` 干预，往某个居民身上放一个事件。
- **标准运行时干预**——`gaworld/kernel/interventions.py` 里的 `set_agent_state`（写一个状态变量）、`update_config`（改配置的某个点）、`remove_agent`（排队删人，下次运行开始时生效），每次调用自动审计。

让干预是**显式的一次调用**，而不是提问的副作用——这样实验的因果链是清楚的：谁在什么时候被做了什么，说得清。

本章小结

- 群体采访一次问一群人同一套问题，个体居民和 cohort 群体智能体可以混着受访，一场会话可以跨城市。
- 连续性只存在于会话记录里，**不写回智能体记忆**——采访是对人口的观测，不是干预。
- 选择题和是非题的回答被解析回选项标签并统计；解析不出来的标 `unparsed`，原话进文档但不计进分母。
- 每个选项给两种计数：**受访者数**（抽样设计）与**代表人数**（人口），占比用代表人数除以有效回答代表的总人数。
- 成本是「人数 × 题数」，面板开跑前用 `POST /api/interview/plan` 把这个数摆出来；执行按城市分进程，档案内串行、提问按人并发（默认 4，上限 16）。
- 导出的是一份自带出处的 Markdown：摘要 → 统计 → 分组分布 → 每个人的原话 → 附录 A（未能作答）→ 附录 B（问题清单）。

动手练习

1. **跑一场带选择题的采访，读懂两种计数**（约 25 分钟） 起面板 `python -m gaworld.apps.dashboard_server --port 8766`，打开 `http://127.0.0.1:8766/site/dashboard/survey.html`。第 1 步在城市选「全部城市」或某一座，**勾 5 位个体居民**，再加**一个 cohort 群体智能体**（看它标注「代表 N 人」）；第 2 步写标题和访谈背景，添加**一道选择题**（选项填「支持 / 不支持 / 说不清」三档）和**一道开放题**；第 3 步开跑。跑完后点「下载完整 Markdown」。 **验收标准**：导出的 Markdown 里，「一、汇总摘要」下的选择题**有一张统计表**，行是选项标签（不是散文），表头同时有**「受访者」和「代表人数」两列**，且「代表人数」列的合计**大于**「受访者」列的合计（差额就是那个 cohort 代表的人）；开放题那节**没有**统计表但有一行「有效回答 N 份」；文首表格里「受访者」写着「6 位（个体 5，群体智能体 1）」、「代表人数」比 6 大；「三、全部回答」里 5 位个体居民各有小节、cohort 那一节标注「代表 N 人」；最后在终端执行 `ls output/interviews/` 找到刚才那个会话目录，确认里面是 `session.json`、`report.md` 和 `round-1/`（`round-1/` 里有 `spec-<城市>.json` 与 `answers-<城市>.json`）。
2. **追问一轮，确认连续性**（约 20 分钟） 接着上一场会话，在结果页点「继续追问」，写一个新问题（例如「你刚才说的那点，最主要的顾虑是什么」），跑完这一轮，再下载一次 Markdown。 **验收标准**：导出的文档里问题数变成 2（表头「问题数」那一行），两题被编成 **Q1 和 Q2**，且**第二轮的编号从 Q2 开始而不是复用 Q1**；展开第一位受访者的「全部回答」，Q2 的回答在内容上**承接了他 Q1 里说过的话**（不是另起一段通用回答）；打开 `output/interviews/<session_id>/session.json`，`rounds` 数组里有两

个条目、各自带 `questions: ["q1"]` 和 `questions: ["q2"]`；再点一次「继续追问」，确认选人区里仍然是原来那 5 位居民加那 1 个 cohort，**没有变成可以重新勾选的空列表**。

ewpage

第 10 章　游戏场：拿真实居民当对手

前面九章做的都是同一件事的不同变体：给居民一份档案，让城市自己跑起来，然后观察。这一章换一种用法——**你亲自下场，跟居民对局**。不是写脚本，不是批量发问卷，是打开一个页面，挑一个具体的对手，跟他聊、让他做选择、当场看到胜负。

10.1 游戏场是什么

游戏场（Agent Playground）是 dashboard 里的一个入口，里面挂着 15 个小游戏。对手不是脚本，而是某座城市里的真实居民档案——职业、性格、日常、价值观都进了提示词。所以同一个问题问两个人会得到两种答案；同一套说辞对一个人管用、对另一个人一点用没有。

它和前面几章的分工很清楚：

- 想跑完整仿真 → `python generative_city_sim.py run`（见第 6 章）
- 想给一群人发问卷并聚合 → 群体采访（见第 9 章）
- 想玩一局、当场看到胜负 → 游戏场

理解游戏场只需要两条硬规则。**这两条是本章的钥匙**——游戏场里所有看起来奇怪的设计，读完这两条就都顺了。

第一条：游戏只读居民，不写回城市包。 一次游戏是沙盒。城市包里的居民档案、状态 CSV、记忆，谁也不能动；游戏状态只在服务端内存里。重启 dashboard，对局记录就清空——居民不记得发生过这事。这是设计，不是 bug。

第二条：胜负与统计量由确定性代码算，模型只负责扮演与表达。 模型做的事只有两件：作为这个人说话，以及按照固定词表给出结构化选择。真正的判定——行动分布直方图、基尼系数、投票分歧率、三轴评分、陪审的裁决票——都是 Python 代码算出来的。

第二条规则带来一个直接后果：**同一个游戏的输出里，数字的可信度是不均匀的。**代码算出来的数字换个模型、换个时间重跑还一样（位比特一致），可以直接引用；模型给的结论（说服游戏的裁判判词、双队评审的分数）需要人来复核。这条区分在 10.4 节展开。



图 10–1　游戏场大厅 顶部一句话写清了规则：每个游戏都拿真实居民当对手，他们的职业、性格、价值观都会影响结果。每张卡片底部那一行是这一局的规模——「1 局 · 最多 12 人 · 需要 LLM」和「长会话 · 2–6 人同居 · 需要 LLM」是两

种完全不同的开销量级，先看这行再点进去。

10.2 四个家族：按问题形状选游戏

15 个游戏按「问题形状」分四组。分组的意义在于：同一组里的游戏可以互相替换或互补，跨组的游戏回答的根本不是同一个问题。



图 10–2　游戏场的四个家族 横向四条带子就是四个家族，每张卡片写了这个游戏在玩什么、挂在哪组 API 下。注意「一对一」那一组有四个游戏，它们测的是同一件事的不同侧面——人格到底可不可塑。

家族	回答的问题	成员
一对一	一个人能不能被改变、能不能被猜中	说服游戏、斗兽场、读空气（猜人局）、陪审团
一群人同时面对一件事	群体在外部冲击下怎么分化	灾害模式、围城、谣言扩散局、公投局、新闻评论
任务协作 / 创意	同一个任务两条相反路线哪个赢	双队竞赛、小说局
长期同居 / 多人	同一屋檐下的关系动力学	同居模式、谁是真人、穿越历史、连续剧

反过来按研究问题反查，可以得到一张更实用的表：

你的研究问题	先玩哪个
一个居民在压力下会不会改主意	说服游戏（双向）、读空气（看档案稳不稳）

你的研究问题	先玩哪个
群体在外部冲击下怎么变	灾害模式、围城
信息在熟人网络怎么传、谁挡得住	谣言扩散局
私下表态和公开表决之间怎么漂	公投局
同一个任务两条相反路线哪个赢	双队竞赛
一群人对一个案件审议后是否收敛	陪审团
一群人看到同一条新闻的分歧度	新闻评论
居民能不能骗过真人	谁是真人
居民在异时代场景下能不能转换语境	穿越历史

10.3 三个游戏跑通

下面三个游戏分别代表三种典型形态：一个可以交互对话的（说服）、一个一次调用出结构化结果的（灾害）、一个沿网络传播的（谣言）。

10.3.1 说服游戏：让一个人改口

这是整个游戏场里唯一一个你真的在**说话**的游戏。流程是：提问 → 聊天 → 复问，裁判模型比较首尾两份答案，**核心结论真的变了**才算赢。

五步跑通：

1. 在大厅点「说服游戏」，左边选**城市**和**对手**。下拉里带年龄、性别、职业，方便挑一个你觉得好说服的。
2. 写一个**有立场可选**的问题——「支持还是反对」「搬还是不搬」。开放式问题（「你怎么看城市化」）复问时判不出改没改口。
3. 设**最大聊天轮数**（默认 5，上限 20）。一轮 = 你说一句 + 他回一句。
4. 点「开始这一局」。他的第一答案显示在对话区顶部，**整局都以它为基准**。
5. 聊天。`Ctrl/⌘ + Enter` 发送，右上角显示剩余轮数。轮数用完会自动复问并结算；想提前收手就点「复问并结算」。

结算面板给出三样东西：最初的回答、复问的回答、裁判的一句话理由，以及胜负。

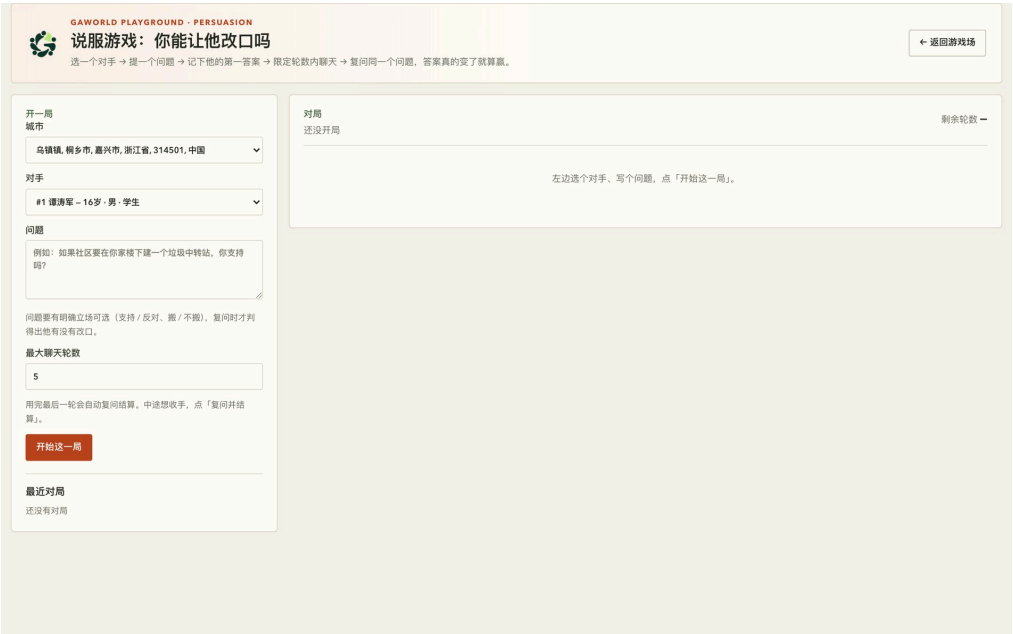


图 10–3　说服游戏的开局面板 注意问题框下面那行提示：「问题要有明确立场可选（支持 / 反对、搬 / 不搬），复问时才判得出他有没有改口。」这是这个游戏最容易踩的坑——不是模型不听话，是问题本身没法比较。底部「最近对局」只列本次 dashboard 运行期间打过的那几局。

判定怎么做的。 三个提示词都在 `gaworld/apps/games_api.py` 里：

- **开局**：档案 + 「按你自己的立场回答，先给答案再给理由，120 字以内」。
- **每轮回应**：档案 + 最初的问题与答案 + 完整对话记录，外加一句明确的态度约束——「只有理由真正贴合你的处境、价值观、利害关系才松口，否则坚持己见并说清你为什么不认同，不要为了客气而附和」。没有这句，模型会一路顺着玩家说，游戏就没有难度了。
- **复问 + 裁判**：重新问一遍最初的问题，再由裁判比较首尾两个答案，只输出 `{"changed": bool, "reason": "..."}`。

裁判的判定规则写死在提示词里：**核心结论变了**（是→否、支持→反对、选 A→选 B、拒绝→接受）才算改；只是措辞、语气变了，或者做了让步但结论不变，一律不算。裁判调用失败或返回里读不出 `changed=true`，一律按「没说动」处理——宁可漏判，不可白送一局。

温度是分开的：居民说话 **0.7**（要像人），裁判 **0.0**（要稳定）。

为什么同一套说辞对不同的人效果不同。 档案来自 `gaworld.interview.roster`：`load_population()` 给身份行，`profile_block()` 给 profiles.md 里那个人的整段档案，截断到 1500 字——性格和价值观正好排在前面，它们决定这人好不好说服。提示词里明写着「不要给一个『正确』的标准答案，给你这个人真的会做的事」，所以做小生意的会先想店里的货，退休教师会先想会不会影响晚年。

一局的模型调用数是 `1（开局）+ 聊天轮数 + 2（复问 + 裁判）`。

10.3.2 灾害模式：同一场灾难，一批人各走各的

一批居民过同一场灾难，stage by stage。**不需要裁判**——每人一份结构化 JSON，直方图自己就出来了。

五步跑通：

1. 在大厅点「灾害模式」，左边选**城市**，再勾**参演居民**（最多 12 人，「🎲 随机 6 人」也行）。人挑得越杂越好——退休老人、带孩子的、做小生意的，反应会差得很远。
2. 选一场**灾难**：地震 / 疫情 / 战争 / 洪水 / 大停电，每场都写好了三幕；或者选「✍️ 自定义灾难」，自己写名称和情境（**一行一幕**，最多 3 幕）。
3. 选**推演几幕**（默认 2）。一幕 = 每个人一次模型调用。
4. 点「开始推演」。这是后台作业，进度条按 `人数 × 幕数` 走。
5. 跑完右边给出：城市简报、四个总览指标（人数、平均恐慌值、互助率、行动分布），以及每一幕的情境原文、行动分布条、每个人的卡片（行动标签 / 具体做法 / 原话 / 恐慌值 / 是否顾得上别人）。

判定怎么做的。 每个人每一幕只问一次，要求只输出一个 JSON：

```
{"action": "避险逃离", "detail": "摸黑叫醒家人往楼下跑", "panic": 4, "h
        elp": true, "say": "快起来，别拿东西了！"}
```

`action` 必须落在固定的六类里——**避险逃离 / 囤积物资 / 救助他人 / 求助求援 / 照常生活 / 打探消息**——所以行动分布图不需要再花一次钱去分类。`panic` 是 1–5 的整数，`help` 是布尔值。解析不了（模型讲了一堆话、JSON 坏了）时这一条降级成「其他」+ 恐慌值 3，**只毁一格，不毁整场**；原文留在卡片上，你能看见模型到底说了什么。

第二幕起，围人的选择进提示词。 每一幕结束后已经有了行动分布，下一幕就在提示词里加一句「你看到身边的人：4 人囤积物资、2 人避险逃离」——不额外调用模型。同时每个人还会看到**自己**前几幕做过什么，所以行为是连贯的，不会第一幕逃了第二幕又在家做饭。没有那句提示词，每一幕都是独立抽样，灾难里最值得看的东西——从众、跟风抢购，或者偏偏有人不跟——根本不会出现。

城市简报是最后一次调用（150 字以内的整体观察），失败不影响整场结果。温度：居民 0.7，简报 0.3。一场的调用数是 `人数 × 幕数 + 1`，6 人 2 幕 = 13 次。

10.3.3 谣言扩散局：一条传闻能走多远

一条传闻沿熟人关系网往外传，跑完给一棵传播树。这个游戏里最有意思的一段是**波次（wavefront）的边界**，下面单独讲。

五步跑通：

1. 在大厅点「谣言扩散局」，左边选**城市**，点「🎲 随机两个小区」。居民列表按**小区**分组，点组标题可以整组勾上。
2. 右边立刻画出这批人的**关系网**——这一步不花任何模型调用，勾人的时候就能看见图。下面会写「N 条关系」，有人没有任何关系也会直接告诉你（消息传不到他们那儿）。
3. 选一条**传闻**：自来水污染 / 银行要倒闭 / 粮价翻倍 / 学校要搬 / 台风瞒报，或者「✍️ 自己写一条」。
4. **最先听到的人**可以不选：默认挑一个关系最多的，再挑一个**不是他熟人**的，这样两个小区都会被点着。
5. 点「放出这条传闻」。跑完右边给出：关系网（叠上传播箭头，可按轮次筛）、触达 / 相信 / 平均信任度 / 最大传播者四个指标、逐轮的扩散条、每个人的卡片和原话，以及一段传播简报。



图 10–4　谣言扩散局的关系网预览 关系图是**按档案推导**出来的，不是仿真长出来的边：同小区＝邻居，同小区同姓＝亲属，同行、同学、同龄熟人各算一种。灰线是熟人关系，勾人时就能看见谁连着谁——消息只沿着这些线走。

模型决定传不传，图决定传给谁。 每个人只回一个小 JSON：

```
{"belief": 72, "action": "转发", "say": "听说那家小银行下周就要关门了，有
        存款的赶紧去取。"}
```

`action` 只能是**转发 / 私下求证 / 辟谣 / 不管**，然后由图决定落到谁头上：

- **转发** → 亲密度最高的、**还没听说**的最多 4 个熟人；
- **私下求证** → 最亲近的、还没听说的那**一个**人（所以「求证」本身也是一种扩散——被问的人就此知道了这件事）；
- **辟谣** → 所有熟人；
- **不管** → 链条到此为止。

让模型自己点名字只会带来编造的人名；这样拆开之后，传播树是这座城的性质，不是模型记名字的能力。

波次的边界：跑 1–2 轮就停是预期行为。 「轮」是波次，不是固定的回合。每轮处理上一波新触达的人，他们的决定把消息送到下一波新认识的人；**下一轮没有人被新触达就停**。加上「每人只开口一次，例外只有一个：信了这条、随后被人当面辟谣的人可以再开口一次」这条约束，开销因此封在 `人数 × 2 + 1`，与轮数无关。

所以界面上那个「最多传几轮」是**上限**，不是承诺。一小区的人互相都是邻居，一跳就全覆盖，典型就是 1–2 轮；想看到 3–4 轮，需要两小区 + 关系稀薄的那批人，或者至少一个辟谣者把消息再送回某个已经听过、但还没真正信的人身上。**看到只跑 1–2 轮就停，不要当成 bug 去查日志。**

关系网是确定性推导的：同小区同姓 0.9（亲属）、同小区 0.55（邻居）、都是学生且年龄差 ⩽ 3 是 0.5（同学）、同职业 0.5（同行）、年龄差 ⩽ 3 跨小区 0.25（同龄熟人）；每人最多留 6 条。「无业 / 退休 / 学生」不算同事——不然一个典型城市包里四分之一的人会变成同一个办公室的。同一批人永远画出同一张图，两局之间可比。

10.4 判定的两种模式

回到 10.1 的第二条规则。游戏场的判定分两种，混用之前先看清楚哪些数字能直接写进报告。

判定方式	用在哪些游戏	怎么算	可信度
代码算分	灾害模式、围城、公投局、陪审团、穿越历史、双队竞赛	固定词表的结构化回复 → 计数、求和、排序、基尼系数	可复现，可直接引用
裁判模型判	说服游戏、双队竞赛的评审环节	一个模型调用读两份答案后给出 `changed` / 分数	需人工复核

代码算分这一侧，每个数字都有明确的算式：

- 灾害模式的行动分布，就是六类动作各多少人；平均恐慌值是 1–5 的均值；互助率是 `help=true` 的人除以总人数。
- 围城的**基尼系数**是对整局累计的食物分配按居民算的标准 Gini（0 = 每人一样多，1 = 一个人吃了所有人的）；**投票分歧率**是票非 `yes` 的占比；**死亡率**是死人数除以总人数。这三个数全是确定性代码，而且死亡本身也是代码判的——饱腹归零每天扣 1 点健康，健康归零就出局。
- 公投局的票数、差额、位移、极化度都从票面（`stance` / `strength` / `say`）直接算，**不需要裁判模型**。
- 陪审团的判决是**得票最多的一方**（plurality），信心是**中位数**，分歧度是最大信心减最小信心。默认无罪推定——票数刚好打平时，裁决卡上也是「无罪」，让一场陪审只能通过多数定罪。
- 穿越历史的三个轴（`era_fit` 时代契合 / `stability` 稳定度 / `inner_conflict` 内在冲突）是**启发式代码**算的：按朝代关键词命中加分、按禁词减分、台词与内心独白的字数为基准加减。跑十次穿越同八个人，分数分布完全一致。引用时要说明这是启发式、不是人给的。

- 双队竞赛的四个维度各 0–10 分（目标达成、可行性、代价、副作用），满分 40，**赢家由总分决定**；模型自己说的赢家只作为 `judge_said` 留在旁边，两者对不上时页面标红。

裁判模型判这一侧只有两处：说服游戏的「首尾对比」，和双队竞赛的评审。它们都做的事是「读两段文本，判断有没有实质变化」，这件事没法用计数表达，只能交给模型——所以要按「需要人审」的标准对待。

工程上还有一个共同约定：**解析失败 = 单条降级**。模型吐了乱码、JSON 坏了，这一条降级成中性默认值（灾害模式是「其他」+ 恐慌 3，谣言局是「其他」+ 信任度 0 且**不往下传**），整体仍然出结果。但表决例外：公投局解析失败会**重问一次**（别的游戏不重问），两次都失败才记成「其他」，且不计入任何一边——因为在表决里，一票读不出来会同时污染票数、差额和位移三个数。

10.5 通用操作流程

不管哪个游戏，流程都是这五步：

1. **选城市与居民。** 城市下拉框默认是当前世界（`dashboard_config.json` 里的 `{"city": slug}`），也可以切到别的城市包。居民列表来自 `gaworld.interview.roster`，带年龄、性别、职业、小区。
2. **选玩法。** 内置的灾难、传闻、议案、剧本、朝代、题材、案件都是可以直接选的，也可以自己写一段。
3. **跑。** 一次两次调用的游戏（说服、猜人）同步返回；几十上百次调用的游戏（灾害、围城、谣言、双队、穿越、连续剧）开一个后台作业，给你 `job_id`，前端按进度条轮询。
4. **看结果。** 面板右上角可以「⬇ 下载 Markdown」，把整场带走。
5. **导出。** 导出文档里会分清哪些是代码算的统计量、哪些是模型写的文字。

「最近对局」只在内存里。 每个游戏页左下角的「最近对局 / 最近推演 / 最近几场」列表，来源是服务端的内存表（说服局保留最近 50 局，后台作业的 store 保留最近 20 个）。**重启 dashboard，这些列表就清空了。**

但有几个游戏会另外往磁盘写一份存档：`output/games/<kind>/<时间>-<job_id>.json`，整场结果加上这局调用路由到的 provider（多人服务器上还记着谁玩的）。这些存档是给 GAWorld-Bench 评测读的——`benchmark/gaworld_bench.py --track B` 要把完成的局汇总起来检查三条事先定好的规律，不够 5 局就弃权——**界面上的「最近对局」照旧只看内存，存档不给你看**。想留一份自己的结果，用「下载 Markdown」。

10.6 教学建议：游戏场是概念直觉的入口

如果你要给学生上课，游戏场最适合当**第一课**。

理由很简单：平行世界那一套（第 12 章）要先把种子、预注册协议、对照组、估计量想清楚，学生第一次面对时往往还没建立起「社会行为是可以被生成出来的」这个直觉。游戏场解决的是更前置的问题——**让一个具体的人，在具体的处境里，做出具体的反应**。

一个学生先在说服游戏里对着一个 16 岁的学生说「你家楼下要建垃圾中转站」，发现怎么讲都不松口；换成一个退休教师，换一套说法就动摇了。这一个瞬间比十页方法论都管用。同理，灾害模式里那个 6 岁孩子写的「妈妈我害怕，拉着我别松手」，和谣言局里「所有人都选了私下求证」的空场，都是一眼就懂的概念。

但游戏场本身不是实验设计。 三条理由：

- **没有对照。** 你没法在游戏场里跑一组对照——同一批居民、同一场景、只有一个条件不同。你只能一局一局地手动重开。
- **没有随机化。** 谁被选中、问题怎么写、轮数设多少、什么时候点结算，全是人在环上的选择。
- **没有统计。** 面板给你的是**一局**的读数，不是一个样本上的分布。

所以正确的用法是：**游戏场负责建立直觉，平行世界负责出可发表的结论。** 先在游戏场里把概念跑通、把手感练出来、把想观察的现象说清楚；然后带着一个明确的问题去第 12 章，在那里设计种子、组、干预和估计量。顺序反了，学生会在游戏场里找「结论」，然后抱怨「为什么跑两次结果不一样」——那正是游戏场该让人发现的事。

10.7 常见困惑

这六条来自仓库的排查清单，都是实际会被卡住的地方。

问：对手下拉显示「这座城市暂无居民」。 这座城市的 `agents.csv` 是空的。修法：`python -m gaworld.city add-agents <city> --size 100` 补一批人，或者换一座有居民的城市。

问：对手的回答像客服，不像居民。 那个人的 profiles.md 档案缺失，只剩 CSV 那一行——没有档案，居民就退化成一段通用人格。修法：补档案，或换一个档案完整的居民。小说局和新闻评论也会遇到同样的现象。

问：明明被说动了，却判「没说动」。 多半是问题太开放，复问答案里没有可比的结论。把问题改成二选一的形式重开一局，比如把「你怎么看城市化」改成「支持还是反对建垃圾中转站」。

问：谣言局显示「0 条关系」。 勾的人分散在不同小区，又没有同行或同龄的巧合，所以连不上。点「🎲 随机两个小区」，或按小区分组整组勾。反过来，**只勾一个小区的话，传闻一跳就传遍**——同小区的人互相全认识。

问：公投的票里出现「其他」。 重问一次之后仍然解析不出来。换个更听话的 provider。这票不计入任何一边，所以看到「其他」时，要意识到分母比你以为的小。

问：聊天轮数用完了还想继续。 一局的轮数上限是开局时定的（默认 5，上限 20），不是可以中途加的。想聊更久就换个更大的 `max_turns` 重开一局。

另外三个值得预先知道的：灾害模式里一堆人被标成「其他」，是模型没按 JSON 输出（小模型常见），原文还在卡片上；每一幕的行动分布几乎一样，多半是灾难情境本身没升级——内置灾难三幕是递进的，自定义灾难要让后面的幕**真的更糟**；投票里 14:0 通过没有分歧，是议案没人吃亏，改成有人得利、有人掏钱的版本才有信息量。

10.8 HTTP API

每个游戏都有一组 `/api/games/<game>/*` 端点。斗兽场是历史最久的入口，仍走它自己的 `/api/arena/*` 命名空间；其余游戏统一挂在 `/api/games/<game>/` 下，由 `gaworld/apps/games_api.py` 转发到各自的后端模块（`disaster_api.py` / `rumor_api.py` / `warzone_api.py` …）。

端点大体是同一套形状：

方法	路径	作用
`GET`	`/api/games/agents?city=<slug>`	可选居民列表（`city` 留空 = 默认世界）
`GET`	`/api/games/<game>/catalogue`	内置剧本 / 传闻 / 朝代，以及人数、幕数、轮数上限
`POST`	`/api/games/<game>/run`	开一局，返回 `{"job_id": "..."}`
`GET`	`/api/games/<game>/jobs/<job_id>`	作业状态；跑完 `result` 是整场

方法	路径	作用
`GET`	`/api/games/<game>/runs`	本次运行跑完的场次（≤ 20），按时间倒序

有两个例外值得单独记：**说服游戏**不提供 `run`/`catalogue`，它是 `start` → `say` → `settle` 加一组 `sessions` 列表（因为它是一局一局聊出来的，状态在会话里）。**猜人局**是同步返回的，不走后台作业——一局只花一次调用。同居模式和谁是真人也不开作业：前者把 session 留在服务端内存里由前端按 tick 反复拉，后者是真人拿座位链接入座的房间。

一个完整的 curl：

```
curl -s -X POST http://127.0.0.1:8766/api/games/rumor/run \
  -H "Content-Type: application/json" \
  -d '{"city": "wuzhen", "agent_ids": [7, 14, 56, 64], "rumor_id":
        "bank", "rounds": 3}'
```

```
{"job_id": "rumor-b5eda212"}
```

两条必须记住的性质：游戏状态在服务端内存里，**不写城市包**；作业表也是内存的，进程重启就没了。要留结果就用 `output/games/` 里的存档，或在面板上导出 Markdown。

需要程序化调用时，从 `gaworld/apps/games_api.py` 里找端点、到 `gaworld/apps/openapi.py` 里找 operationId。operationId 的规则是 `<method>_<路径各段>`——`POST /api/games/persuasion/start` 就是 `post_games_persuasion_start`。Python 客户端可以直接按 operationId 调任意一个：

```
from gaworld.client import GAWorldClient

gw = GAWorldClient("http://127.0.0.1:8766")
job = gw.call("post_games_disaster_run", body={
    "city": "wuzhen", "agent_ids": [3, 7, 19],
    "disaster_id": "earthquake", "stages": 2,
})
```

`gaworld/apps/openapi.py` 是路由的唯一登记处，`tests/test_openapi.py` 会扫全部路径——新增端点没登记会直接 fail。

本章小结

- 游戏场是拿真实居民当对手的 15 个小游戏，分四族：一对一（说服、斗兽场、读空气、陪审团）、一群人同时面对一件事（灾害、围城、谣言、公投、新闻评论）、任务协作与创意（双队、小说）、长期同居与多人（同居、谁是真人、穿越历史、连续剧）。
- 两条硬规则：**游戏只读居民，不写回城市包**；**胜负与统计量由确定性代码算，模型只负责扮演与表达**。这决定了哪些数字能直接引用、哪些需要人审。
- 说服游戏的关键是「核心结论真的变了」才算赢——措辞变化和让步但结论不变都不算，所以问题必须写成有立场可选的形式。
- 灾害模式用一次调用拿结构化 JSON，行动标签落在固定六类里，直方图不需要裁判；第二幕起身边人的选择会进提示词。
- 谣言局是波次扩散：每人只开口一次，没人愿意再传就停，**跑 1–2 轮是预期行为**。

- 代码算分（直方图、基尼、投票分歧率、三轴评分、多数票）可复现可直接引用；裁判模型判（说服首尾对比、双队评审）需要人工复核。
- 「最近对局」只在内存里，重启 dashboard 就清空；部分游戏另有 `output/games/` 存档，那是给评测读的。
- 游戏场建立概念直觉，平行世界（第 12 章）才出可发表的结论——**游戏场本身不是实验设计**。

动手练习

1. **对同一位居民用同一个问题跑一次说服游戏，说清它为什么还不是一个实验**（约 25 分钟） 起面板 `python -m gaworld.apps.dashboard_server --port 8766`，打开 `http://127.0.0.1:8766/site/dashboard/persuade.html`。选一座城市，挑一位居民，写一个**有立场可选**的问题（例如「社区要在你家楼下建垃圾中转站，你支持吗」），把最大聊天轮数设成 5，开一局，聊满几轮后点「复问并结算」。把最初的回答、复问的回答、裁判的理由抄下来。然后**换个说法再开一局**（同一个居民、同一个问题、只换你说话的方式）。 **验收标准**：结算面板上能看到三段文字（最初的回答、复问的回答、裁判的一句话理由）和一个胜负判定；你能指出裁判判定规则对应的是「核心结论变了」还是「只是措辞变了」（对照 `gaworld/apps/games_api.py` 里的 `_judge_prompt`）；你能用两局的结果说明「没有对照、没有随机化、没有统计」中的至少两条，因此这还不算实验——并说出要让它变成实验，第 12 章的平行世界需要补上哪三样东西（种子、组、估计量）。
2. **跑一局谣言扩散，在传播树里找出「没听说」的人，解释他为什么没被触达**（约 20 分钟） 打开 `http://127.0.0.1:8766/site/dashboard/rumor.html`，选城市，点「🎲 随机两个小区」，选一条传闻（例如「银行要倒闭」），放出它。跑完后看关系网和传播树。 **验收标准**：结果区的「触达 / 相信 / 平均信任度 / 最大传播者」四个指标有值；你能从「参演居民」名单里指出**至少一个没听说**的人，并在关系网里说出他为什么没被触达（是他和传播链上的任何人没有关系，还是有关系但那一轮没人转发给他）；你能解释为什么这一局只出现了 1–2 个轮条（对照「每人只开口一次，没人愿意再传就到此为止」这条规则），以及要让它多跑几轮可以换什么（换人 / 换一个社区 / 加一个倾向辟谣的人）。

ewpage

第 11 章　研究工作台：把研究想法变成可执行的方案

前面几章都在教「让城市跑起来」：给居民一份档案，给世界一套规则，看它演化。这一章讲方向相反的用法——**你提出一个研究问题，GAWorld 把它变成一份能跑、能复现、能被判定对错的实验方案**。研究工作台是整个平台里最面向研究的一页：别的页面产出观察，这一页产出的是**得到证据的条件**。

它分两段：前一段产出文档（方案），后一段产出证据（研究）。把文档变成证据的那个中间环节，叫**预注册协议**。

研究工作台是什么

一句话：**把「我有一个研究想法」变成「我有一个能被否证的设计」**。

想法和方案的区别不在于写得详不详细，而在于四个问题能不能答上来：干预是什么、对照是什么、看哪个指标、多大才算有效、判定由谁按什么规则做。研究工作台强制把这四个问题在跑之前写下来。写不下来的部分会被**丢掉并留下记录**（协议里的 `dropped` 字段），而不是悄悄留到跑完变成一句「未测到」。这就是它和「让模型写一份研究计划」的差别。

三条原则贯穿始终，`gaworld/research/` 下 `workbench.py`、`protocol.py`、`evaluate.py` 三个模块的头注释里写着同样的意思：

1. **模型只能引用存在的东西。** 方案只能映射到功能目录里的功能（`docs/FEATURES.md`），协议只能用可测量目录里的指标（`gaworld/research/measures.py`）。
2. **预测先于结果。** 方向和最小效应在跑之前就写进协议，跑完不能改。
3. **判定在代码里，解释在模型里。** 模型不能推翻代码的判定，也不能把「不确定」写成「有效应」。

打开面板：

```
python -m gaworld.apps.dashboard_server --port 8766
```

然后访问 `http://127.0.0.1:8766/site/dashboard/research.html`（控制台里也有「研究工作台」入口）。

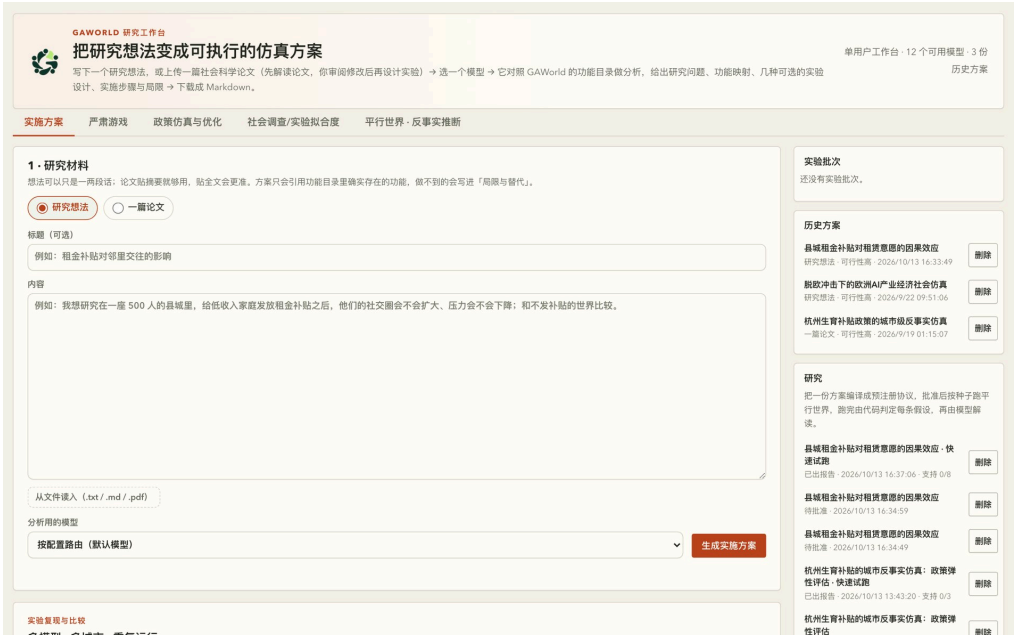


图 11–1　研究工作台的「实施方案」页签 左边是「1 · 研究材料」表单：材料类型选「研究想法」还是「一篇论文」，写标题、写内容，或者点「从文件读入（.txt / .md / .pdf）」把论文贴进来；最下面是「分析用的模型」下拉框和「生成实施方案」按钮。右栏三块：实验批次、历史方案、研究——方案和研究都存在这里，可以随时回看。五个页签就挂在这张页面的顶部。

只想要一份方案文档的话，做完第一段就可以停，方案随时能下载成 Markdown。

五个页签的分工

研究工作台把五类研究任务放在同一个页面下，用标签页分开：

页签	它做什么	详见
实施方案	方案 → 预注册协议 → 跑 → 判定 → 报告	本章
严肃游戏	回合制角色扮演：角色可由居民或真人扮演	见第 13 章
政策仿真与优化	一段政策 → 居民反应 → 模型提出修改 → 得分最高的一版作为建议	见第 13 章
社会调查 / 实验拟合度	真人调查或实验的规格 → 居民照着做 → 代码比对与真人结果的差距	见第 13 章
平行世界 · 反事实推断	手写实验设计 → 效应量、区间、异质性	见第 12 章

分工的逻辑是「产物形态」：实施方案产出的是**一份可判定的实验**；严肃游戏产出的是**一局过程数据**；政策仿真产出的是**一个被推荐的方案**；拟合度产出的是**一组对照真人的偏差**；平行世界产出的是**一个反事实估计**。本章只讲第一个。

（当前代码里的标签栏还有第六个「组织智能」，走 `?tab=organization-intelligence`，文档在 `docs/ORGANIZATION_INTELLIGENCE_TUTORIAL.md`，本章不涉及。）

实施方案：从想法到一份能判定的报告

六步



图 11–2　研究工作台的主线流程 六步从「想法或论文」走到「评分」，每一步的产物都能单独拿到：精读产出可编辑的解读，方案产出多种备选设计，选定并预注册产出写死判定规则的协议，执行产出平行世界，评分产出四种判定之一。注意最后一步写着「代码算分，不由模型评」——这是整条链的关键。底部那行标出了两个落盘目录：方案在 `output/research/` 下，实验在 `output/research/studies/` 下。

第 1 步：写下研究想法，或上传一篇论文

材料类型二选一：**研究想法**（一两段话就够）或**一篇论文**。论文可以点「从文件读入（.txt / .md / .pdf）」：纯文本和 Markdown 直接在浏览器里读，PDF 在服务器端用 `pypdf` 抽文字，接口是 `POST /api/research/extract`。扫描版 PDF

没有文字层，会提示你改成粘贴文本。

材料超过 80,000 字会被截断（`workbench.py` 的 `MAX_MATERIAL_CHARS`），截掉的是尾部——通常是参考文献和附录。论文只贴摘要也够用，但贴全文更准：方法和变量定义往往只在正文里。

第 2 步：`/digest`——先读论文，读完要人来改

论文走两步。第一步点「① 解读论文」，这一步只做一件事：读懂论文。提示词里**不带功能目录**，所以比完整分析快得多，也更不容易分心。

完成后表单下方出现「2 · 论文解读（可修改）」卡片，六个字段全是可编辑的文本框：理论框架、方法与数据、关键变量、主要发现、研究问题、假设。

这个卡片是整条流程里最需要你花时间的地方。 读错的方法、编造的发现，在这里改只要几秒；等三种实验设计都建立在它上面之后再发现，就得全部重来。

改法上有两个实用建议：

- 把论文里的测量改写成 GAWorld 能测的指标名（`stress`、`econ_security`……，完整目录见后面「预注册的价值」一节），后面协议编译时的丢弃会少很多。
- 假设要写清方向：`H1：发放租金补贴后，低收入家庭的压力下降` 远比 `H1：补贴影响压力` 好用——协议里的 `direction` 字段要的就是前者。

不满意可以点「重新解读」；改了材料或切换了类型，旧解读自动作废。表单下方还有一个「跳过解读，直接生成方案」——解读和方案在同一次模型调用里完成，**这一步丢掉了人工审阅的窗口**，后面会说清代价。

第 3 步：`/analyze`——对照功能目录做映射

点「③ 据此设计实验」。这一步模型拿到三样东西：论文原文、**你确认过的解读**、整份功能目录（`docs/FEATURES.md`，最多读 60,000 字）。

功能目录是**事实来源**：`workbench.py` 里 `CATALOGUE_PATH = "docs/FEATURES.md"`，每次分析都重新读文件。给 GAWorld 加了新功能并写进 `FEATURES.md`，下一次分析就会用上，不用重启面板。

方案落成 `output/research/<plan_id>.json`，主要字段是这些：

字段	内容
`kind`	`idea` / `paper`
`paper_digest`	论文解读四段：理论 / 方法 / 关键变量 / 发现
`research_questions`、`hypotheses`	字符串列表
`feasibility`	`score`（0–100）、`verdict`（high / medium / low）、依据
`capability_map`	`[{need, feature, how, entry}]`：研究需要 → GAWorld 功能 → 用法 → 入口
`designs`	几个思路不同的实验设计，外加 `recommended_design` 下标
`steps`	逐步实施步骤，每步带命令和对应面板
`validation`	可信度检验

字段	内容
`gaps`	`[{limitation, workaround}]`：GAWorld 做不到的地方与替代做法
`estimated_cost`	模型调用估算

一个设计里有思路、优势、短板、城市、人口、Agent 设定、时间跨度、实验条件、事件、测量指标。模型被要求给出**思路实质不同**的几个设计（常见是「忠实复现 / 机制检验 / 延伸情境」），并标一个推荐。

「局限与替代」那一节要认真读。 它列的是 GAWorld 做不到或做不好的地方（比如没有住房市场），以及替代做法或需要新写的插件。这一节比概要更能决定这项研究能不能做。

第 4 步：选定一个设计，编译成预注册协议

在结果卡片右上角点「转成研究」。编译的是**当前选中的那个设计**，不一定是推荐的那个——`POST /api/research/studies` 的 `design_index` 就是干这个的，下标从 0 开始。编译是一次模型调用，完成后右侧「研究」列表多出一项，阶段是**待批准**。

编译出来的协议长这样：

字段	内容
`conditions`	最多 8 个世界：`{id, label, role, events[], config}`，角色是 `baseline` / `treatment` / `placebo`
`hypotheses`	最多 20 条：`{id, statement, measure, treatment, control, direction, min_effect, aggregation}`
`validity`	`seeds`（最多 5 个，默认 `[42, 43]`）、`placebo`
`dropped`	编译时丢弃的内容和原因
`notes`	测不到的东西、简化之处

`direction` 只有 `increase` / `decrease` 两种，`aggregation` 只有 `final`（最后一步的值）和 `mean`（全程均值）两种。

协议要求有安慰剂时，代码会自己补一个「市政通告」式的无关事件世界。它就是**噪声底线**：`|安慰剂 − 基准|` 的最大值。

第 5 步：跑前检查，然后批准

批准之前有一道闸门。**错误**（有任何一条就不能批准）：

- 没有处理条件（至少一个 `role = treatment` 的世界）；
- 没有一条可评估的假设；
- 仿真天数不晚于最后一个事件日——事件还没来得及起作用；
- 估算调用量超过上限（默认 50,000）。

警告（不拦，但要看）：只有一个种子、没有安慰剂世界、每一条编译时丢弃。

「只有一个种子且没有安慰剂」这一条值得展开：这种情况下判定**最高只能到「不确定」**。这是刻意的——单种子、无对照，正是「下一个自信的结论」最容易冒出来的地方。

成本是这么估的：

```
估算调用量 = 居民数 × 仿真天数 × 每人每天调用数 × 世界数 × 种子数
每人每天调用数：完整 tick 模式 198、lean_tick 模式 100、daily_brief 模式
1
```

所以常见的省钱做法是**快速模式 + 只选十几位居民**先跑通，看判定和数据质量，确认值得做再加大规模。面板上「参与居民 id」留空 = 当前城市配置里的全部居民。

第 6 步：跑

每个种子跑一次普通的平行世界实验，落在 `output/parallel_worlds/study_<研究 id>_s<种子>/`。运行中可以暂停、继续、停止，停止会保留已经生成的世界产物。一次只能跑一项研究——每项都是好几个完整仿真，同时跑会互相拖慢。

跑完之后阶段依次变成**已判定** → **已出报告**。

第 7 步：读判定与报告

结果卡片上多出三块：**结果表**（每条假设的各种子效应、均值效应、噪声底线、判定徽章与判定依据）、**数据质量**（哪些世界没产出数据，哪些指标在所有世界里都不变）、**解读**（模型写的发现、局限和后续研究建议）。判定徽章有四种，下一节专门讲。

报告的顺序本身就是论证：先是预注册（样本、条件、假设与预测、指标来源、编译时丢弃、跑前检查），然后才是执行记录、结果、数据质量和解读。读者在看到任何数字之前先看到预测。

接口速查

面板背后全在 `/api/research/` 下，长操作都是后台任务：POST 返回 `job_id`（202），轮询 `GET /api/research/jobs/<job_id>` 直到 `status` 变成 `done`。

方法	路径
POST	`/api/research/extract`
POST	`/api/research/digest`
POST	`/api/research/analyze`
GET	`/api/research/plans`、`/plans/<id>`、`/plans/<id>/export`
POST	`/api/research/studies`

方法	路径
GET	`/api/research/studies`、`/studies/<id>`、`/studies/<id>/repor`
POST	`/api/research/studies/<id>/<动作>`

用 curl 走一遍论文的两步流程：

```
curl -s -X POST http://127.0.0.1:8766/api/research/digest \
  -H 'Content-Type: application/json' \
   -d '{"title":"租金补贴与心理压力","text":"（论文正文）","provide
       r":""}'
```

拿到 `job_id` 后轮询，把返回的 `result` 改好，作为 `digest` 字段带回去：

```
curl -s -X POST http://127.0.0.1:8766/api/research/analyze \
  -H 'Content-Type: application/json' \
   -d '{"kind":"paper","title":"租金补贴与心理压力","text":"（论文正
       文    ）     ","digest":{"paper_digest":{"theory":"…","metho
       d":"…","key_variables":["补贴","压力"],"findings":"…"},"rese
       arch_questions":["…"],"hypotheses":["H1：补贴后压力下降"]}}'
```

`digest` 只对 `kind: "paper"` 生效，字段全空时会被忽略。`design_index` 不是整数或越界时返回 400；不传就编译推荐的那个设计。跑前检查没通过时 `approve` 返回 400。

为什么「先读论文再设计」的顺序不能反

模型解读必须有人工审阅环节，否则整条链的第一环就不可信。理由有三层。

第一，解读是整条链的地基，不是它的一段摘要。 后一步 `/analyze` 拿到的解读以你确认过的为准——`workbench.py` 里那句注释写得很直白：用户确认过的解读压过模型自己的复述（`A digest the user confirmed outranks the model's restatement of it`）。也就是说**你改的那份才是地基**。地基错了，上面几种设计、预注册协议、跑出来的数字，全都继承了这个错，而且错得很体面：它会出现在一份格式完整、逻辑自洽的方案里。

第二，模型最常犯的错是把论文里没有的发现写得像有。 「主要发现」那一栏最容易出错：把相关写成因果、把样本里的次要结果说成主要结论、把方向写反。这类错误不会被格式检查拦住，因为输出格式完全正确。人工审阅是唯一能拦住它的地方。

第三，成本是不对称的。 改解读是改几个文本框；改实验设计是重新生成一份方案；改一份已经跑完的研究，是重新花钱跑一遍。

所以「审阅修改」是流程的一部分，不是可选步骤。跳过解读在赶时间的探索里可以用，但**不要拿它的产物直接当实验材料**——它的解读和方案出自同一次调用，你没有机会在中间插进去核对。

预注册的价值：把判定写死在跑之前

方案是写给人看的文档：条件、事件、指标都是文字。协议是同一项研究写给执行器看的版本——每个条件已经是一个平行世界，每条假设都点名了一个可测量指标、两个要对比的条件、一个方向和一个最小效应。这样跑完可以**不经任何人判断**直接打分。

预注册要防的是**事后挑指标**：跑完了，看哪个指标动了，就把它写成「发现」。协议在跑之前就把「看哪个指标」定死，跑完只能对着它打分；报告的顺序（预注册在前）又让读者先看到预测再看数字。

「指标只能选自可测量目录」这条约束，是让判定**可执行**的关键。目录是代码（`gaworld/research/measures.py` 的 `registry()`），不是提示词里的一段文字：

类别	指标 id（名称）
九维状态	`emotion`（情绪）、`stress`（压力）、`econ_security`（经济安全感）、`city_identity`（城市认同）、`policy_sensitivity`（政策敏感度）、`platform_dependence`（平台依赖）、`risk_preference`（风险偏好）、`voice_propensity`（表达倾向）、`mobility_intent`（流动意愿）
干预插件	`stance_score`、`toxicity_score`、`misinformation_risk`、`cross_viewpoint_exposure`、`intervention_reward`（需要开 `CONFIG["intervention"]["enabled"]`）
生理 / 需求	`energy`、`hunger`、`fatigue_debt`、`self_control`、`social_need`、`time_pressure`

一共 20 个，来源都是每个世界的 `state/agent_state_history.csv`，取值 0–1（`stance_score` 是 −1–1）。

指标不在目录里的假设，会在编译时被**丢掉并记下原因**——因为判定器要读的正是提示词里承诺的那一列；承诺了一个代码读不到的指标，跑了也判不了。同理被丢掉的还有：处理条件不存在、对照缺失或与处理相同、方向不是上升 / 下降、条件超过 8 个、事件或 config 补丁格式不对。**丢弃会被记下来并显示，不会被悄悄修正**——否则你会在一次昂贵的运行之后才得到一句「未测到」。

还有一个读数时的注意点：每个指标都带一个**来源等级**，沿用 `benchmark/MECHANISM_PROVENANCE.md` 的 (a) 真实数据校准 / (b) 文献标准机制 / (c) 自定或标定的旋钮。**目前目录里的 20 个指标全是 (c) 级**：九维状态对事件的反应由一次模型调用给出，生理指标是手写规则，干预指标是关键词匹配打分。(c) 级指标的结论**只读方向、大小不作数**——报告里每个效应旁边都写着这句话，模型解读如果给 (c) 级指标写了具体大小，会被标出来。判定规则本身不因等级改变。

产物落盘在哪里

内容	位置
方案	`output/research/<plan_id>.json`（含渲染好的 `markdown`）
研究记录	`output/research/studies/<study_id>/study.json`
研究报告	`output/research/studies/<study_id>/report.md`
每个种子的运行	`output/parallel_worlds/study_<study_id>_s<种子>/`（重置重跑后为 `…_r<次数>_s<种子>/`）

论文解读**不单独落盘**：它是任务结果，你确认后随方案一起存在 `paper_digest` / `research_questions` / `hypotheses` 里。

方案和研究都不在任何一座城市的运行目录下——**方案在被运行之前不属于任何城市**。

四种判定的读法

每条假设在每个种子上算一个效应：`处理条件的值 − 对照条件的值`（按 `aggregation` 聚合）。有安慰剂世界时，`|安慰剂 − 基准|` 的最大值作为噪声底线。然后按固定规则判定（`gaworld/research/evaluate.py`）：

判定	条件	怎么读
supported（支持）	所有种子方向都与预测一致，均值效应 ≥ 最小效应，且 > 噪声底线	方向一致且幅度够；先看指标的来源等级
contradicted（反向）	所有种子都朝预测的反方向走，幅度同样过线	这不是失败，是信息：预注册的预测被数据否掉了
inconclusive（不确定）	其他一切情况——方向不一致，或幅度不够	见下
unmeasured（未测到）	没有任何种子同时拿到处理和对照的值	缺的是证据，不是「无效」

第二道关是**居民配对检验**：每个种子的报告里还有逐居民的配对估计（同一居民在处理世界与对照世界之差）。当假设的对照条件就是这些估计的对照世界时，「支持 / 反向」还要求**每个种子**的配对检验在所判方向上显著（$q < 0.05$ 且 95% 区间不含 0），否则降为「不确定」，并在依据里写明是哪个种子没过。这道关只会让判定更严，不会更松。另一条容易忽略的降级规则：几个种子的可比性版本或执行模式不一致时，即使方向一致也会被降为「不确定」。

什么时候该接受 inconclusive，而不是硬解释。 「不确定」是合法结论，不是失败，理由有三：

1. **它通常指向具体的东西。** 判定依据里写着几个种子方向一致、均值效应多少、噪声底线多少——一眼能看出是方向不一致，还是幅度不够。前者多半是效应本身不稳（或者机制里有个开关没打开），后者多半是最小效应定得太高。
2. **硬解释会把噪声写成发现。** 最常见的情况是：单种子、无安慰剂、效应刚好没过噪声底线。这时候写「有影响」和写「无法排除运行噪声」之间，差的是整项研究的可信度。
3. **数据质量问题要先看。** 如果某个指标在所有世界里都不变，它什么也说明不了——这时「不确定」的真实原因可能不是统计，是设计。

反过来，「未测到」和「不确定」也不能混着用：「未测到」表示压根没拿到对照值，缺的是证据，不能据此说政策有效或无效。

严肃游戏：一句话指引

「严肃游戏」页签把一段情景描述变成一个回合制角色扮演游戏：**角色落座**（每个角色由一位居民智能体或一位真人扮演，可以混坐一桌），**每一轮所有角色同时行动，一次主持人调用裁定本轮结果并调整指标**，最后**按评分维度复盘打分**。真人玩家拿一个座位链接在自己的浏览器里进桌，只能看到自己角色的私密信息。完整流程、对局接口和落盘位置见第 13 章。

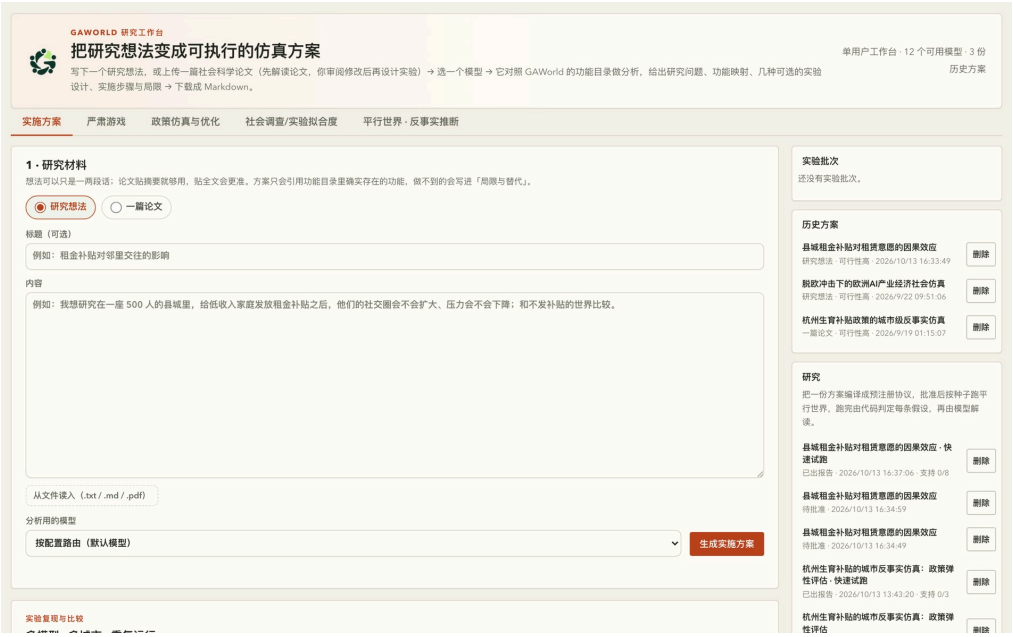


图 11–3　工作台顶部的页签栏 从左到右是实施方案、严肃游戏、政策仿真与优化、社会调查/实验拟合度、平行世界 · 反事实推断，本章讲的流程都发生在第一个页签里。（这张实拍与图 11–1 是同一张截图的副本，画面停在「实施方案」页签上；严肃游戏页的实际界面见第 13 章。）

一条方法论提醒：产物是研究材料，不是结论

研究工作台给你的是**研究材料**，不是**结论**。它的价值在于把「我有一个想法」变成「我有一个能被否证的设计」。

这句话有两层意思，学生最容易在第一层上出错。

一次 supported 不等于「现实里补贴有效」。 当前目录里的指标全是 (c) 级，效应大小不作数；九维状态对事件的反应本身就是一次模型调用给出的判断。所以「支持」说的是：*这个模型在这个机制下的这个判断，穿过了状态动态，在多个种子上方向一致*。这是关于**仿真**的结论，不是关于现实的结论。要把仿真结论搬到现实，需要外部效度论证，那是另一个工作台的事。

一份未经审阅的模型解读直接跑实验，等于把第一环的错误放大到所有环节。 这是实践中最常见的误用：拿到一篇论文，点「生成实施方案」，拿到方案，转成研究，跑完写报告。这条路没有错，但它跳过了唯一一个人能看见模型误读原文的窗口——**这个窗口是流程的一部分，不是可选步骤**。

本章小结

- 研究工作台把「一个想法」变成「一份可执行、可复现、能被判定」的实验方案：前段出方案（`output/research/<id>.json`），后段出证据（`output/research/studies/<id>/`），中间是预注册协议。
- 论文要走两步：先 `/digest` 单独解读，再由研究者审阅修改，然后才 `/analyze` 对照 `docs/FEATURES.md` 做功能映射并给出几种备选设计。审阅

是流程的一部分，不是可选步骤。

- 方案只能映射到功能目录里的功能，协议只能用可测量目录里的 20 个指标；选不到的会被**丢弃并留下原因**，不会悄悄留到跑完变成「未测到」。
- 预注册的价值在于把假设、指标、条件、方向、最小效应和判定规则在跑之前固定下来；报告的顺序（预注册在前、结果在后）本身就是防「事后挑指标」的论证结构。
- 判定由代码按固定规则做，只有四种：`supported` / `contradicted` / `inconclusive` / `unmeasured`；模型只负责解读给定数字，不能推翻判定，也不能把「不确定」写成「有效应」。
- 产物是研究材料不是结论：当前指标全是 (c) 级，只读方向、大小不作数；一次「支持」说的是模型在这个仿真里的判断，不是现实里的事实。

动手练习

1. **走完一次「想法 → 方案 → 预注册协议」**（约 25 分钟） 起面板 `python -m gaworld.apps.dashboard_server --port 8766`，打开 `http://127.0.0.1:8766/site/dashboard/research.html`。材料类型选「研究想法」，写一个你自己的问题（例如「在 500 人的县城里给低收入家庭发租金补贴后，他们的压力会不会下降」），选一个模型，点「生成实施方案」。方案出来后，点「转成研究」把当前选中的设计编译成协议。 **验收标准**：右侧「历史方案」多出一条方案，点「下载 Markdown」能拿到方案文档，里面有「功能映射」和「实验设计」两节、几个设计标签、每条假设写明了方向；右侧「研究」列表多出一项，阶段是**待批准**，卡片上能看到**条件**（每个世界一个角色：基准 / 处理 / 安慰剂）和**假设与预测**（指标、对比的两个条件、方向、最小效应）两段，以及「编译时丢弃」——要么为空，要么每一条都写清了原因。文件层面：方案在 `output/research/<plan_id>.json`，研究记录在 `output/research/studies/<study_id>/study.json`。
2. **给一篇真论文做一次解读，并手工核对**（约 30 分钟） 找一篇你熟悉的实证论文（PDF 或 .txt 都行），材料类型选「一篇论文」，读入文件，点「① 解读论文」。拿到「2 · 论文解读（可修改）」卡片后，逐字段对照论文原文核对：方法与数据（样本、识别策略）、关键变量（自变量、因变量）、主要发现（有没有编造或写反）、假设（有没有把方向写清）。 **验收标准**：你至少能指出**一处**模型解读不准确或漏掉的地方，并把它改掉（直接在文本框里改）；改完点「③ 据此设计实验」，生成的方案里「论文摘要 / 研究问题 / 假设」三节显示的是**你改过的那一版**，不是模型重新表述的版本；在 `output/research/<plan_id>.json` 里打开 `paper_digest` 字段，确认落盘的是你改过的文字。如果某条假设你改不动（比如论文的因变量在那 20 个可测量指标里根本没有），把这条记进笔记——它大概率会在编译时被丢弃并给出原因，这正是你要亲手验证的那条约束。

ewpage

第 12 章　平行世界：把同一段历史分叉成 N 条

前十一章讲的是怎么把一座城跑起来：居民是谁、模型怎么调、仿真怎么跑、结果怎么读。这一章讲的是 GAWorld 里最接近「实验」的那一块——**反事实**：如果那件事没发生，这群人会过成什么样？

这不是一个比喻。GAWorld 的做法是把这个问句变成一次可执行的仿真：一座城，同一批居民，同一个随机种子，同一个天数，同一个模型，跑 N 遍；每一遍唯一的区别是**发生了什么**。N 段历史于是从同一根主干上分叉出去，谁离谁多远、什么时候分开的、这件事最终落在谁身上，全部可以读出来。

本章讲的四件事：一座城怎么分叉（`spec` 写什么）、分叉后怎么测（逐步偏离度与分叉点）、测出来的数字算什么（`causal.py` 里的确定性估计量）、以及怎么把结论交出去（解读接口与产物落盘）。

12.1 一个思想：除了「发生了什么」，其它一切都不变

反事实难在**反事实的那一段历史没法观测**。真实世界里没发生的那条线不存在，你只能拿「发生的人」和「没发生的人」比——但这两拨人本来就不同，制度、经济处境、性格都不一样，比出来的差异里混着「他本来就那样」的部分。

平行世界把这个问题绕开了：**不给两组人，一组人跑 N 遍**。

于是每个世界都共享同一份底座：

共享的东西	具体是什么	为什么必须一样
同一批居民	同样的档案、人格、记忆起点、职业技能	否则比的是两组人，不是两条历史
同一个随机种子	`seed`	随机抽样的差异不能混进结论
同一天数	`sim_days`	观测窗口不同，终局不可比
同一个模型	`llm_provider`	模型不同 = 换了一套生成机制
同一份其它配置	除世界补丁外全部一致	规则不同 = 换了实验对象

唯一被允许变化的是**事件表**（发生了什么）和**配置补丁**（规则改了哪一条）。这意味着：终局出现的任何差异，都只能归到那件事头上。



图 12-1　平行世界的三层结构 这张图把方法拆成三层：最上面是所有世界共享的底座（同一批居民、同一个随机种子、同一个天数与模型），中间是分叉——每个世界一张自己的事件表或配置补丁，最多 8 个，最下面才是逐步测量：逐日偏离度、分叉点、逐人影响、剂量—反应。底座那一层是整章的地基：它一旦不成立，后面的所有数字都只是「两组人之间的差别」，不是「一件事造成的后果」。

这张图也标了工程上的分工：估计量在 `gaworld/parallel/causal.py` 里，是确定性代码，不调模型。

12.2 两个层级：`compare-event` 与 `parallel-worlds`

GAWorld 有两个层级的对照实验，别混用。

`compare-event` 是最简对照：有事件 / 无事件两条历史，跑完比较**最后一行**。它固定两个世界、每个世界一个事件、不支持改配置，产物在 `output/comparisons/`。

```
python generative_city_sim.py compare-event \
  --event-name "临时交通限行" \
  --event-description "主干道限行导致通勤时间上升并影响出行决策" \
  --event-day 2 --event-time 09:00 --sim-days 3 --seed 42
```

`parallel-worlds` 是它的一般化：2–8 个世界，每个世界一张自己的事件表（或一个配置补丁），并且**不只比终局**。

	`compare-event`	`parallel-worlds`
世界数	固定 2	2–8
每个世界的事件数	1	任意，按 `(day, time)` 自动排序
世界之间还能差什么	只有事件	事件 **或** config 补丁
比较方式	终值 + 均值	逐步偏离度 + 分叉点 + 逐人影响
统计判定	无	配对 ATE、区间、p / q、DiD、安慰剂噪声地板
界面	仅 CLI	面板 + CLI（同一个引擎、同一套 spec 格式）
产物目录	`output/comparisons/`	`output/parallel_worlds/`

`compare-event` 没有被替换，也没有改动——它的输出布局被 benchmark 依赖。面板还会把旧的 `compare-event` 结果**当场适配**成两世界实验读出来，磁盘上什么都不改。

什么时候用哪个：只想知道「有没有影响」，`compare-event` 更省事；想知道**从哪一步**开始有影响、影响**落在谁**身上、两种强度**哪个更狠**，就得用 `parallel-worlds`。本章讲后者。

12.3 世界规格：一份 spec 的真实结构

`gaworld/parallel/spec.py` 把一份 JSON 校验成 `ExperimentSpec`。字段照抄代码，不是照抄想象。

实验级字段（所有世界共享）：

字段	类型	默认	说明
`name`	str	`平行世界实验`	实验名，同时决定输出目录名
`sim_days`	int	跟随 `CONFIG`	每个世界跑几天
`seed`	int	`42`	所有世界共用
`agent_ids`	int[]	跟随 `CONFIG`	参与的居民；留空按配置
`llm_provider`	str	跟随路由	强制所有世界同模型
`fast`	bool	`false`	确定性认知 + 跳过日总结/日记 + 默认 3 人
`max_parallel`	int	`2`	并发进程数，取值 1–4（不是世界总数）
`baseline_id`	str	第一个无事件的世界	对照基准
`worlds`	list	—	2–8 个世界

世界级字段：`id`（缺省由 `label` 生成）、`label`、`events[]`、`config{}`、`note`、`role`（`treatment` / `placebo` / `baseline` / 空）、`dose`（数字）。

事件对象只有四个键：`day`（≥1 的整数，必填）、`time`（形如 `09:30`，缺省 `10:00`）、`name`（必填）、`description`（居民当天会「看到」的那段话）。

一份可以直接改的 spec：

```
{
  "name": "限行强度剂量反应",
  "sim_days": 4,
  "seed": 42,
  "agent_ids": [1, 2, 3, 4, 5],
  "max_parallel": 2,
  "worlds": [
    {"label": "基准世界", "events": []},
    {"label": "轻度限行", "role": "treatment", "dose": 1, "events":
        [
      {"day": 2, "time": "07:00", "name": "临时交通限行",
       "description": "早晚高峰单双号限行，通勤时间小幅增加。"}]},
    {"label": "重度限行", "role": "treatment", "dose": 2, "events":
        [
      {"day": 2, "time": "07:00", "name": "全面交通管制",
        "description": "主干道全面管制，通勤时间显著增加，部分人无法到
        岗。"}]}
  ]
}
```

写错时的报错也是有用的：事件缺 `name` 或 `day`、`time` 不是 `HH:MM`、`dose` 不是有限数字、`role` 取值不认识、基准世界 `baseline_id` 不在列表里——都会直接指出是哪个世界的哪一项，不会默默吞掉。

有几个键世界不能改

事件回答「发生了什么」，`config` 补丁回答「规则变了会怎样」——比如个税起征点、裁员概率、环境事件的生成频率。一个世界除了 `events` 还能带一个配置补丁：

```
{
  "label": "低起征点世界",
  "events": [],
  "config": {"economy": {"tax": {"monthly_exemption": 3000}}}
}
```

补丁最终**深合并**进配置：只把起征点从 5000 改成 3000，`economy.tax` 下的税率级距、专项扣除原样保留，其它 `economy` 键也不受影响。所以只写你要改的那一片叶子就行。

但有一批键是**保留键**，写在世界的 `config` 里会直接报错（不会默默忽略），因为它们要么是对照基准、要么是磁盘隔离的边界：

```
random_seed · sim_days · agent_ids · policy_events · stateful · dis
tributed
memory_dir · log_dir · vector_db_path · state_output_dir
network_output_dir · environment_output_dir · diary_output_dir · vi
sualization
```

前半截是**对照基准**：世界之间种子或天数不一样，报告里的差异就没法归因了。后半截是**磁盘隔离**：每个世界有自己的记忆、日志、状态、日记目录，两个世界共用任何一个都会互相污染——而污染的表现形式恰恰是「这个干预看起来没效果」，正是实验本该诚实给出的结论。

12.4 怎么跑：命令行与面板

命令行参数是 `python3 generative_city_sim.py parallel-worlds --help` 的实际输出：

参数	说明
`--spec SPEC`	实验定义 JSON；不给 `--sweep` 时必填
`--sweep PATH=V1,V2,…`	参数扫描：基准 + 每个取值一个世界
`--placebo`	配合 `--sweep`：加一个基准的原样副本作噪声底线
`--sim-days N`	覆盖 spec 里的天数
`--seed N`	覆盖随机种子（所有世界共用）
`--llm-provider NAME`	强制所有世界用同一个 provider
`--fast`	快速模式：确定性认知 + 跳过日总结/日记 + 3 人队列
`--output-root DIR`	输出根目录，默认 `output/parallel_worlds`

命令行参数**覆盖** spec 里的同名字段，所以同一份 spec 可以先 `--fast --sim-days 2` 跑通流程，再原样跑正式配置。

```
python generative_city_sim.py parallel-worlds --spec worlds.json --
        fast --sim-days 2
```

面板是研究工作台的「平行世界 · 反事实推断」页签，底下就是上面那个引擎。左边设计、右边观测：左边定义世界之间差什么，右边回答于是差出了什么。面板提供四个模板（裁员冲击 / 交通限行强度 / 安慰剂对照 / 严谨对照），其中「严谨对照」= 基准 + 处理 + 安慰剂，且重复 3 个种子——**要下结论时从它开始**。面板上设计好的实验可以照抄成 JSON 走 CLI，CLI 跑出来的实验也会出现在面板的历史列表里。

运行层面：每个世界是**独立子进程**（`python generative_city_sim.py run`），`max_parallel` 决定同时跑几个。一个世界失败不影响其它世界跑完，报告照常生成，失败的世界标成「无状态数据」，原因原样显示，完整日志在 `worlds/<world_id>/run.log`。同一时刻只允许一个实验在跑，第二次 `start` 拿到 `409`。

N 个世界就是 **N 次完整仿真**，没有任何共享或折扣。5 人 × 4 天 × 3 个世界 ≈ 3 次普通运行；`max_parallel` 只影响墙钟时间，不影响总调用量。

12.5 逐步测量：偏离度、分叉点、逐人影响

这是本方法真正比「只比终局」多出来的东西。`gaworld/parallel/analysis.py` 从每个世界的 `state/agent_state_history.csv` 重建整条轨迹，给出三个视角。

逐日偏离度。 每一步，该世界所有指标与基准世界同一步的**平均绝对差**，一条曲线一个世界。基准世界自己就是那条主干（按定义偏离恒为 0）。

分叉点。 判定规则是「**越过并且待住**」：某一步越过阈值、且随后连续几步都在阈值之上，才算分叉。只越一步就掉回去的，那是 LLM 的随机性，不是两段历史真的分开了。默认阈值 `DEFAULT_SPLIT_THRESHOLD = 0.02`；如果跑出了安慰剂世界、且它的偏离峰值高于这个默认值，阈值会被**安慰剂的偏离峰值**接管（`split_threshold_source` 记为 `placebo`），因为那个数才是这次运行的真实底噪。

逐人影响。 同一个居民在两个世界里的终局差距，按大小排序，最多 60 人（`_MAX_MOVERS`）。人群均值会把个体抹平——同一场冲击落在通勤最长的人和落在家里的人身上，是两回事。

磁盘隔离不是洁癖

每个世界跑在自己的目录树里：记忆、日志、状态、日记、人生事件、干预队列、向量库、可视化产物，全部重定向到 `worlds/<world_id>/` 之下。`gaworld/parallel/spec.py` 的 `world_overrides()` 是**唯一**知道这件事的地方。

理由是 `spec.py` 注释里写的那句：两个世界共用记忆目录会互相污染历史，而污染的表现形式是「这个干预看起来没效果」——正好是实验本该诚实给出的那个答案。副产品是 `reset` 永远不会清掉你正在跑的主仿真的记忆；并行世界也强制关掉 Moltbook 投递（N 个世界会以基础人口的账号发 N 次）。



图 12-2　平行世界面板的一次真实实验 这是研究工作台「平行世界 · 反事实推断」页签的一次「快速试跑」（县城租金补贴对租赁意愿，5 天、种子 42、单一模型 MiniMax-M3、快速模式）。上面是面板直接给出的结论摘要：普惠低额补贴（150 元/月）第 49 步开始分叉、终局距离基准 0.0393；普惠高额补贴（500 元/月）第 69 步才分叉、终局距离 0.0265；定向补贴（300 元/月）全程未明显分叉；安慰剂（仅公告、无实际补贴）终局距离 0.0131、峰值 0.0415。下面是分叉图：每条线是一个世界，实心点是事件位置，空心圆圈是分叉点。这张图真正值得看的地方是**对照关系**——安慰剂那条线的抖动幅度，就是其它任何一条线想被称为「效应」时必须超过的门槛。

12.6 估计量：`gaworld/parallel/causal.py` 里的确定性代码

`analysis.py` 回答「差了多远」，`causal.py` 回答研究真正要问的：**这件事造成了什么、有多确定、落在谁身上**。

平行世界在设计上特别干净：每个世界里是同一批居民、同一个种子，所以每个人在处理世界和对照世界里**各有一段历史**——个体效应是一次减法，而不是一次插补。全部计算在 `gaworld/parallel/causal.py`，固定随机种子（`RNG_SEED = 20261002`），同样的产物永远给出同样的区间。它**不调用模型**。

估计量	它回答什么问题
个体效应（ITE）	同一个居民在两个世界里差多少？取事件之后窗口的均值（只改 config、没事件的世界取全程）
ATE	所有人的个体效应平均下来是多少？括号里还给相对对照水平的变化
95% 置信区间	这个均值有多稳？按居民重抽样 2000 次的 bootstrap 百分位区间
p（符号翻转检验）	没有效应时这个均值可能来自偶然吗？零假设下每个个体效应的正负号可互换；≤12 人精确枚举，否则 5000 次蒙特卡洛
q（BH 校正）	一次实验里检验了很多「世界 × 指标」，会不会有假阳性？Benjamini–Hochberg 校正全部检验
d_z	效应量多大？配对效应量 = ATE ÷ 个体效应的标准差
事件前差 / DiD	事件发生前两个世界就已经不一样了吗？差分中的差分 = 事件后差距 − 事件前差距
安慰剂噪声地板	这个效应是不是只是仿真自己抖出来的？拿安慰剂世界与基准的距离当上限
起效顺序	哪个指标先动？经济安全感先动、压力随后、情绪最后——因果链的**线索**，不是证明
持续度 / 半衰期	这件事的影响留下来了，还是过几天就消回去了？持续度 = 终值 ÷ 峰值；半衰期 = 峰值后跌回一半用的步数

估计量	它回答什么问题
异质性	效应落在谁身上？按居民属性分组估计条件平均效应（种子表里的性别、年龄段、户籍，以及任何 2–8 个取值的文本列），外加「事件前水平」中位数分组（天花板效应、回归均值最常从这儿冒出来）
剂量—反应	强度加大，效应是变大、变小、还是到某个点就饱和了？拟合「效应 ~ 剂量」的直线，报告斜率、R^2、是否单调
跨种子复现	换一个随机种子，结论还站得住吗？以**种子**为单位合并，报告方向一致的种子数与种子层面的 t 区间

判定规则是固定的，可以手算复核

`causal.py` 的 `_verdict()` 只有五条分支，代码与面板上写的一致：

判定	条件
稳健	q < 0.05、置信区间不含 0、绝对值超过安慰剂噪声上界、事件前平衡
噪声内	统计显著，但 |ATE| 没超过安慰剂世界的区间上界
事件前不平衡	显著，但事件前差距 > max(0.01, ¼|ATE|)；此时以 DiD 为准
提示性	未校正 p < 0.05 或区间不含 0，但没通过 FDR
无效应 / 样本不足	区间含 0 且不显著 / 两边都有数据的居民少于 3 人

「事件前平衡」的容差是 `max(BALANCE_TOLERANCE = 0.01, 0.25 × |ATE|)`。安慰剂世界自己的行不参与噪声判定；如果安慰剂本身「稳健」地动了，报告会给出警告——这次运行的噪声不小。

噪声地板是这套方法的地基

GAWorld 的认知由 LLM 驱动，除非开确定性认知，否则**两个配置完全相同的世界也不会跑出完全相同的历史**。偏离度里天然含有一份与事件无关的噪声。不先量它，你会把噪声读成效果。

做法是加一个**安慰剂世界**：一个真实发生、但对居民生活没有实质影响的事件（比如「第 2 天 10:00 市政通告，不涉及任何居民的实际生活」），并把它的 `role` 设为 `placebo`。它给出的偏离度就是噪声底噪，真实干预必须明显高过它，结论才成立。

想把底噪压下去有两个办法：勾**快速模式**（感知、计划、反思都走确定性模板，随机性大幅下降但**不为零**）；加长天数、加多居民（噪声是随机的会被平均掉，效果是系统性的不会）。无论压到多低，安慰剂世界仍然该跑——它给的是**这次运行、这个模型、这套配置下的实测底噪**，比任何先验假设都可靠。

12.7 剂量—反应与参数扫描

「这个参数取多少合适、结果对它有多敏感」不需要手写一串只差一个数的世界。`--sweep` 一次扫一个数值配置项：

```
python generative_city_sim.py parallel-worlds \
  --sweep economy.shocks.layoff_base_prob=0.01,0.02,0.04 --placebo
       --fast --sim-days 30
```

`economy.shocks.layoff_base_prob`（面板里叫「裁员基准概率」）当前默认是 `0.001`，按 Epoch 7 的说明它现在**按月**解释。这条命令展开成 5 个世界（已实际验证）：

世界 id	标签	角色	剂量	配置补丁
`baseline`	基准：裁员基准概率 = 0.001	baseline	0.001	无
`v1`	裁员基准概率 = 0.01	treatment	0.01	`economy.shocks.layoff_base_prob = 0.01`
`v2`	裁员基准概率 = 0.02	treatment	0.02	`economy.shocks.layoff_base_prob = 0.02`
`v3`	裁员基准概率 = 0.04	treatment	0.04	`economy.shocks.layoff_base_prob = 0.04`
`placebo`	安慰剂：裁员基准概率 = 0.001	placebo	0.001	无

每个取值一个世界、补丁只改这一片叶子、取值直接当**剂量**——`causal.py` 里现成的剂量—反应拟合因此能直接读这条曲线：效应对参数值作直线拟合，给斜率、R^2 和是否单调。

`--placebo` 加的那个世界是**基准的原样副本**（同样的配置、同样的事件、同样种子）。它与基准之间的差就是仿真本身的波动，作为噪声底线。

扫描有护栏，不合格会直接报错，而不是跑出 N 个一模一样的世界再报告「没有效应」：

情况	结果
路径在配置里不存在，或不是数值	报错，指出断在哪一级
实验级设置（保留键）	报错
整数参数给了小数	报错
取值等于当前值，或重复	跳过并说明（当前值就是基准世界）
不同取值少于 2 个	报错——剂量反应至少要三个点
基准 + 取值（+ 安慰剂）超过 8 个世界	报错

一次只扫一个参数：多参数网格的世界数按乘积增长，8 个世界装不下，也很难读。面板上可扫的数值参数有 500 项，都带中文说明（`gaworld/settings/config_docs.py`），取值是否在合理范围内不自动检查——概率类参数自己把握。

同时给 `--spec` 时，spec 提供共享设置（名称、居民、并发数……）和**基准世界的事件**，世界列表由扫描生成。也就是说「经济下行期里裁员率的影响」可以这样设计：先在基准世界加一个下行事件，再生成扫描，基准的事件会复制到每个世界。

12.8 怎么读结果

拿到报告按顺序问四个问题。

一、分叉了吗？ 看偏离曲线有没有**稳定**越过阈值。分叉点落在事件之后不久，说明事件确实推动了系统。分叉点落在事件之前——事件还没发生两条线就分开了——说明差异里混着噪声，这时候先看安慰剂世界。

二、分叉点早 vs 晚意味着什么？ 早，意味着传导快：事件当天或次日就改变了某些指标，机制是直接的（通知、涨价、当天通勤）。晚，意味着传导慢：消息先在社交网络里传开，或者要等某个周期（发薪日、结算日）才落地，机制是间接的。注意分叉点读的是「**指标**分开了」，不是「事件被注意到了」——这两件事之间可以隔好几天。

三、剂量单调吗？ 比较两个世界的 `divergence_final`。重度显著大于轻度 = 单调；两者接近 = 存在饱和，管制加码没有换来更多变化。再看两者的第一行变化是不是同一个指标：如果不是，说明强度变化改变了**作用机制**，而不只是幅度。

四、落在谁身上？ 「谁被改变了」把同一个居民在两个世界里的终局差距排序，通常排在最前面的是通勤距离最长、工作最不能远程、或者收入缓冲最薄的那几个。点开他们的 profile 验证这个解释——这一条比平均效应有信息量得多。

两个容易读错的地方

「全程未明显分叉」不等于「没有效应」。 它只说明这条指标上没有稳定越过阈值。可能是效应确实很小、可能是效应的方向在不同指标上互相抵消、可能阈值对这次运行太高（安慰剂世界的峰值抬高了阈值）、也可能是效应落在了别的时间尺度上。正确做法是去看 `causal_effects.csv` 里的那一行：ATE、区间、p / q、噪声、判定，五样都摆在那里。

「效应显著」不等于「效应是你以为的那个机制」。 事件描述写什么，居民当天就「看到」什么——描述越具体，反应越有依据。反过来，如果你想要的是「一条政策通过改变某条规则起作用」，就别用事件，用 `config` 补丁；用事件去模拟政策，测的是「居民听说有这条政策」。

另外，**不要只看终值**。事件当天猛跳、第三天就回到基准，和一路走低到最后，终值可能一样，但结论完全不同——这正是逐步偏离度存在的理由。

12.9 解读接口：一次模型调用，每条发现都被绑回一行估计

`POST /api/parallel-worlds/interpret` 让模型把估计表读成一段人话。请求体 `{root, baseline?, provider?, language?}`，写一次解读并缓存。

它的约束写在代码里（`gaworld/parallel/interpret.py`），不只在提示词里：

- 提示词里只有**表**，没有原始数据：估计表（按判定排序的前 40 行）、起效顺序、跨种子重复、剂量反应、警告。
- 每条发现必须写明 `world`（世界 id）和 `metric`（指标 id），且这一对必须出现在估计表里。代码用 `check_claims()` 把它对回估计表——模型用中文标签作答（比如把「压力」写进 `metric`）也能对上。
- 对上了，就把这行的**判定**一起附上；指向的行不是「稳健」的，界面上会提示「不能当作确定的效应」；指不到任何一行的发现**保留显示**，但标为「不作数」。

这个设计是防止模型自由发挥的关键：模型可以组织语言、可以指出机制线索、可以提下一步实验，但它**不能自己发明一个效应**。它写的每一个数字都必须能指回一行有判定背书的估计。

结果按对照世界缓存在实验目录的 `interpretation.json` 里，重新打开不花调用；换一个对照世界就是另一份解读——「重度 vs 轻度」和「重度 vs 基准」是两张不同的表，值得两段不同的文字。

12.10 产物落盘与复现要求

一次实验的目录长这样：

```
output/parallel_worlds/<时间戳>_<实验名>/
├── experiment.json            ← 清单：spec + 每个世界的路径 + 运行状态
├── report.json                ← 完整报告（面板读的就是它）
├── divergence_metrics.csv     ← 逐世界 × 逐指标的 baseline / 本世界 / Δ 明细
├── divergence_summary.md      ← 人读的结论摘要（最先看这个），含「反事实推断」一节
├── causal_effects.csv         ← 逐世界 × 逐指标的 ATE / 区间 / p / q / DiD / 判定
├── interpretation.json        ← 模型解读，按对照世界缓存
└── worlds/
    ├── <world_id>/
    │   ├── run.log             ← 这个世界的完整运行日志（进度和报错都从这读）
    │   ├── reset.log
    │   ├── state/agent_state_history.csv    ← 分析的原始输入
    │   ├── memory/ logs/ network/ environment/ diaries/ life_events/ intervention/
    │   └── visualization/simulation_trace.json   ← 可在「仿真回放」里逐帧看
    └── <world_id>/ ...
```

每个世界都是**一棵完整的仿真产物树**——它和一次普通 `run` 的产物结构完全一样，所以任何针对单次运行的分析工具都能直接对着它跑。

`divergence_metrics.csv` 的列是 `world_id, world_label, metric, baseline_final, final, delta_final, baseline_mean, mean, delta_mean`；`causal_effects.csv` 的列是 `world_id, metric, n, ate, ci_low, ci_high, p_value, q_value, d_z, ate_final, pre_gap, did, noise, verdict`。想用 pandas 读结论，读这两个就够了。

复现四件套

同一个结论要能被重跑出来，需要四样东西同时对上：**同种子 + 同模型 + 同天数 + 同居民**。`seed` 写进 spec 并盖在每个世界的配置上；`llm_provider` 强制所有世界走同一个 provider（否则模型路由会把不同任务分给不同模型）；`sim_days` 和 `agent_ids` 是实验级字段，保留键挡住了「某个世界悄悄用了不同的天数或居民」。

变更默认配置后必须追加 Epoch

`gaworld/core/comparability.py` 记录着**可比性版本**：货币系统改造、人格语料重写、收入以 profile 锚定、出行方式阶梯……每次变更都让此前的运行与此后的运行不可比，于是每个 Epoch 记下变更日期、影响了哪些量、依据在哪。

平行世界实验在完成时会被盖上 `comparability_epoch`（当前是 Epoch 11）。跨 Epoch 的结果**不可合并**：面板的跨种子复现会把不同 Epoch 的种子跳过并在 `excluded` 里说明原因；研究评估器也不会用不同 Epoch 的种子去判定假设成立与否。

规则很简单：**变更默认配置、导致此前运行不可比时，就在 `gaworld/core/comparability.py` 追加一个 `Epoch`（只增不重编号）**。默认关闭的可选开关不需要——运行清单里的配置快照已经能区分它们。

12.11 与第 11 章研究工作台的关系

研究工作台（见第 11 章）把「研究条件」**编译**成平行世界：写下一个研究想法或上传一篇论文，系统对照 GAWorld 的功能目录做映射，给出两三种可选的实验设计，实施方案一旦定下就编译成预注册协议，按种子跑成平行世界，再由代码打分判定。

本章讲的是另一半：**怎么跑、怎么读**。两者的分工是——

	研究工作台	平行世界
输入	一段研究想法 / 一篇论文	一份 spec / 一个参数路径
产物	预注册协议 + 多种子实验 + 假设判定	一次 N 世界实验 + 逐步偏离度 + 反事实估计
谁做决定	人（选设计、选方案）	代码（跑、分叉、算区间、判判定）
输出	假设是否被支持	事件造成了什么、落在谁身上、剂量如何

严肃游戏（见第 11 章与第 10 章）是**另一种**把研究问题变成互动形式的路子：让居民和真人同场，产出的是对话与判断；平行世界要的是**可归因的差值**。二者可以互补——先用平行世界估出「租金补贴大概能把租赁意愿推动多少」，再用严肃游戏去看这个推动在具体的人身上长什么样——但不要指望严肃游戏给出效应量。

本章小结

- 平行世界的核心是**控制**：同一批居民、同一个种子、同一个天数、同一个模型、同一份其它配置，唯一变化的是事件表或配置补丁；终局的差异因此才归得到那件事头上。
- `compare-event` 是有事件/无事件的最简对照，`parallel-worlds` 把它一般化到 2–8 个世界，并从「只比终局」升级到逐步偏离度、分叉点与逐人影响。
- 世界规格只有实验级与事件级两层：世界可带 `events` 与 `config` 补丁，但种子、天数、居民名单与各世界的输出目录是保留键，写了直接报错。
- 估计量在 `gaworld/parallel/causal.py`，是确定性代码、不调模型：配对 ATE、自助法区间、符号翻转 p、BH q、事件前平衡与 DiD、安慰剂噪声地板、起效顺序、异质性、剂量—反应、跨种子复现，判定规则固定可手算复核。
- **安慰剂世界是地基**：LLM 驱动的仿真即使配置完全相同也会抖，先量出噪声底噪，再谈「没有明显分叉」。
- 解读接口只有一次模型调用，且每条发现都被绑回一条估计行和它的判定，指不到的行标为「不作数」；解读按对照世界缓存。

- 变更默认配置导致结果不可比时必须追加 Epoch；跨 Epoch 的实验结论拒绝合并。

动手练习

1. **三个世界，找出分叉点**（约 30 分钟）

   写一份 spec：三个世界共用 `sim_days: 4`、`seed: 42`、`agent_ids: [1,2,3,4,5]`——「基准世界」无事件，「轻度事件」在 Day 2 09:00 触发一个描述具体的事件（如「本地主要雇主宣布裁员 20%」），「重度事件」同一时刻触发一个强度更大的版本。用 `python generative_city_sim.py parallel-worlds --spec worlds.json --fast` 跑完。

   验收标准：在 `divergence_metrics.csv` 与 `report.json` 里找到两个处理世界的 `split_step`，把步数换算成第几天（`report.json` 的 `steps_per_day` 给出换算比例），写出「轻度第 X 天分叉、重度第 Y 天分叉」；再在 `divergence_summary.md` 里指出哪个世界的 `divergence_peak` 更大。如果两个世界都「全程未明显分叉」，先往 spec 里**再加一个安慰剂世界**（`role: "placebo"`，事件用「第 2 天 10:00 市政通告，不涉及任何居民的实际生活」）或把 `sim_days` 加长再跑，然后读出安慰剂世界的偏离峰值，写明你的噪声地板大概有多大。

2. **跑一条剂量—反应曲线，量出噪声地板**（约 30 分钟）

   ```
   python generative_city_sim.py parallel-worlds \
     --sweep economy.shocks.layoff_base_prob=0.01,0.02,0.04 --plac
           ebo --fast --sim-days 5
   ```

   验收标准：（a）在 `experiment.json` 里数出世界总数，并指出哪个是 `baseline`、哪个是 `placebo`、哪几个带 `dose`；（b）在 `causal_effects.csv` 里找到 `world_id` 为 `placebo` 的那些行，读出它的 `ate`、`ci_low`、`ci_high`，写出**噪声地板的量级**；（c）在 `report.json` 的 `causal.dose_response` 里找到至少一个指标，读出它的 `slope` 和 `monotonic`，用一句话说「剂量加大时这个指标是变大还是变小」；（d）拿一个真实处理世界的 `ate` 和安慰剂的 `ci_high` 比一比，写出「这个效应算不算超过了噪声地板」。

ewpage

第 13 章　三种研究页签：政策仿真、复制拟合与严肃游戏

研究工作台顶部的页签里，除了「实施方案」和「平行世界」，还有三个：政策仿真与优化、社会调查/实验拟合度、严肃游戏。前面那两个入口你可能已经用熟了——一个把研究想法编译成预注册协议再跑一遍，一个自己搭平行世界看反事实效应。这三个是另外三种用法：它们不跑完整的社会仿真，而是把一批居民放到一张桌子前，只问他们一件事。

三个页签的共同点是**判定权在代码手上**。政策仿真里最终推荐哪一版政策，是代码按综合分选的；复制拟合度里「像不像真人」的所有数字，是代码算的；严肃游戏里根本没有统计结论可判，代码守的是回合机制，模型负责扮演、裁定和复盘。一句话概括：模型可以生成内容，但不能自己宣布「结论成立」。

13.1 三个页签在哪、产出到哪

三个页签共用同一个入口页 `site/dashboard/research.html`，但各有一份前端脚本和一份后端模块：

页签	前端脚本	后端模块	产物目录
政策仿真与优化	`policy-sim.js`	`gaworld/apps/policy_sim_api.py`	`output/re`
社会调查/实验拟合度	`replication.js`	`gaworld/apps/replication_api.py`	`output/re`
严肃游戏	`serious-game.js`	`gaworld/apps/serious_game_api.py`	`output/re`

后端模块只干两件事：把请求变成后台任务，把结果读写到磁盘。真正的逻辑在 `gaworld/research/` 下的三个文件里——`policy_sim.py`、`replication.py`、`serious_game.py`——它们不认识 HTTP 也不起线程，模型调用是注入进来的，所以可以脱离服务器直接调用和测试。

产物不会写回任何城市包。三个页签在 `output/research/` 下各占一个目录，文件名带时间戳和随机后缀（`pol-`、`rep-`、`sg-`、`sgs-`），可以直接在磁盘上翻。

13.2 政策仿真与优化：让代码挑出更好的那一版

13.2.1 一轮仿真怎么跑

表单很短：选城市、选居民样本、写政策描述、可选的候选政策，外加一个「用推荐的修订版再仿真一次」的勾选框。点「开始仿真」之后：

1. **抽样**。从所选城市按种子抽一批居民（页面给的档位是 6/9/12/15/20/24/30，默认 12），16 岁以下不参加——政策总得落在某个人头上，那个人得交钱、上班、能投票。抽中的人按 id 排序存进这一次运行。
2. **居民反应**。每位居民对着自己的档案，对**每个版本**各输出一个固定格式的 JSON。
3. **代码汇总**。支持率、反对率、日子变差的比例、各项均值、按年龄段/性别/户籍的分组均值、分组之间生活感受的最大差距，外加一个综合分。
4. **一次模型调用**。把上面的数字、分组和最不满意居民的原话一起读进去，给出效果评估、主要效果、风险与被忽视的人群、逐条修改，以及一份完整的修订版政策。
5. **验证**（默认开启）。修订版回到**同一批居民**上再仿真一次。
6. **代码选推荐**。版本里综合分最高的那个是推荐；平局时按 A→B→R 的顺序取靠前的。修订版跑输原政策时，页面上如实标出来。

模型调用数是 `居民数 × 版本数 + 1`。12 个人、原政策加候选再加修订版，就是 12 × 3 + 1 = 37 次。

13.2.2 居民怎么回答

回答的形状是写死的，在 `policy_sim.py` 的 `_REACTION_SCHEMA` 里：

```
{
  "support": -2,
  "wellbeing": 1,
  "finance": 0,
  "compliance": 70,
  "behavior": "……",
  "concern": "……",
  "suggestion": "……"
}
```

四个数字各自有量程：支持度 −2（强烈反对）到 2（强烈支持），半年后的生活感受 −5 到 5，钱包影响 −5 到 5，遵从度 0 到 100。另加三句话：实际会怎么做、最担心什么、只能改一处会怎么改。

读不出 JSON 的回答**记为无效**，不算作中立。如果把读不出来的回答当成 0 分，模型的格式失败会伪装成「居民没意见」。代码在 `parse_reaction` 里给无效回答打上 `ok: False`，汇总时只统计有效的那一批，无效的条数在结果页上单列。

提示词里还有一条 `_STAKES_RULE`：先想清楚这项政策落到你头上具体意味着什么——你住哪、干什么、家里有谁、手里有多少钱；不要给面面俱到的中间答案，只有确实事不关己时才打 0。这条规则针对的是一个具体的失败模式：被泛泛地问一句「你支持吗」，所有人都会答一个体面的中间值。

13.2.3 综合分：代码算的那一列

`composite_score` 把四个均值各自归一到 0–1 再加权：

分量	归一方式	权重
支持度	(x + 2) / 4	35%
生活感受	(x + 5) / 10	30%
钱包影响	(x + 5) / 10	15%

分量	归一方式	权重
遵从度	x / 100	20%

加权后乘 100，保留一位小数。权重是模块顶部的 `SCORE_WEIGHTS` 常量，不是模型给的，改它就改文件。

除了总分，代码还给出三个比例（支持率、反对率、日子变差的比例）和一张分组表：按年龄段、性别、户籍三个轴各切一次，每格记人数、平均支持度、平均生活感受；再从所有「至少两个人」的分组里找出生活感受差距最大的一对，记在 `widest_gap` 里。政策该动谁、从这张表上看。优化器提示词还会把每个版本最不满意的那批居民的原话摘出来（最多 24 条），让每条修改都能追溯到某个人的某句话。

13.2.4 为什么必须是同一批居民

三个版本跑在同一批居民上，不是省事，是方法要求。

假设 A 版拿到 55.4 分、B 版拿到 44.0 分。这两个数之差里混着两样东西：政策本身的差别，和这 12 个人碰巧是什么样的差别。如果每个版本各抽一批人，「A 比 B 好 11.4 分」就没法区分是政策造成的还是抽样造成的。同一批人挨个评价三个版本，差值变成**配对差**——同一个人对 A 的回答减去对 B 的回答，人这个因素被控制住了。

代价是结论的适用范围：样本小，比例只在「这批人这批问题上」有意义。导出文件末尾固定写着——居民反应由大模型按每位居民的档案扮演生成，是对政策反应的情景推演，不是民意调查。这三个页签里，只有政策仿真会直接给出「支持率 42%」这种容易被误用的数字，所以这句免责声明尤其要看。

结果页还有「以修订版为原政策再优化」按钮，把修订版填回输入框再来一轮。模型写的「预期会改善什么」不能替代下一轮的数字。

13.2.5 落盘与接口

- 记录写 `output/research/policy/<id>.json`，只在运行结束时写盘，跑一半崩了不留半份。
- 接口在 `/api/research/policy/*`：`GET /` 列记录并返回样本上下限，`POST /run` 起一次运行（返回 `job_id`），`GET /jobs/<job_id>` 看进度，`GET /<run_id>` 取完整记录，`GET /<run_id>/export` 拿 Markdown。
- 记录里有三个版本的政策全文、每位居民对每个版本的逐项反应、指标、分组、模型的建议，以及 `best` 字段——那个字段就是代码选出来的推荐版本。

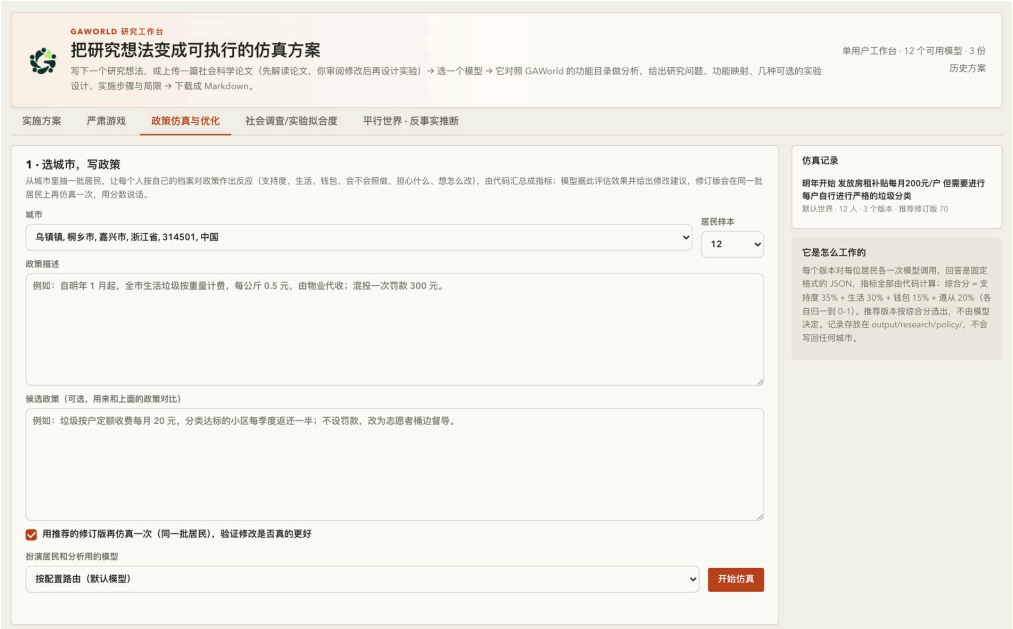


图 13–1　政策仿真与优化页签

左边是表单：选一座城市、写政策描述、可以再写一个候选政策用来对比，底部勾选「用推荐的修订版再仿真一次」。右边「它是怎么工作的」那几行把方法论写在界面上：每个版本对每位居民各一次模型调用，回答是固定格式的 JSON，指标全部由代码计算，推荐版本按综合分选出、不由模型决定。每条仿真记录都带着所在世界、样本人数、版本数和推荐版本的分数。

13.3 社会调查/实验拟合度：这群居民像不像真人

这个页签问的问题很朴素：**能不能拿这批居民当受访者用**。做法是拿一篇报告了真人社会调查或实验结果的文献，让一座城市的一批居民按同样的设计重做一遍，然后由代码比较两边有多接近。

13.3.1 从论文到一份可编辑的规格

第一步是上传文献：粘贴正文，或者从 `.txt` / `.md` / `.pdf` 读入（PDF 需要装 `pypdf`）。方法与结果部分最重要。文本超过 6 万字只读前面，并在返回里标出截断。

然后是**一次模型调用**做抽取，产出的是一份可编辑的规格，不是一段散文：

- 真人受访者是谁、总样本量、年龄范围（居民按这个范围筛选；文献没写就只排除 16 岁以下）；
- 所有人看到的说明，以及**实验条件**：调查只有一个条件，实验每个组间条件一个，**第一个是对照组**，每个条件带受访者看到的材料；
- **题目**：量表（写清 min—max 与端点含义）、单选（选项对所有条件通用）、数值（可带范围与单位）。上限 12 道题、6 个条件；
- **真人结果**：每题每条件的均值 / SD / n，或各选项的占比，**每个数字注明出处**（表号、图号或页码）。

抽出来的规格会在页面上以表单呈现，逐项可改；增删题目或条件可以直接编辑 JSON，应用时按同样的规则校验，丢弃或修正了什么会列在 `warnings` 里。

13.3.2 只报告文献里有的东西

这是这个页签最重要的方法论约束，写在抽取提示词的第一条里：**只抽取文献里确实报告的数字，原样照抄；文献没报告的写 null，绝不估算或编造。**

代码也守着这条：`normalize_spec` 的文档字符串写着「从不发明一个数字」——文献没报告的真人结果就保持缺失，`results` 里的每条记录都带 `source` 字段。文献没报告的题目结果不会被补一个「大概均值」，它在比较时就是空的、不参与打分。

这条约束决定了「拟合度」这个词的含义。它衡量的是**在可比的量上有多接近**，不是「整体像不像真人」。文献只报了三道题的均值，那拟合度就只覆盖那三道题；文献没报某个条件的 SD，那个条件就没有真人 p 值，判定时也不拿它说事。

抽取提示词里还有一条针对实际踩坑的提醒：小模型容易把不同条件的选项合并成一题，比如把收益框架的 A/B 和损失框架的 C/D 并成四个选项。提示词要求选项写成与条件无关的说法（「确定的方案」「有风险的方案」），各条件的具体措辞写进该条件的材料里；核对规格时发现选项文不对题，就照这个思路改。

13.3.3 组间与配对

选定城市、居民规模（5—500，页面档位 10/20/30/50/100/200/300/500，默认 50）、随机种子和分配方式之后，居民开始作答：

- **组间**（默认，同文献）：居民按种子随机、均衡地分到各条件，每人只答一个条件；
- **配对**：每人分别作答每个条件——各条件是互相独立的调用，没有前后影响，效应用同一批人配对检验，更省样本。

每位居民每个条件一次模型调用，按自己的档案作答，输出固定 JSON。**超出量程或读不出的回答记为无效，不会被截断到边界**——一个 7 分量表上答 9，不算 7 分，算无效。

模型调用数是 1（抽取）加居民数（组间）或居民数 × 条件数（配对）。页面在跑之前就把这个数显示出来。

13.3.4 代码比较三件事

比较全部由代码完成，不经模型。

第一件，逐题逐条件的误差。 代码算出居民这一格的均值、SD 和 95% 区间（`均值 ± 1.96 × SD / √n`），与真人的数对照，误差按题目自己的量程归一到 0–1：

- 单选题用两个分布的**总变差距离**（TVD，即两个比例分布之间差的绝对值求和再折半）；
- 有量程的题用 `|居民均值 − 真人均值| / (max − min)`；
- 没有量程但文献给了 SD，用 4 倍真人 SD 当跨度；
- 两样都没有，用相对误差。

第二件，真人值是否落在智能体的 95% 区间内。 这一列是 `within_ci`，逐格给出是或否。落在区间内，说明两格的差距小到可以由抽样波动解释；落在区间外，说明差距比抽样误差大，是真实的系统性偏离。

第三件，处理效应的方向与显著性。 每个处理相对对照组，效应按量程归一（单选题取真人里变化最大的那个选项的占比变化）。居民这一侧的 p 值：组间用 Welch 检验，配对用配对 t 检验，都是正态近似。文献若给了 SD 和 n，代码也顺带算出真人自己的 p 值。判定分四档：

判定	含义
复现	方向一致且在居民一侧显著
方向一致，但不显著	方向一致，p 不小于 0.05
未复现	方向不一致，也不显著
方向相反且显著	方向反了，且显著

真人效应小于量程 1% 的，判为「文献中无差异」，不参与方向一致率的分母——文献自己都没测出差异，就不能要求居民复现出一个差异。

汇总给出几个数：水平准确度（1 − 平均误差）、真人值落在 95% 区间内的比例、单元格水平相关 r 和效应相关 r（各需至少 3 个点才算得出）、平均 SD 比（小于 1 表示智能体比真人更一致），最后是**拟合度分** = 100 ×（50% 水平准确度 + 50% 效应方向一致率）；没有处理效应的纯调查就是 100 × 水平准确度。文献若按人群分组报告了结果，分组还能映射到居民的性别、年龄范围、户籍上（最多 8 个，可组合），分组里居民不足 5 人的标成「样本少」，只当线索用。

13.3.5 模型解读：只能引用，不能新造数字

跑完可以勾选「让模型解读偏差」，多一次模型调用。解读的规矩很硬：提示词里只给代码算好的表格，**每条 finding 必须写 `ref`，而 `ref` 必须是表里某一行的引用**（形如 `Q1/C1`、`Q1/C2−C1`）；不许自己算新数字。代码把 finding 挂回它引用的那一行，检验它声称的模式对不对——在「居民效应比真人还大」的一行上写「效应消失了」，标成冲突；不引用任何行的，标成不作数。什么都不删。

于是你得到的是两种东西的分工：数字和判定来自代码，解释来自模型。



图 13–2　社会调查/实验拟合度页签

第一步是上传文献：可以粘贴正文，也可以从 `.txt` / `.md` / `.pdf` 读入，提示语写明了需要包含什么——受访者是谁、问了什么（或实验材料是什么）、各组的结果数字。右边的「拟合记录」列出了历史运行，每条带所在世界、样本人数和拟合度分；下面那段「它是怎么工作的」把方法写在界面上：逐题逐条件的误差按题目自身量程归一到 0–1，真人数值是否落在居民的 95% 区间内，实验还比较每个处理相对对照组的效应方向与显著性。



图 13–3　两条效度轴，以及拟合度页签落在哪一格

平台把效度拆成两条轴：功能效度问「平台自己做到位了吗」，人类效度问「像不像真人」。拟合度页签属于后者里的「与真实参照比对」——拿真人数据当对照，逐格比误差。它的结论只在这一格里有意义；功能效度那一格的事，得靠别的手段去核对。

13.3.6 落盘与接口

- 抽取出的规格只在页面里，随 `POST /run` 一起发回；记录在运行结束时写入 `output/research/replication/<id>.json`，不会写回任何城市。
- 接口在 `/api/research/replication/*`：`GET /` 列记录并返回规模上下限，`POST /extract` 抽规格，`POST /check` 校验并返回 `warnings`，`POST /run` 跑，`GET /jobs/<job_id>` 看进度，`GET /<run_id>` 取完整记录，`GET /<run_id>/export` 拿 Markdown，`POST /<run_id>/interpret` 单独再解读一次。
- 记录里有规格全文、抽到的居民、逐人回答，以及 `comparison` 里的 `cells`、`effects`、`group_cells`、`group_gaps` 和 `summary`。

这个页签的边界要记住：文献里的人群和这座城市的居民不同——国家、年龄、教育、职业都不一样，差距有一部分来自人群本身，不全是模型的问题；按年龄筛选只是最粗的对齐。居民也没跑日程仿真，回答只基于档案和题目本身，依赖真实激励、多轮互动或长期追踪的实验，这里只能复现其问卷部分。

13.4 严肃游戏：把一段描述变成一张桌子

前两个页签都是居民对着一份材料作答，一个人的作答。严肃游戏不一样：它让一群人坐在一起，每轮各说各的，然后由主持人裁定。

13.4.1 从描述到设计

写一段描述：谁在场、各自想要什么、要决定什么。可以指定轮数（2–8）和角色数（2–8），留「自动」由模型定。

一次模型调用把它变成一份游戏设计：背景、角色（公开身份 / 各自的目标 / 只有自己知道的私密信息）、公共指标、每轮的事件与可选做法、评分维度、复盘问题。角色数少于 2 或没有回合的，代码判定为不可玩，模型自动重试一次；两次都不行就报错，让你换模型或再试一次。

公共指标最多 5 个，每个有初始值和上下限，主持人每轮根据大家的行动调整它——比如「方案支持率」「三方信任度」「项目拖延风险」。这些数字是游戏里的公共局面，不是研究结论。

13.4.2 座位：居民与真人混坐

每个角色要选一种扮演者：

- **居民智能体**：指定某位居民或随机挑一位，来自所选城市，同一局不会重复入座；
- **真人**：填个称呼，会生成一条座位链接。

居民扮演角色时，**自己的档案在前、角色简介在后**：一个谨慎的理发师去演物业经理，还是谨慎的理发师。提示词里明说「用你自己的性格、经历和说话方式来演这个角色——你会怎么做就怎么做，不要演成教科书里的标准答案」。

真人座位是「人在环里」的关键。把链接发给参与者，他们在自己的浏览器里打开，只看得到公开信息和自己角色的私密信息；本轮别人的行动在结算前看不到。座位链接用的是随机 token，但**它只是区分身份，不是鉴权**——它适合在一台本地面板上组织的课堂或工作坊，不适合直接放到公网上。

13.4.3 同时行动，主持人裁定

每一轮所有角色**同时**行动。居民由模型按档案来演，真人在浏览器里提交。代码守着这条：它记录谁已经行动、谁还没行动，轮次只由它推进。

同时是刻意的。两个原因：坐在桌前的人没法被要求连等六次模型调用再轮到自己；而且如果居民按顺序行动，他们就能看到本轮别人的动作，而真人看不到——那就不是同一场对局了。大家基于同一段历史行动，主持人整轮一起裁定。

人齐了，主持人（一次模型调用）裁定本轮结果：发生了什么、各方得失、指标怎么变。指标变化量由代码夹在各自的上下限内，行动没触及的指标保持 0。主持面板上还有「不等了，结算本轮」，把缺席的真人记成「(这一轮没有行动)」；居民还在思考时这个按钮不生效。

13.4.4 复盘

最后一轮结束后是一次复盘调用：按评分维度给每个角色打 1–5 分，逐条评估学习目标是否达成（yes / partly / no），回答复盘问题，最后给研究者几条洞见。可以下载 Markdown。

这里和前两个页签有个真实的差别，值得说清楚：严肃游戏**没有统计结论**——没有 ATE、没有拟合度分、没有「哪一版更好」要选。所以代码没有可判的东西，它守的是回合机制（谁还没行动、指标不越界、同时行动、轮次怎么进）。裁定和复盘

是模型的活。实际产物里能看到这一点：真实会话文件的 `debrief.scores` 数组有时是空的，因为复盘分数是模型回填的、不是代码保证的。读复盘的时候，别把模型给的分数当成测量值。

13.4.5 谁能看到什么

视图	打开方式	能看到
主持	侧栏「对局」或 `?tab=game&session=<id>`	所有角色的私密信息、居民的心里话、本轮已提交的行动、真人座位链接
玩家	座位链接 `?tab=game&session=<id>&seat=<token>`	公开信息和自己角色的私密信息；本轮别人的行动在结算前看不到

13.4.6 落盘与接口

- 设计存 `output/research/serious_games/<id>.json`，对局存 `output/research/serious_games/sessions/<id>.json`，每次状态变化都写盘，不会写回任何城市。
- 进程重启时正在进行的模型调用会丢失；下次打开该对局会从「裁定中 / 复盘中」继续。
- 接口在 `/api/research/games/*`：`GET /` 列游戏与对局，`POST /design` 设计（返回 `job_id`），`GET /jobs/<job_id>` 看进度，`GET /<game_id>` 取设计，`POST /<game_id>/update` 覆盖设计，`POST /sessions` 开一局，`GET /sessions/<id>` 看对局（带 `?seat=` 就是玩家视图），`POST /sessions/<id>/act` 提交行动，`POST /sessions/<id>/resolve` 强结本轮，`GET /sessions/<id>/export` 导出。

13.5 三个页签怎么选

页签	它回答的问题	判定由谁做	产物
政策仿真与优化	这条政策怎么改更好	代码按综合分选版本	`output/research/policy/`
社会调查/实验拟合度	这群居民像不像真人	代码算误差、区间、p 值和拟合度分	`output/research/replication/`
严肃游戏	人在这个情境里会怎么想、怎么谈	没有统计结论；代码守回合机制	`output/research/serious_games/`

按问题类型选：

- **优化问题**用政策仿真：「这条政策怎么改更好」。前提是你接受「居民反应是情景推演、样本小、适合比版本不适合给比例」这条限制。
- **验证问题**用复制拟合度：「这群居民像不像真人」。你手上有文献，想知道智能体能不能顶替真人受访者。它的答案只在文献报告过的那些量上有效。
- **过程问题**用严肃游戏：「人在这个情境里会怎么想、怎么谈」。你要的是协商过程、策略和话术，最好还能把人拉进来一起玩。产出是叙事和洞见，不是分数。

三个页签都遵守同一条底线，但落点不同：政策仿真是「模型提建议、代码挑赢家」，复制拟合度是「代码算全部数字、模型只许引用不许新造」，严肃游戏是「模型扮、模型裁、代码守规则」。凡是要下结论的地方，结论都不由模型宣布。

本章小结

- 三个页签都在 `output/research/` 下各占一个目录（`policy/`、`replication/`、`serious_games/`），都不写回城市包，后端只做任务调度和落盘。
- 政策仿真里所有版本跑在**同一批居民**上，比的是配对差；综合分 = 支持度 35% + 生活 30% + 钱包 15% + 遵从 20%，由代码算，推荐版本由代码选，模型只提建议。
- 复制拟合度只使用**文献报告过的**量，文献没报告的留空、不估算，因此「拟合度」衡量的是在可比的量上有多接近，而不是整体像不像真人。
- 拟合度的三组数字（逐格误差、真人值是否落在 95% 区间内、处理效应的方向与显著性）全部由代码计算；模型解读必须引用代码给的行，不许自己算新数字。
- 严肃游戏让居民智能体和真人混坐一桌，每轮同时行动、由主持人裁定；座位链接只是身份标记不是鉴权，复盘分数是模型给的、不是测量值。
- 三者的共同底线：模型负责扮演与解读，判定和打分由代码做。

动手练习

1. **跑一轮政策仿真，并解释为什么必须是同一批居民**（约 30 分钟） 打开研究工作台的「政策仿真与优化」页签，选一座城市，用一句话写一条原政策（例如「自明年 1 月起，全市生活垃圾按重量计费，每公斤 0.5 元，混投一次罚款 300 元」），再写一个候选政策（例如「按户定额每月 20 元，分类达标的小区每季度返还一半，不设罚款」），保留「用推荐的修订版再仿真一次」的勾选，跑一轮。 在结果页找到三个版本（原政策 / 候选政策 / 推荐修订版）的综合分和推荐标记。**验收标准**：能说出推荐版本是哪一个，并指出它是代码按综合分选出来的而不是模型说的；能解释「如果每个版本各抽一批居民，A 比 B 高出的那几分里就混进了抽样差异，无法归因给政策」；能指出导出文件末尾那句「不是民意调查」的免责说明，并说明为什么在十几人的样本上不能把支持率当成民调结果。
2. **走一遍复制拟合，找出真人值落在智能体 95% 区间内还是区间外**（约 40 分钟） 找一篇报告了真人社会调查或实验结果的论文（要有题目、条件和各组的结果数字），在「社会调查/实验拟合度」页签上传或粘贴正文，抽取研究设计，**逐项核对**抽出来的数字与出处（至少核对一个真人结果的数值和它的 `source`），然后选一座城市、选 20–50 位居民、选组间或配对，跑一次。在「逐题对照」表里逐格找一题，看「真人值在 95% 区间内」那一列是「是」还是「否」。**验收标准**：能指出至少一题是「否」（在 95% 区间外），并解释这意味着这批居民与真人的差距大于抽样波动能解释的范围，属于系统性偏离；能说出拟合度分在这次运行里是多少、它由哪两项构成（水平准确度与效应方向一致率，各占一半）；如果文献报了处理效应，能在「处理效应」表里指出每个效应被判成了「复现 / 方向一致但不显著 / 未复现 / 方向相反且显著」中的哪一档，并说出这个判定的 p 值从哪来。

ewpage

第 14 章　大规模人群：参数化人口与群体模拟

50 个居民跑一次个体仿真，是一次实验；5 000 个居民按个体跑，是一次算不完的实验。GAWorld 为这件事准备了两条独立的路：一条是**参数化人口合成**——用一组人口学旋钮造出一整座小镇，让「造得出来」本身可控可复现；另一条是**群体（cohort）模拟**——把人口压成几十个群体，每群每天只花一次模型调用，让「跑得便宜」成立。本章讲清两条路各自解决了什么、边界在哪里，以及那条把它们串起来的验证门。

为什么需要这一层

个体模式下，每个居民每天要走多次完整认知管线。GAWorld 按每人每天 198 次调用核算成本（48 个 tick × 4 次调用 + 6 次日界调用），所以 500 人跑一天是 10 万次调用，5 000 人跑一天就是一百万次。这个数量级下，研究者能做的不是「设计一个大样本实验」，而是「在一小撮人上反复试参数」。

规模问题其实分成两个独立的难题。第一个是**造得出**：「我想要一个有 34% 老年人、基尼 0.36 的县城」——光靠 LLM 一个个捏居民，规模和结构都不可控，捏 500 个和捏 50 个的边际成本也差不多。第二个是**跑得起**：多数研究问题（政策会怎样改变整体分布、谁落在分布尾部）根本不关心每个人的第 37 个 tick 在想什么。

参数化人口合成解决第一个，群体模拟解决第二个。顺序不能反：群体模拟要划分群体、挑实体化个体、用社交图做耦合，都依赖一份结构已知的人口。

参数化人口合成

旋钮契约：把面板级参数变成一座完整的小镇

旋钮的唯一事实源是 `gaworld/population/schema.py` 里的 `PopulationSpec`，分七组：人口结构（`demography`）、家庭（`household`）、教育与就业（`education_work`）、收入（`income`）、地理（`geography`）、心理状态初值（`psychology`）、社交网络（`social_network`）。面板通过 `GET /api/population/schema` 读它，不在 JavaScript 里另抄一份——九个状态变量本来就已经在两个地方各声明过一次。

旋钮组	代表什么	关键字段
`demography`	年龄金字塔与人口流动	`median_age`、`share_under_18`、`share_over_65`、`migrant_share`
`household`	家庭结构	`mean_size`、`share_single_person`、`share_multigen`、`share_shared_rental`
`education_work`	教育与就业	`tertiary_rate`、`employment_rate`、`unemployment_rate`、`industry_mix`
`income`	收入分布	`median_monthly`、`gini`、`pareto_tail_alpha`
`social_network`	社交图形态	`mean_degree`、`homophily_strength`、

旋钮组	代表什么	关键字段
		`geo_decay`、`workplace_size_alpha`

规模范围被夹在 20–5 000 之间，年龄下限是 6 岁（`min_agent_age`）：GAWorld 的居民要排日程、持观点、做反思，把幼儿交给认知管线没有意义，所以 0–17 那一档从 6 岁起采样，以下的人口不表示。

可行性预检：参数不是互相独立的

`check_feasibility()` 是一份纯数学检查，廉价到可以在面板每次按键时跑一遍，它从不抛异常，只返回 error 与 warning 两类问题。原因是这些旋钮之间存在数学约束，最典型的是户均规模：

```
1·s + 2·(1−s)  ≤  户均规模  ≤  1·s + max_size·(1−s)
```

`s` 是单人户占比。单人户占 60% 时，户均规模的可行区间是 [1.4, 3.0]，要求 3.2 就不可能。我用一份故意冲突的 spec 实跑了一次，报错长这样：

```
[error] household.mean_size: 单人户占比 59% 且最大户规模 6 时，户均规模最多是 3.03，但设定为 3.20。
        建议：把 mean_size 降到 3.03 以下，调低 share_single_person，或放大 max_size。
```

同类检查还有：中位年龄在两个年龄占比被钉死后并非自由（可达区间由 `median_age_bounds()` 算出）、就业率与失业率之和不能超过 100%（两者都按劳动年龄人口定义）、三代同堂户不能超过老人供给。工具报的不是「你的参数不对」，而是「这组参数互斥，该动哪一个」。

三个技术要点

复述得出来就够用的版本：

IPF 拟合联合分布，含结构性零。 合成器先建一张 `年龄档 × 性别 × 教育 × 就业 × 行业` 的五维联合表，再用迭代比例拟合（IPF）把它的边缘分布压到目标值上。关键在于表里的**结构性零**——未就业的人没有行业，未成年人不进劳动年龄段也不该有大专学历，65 岁以上的人状态是「退休」而不是「失业」。IPF 会保持零值不变，所以把这些不可能的组合在拟合前就编码成 0，是让「8 岁的软件工程师」不出现的唯一办法。边缘目标里 `employment` 用的是**（年龄档 × 就业状态）联合边缘**而不是一维边缘，否则 IPF 会靠把老人拉去上班来凑就业总数，工作年龄就业率就悄悄掉下来了。

最大余数法整数化，让边缘精确命中。 拟合出的联合分布是概率表，要变成 500 个人。如果独立抽 500 次多项分布，每个边缘的偏差是 `√size` 量级——N=500 时每个类别差 ±11 人，面板上写着「就业率 68%」结果数据里是 62%。所以合成器用最大余数法分配整数名额：先向下取整，剩下的名额按小数部分从大到小补齐（同分用带种子的抖动打破），最后打乱顺序。

收入用秩变换，中位数与基尼严格成立。 每个在业者先算一个属性驱动的隐含分（教育等级、行业溢价、经验、噪声加权），再把这个分**排序转成秩**（`ranks[i] = (rank + 0.5) / n`），用秩去查一条预先标定好的收入分位函数：主体是对数正态，超过 `tail_threshold_pct`（默认 95%）后接一段帕累托尾、在拼接点按水平对齐保持连续。收入分布的中位数与基尼由这条分位函数**保证**，而个人收入仍与教育、行业、年龄相关——因为决定名次的那个分就是属性驱动的。基尼还有个额外处理：`lognormal_sigma_from_gini()` 直接反解闭式解会偏高几个点（帕累托尾额外贡献了不平等），所以合成器改在对**拼接后**分布的实测基尼上二分 σ，面板上的数字就是数据里的数字。

家庭、工作单位与社交图

儿童优先建户。 家庭先建：儿童先进单亲户或核心户，父母按配偶年龄差配对进核心户或三代同堂户，其余成年人进单人户、夫妻户或合租户。顺序反过来就得靠回填修补，容易出现「有小孩但没有成年人」这种一户无依的怪家庭。

工作单位的规模是幂律的。 `build_workplaces()` 在每个行业内部按帕累托抽样切分公司规模（指数由 `workplace_size_alpha` 控制），少量大雇主加一长尾小公司。工作单位存在的意义是**生成关系**——一个行业一坨 blob 造不出真实的同事图。

社交图是同质性的。 边的来源有四类，按优先级排：家庭（亲属/室友）→ 工作单位（≤6 人的公司近完全连接，更大的切成 5 人小队，免得 60 人的雇主造出 1770 条边）→ 同址邻居 → 选择性友谊。最后一类按 `homophily × 地理` 抽样，并做 Watts–Strogatz 式随机重连（`rewire_p` 默认 0.10），让图保住小世界路径长度而不是碎成一堆团。同质性打分是年龄 0.30 + 教育 0.20 + 同行业 0.25 + 同籍贯 0.25，再按 `homophily_strength` 在「人人平等」和「相似者抱团」之间插值，最后过一遍 Dunbar 上限。

产出：与现有格式完全一致

```
# 预览，不写任何文件
python -m gaworld.population --size 500 --seed 42 --check

# 写出三个文件
python -m gaworld.population --size 500 --seed 42 --name my_town --
        out data/town
```

`--check` 的真实输出（我在本机跑过）：

```
生成完成：500 人 | preset=cn_county_town seed=42
  IPF：2 轮，收敛，最大边缘偏差 0.0006
  关键指标（目标 → 实际）：
    median_age                36.000 →     36.000
    employment_rate            0.680 →      0.692
    tertiary_rate              0.350 →      0.355
    income_median           6500.000 →   6500.000
    income_gini                0.420 →      0.414
    household_mean_size        2.600 →      2.604
    mean_degree               12.000 →     12.000
  社交网络：3000 条边，平均度 12.0，聚类 0.115（随机图 0.024），平均路径 2.8
3，small-world σ=4.2
  被松弛的旋钮（相对误差最大的几个）：
    share_over_65          要求 0.14 → 实际 0.132  （偏差 5.7%）
 （--check：未写出文件）
```

「被松弛的旋钮」是这一层的诚实之处。 达不成的参数会被显式列出来，而不是假装达成。上例里 65 岁以上占比差 5.7%，是整数化取整造成的，可以接受；若某个旋钮偏差 30% 以上，说明参数组合本身有内在冲突，该改参数。

写出时落三个文件：`<name>_state_init.csv`（带 BOM，编码 utf-8-sig，因为仿真器用 BOM 感知的编解码器读它）、`<name>_profiles.md`、`<name>_manifest.json`。CSV 的列序是 `id,name,gender,age,hukou,residence,employment,industry,monthly_income` 加九个状态变量，和 `data/hangzhou_agents_state_init.csv` 完全一致——**这就是为什么 `build_agent` 不需要任何改动**。manifest 存了完整 spec、IPF 收敛报告和校验结果，是复现这批人唯一的凭据。

写完之后还有一道**校验门**（`report.py` 的 `validate_population`）：未成年在业、未成年人有大专学历、未就业却有工资性收入、在业但无行业归属、状态变量越界、id 重复、家庭分配不完整、户里有小孩却没成年人、关系数超 Dunbar 上限——任何一条 error 都会让输出**不落盘**。我故意造了一份冲突参数，它确实被拦下了：

```
校验未通过，未写出文件。
...
  [error] household.mean_size: 单人户占比 59% 且最大户规模 6 时，户均规模最多是 3.03，但设定为 3.20。
  [error] dunbar_cap_exceeded: 关系数超过 Dunbar 上限（enforce_dunbar 未生效）（179 例）
```



图 14–1　Population Studio：先定人口结构，再谈模拟 这是控制台「人口与群体」页签的第 1 步。选好人口模板、居民人数和随机种子后，右侧的「参数体检」面板会实时给出可行范围——图里写着「户均人数只能在 1.8–4.8 之间；中位年龄只能在 26–60 岁之间」，并注明超出范围是因为这些参数互相牵制、不是 bug。这块面板背后就是上面那个 `check_feasibility()`，纯数学、不生成任何人、跟着滑块即时更新。整条路径可以零成本走完：第 1 步选模板，第 2 步看人口结构，第 4 步跑模拟（默认不调真实 LLM），第 5 步看验证结论。

预设：每一种代表一种社会结构

`--preset` 的可选值从 `--help` 里抄是：`aging_community`、`cn_county_town`、`cn_tier1_district`、`college_town`、`custom`、`us_suburb`。它们各自的含义：

preset	中位年龄	就业率	收入中位数 / 基尼	什么社会结构
`cn_county_town`	36	0.68	6500 / 0.42	基准线：普通县城，服务与商贸占一半以上
`cn_tier1_district`	34	0.74	11000 / 0.46	一线城区：更年轻、高教育、tech 30% + finance 14%
`aging_community`	52	0.42	4200 / 0.36	老龄社区：65+ 占 34%，医疗

preset	中位年龄	就业率	收入中位数 / 基尼	什么社会结构
				18%、服务 46%
`college_town`	27	0.35	4500 / 0.34	大学城：高教 80%、教育业 34%、合租 45%
`us_suburb`	39	0.63	5800 / 0.48	美国郊区：少儿 23%、高基尼、合租仅 5%

每个预设自身都通过可行性检查，有测试锁住。`custom` 是空白起点：`normalize_spec()` 会把预设默认值、你的覆盖、范围夹取依次套上，未知键被忽略而不是报错，旧版面板连新版后端时优雅降级。

群体（cohort）模拟

一个群体同时携带均值与离散度

群体是属性空间里的一个格子。默认按 `年龄档 × 行业 × 户籍` 三轴划分——年龄决定日常节奏，行业决定经济暴露，户籍决定权益与归属，这三轴是现有状态变量最敏感的维度。500 人的镇落进 30–40 个群体（我实跑得到 38 个，规模 4 到 50 人）。可用轴还有 `employment`、`gender`、`district`。

关键在 `Cohort` 同时带两个统计量：`centroid`（各状态变量的组内均值）和 `dispersion`（标准差，`ddof=0`，因为这是一个被完整观测的群体的总体统计量而非对更大总体的估计）。只留均值就是 Kirman (1992) 批判的「代表性个体」：异质个体的聚合一般不等于任何单个个体的行为，一个忘掉自己离散度的群体会做出成员谁都不会做的决定。留着离散度，群体简报才能说「这个组大约三分之一经济压力明显偏高，其余还算安稳」，而不是「本组 econ_security = 0.5」。每个居民恰好落进一个群体（划分是个覆盖，不重复、不遗漏）；少于 4 人的格子并进质心距离最近的群体——一个人的「群体」只是白付开销，离散度也没有意义。

每群每天一次 LLM 调用

`simulate_cohort_day()` 对每个群体发一次调用，拿到一份归一化的当日摘要（`brief` / `divergence` / `memory` / `state_changes` / `share_affected`）。摘要里的变化量还要乘以 `share_affected`——模型说「冲击影响了组内 40% 的人」，组均值就该只移动 0.4 倍的单人效应；原样施加会把影响高估 2.5 倍，而这种误差每天累积。

摘要推给成员时，每个人拿到的是**同一个位移**而不是组均值。加上同一个常数只会平移分布、不改变标准差，所以群内离散度天然保住；要是把组均值赋给每个成员，一天下来整个群体就塌成一个点。调用失败不会抛异常，会降级到确定性 fallback——一个群体的日子崩掉不该带走一整片子人口。

实体化预算：谁被提升到完整个体保真度

每天从群体里挑一小批人按完整保真度跑，四类，优先级从高到低：

类别	谁	说明
`focal`	你点名跟踪的居民	每天都实体化，预算被占满也不会被挤掉
`event`	今天卷入关键事件的	事件系统当天给出的名单

类别	谁	说明
`audit`	按 `audit_fraction` 分层随机抽样	**用来量误差，不是用来减误差**
`tail`	群体均值代表不了的离群者	按到群体质心的距离排序，远的优先

`focal` 与 `event` 即使超预算也照常实体化：悄悄丢掉研究者点名跟踪的人，是一种很难发现的失效。审计样本在各个群体间**分层**抽取，免得最大的群体被超采；它优先于尾部选择，因为一个没被测量的近似比一个稍微没那么精准的近似更糟。

审计残差是这套机制的核心度量：审计样本自己的变化减去它所属群体对它的预测，差多少。残差超阈值（`residual_alarm` 默认 0.08）就说明这个群体没描述好它的成员，第二天自动给它加采样（`audit_boost_factor` 默认 2，上限 8 倍），连续几天不告警再降回来。加样本回答的是告警引出的那个问题：残差大，是群体真没描述好，还是今天恰好抽到了那几个人？

群内零均值的社交图耦合

这一节是群体模拟能做网络相关问题的关键。不开耦合时，群体层给每个成员同样的位移，邻居之间的共变在聚合后就不存在了——验证门 L2 一定不通过。

`NetworkCoupling` 的设计是**所有权清晰切分**：

- **群体层拥有均值**——群体摘要预测了聚合怎么动，聚合就怎么动。
- **图拥有群内离散**——每个成员额外被「自己的邻居昨天怎么动的」推动一下，但**相对于本群体整体怎么动的**。

「相对于群体」这个框架是安全性的来源：耦合项在群体内**居中**，求和为零，因此它移动不了群体的均值。若直接加一个原始的邻居项，图就能悄悄推翻群体的预测，把本来已经通过的层（L1 分布级、L4 因果响应）置于风险中去救那层失败的。耦合项还有上限（`max_multiple` 默认 1.0 倍 `max_delta`），免得一个病态的邻域压过群体自己的预测。代价为零——纯是在已有社交图上做算术。

开启方式：

```
python -m gaworld.group --size 500 --days 7 --network-coupling 0.7
        --no-llm
```

两个防呆是刻意做的。`network_coupling > 0` 却不传社交图会直接抛 `ValueError`；CLI 用 `--population` 读 state CSV 时开耦合同样明确报错，因为 **CSV 里没有边**——我实跑过：

```
--network-coupling 需要社交图，而 state CSV 不含边。请改用 --size 现场生成人口。
```

传了强度却静默不生效，会让人以为修复起作用了，其实没有。

成本与产物

```
python -m gaworld.group --size 500 --days 7 --no-llm
```

真实输出：

```
群体模拟完成：500 人 → 38 个群体，7 天
  LLM 调用（实测）：0
  实体化个体-天：140
  全个体模拟估算：693000（按 198 次/人/天）
  成本比：本次未调用 LLM（--no-llm），故不可比
  审计残差 L1 最大：0.0000
```

`--no-llm` 下群体简报走确定性 fallback，整轮零成本。**审计残差恒为 0 是正确的**——独立驱动器的个体层是占位（诚实地返回零变化，而不是编造看起来合理的移动），预测为零、实际为零、残差为零。 产物方面，`--out` 写出一份 run 记录 JSON（`group_run.json`），含分群、每日名单、群体变化、审计残差、调用数与成本核算。

接进主运行

`python -m gaworld.group` 是独立驱动器，个体层是占位产物。要让控制台、平行世界、研究工作台直接读一次大规模运行，得在主运行里开群体模式：

```
# dashboard_config.json（或 GAWORLD_CONFIG_OVERRIDES）
#   "csv_path": "data/town/my_town_state_init.csv",
#   "md_path":  "data/town/my_town_profiles.md",
#   "agent_ids": 500,
#   "simulation_mode": "group",
#   "group": {"materialization_budget": 20, "audit_fraction": 0.03,
        "focal_ids": [7, 42]}
python generative_city_sim.py run --fast-forward --sim-days 30
```

只支持按天快进（`long_run.enabled = true`、`unit = day`），不满足就在开跑前报错——逐 tick 循环里每个人都要参与感知、社交和场所占用，没法只让一部分人走完整管线。每天的顺序是：日初挑出今天单独跑的人（焦点 → 审计样本 → 尾部），每个群体一次调用、把其他成员按群体变化一起平移；快进简报只给这几个人跑（普通快进的个体层，不是占位）；日终把结果并回群体统计，审计样本自己的变化减去群体预测就是当天的审计残差。

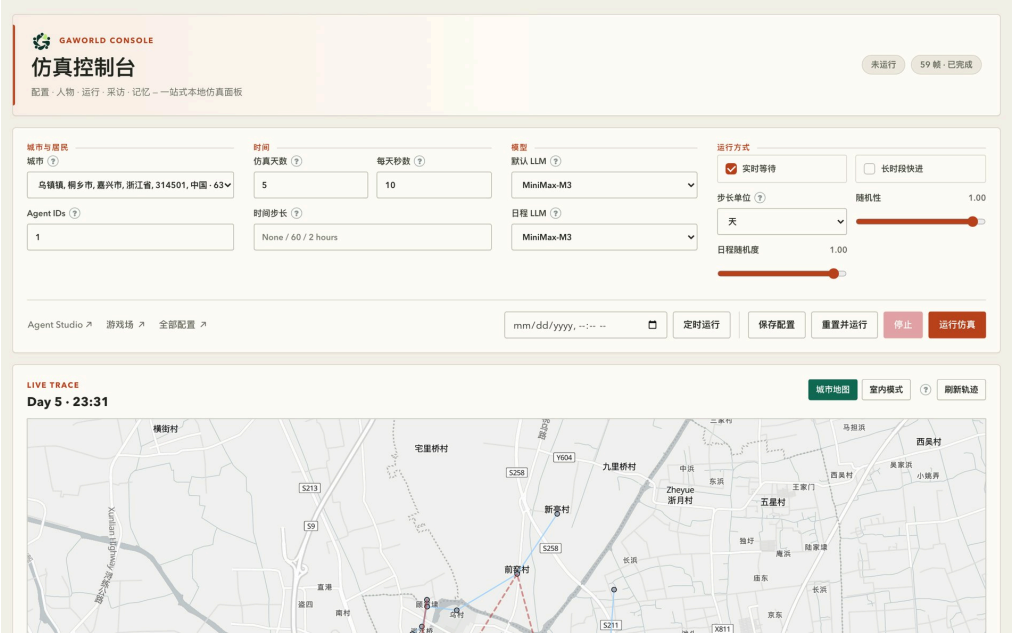


图 14–2　控制台：群体模式就接在这块配置面板里 这是仿真控制台展开后的全貌。左上是城市与 Agent IDs 选择，中间是时间步长与模型，右侧「运行方式」里的「长时段快进」和「步长单位」正是主运行群体模式要求的按天快进开关。把 `simulation_mode` 设为 `group`、Agent IDs 填满 500，这块面板跑出来的就是一次大规模群体运行——控制台、平行世界、研究工作台读它跟读普通运行没有区别，因为产物结构是一样的。

产物与普通运行相同：每个居民的状态历史（`agent_state_history.csv` 里 500 人都在）、运行清单，外加三张记录表——`group.partition`（开跑时的分群）、`group.day`（每天的名单、群体变化、审计残差、告警、当天调用数）、`group.summary`（全程统计）。

主运行里的群体模式不能用来研究网络扩散：群体里的人一起动，不看社交图——这正是 L2 不通过的原因，能修它的耦合项要用人口合成器给的社交图，主运行里没接。分布与政策效应（L1 / L4）可以看。

验证门：这个近似能回答哪类问题

这是整套流程里最重要的一步。 只看到「省了 25 倍」而不看判定结论，等于把群体模式当成免费午餐。验证门做的是**配对实验**：同一份人口、同一段日子，逐个体跑一遍当参照层，群体跑一遍当被测层。

```
python -m gaworld.group.validate --size 100 --days 14 --network-cou
        pling 0.7
```

四层判定

层	测什么	为什么单列
L1 分布级	关键变量边缘分布的 Wasserstein-1 距离	比均值会放过「分布塌成一个点但位置正确」的近似
L2 网络级	邻居间状态共变（Moran's I）	度分布、聚类是静态图属性，两层天然一样，测了同义反复
L3 尾部	极端个体占比、分位区间宽度、首次越阈时间	聚合近似的已知失效模式正是保住主体、压扁尾部
L4 因果响应	同一政策冲击下的 ATE 方向、量级、子群异质性	基线全对但 ATE 反号的近似，用于政策研究会得出相反结论

L2 与 L4 是**分水岭**：这两层不过，`gate_passed` 就是 false，退出码 1，可以直接进 CI。

阈值不是拍脑袋的常数

L1 和 L2 的判定都**相对于实测基线**：先把参照层跟它自己跨种子跑几遍（默认种子 1、2、3），量出它本身的 run-to-run 波动，近似误差落在同一量级内才算通过。

```
判定 = |群体 I − 参照 I| ≤ 2.0 × 参照层跨种子标准差
```

量难测时容差自动放宽，好测时自动收紧。绝对常数（比如「比值必须在 0.5–2.0 之间」）在这里行不通——设计文档里记了实测：那个带宽跟估计量本身的噪声同量级，判定会随抽到的种子翻转，有一版固定耦合下 5 组种子过了 3 组，那是掷硬币装成测量。

L2 还有一层保护：参照信号低于噪声地板（`0.05`）的变量不参与判定——拿两个接近 0 的数相除毫无意义。四个变量全部低于地板时，整层报「无法判别」：既不是通过也不是失败，而**分水岭层无法判别时关口不算通过**。

实跑一次的结果

我在这台机器上跑了上面那条命令（100 人、14 天、耦合 0.7），四层全过，退出码 0：

```
═══ Group 模式验证门 ═══  人口 100 | 14 天
参照层跨种子基线（Wasserstein-1 最大）：0.0441 | 配对实验种子：[1, 2, 3]

✅ 通过  L1  分布级：边缘分布距离是否落在参照层自身的种子间噪声量级内
        ✓ emotion          W1=0.0240（预算 0.0335）sd比=0.86
```

```
        ✓ stress            W1=0.0229 (预算 0.0413) sd比=0.85
        ✓ econ_security     W1=0.0268 (预算 0.0548) sd比=0.89
        ✓ city_identity     W1=0.0278 (预算 0.0653) sd比=0.89

✅ 通过  L2  网络级：邻居间的状态共变（Moran's I）是否落在参照层自身噪声量级内
          emotion           参照 I=+0.035  群体 I=+0.139  （参照信号低于噪声地板，不参与判定）
          stress            参照 I=+0.069  群体 I=+0.090  比=1.28 z=0.87（容差 2.0σ，σ=0.025）
          econ_security     参照 I=+0.081  群体 I=+0.059  比=0.98 z=0.88（容差 2.0σ，σ=0.025）
          city_identity     参照 I=+0.079  群体 I=+0.089  比=1.07 z=0.43（容差 2.0σ，σ=0.025）

✅ 通过  L3  尾部与稀有事件：极端个体占比、分位区间宽度、首次越阈时间
          stress            分位宽度比=0.89  低尾 0.003→0.000  高尾 0.037→0.017

✅ 通过  L4  因果响应：同一政策冲击下的 ATE 方向、量级与子群异质性
          ATE 参照=-0.3919  群体=-0.3997  同号=True  相对误差=2.0%
          子群异质性保留=1.13  子群符号一致率=100%

关口结论（L2 + L4 为分水岭）：✅ 通过
全部四层：✅ 通过
```

读法：`z` 是标准化偏差，**这才是判据**；`比` 只是给人看的诊断。L4 那行最要紧——ATE 参照 −0.3919、群体 −0.3997，同号、相对误差 2.0%，子群符号一致率 100%，这意味着「用群体层做政策实验，得到的冲击效应和逐个体跑出来的是同一个东西」。

这个门验证了什么、没验证什么

参照层是**逐个体**跑同一个透明的传染过程。所以它衡量的是「把个体聚合成群体」这一步的误差，**不等于**群体模式对完整 12 阶段 tick 流水线的复现度——后者要读只有 `run_simulation` 才凑得齐的整轮运行状态，从外部调不到，也没法进 CI。换句话说，验证门过了说明**聚合这个操作**本身可靠；真实 LLM 驱动的个体层跟参照层的行为是否一致，是另一个问题。

耦合强度怎么定

耦合强度不是越大越好。N=100、14 天、预算 20、3 种子下扫一遍（教程里的实测）：

耦合强度	0.0	0.5	0.7	0.9
L2 最差 z	3.87	2.26	**0.88**	4.14
关口	❌	❌	✅	❌

太小是传不动（欠传播），太大是传太猛（过传播），两头都算失败。**0.7 不是普适常数**——它是针对验证门内那个参照过程（传染权重 0.6）标定出来的。换成真实 LLM 驱动的个体层后必须重新标定，用同样的方法扫一遍，找 z 最小的那档。默认值是 `0.0`（关闭），需要显式开启。

什么时候能用、什么时候不能

这一节最要紧。群体模拟是一次**有结构的近似**，不是免费的个体模拟。它能不能用，不看你有多想要这个答案，看你要问的问题落在哪一层。

研究问题	适合吗
这个政策会怎样改变整体分布（收入、情绪、压力）	✅ 适合
谁在分布尾部、极端个体占比多少	✅ 适合
政策冲击的处理效应与子群异质性	✅ 适合（需 L4 通过）

研究问题	适合吗
观点扩散、极化、少数派引爆、信息级联	⚠️ 必须开网络耦合并跑验证门确认
单个居民的完整生活叙事	❌ 用个体模式，或把 TA 设为 focal

能：分布层面的问题。 这是群体模式的主场。「这项补贴会把这座城的压力分布推成什么形状」「尾部有多少人从安全掉进不安全」——问的是分布，而分布恰恰是群体层能忠实复现的东西。L1 与 L3 守的就是这个。

不能：网络扩散。 群体模式保不住个体级的传播路径。信息从谁传到谁、经过几跳、哪条边是关键路径——这些在群体层根本不存在，因为群体里的人一起动。开了耦合也只是让「邻居共变」这个二阶统计量在聚合后还在，不等于保住了路径。**个体被实体化之前不是真实的个体**，他们的行为轨迹是群体的平移，谁先被影响、沿什么路径扩散，全都读不出来。

不能：个体级因果效应。 同一原因。实体化之前，一个人在群体里扮演的是「这个格子的典型成员」加上「他离格心有多远」，不是一个有自己经历、记忆和关系史的主体。所以「政策对**这个人**的影响」在群体模式下是不可问的——不是问得不准，是问的东西不存在。想跟住某个人，用 `--focal 7,42` 把他设成焦点居民，他每天都按完整个体保真度跑。

这个门测的是什么，就只能用什么。 L1/L3 过了说明分布可以信，L2 过了说明网络级共变可以信，L4 过了说明政策效应可以信。哪层没过，那类问题就不要用这次运行的结果下结论。

成本对比与零成本预演

同样 500 人、30 天，逐次实测（基准：全个体 500 × 30 × 198 = 297 万次）：

实体化预算	cohort 层调用	实体化层调用	合计	相对全个体
0	1,140	0	1,140	**2605× 省**
10	1,140	59,400	60,540	49× 省
20（默认）	1,140	118,800	119,940	**25× 省**
50	1,140	297,000	298,140	10× 省

结论：cohort 层几乎免费，成本几乎完全由 `--budget` 决定。 调群体粒度对成本影响很小——群体数是几十量级，实体化人数是几十量级，但后者乘的是 198 倍的个体开销。

零成本预演的做法是 `--no-llm`：群体简报走确定性 fallback，个体层是占位，整轮零 LLM 成本。它适合调参数、看划分形状、估成本、看审计残差随预算怎么变，**不适合**用它的残差判断近似好不好——个体层没在动，残差恒为 0 是正确的。

另外一条与群体无关但同样值得知道的性能修复：个体模式的 tick 数随 agent 数**超线性**增长，因为主时间线是固定网格与每个 agent 的 LLM 自拟日程时间的**并集**，而 LLM 生成的 `HH:MM` 没有对齐逻辑。开 `CONFIG["time_grid_snap"] = True` 并同时设 `CONFIG["time_step_minutes"] = 30`，tick 数恒等于 `1440 / step`，与人口规模无关。默认关闭，因为它会改变日内时序。

实践建议：三步走

1. **先 `--check` 预览人口结构，不花钱。** `python -m gaworld.population --size 500 --seed 42 --check` 几秒钟出结果。读三样东西：IPF 收敛了吗（几轮、最大边缘偏差多少）、目标到实际的偏差（哪些旋钮被松弛了、偏差多大）、社交网络指标（平均度、聚类、平均路径、small-world σ）。偏差在几个百分点内是取整造成的正常现象；某个旋钮差 30% 以上，回去改参数。

2. **再用 `--no-llm` 跑通群体逻辑，仍然不花钱。** `python -m gaworld.group --size 500 --days 7 --no-llm`。看群体数对不对（`--axes` 决定）、成本账算出来对不对、`--focal` 指定的人是不是天天在名单里、审计残差是不是如预期为 0。这一步把「命令写错了」和「模型行为不对」彻底分开。
3. **最后开 LLM 跑正式实验。** 先跑一次验证门（`--size 100 --days 14 --network-coupling 0.7`），读它的四层判定，再扫一遍耦合强度找 z 最小的档。验证门不通过就**不要**拿群体结果下结论——那不是「精度不够」，是「这层问题这个近似答不了」。

本章小结

- 大规模问题的两半是分开的：参数化人口合成解决「造得出」（旋钮契约 + 可行性预检 + IPF 拟合 + 最大余数整数化 + 收入秩变换），群体模拟解决「跑得起」（每群每天 1 次调用 + 按预算实体化少数个体）。
- 产出与现有格式完全一致：状态 CSV（带 BOM）、profile Markdown、manifest 三件套，`build_agent` 无需改动，所以生成的人口可以直接填进 `CONFIG["csv_path"]` / `CONFIG["md_path"]`。
- 一个群体同时携带均值**与**离散度；只留均值就是「代表性个体」，会让群体做出成员谁都不会做的决定。
- 群内零均值的社交图耦合是群体模式能做网络相关问题的前提：它让邻居共变在聚合后依然存在，且因为在群体内居中，**移动不了群体均值**，不会把已经通过的层拖下水。
- 验证门 L1–L4 用参照层自身的跨种子噪声当尺子，而不是拍脑袋的常数；L2（网络级）与 L4（因果响应）是分水岭，不过时退出码为 1，可直接进 CI。
- 分布层面的问题可以用群体模式；**网络扩散**与**个体级因果效应**不可以——个体被实体化之前不是真实的个体。

动手练习

1. **预览一座 500 人的镇，并解释它的数字从哪来**（约 20 分钟） 跑 `python -m gaworld.population --size 500 --seed 42 --check`，把输出读懂。记录：IPF 几轮收敛、最大边缘偏差多少；年龄分布与目标中位年龄的差距；收入中位数与基尼的实际值；「被松弛的旋钮」里偏差最大的几项及其百分比。然后用 `--preset aging_community --size 500 --seed 42 --check` 再跑一次，对比两份输出里被松弛的旋钮怎么变。 **验收标准**：能说清「中位年龄为什么 36 变成 36 而 `share_over_65` 14% 变成 13.2%」（前者是分段均匀中位数在年龄段内可调、后者是 500 人里整数化后 66 人而不是 70 人）；能解释为什么 IPF 需要结构性零（否则会抽出 8 岁的软件工程师）；能解释秩变换为什么能同时做到「中位数与基尼严格成立」和「收入仍与教育行业相关」；能说出最大余数法若换成独立多项抽样，边缘偏差会有多大（N=500 时每个类别约 ±11 人）。
2. **跑通群体逻辑，再让验证门给结论**（约 40 分钟） 先跑 `python -m gaworld.group --size 500 --days 7 --no-llm`，记下群体数、实体化个体-天、全个体估算。再跑 `python -m gaworld.group --size 500 --days 7 --focal 7,42 --no-llm`，确认焦点居民天天在名单里。最后跑 `python -m gaworld.group.validate --size 100 --days 14 --network-coupling 0.7`，记录 L1–L4 各层的判定与关口结论，并用 `echo $?` 确认退出码。再把 `--network-coupling` 改成 `0` 重跑一次，对比 L2 的 z 值怎么变。 **验收标准**：能指出群体数（38 左右）与 `--axes` 默认三轴的对应关系；能解释为什么 `--no-llm` 下审计残差恒为 0 是正确的而不是「近似完美」；能指出 L2 在耦合为 0 时是否通过、z 值变成多少；能解释为什么 L4 那行的「同号=True」比「相对误差 2.0%」更关键；能说明退出码为什么能直接进 CI。

ewpage

第 15 章　插件开发：写一个新子系统，不改主循环

前面十几章讲的都是怎么用：装平台、接模型、配城市、跑仿真、进面板。这一章讲另一件事——当 GAWorld 现成的子系统不够用时，你想加一种新的环境事件、一条新的信息流、一套新的动作约束，怎么加，才不会把主循环改成一团意大利面。

答案在 GAWorld 里是写死的：**新子系统一律写成 `gaworld.kernel.Plugin` 的子类，挂在事件总线上，主循环一行不改。** 这不是一句口号，仓库里有测试守着：文件 `generative_city_sim.py` 和函数 `run_simulation` 都有行数上限（`tests/test_line_budget.py`，预算在 `tests/line_budget.json`），超了 CI 直接失败；把代码搬出去之后如果空出来的行数超过上限 50 行，测试还会反过来要求你把上限调低，防止空间被悄悄填回来。

15.1 架构约定：微内核，以及”主循环零改动”为什么是硬规矩

自 2026-07 起，GAWorld 运行在 society-centric 微内核架构上（设计文档 `docs/proposals/2026-07-11-microkernel-plugin-architecture.md`，参考 Agent-Kernel 技术报告 arXiv:2512.01610）。整个仓库被切成四层：界面层（`site/`）、API 层（`gaworld/apps/`）、微内核（`gaworld/kernel/`）、认知管线（`gaworld/sim/pipeline.py`），领域子系统全部以插件形式挂在最下面一层。



图 15-1　GAWorld 的分层：领域子系统全部挂在事件总线上 微内核层只有六件东西——`Clock`（仿真时钟）、`EventBus`（事件总线，三种钩子语义）、`PluginRegistry`（插件注册表）、`Controller`（动作校验 + 可审计运行时干预）、`Recorder`（统一事件流）、`SimContext`（运行时上下文）。内核里不允许出

现任何领域逻辑，`gaworld/kernel/__init__.py` 的模块文档里写得很直白："Domain logic must never be imported here." 所以你写一个经济模型或一套家庭规则，内核既不知道也不需要知道。

内核的六件东西各自对应一个问题：`Clock` 回答"现在几点"（只有主循环能推进它）；`EventBus` 回答"谁想知道刚刚发生了什么"；`PluginRegistry` 回答"谁来了、谁先装配"；`Controller` 回答"这个动作准不准做、谁有权临时改世界"；`Recorder` 回答"把发生过的事写成一条统一格式的时间线"；`SimContext` 回答"运行时你能合法碰到的所有东西"。

规矩有两条，都写在 AGENTS.md 里

第一条：**新代码只写进 `gaworld/` 包，不添加新的根目录模块。** 仓库根目录只允许有 `generative_city_sim.py` 这个 CLI 入口，旧版 flat 模块搬到了 `legacy/`（不参与构建）。你写的插件应该放在 `gaworld/<你的子系统>/plugin.py` 这样的位置。

第二条：**新子系统写成插件，不在 `generative_city_sim.py` 里加内联逻辑。** 这条规矩和行数上限是同一件事的两面。行数上限只是症状，真正的病是"主循环内联调用一切"——早期架构里经济、行为、兴趣这些子系统全靠 `if CONFIG.get("xxx", {}).get("enabled")` 加直接函数调用接进 `run_simulation()`，新增一个子系统就等于改一次主循环源码，改到最后没人敢动。插件化把这件事反转过来：主循环只负责按顺序发射事件，谁来听、听了做什么，是插件自己的事。

这条规矩对写插件的人意味着一个很具体的约束：**你的逻辑不能长在主循环里。** 想在某个时刻做事，就去订阅那个时刻对应的事件；想改某个东西，就去注册一个校验器或干预；想加新的一步认知，就在 `CONFIG["pipeline"]` 里插一个阶段名，而不是去改 `_stages.py`。

15.2 最小插件：一份能直接读懂的代码

插件的基类在 `gaworld/kernel/registry.py`，它给你的东西很少，也就够了：

成员	作用
`id: str`	唯一标识。同时是配置命名空间、per-agent 状态命名空间、输出目录名、Recorder 表名前缀
`requires: tuple[str, ...]`	依赖的其他插件 id，决定 `setup` 的先后顺序
`setup(ctx)`	装配时调用一次。订阅事件、注册校验器、注册干预都在这里做
`teardown(ctx)`	仿真结束时调用一次，逆序执行。用来关文件、停线程
`output_dir(ctx)`	返回本插件独占的输出目录 `output/<id>/`

`ctx` 是 `SimContext`（`gaworld/kernel/context.py`），一个 dataclass，字段是：`config`（当前配置）、`clock`（时钟）、`bus`（事件总线）、`registry`（注册表）、`controller`（校验与干预）、`recorder`（事件流）、`agents`（居民列表）、`agents_by_id`（按 id 索引）、`llm`（模型调用入口）、`rng`（本插件的随机流）、`extras`（城市地图之类的共享对象）。

下面这份代码是可以直接跑的最小插件。字段名和上面这张表、以及 `SimContext` 的字段完全一致：

```python
# my_pkg/rumor.py
from gaworld.kernel import Plugin


class RumorPlugin(Plugin):
    id = "rumor"                    # 唯一标识 = 配置/存储/agent 状态命名空间
    requires = ()                   # 依赖的其他插件 id（决定 setup 顺序）

    def setup(self, ctx):
        # ctx 是 SimContext：config / clock / agents / bus / controller /
        # recorder / registry / llm / rng
        ctx.bus.on("on_day_start", self.on_day_start)
        ctx.bus.on("perception.compose", self.inject_rumor)

    def teardown(self, ctx):
        pass

    def on_day_start(self, hook_ctx):           # observe：返回值被忽略
        pass

    def inject_rumor(self, hook_ctx):           # collect：返回 0..n 条贡献
        agent = hook_ctx["agent"]
        ctx = hook_ctx["sim"]                   # 每次分发都携带 SimContext
        state = ctx.agent_ext(agent, self.id)   # 本插件的 per-agent 命名空间
        if state.get("heard_rumor"):
            return ["有人跟你提起：城东要修新地铁线（真实性存疑）"]
        return None
```

它只做了三件事，正好对应插件的三个职责。

第一件：声明订阅什么。 `ctx.bus.on(event, fn, priority=10)`。事件名是字符串，写错了不会报错，只是永远等不到——这是插件最隐蔽的坑之一，事件名一定要照着事件目录抄（见 15.5）。`priority` 越大越先执行，同优先级按注册顺序。

第二件：收到事件做什么。 处理器拿到的 `hook_ctx` 是一个普通 dict，除了事件自己的键，还一定带一个 `sim` 键——那就是 `SimContext` 本身（内核在 `build_kernel` 里把 `bus.base_context["sim"] = ctx` 设好，每次分发都带上）。所以插件里不需要到处传 `ctx`，`hook_ctx["sim"]` 就有。

第三件：把结果交回总线。 交回什么，取决于这个事件是什么语义。`on_day_start` 是 observe 语义，返回值被丢掉，所以 `on_day_start` 返回什么都不影响运行；`perception.compose` 是 collect 语义，返回的字符串会作为一条"环境上下文片段"追加到这一步的感知文本里，进了下一阶段的提示词。上面这段代码里 `return None` 是"我这次不贡献"，`return ["……"]` 是"我贡献一条"。

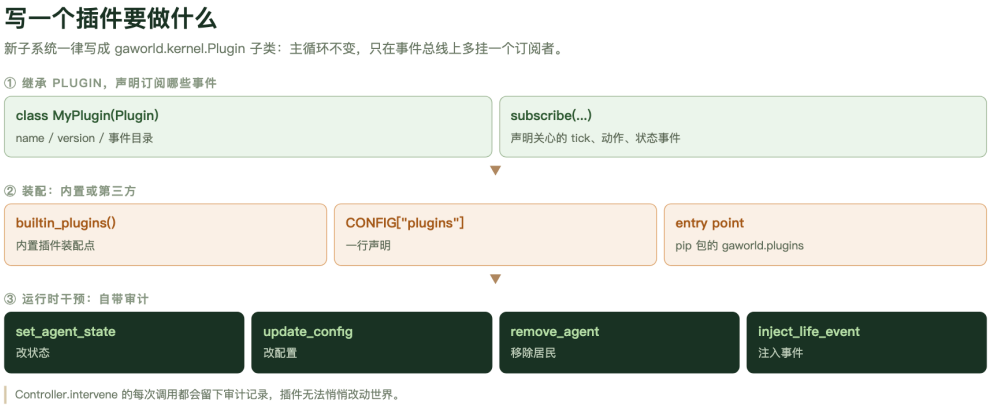


图 15–2　插件的三段式：声明、装配、干预 这张图是流程示意，不是 API 清单——真实接口比图上更朴素：没有 `subscribe(...)` 方法，订阅就是在 `setup()` 里调 `ctx.bus.on(...)`；没有 `name`/`version` 字段，标识只有 `id` 一个。图上第三层那四个方块（`set_agent_state` / `update_config` / `remove_agent` / `inject_life_event`）是运行时干预，15.8 展开讲。

15.3 三种钩子语义：选错了会发生什么

事件本身决定了钩子的语义，**不是你返回什么决定**。同一个 `fn(ctx)` 签名，用在 observe 事件上返回值被丢弃，用在 collect 事件上返回值会被合并。搞混的后果不是报错，而是"插件明明装配了，就是没效果"——这是插件开发最常见的 bug 来源。

语义	处理器签名	返回值	典型用途
observe	`fn(ctx_dict)`	忽略	旁观生命周期：统计、落盘、外部同步
collect	`fn(ctx_dict)`	`None` / 单项 / 列表，内核合并	感知注入、候选征集
filter	`fn(value, ctx_dict)`	新值；`None` 表示保持原值	改写流经的数据

observe：返回值被丢掉

`agents.built`、`on_simulation_start`、`on_day_start`、`on_time_tick`、`on_agent_post_step`、`memory.consolidate` 这些事件都是 observe。你在里面统计、写文件、发请求都随你，但**你 return 的东西没人看**。想做统计就在函数体里写，别指望 return。

collect：返回 0 到 n 条贡献

`perception.compose`、`perception.sections`、`env.events.compose`、`env.events.tick` 是 collect。内核把每个处理器的返回值合并成一个列表：返回 `None` 表示不贡献；返回一个字符串当作一条；返回一个列表，整个列表被摊平合并进去。

坑在这里：**返回一个 dict 会被当成"一条贡献"，而不是被展开。** 收集上来的东西最后会被 `str()` 转成文本拼进提示词，于是你的提示词里会多出一段 Python 字典的字面量。要传结构化数据，就把它放进 `ctx.agent_ext(agent, self.id)` 里，在别的地方取用——内置的 `InterventionPlugin` 就是这么干的：它把整份推荐 feed 存进自己的命名空间，只把一句人话交给总线。

filter：改写载荷

`action.selected`（动作字符串）、`env.events.reach`（本 tick 的环境事件列表）、`day.routine.skip`、`tick.agent_order`、`fast_forward.digest_agents` 是 filter。filter 的处理器签名多一个参数：`fn(value, ctx_dict)`。返回新值就替换，返回 `None` 表示"我没意见，保持原值"。

"保持原值"必须写 `return None`，不能写 `return []`。 内核的判断是 `if out is not None: value = out`——空列表不是 `None`，会被当成"新的值"覆盖上去。想表达"清空"和"不干预"，是两件不同的事。

选错语义的四种典型症状

第一种，**在 observe 事件上以为自己在注入**。你写了 `return ["给agent的一句话"]`，挂在 `on_agent_post_step` 上——返回值被丢弃，提示词里什么都没有。表现是插件装了、钩子也进了，但模型行为一点没变。

第二种，**在 filter 事件上按 collect 写**。你写了 `def handle(self, hook_ctx)`，事件是 `action.selected`，内核调 `fn(value, ctx)`，参数个数不对，抛 `TypeError`。总线会把它当成普通异常捕获，记一条 warning 就继续跑——**过滤器静默不生效**。日志里那条 `hook ... raised: ...` 就是现场。

第三种，**在 filter 事件上返回了"空"**。想表达"这个动作我不干预"却写了 `return []`，原值被空列表替换，下游拿到的是空的。

第四种，**原地改了列表**。`env.events.reach` 的文档明确要求"返回新列表，别原地改"。你在过滤器里对传进来的列表做了 `append` 或 `remove` 再 `return None`——值确实变了，但链上排在后面的过滤器拿到的是已经被你改过的对象，顺序和优先级就乱了。改就返回新列表。

顺序与信任边界

同一事件的处理器按 `priority` 降序执行，同优先级按注册顺序。内置插件里能看到这个约定的实际用法：本地物理感知在 `perception.compose` 上用 `priority=30`，人生事件用 `priority=20`，干预插件用 `priority=10`；`MultiplayerPlugin` 挂在 `action.selected` 的最后，这样真人在玩的时候，他的话是最后一道。

总线是**信任边界**：处理器抛异常会被捕获、记 warning、继续跑，不会拖垮整个仿真。想让异常传播，得显式声明：

```
ctx.bus.on("perception.compose", self.inject_rumor, critical=True)
```

只有当"这个处理器失败，后续继续跑就不对了"的时候才用 `critical=True`。认知管线里的阶段则相反——**阶段的错误会传播**，因为阶段是控制流本体，吞掉一个失败会静默地污染居民状态。

15.4 装配方式三选一（可叠加）

一个插件有三条装配路径，选一条就行，也可以叠加。

方式一：内置插件

内置子系统的装配点只有一个文件：`gaworld/plugins/__init__.py` 里的 `builtin_plugins()`。这个函数返回一组插件实例，模拟器在 `run_simulation` 里逐个 `sim_ctx.registry.register(...)` 注册。仓库里现有的干预、技能、兴趣成长、经济、家庭、物理感知等子系统全在这里面。

加一个内置插件，就是在这个列表里加一行：

```
    from gaworld.rumor.plugin import RumorPlugin       # 文件顶部随其
        它 import
    ...
    return [
        BigFivePlugin(),
        InfoSourcesPlugin(),
        InterventionPlugin(),
        RumorPlugin(),          # 新增：装配点只有这一处
        ...
    ]
```

顺序有意义。这份列表是注册顺序，也是拓扑排序的回退顺序（`requires` 才是真正的依赖声明，注册表用 Kahn 算法按 `requires` 排，检测到环就退回输入顺序）。列表里的注释写得很清楚——大五人格排第一是因为它是只读前置层，信息源读大五的开放性所以紧随其后，公共部门要改经济规则所以在经济之后，家庭在经济之后是因为它的日终结算要落在一个已经结清的日子上。

方式二：`CONFIG["plugins"]` 一行声明

这是第三方扩展最常用的方式，不需要安装任何包，只要那个类所在模块"可导入"就行：

```
from gaworld.settings import CONFIG

CONFIG["plugins"] = [
    {"class": "my_pkg.rumor:RumorPlugin", "enabled": True},
]
```

类路径的格式是 `模块:类名`（**冒号**），和认知管线阶段用的 `模块:函数名` 是一套风格。`enabled` 缺省为真，写 `False` 就是跳过。

不想改源码的话，同一份 JSON 可以从两个地方进去（`gaworld/settings/overrides.py`）：仓库根目录的 `dashboard_config.json`，或者环境变量 `GAWORLD_CONFIG_OVERRIDES`。注意控制台的配置面板自己不会写 `plugins` 这个键——它只写有效配置里存在的键——所以这条路得手改文件或走环境变量：

```
export  GAWORLD_CONFIG_OVERRIDES='{"plugins":[{"class":"my_pkg.rumo
        r:RumorPlugin","enabled":true}]}'
```

方式三：pip 包的 entry point

把你的插件打成 pip 包，在 `pyproject.toml` 里声明一个 entry point：

```
[project.entry-points."gaworld.plugins"]
rumor = "my_pkg.rumor:RumorPlugin"
```

装上之后内核会自动扫描这个组并装配，**没有白名单**（这是项目的一个明确决定）。`PluginRegistry.load_entry_points()` 会把每个 entry point 加载成实例并注册，不是 `Plugin` 子类的会被跳过并记一条 warning。

第四种：代码注册

`ctx.registry.register(RumorPlugin())` 直接注册一个实例。测试和脚本里常用，比如 `tests/test_kernel_plugin_e2e.py` 里的 `ProbePlugin` 就是从 `CONFIG["plugins"]` 的类路径被装起来的。

装不上不会炸

三种方式下，装配失败——导入报错、不是 `Plugin` 子类、`setup` 抛异常、依赖缺失——都只会 `logger.warning` 并停用这个插件，模拟照常跑。这是内核把插件当信任边界的一部分。代价是：**你的插件可能根本没装上，而你知道这件事的唯一途径是看日志。** 跑一次仿真，开头那行 `🧩 已装配插件：…` 会列出所有生效的插件 id，对一下你的 `id` 在不在里面。

15.5 当前可订阅的事件目录

下面这张表来自 `docs/PLUGIN_AUTHORING.md`，是写插件时该照着抄的目录。**每个分发出去的 `hook_ctx` 都额外带一个 `sim` 键**（`SimContext`），表里没列；最后一列列的是关键键，不是全部键——实际载荷里还有 `config`、`agents`、`city_map` 之类的通用字段，处理函数用 `hook_ctx.get(...)` 取就行。

事件	语义	上下文关键键
`agents.built`	observe	`agents` `config`（居民构建后、初始快照前——做 per-agent 状态播种用这里）
`on_simulation_start` / `on_simulation_end`	observe	`config` `agents` `agents_by_id` `city_map`
`on_day_start` / `on_day_end`	observe	`day` + 同上
`day.routine.skip`	filter（value=`False`）	`agent` `day`（日程生成前逐居民问一次；返回 `True` 则沿用基础日程、不调 LLM 生成当天日程）
`fast_forward.digest_agents`	filter（value=本步要出简报的居民列表）	`day`（按天快进时问一次：今天谁跑快进简报。默认全部。返回新列表，别原地改）
`on_time_tick`	observe	`day` `time_str`
`env.events.reach`	filter（value=本 tick 的城市环境事件列表）	`agent` `day` `time_str`（逐居民问一次：哪些事件到得了他。返回新列表）
`tick.agent_order`	filter（value=本 tick 的居民列表）	`day` `time_str`（谁先行动。返回新列表，别原地改）
`agent.moved`	observe	`agent` `activity` `movement` `city_map` `day` `time_str`
`on_agent_pre_step` / `on_agent_post_step`	observe	`agent` `day` `time_str` `step`
`perception.compose`	collect	`agent` `day` `time_str` `scheduled_activity` `env_context`

事件	语义	上下文关键键
		`social_context` `env_events` `policy` `policy_desc` `news`
`perception.sections`	collect	`agent` `day` `time_str` `scheduled_activity` `social_context`（贡献渲染在感知 prompt 内部的专属段落，不污染环境上下文）
`action.selected`	filter（value=动作字符串）	`agent` `activity` `day` `time_str` `location`
`memory.consolidate`	observe	`agent` `day`（日终逐 agent 发射，插件自管节奏门控）
`episode.compose`	observe（可改写 episode）	`agent` `episode` `step_minutes` `day` `time_str`
`env.events.compose`	collect	`agent` `day` `time_str` `step` `daily_logs`
`env.events.tick`	collect	`day` `time_str`
`state.effects`	observe	`agent` `step` `day` `time_str`（update_state 阶段内、社会影响之前）

这张表不是永远完整的。代码里还跑着几个没写进文档表的事件：`interrupts.compose`（filter，中断思考的产出）、`action.candidates`（collect，候选动作征集）、`action.outcome`（filter，动作结果）、`location.resolve`（filter，地点解析）、`interrupt.applied`、`life.event.applied`、`collaboration.event`（后三个是 observe）。想知道当下确切有哪些，自己扫一遍最保险（下面这段是本章成稿时实际跑过的命令）：

```
python3 - <<'PY'
import re, pathlib
pat = re.compile(r'(?:hook_bus|bus)\.(emit|collect|filter)\(\s*"([a
        -z_.]+)"', re.S)
files = list(pathlib.Path('gaworld').rglob('*.py')) + [pathlib.Path
        ('generative_city_sim.py')]
for p in files:
    for kind, ev in pat.findall(p.read_text(encoding='utf-8', error
        s='ignore')):
        print(f"{ev:26s} {kind:8s} {p}")
PY
```

事件名的分布有个规律值得记住：**生命周期事件在 `generative_city_sim.py` 里发射（`agents.built`、`on_day_start`、`on_time_tick`、`on_day_end`、`on_simulation_start`/`_end`），认知过程内的事件在**

`gaworld/sim/_stages.py` 里发射（`perception.compose`、`action.selected`、`state.effects`、`env.events.compose`、`agent.moved`、`episode.compose`）。你要写的插件多半要同时用到两边。

15.6 认知管线 K2：替换、插入、消融

每个 agent 的一步认知被切成 12 个命名阶段，顺序写在配置里，代码在 `gaworld/sim/pipeline.py`：

```
prepare → perceive → interrupts → plan → adjust_activity → move →
select_action → reflect → update_state → broadcast → memorize → rec
ord
```



图 15–3　12 阶段认知管线：每个阶段都是 `fn(agent, step, ctx)` `step` 是这一步的数据总线（一个普通 dict）：钩子可见的键沿用旧名（`scheduled_activity`、`action`、`outcome`、`reflection`），阶段之间传工作用的键以下划线开头（`_perception`、`_plan_text`、`_act`）。消融、替换、插入某个阶段，都只是改 `CONFIG["pipeline"]["agent_step"]` 这一个列表，主循环不用动。

替换：把某个内置阶段换成你自己的函数，路径写成 `模块:函数名`：

```
CONFIG["pipeline"]["agent_step"] = [
    "prepare", "perceive", "interrupts", "plan", "adjust_activity",
        "move",
    "my_pkg.stages:deliberate",   # 替换 select_action
    "reflect", "update_state", "broadcast", "memorize", "record",
]
```

替换意味着**你得自己产出那个阶段本该产出的工作键**。`select_action` 写出的是 `step["_act"]` 和 `step["_effective_activity"]`，下游阶段会读；你的函数不写，下游就拿到空值。插入没有这个负担——插入的阶段只管往 `step` 里加东西，不影响别人。

插入：在任意位置插一个自定义阶段：

```
CONFIG["pipeline"]["agent_step"] = [
    "prepare", "perceive",
    "my_pkg.stages:deliberate",   # 插在 perceive 之后：此时 step["_p
        erception"] 已就绪
    "interrupts", "plan", "adjust_activity", "move",
    "select_action", "reflect", "update_state", "broadcast", "memor
        ize", "record",
]
```

消融：把某个阶段从列表里删掉。省略 `reflect`，就是一个**不反思的 agent**——它照样感知、计划、行动、更新状态，只是不再问"这次经历意味着什么"。这是认知消融实验最干净的形态，因为下游阶段对缺失数据是容错的（去掉 `reflect`

后 `reflection` 为空串），其余流程照常：

```
CONFIG["pipeline"]["agent_step"] = [
    "prepare", "perceive", "interrupts", "plan", "adjust_activity",
        "move",
    "select_action",              # "reflect" 被省略 = 消融反思
    "update_state", "broadcast", "memorize", "record",
]
```

两个注意点。**`prepare` 和 `record` 是结构性阶段**（前者的 pre-step 钩子在这里发射，后者写日志和最终的 step 键），消融目标应该是中间那些认知阶段，消融掉 `record` 你会丢掉运行记录而不是改变认知。**未知阶段名会被跳过并记一条 warning**（`pipeline stage %r not resolvable; skipped`），不会让整个仿真停摆——但一个拼错的名字会静默地让你的实验跑成了另一套配置。跑之前先 `print(CONFIG["pipeline"]["agent_step"])` 确认一遍。

这一节最该记住的是：做认知消融实验，一行插件代码都不用写。 改配置就够了。仓库里已经有现成的验收测试（`tests/test_pipeline_ablation.py`）：它断言去掉 `reflect` 之后模型调用计数里的 `reflection` 归零、而 `planning` 照常发生。你要做消融研究，照着这个测试改一行配置就能起步。

还有一条更细的机制叫**按步跳过**：在 `step["_skip_stages"]` 里列出的阶段，只在这一步被跳过（每个阶段执行前读一次，所以插件可以在 `on_agent_pre_step` 里设置，而 `on_agent_pre_step` 正是 `prepare` 阶段发射的）。离城插件用它让在外的居民跳过感知、计划、选动作、反思这些逐步的 LLM 调用。代价是：跳过阶段的人要负责补上被跳过阶段本该产出、后续阶段会读的工作键（比如 `_act`、`_effective_activity`）。

15.7 状态与数据所有权

插件之间要传状态，最容易出的问题是隐式耦合：两个子系统都往 `agent` 字典顶层塞裸键，谁也说不清那个键归谁。内核用四种方式把所有权摆平。

per-agent 状态用 `ctx.agent_ext(agent, self.id)`，它返回 `agent["ext"]["<你的 id>"]` 下的那个 dict，跟着居民的生命周期存在。所有插件的 per-agent 状态都收在 `agent["ext"]` 这一个命名空间下，**不要往 agent 字典顶层塞裸键**。

模拟级共享状态用 `ctx.plugin_state(self.id)`。别的插件可以用同一个调用只读访问，但禁止 import 你的内部模块——需要协作就用 `requires` 声明依赖，让装配顺序替你保证。

输出文件用 `self.output_dir(ctx)`，它返回 `output/<你的 id>/`，格式自定。这是 Database-per-Plugin：每个插件一个目录，互不踩脚。内置的信息源插件就把每个居民的信息食谱写在 `self.output_dir(ctx) / "diets.json"`。

统一事件流用 `ctx.recorder.record("<id>.<table>", {...})`，落到 `output/records/<id>.<table>.jsonl`，每行自动带上 `_day` 和 `_time`，用来跨插件对齐时间线和审计。你自己的私有文件随便放，但**凡是希望别人事后能对齐时间线看的东西，就走 Recorder**。

一句话：你想留的东西分两堆——**结构化的、需要被别人读懂的进 Recorder，格式随你定的进自己的输出目录。** `output/records/` 下那些 `family.*.jsonl`、`home.observation.jsonl`、`big_five.seeded.jsonl` 就是这么来的。

15.8 动作校验与运行时干预

`Controller`（`gaworld/kernel/controller.py`）是一个无状态的中介，管两件事：动作校验，和运行时干预。

动作校验链

每个结构化的动作请求在执行前都要过一遍按优先级排序的校验器链。校验器返回 `Verdict`，或者返回 `None` 表示"没意见，交给下一个"：

```
from gaworld.kernel import Verdict


class RumorPlugin(Plugin):
    id = "rumor"

    def setup(self, ctx):
        ctx.controller.register_validator(self.check_curfew, priori
        ty=5)
        ctx.controller.register_intervention("rumor.plant", self.pl
        ant)

    def check_curfew(self, request, ctx):
        # request: ActionRequest(agent_id, name, params, raw_text)
         if request.name == "move" and ctx.clock.time_str >= "23:0
        0":
            return Verdict.deny("宵禁时段不能外出")
        return None             # None = 无意见，交给下一个校验器

    def plant(self, ctx, agent_id=None, text=""):
        ctx.agent_ext(ctx.agents_by_id[agent_id], self.id)["heard_r
        umor"] = text
```

`Verdict` 有三种：`allow()`、`deny(reason)`、`rewrite(request)`（改写请求而不是否决）。第一个 `deny` 生效，后面的校验器不再跑。**被否决的动作会写进 `output/records/action.denied.jsonl`，而且理由会在这个居民下一 tick 的感知里以"刚才的行动受阻：…"出现**——这是设计上的关键一笔：agent 会知道自己被拦了，下一步可以换个做法，而不是莫名其妙地什么都没发生。

仓库里已经装了两个内置校验器：`location_exists` 默认开（拦住"去一个这座城市里根本不存在的地方"），`venue_open` 默认关（拦截非营业时间的地点；默认关是因为硬拦会改变动力学，闭店场景由 P0/P2 层反应式处理）。开关在 `CONFIG["controller"]["validators"]` 里。

四个标准运行时干预

名称	参数	语义
`set_agent_state`	`agent_id` `key` `value`	立即写入 agent 状态变量
`update_config`	`path`（点号路径）`value`	立即写入运行中 CONFIG
`remove_agent`	`agent_id`	排队，下个日边界生效（含社交邻居清洗）
`inject_life_event`	`event` dict 或散参	入队人生事件（由人生事件插件注册）

前三个是内核自带的（`gaworld/kernel/interventions.py`，`build_kernel` 时无条件注册）；第四个由 `LifeEventsPlugin` 注册（`gaworld/events/plugin.py`），因为它要认识人生事件的数据结构——这是内核不碰领域逻辑的又一个体现。运行时造人（`add_agent`）**还没实现**，它需要一条种子摄取路径（画像 + 状态行 + 社交名册 + 记忆初始化）。

几个必须知道的细节。`set_agent_state` 写的是 `agent["state"][key]`，也就是九个归一化状态变量（`energy`、`hunger`、`social_need`、`fatigue_debt`、`self_control`、`time_pressure`、`emotion`、`stress`、`econ_security`）那一套，**值会被 `float()` 强转**。`update_config` 写的是运行中的 CONFIG，但对那些在 `setup` 时就把配置快照下来的消费者，要下一次运行才看得到。`remove_agent` **不在当前 tick 生效**——它把 id 排进队列，主循环在下一个日边界统一处理，同时把被移除的 id 从其余居民的 `social_neighbors` 里洗掉；中途删人会破坏正在进行的 step 管线。

每次调用都留审计

`Controller.intervene` 每调用一次，都会往 `controller.intervention` 表写一条记录（`output/records/controller.intervention.jsonl`），带干预名和参数。

这就是"插件不能悄悄改世界"这条设计的落点。 一个插件想在运行中途推某个居民一把，可以，但它必须走 `intervene`，于是这次干预一定会在审计表里留下名字、参数和时间。想知道"这个结果是不是被人动过手脚"，翻一下这个 jsonl 就知道。同一套 API 也是面板的入口：仪表盘是另一个进程，它不能直接调 `ctx.controller.intervene`，所以走 `gaworld/kernel/remote.py` 的跨进程队列（`output/kernel/interventions.json`），模拟器在日边界和每个 tick 把队列排干，走的还是同一条 `intervene` 路径，因此审计记录一条不落。

插件自己注册干预也一样：

```
ctx.controller.register_intervention("rumor.plant", self.plant)
```

注册名不要和标准四个撞车。

15.9 从零到一：写、装、跑、验

下面是一条完整路径，按顺序走。

第一步：挑一个最接近的参考实现抄。 这一条放在最后单独讲，但它应该排在最前面做。

第二步：写插件。 新建 `gaworld/<你的子系统>/plugin.py`，继承 `Plugin`，定 `id`，在 `setup()` 里订阅事件。业务逻辑单独放一个模块，插件类里只做 import 和转发——内置插件都是这么写的（`InterventionPlugin` 在 `setup` 里 `from gaworld.policy import intervention as impl`，把领域逻辑挡在内核装配之外）。

第三步：装配。 内置就加进 `gaworld/plugins/__init__.py` 的 `builtin_plugins()`；第三方就往 `CONFIG["plugins"]` 加一行，或者用 entry point。

第四步：跑一次。 用控制台跑一次，或者 `python generative_city_sim.py run`。开头那行 🧩 `已装配插件：…` 里要能看到你的 id——看不到就是没装上，去日志里找那条 warning。

第五步：确认产物。 你的插件应该在 `output/` 下留下自己的东西：`output/<你的 id>/` 里的私有文件，或者 `output/records/<你的 id>.<table>.jsonl` 里的事件流。**没有产物等于没跑起来**，这是最直接的验收。

第六步：写测试。新代码必须带测试，在同一个 PR 里。 仓库有两个套件：`pytest --suite core` 是冒烟套件（跑 `tests/suites/core.txt` 里列的那些，约半分钟到一分钟），`pytest --suite full` 是全量。核心那套里已经包含了 `tests/test_kernel.py`、`tests/test_kernel_plugin_e2e.py`、`tests/test_sim_pipeline.py`、`tests/test_pipeline_ablation.py`——也就是说，插件机制本身是被持续验证的，你的插件要按同样的方式被验证。`tests/suites/core.txt` 里多一条陈旧项（指向已经不存在的测试）整个套件就会失败，所以改了测试名字记得同步改这个清单。

第七步：过静态检查。 三条命令：

```
ruff check .                        # lint
python scripts/lint_ratchet.py      # 棘轮：改过的文件不能增加 lint 发现数
mypy gaworld                        # gaworld/ 树是严格类型检查
```

棘轮脚本的规矩是：CI 跑 `python scripts/lint_ratchet.py --base origin/<目标分支>`，**一个被你改过的文件不得增加发现数，新增的文件（或者已经被格式化过的文件）必须已经是格式化状态**，并且仓库总数不得高于 `scripts/lint_baseline.json`（清理过之后用 `--update-baseline` 记录新基线）。本地不带参数跑就是检查未提交的改动。`mypy` 对 `gaworld.*` 开了 `disallow_untyped_defs = true`，所以你的函数要有类型标注。

第八步：判断要不要追加 Epoch。 如果你的改动**改变了默认配置下跑出来的结果**，让旧的结果不再可比，就必须在 `gaworld/core/comparability.py` 的 `EPOCHS` 里**追加**一个 `Epoch`（只增不重编号、不改旧的）。每个 epoch 是一个 `Epoch(number, since, change, affects, source)`。当前 epoch 是 `EPOCHS[-1].number`，运行、compare-event 目录和平行世界实验都会被打上这个标记，跨 epoch 的合并会被拒绝——平行世界的复现池不收别的 epoch 的种子，研究评估器不会用不同 epoch 的种子下"支持/不支持"的结论。**默认关闭的可选开关不需要 epoch**：运行清单里的配置快照已经能区分它们。

判断标准很简单：如果一个拿着你三个月前的结果写论文的人，现在跑同样的默认配置，结果不再能直接比较，那就是该加 epoch 的时候。

15.10 给工程师的一条建议：别从零写

插件机制看起来简单，但真正写起来有几十个隐含约定——钩子语义、事件载荷的键名、状态的归属、产物的落点、装配的顺序。**从零写一遍，等于把这些约定重新踩一遍坑。**

仓库里有三个可以直接抄的参考实现，覆盖了三种典型形态：

- `tests/test_kernel_plugin_e2e.py::ProbePlugin`——CI 持续验证的完整示例：经 `CONFIG["plugins"]` 装配、感知注入真的到达 LLM 提示词、动作过滤器观察到每个被选中的动作，还注册了一个自己的干预。这个文件是"新子系统零侵入"的验收测试，抄它最省事。
- `gaworld/policy/plugin.py`——第一个迁移到插件面的内置子系统（干预），**三种钩子用法齐全**：`agents.built` 播种 per-agent 状态、`perception.compose` 注入感知、`on_agent_post_step` 落盘指标。
- `gaworld/skills/plugin.py`——两个钩子、两个语义：`perception.sections` 贡献提示词里的专属段落，`memory.consolidate` 按自己的节奏门控做技能蒸馏。

选一个形态最接近的抄下来改，比读十页文档更快。文档本身在 `docs/PLUGIN_AUTHORING.md`，里面还有完整的最小插件、事件表和干预表。

本章小结

- 新子系统一律写成 `gaworld.kernel.Plugin` 子类，挂在事件总线上；`generative_city_sim.py` 和 `run_simulation` 有行数上限守着，主循环不允许长出新逻辑。
- 插件只做三件事：在 `setup()` 里声明订阅什么、在事件到来时做事、按事件的语义把结果交回总线。
- 钩子语义由**事件**决定，不由返回值决定：observe 看返回值为空气，collect 收 0..n 条贡献，filter 改写载荷；选错的典型症状是"插件装了但没效果"。
- 装配有三条路：内置进 `gaworld/plugins/__init__.py` 的 `builtin_plugins()`、`CONFIG["plugins"]` 一行声明、pip 包的

`gaworld.plugins` entry point；装不上只会 warning，看日志和那行"已装配插件"。

- 认知消融、阶段替换、阶段插入都只是改 `CONFIG["pipeline"]["agent_step"]`，**不需要写任何插件代码**——这是做认知实验最快的路。
- 状态各有归属：per-agent 进 `ctx.agent_ext(agent, self.id)`，共享进 `ctx.plugin_state(self.id)`，私有文件进 `self.output_dir(ctx)`，跨插件对齐用 `ctx.recorder.record(...)`；运行时改动一律走 `Controller.intervene`，每次调用都留审计。

动手练习

1. **写一个自己的插件并装上**（约 30 分钟） 照着本章 15.2 的最小插件写一个 `gaworld/practice/plugin.py`，`id = "practice"`，做一件小事：在 `perception.compose` 上给每位居民贡献一句你自己的环境观察（例如"练习插件：今天是第 N 天"）。然后在 `CONFIG["plugins"]` 里加一行 `{"class": "gaworld.practice.plugin:PracticePlugin"}`，跑一次仿真。 **验收标准**：仿真开头那行"已装配插件"里出现 `practice`；`output/records/` 下出现你的插件写出的记录文件（如果你用了 `ctx.recorder.record("practice.log", {...})`），或者 `output/practice/` 下出现你写的文件；随机抽一个居民，打开 `output/logs/agent_<id>.log`，能看到 `Env:` 那一行里多出你贡献的那句话（插件贡献的片段会拼进这一步的环境上下文，而环境上下文会写进当天日志）。
2. **只改配置做一次认知消融**（约 20 分钟） 不写任何插件代码，只把 `CONFIG["pipeline"]["agent_step"]` 里的 `reflect` 去掉，跑一次仿真；再保持默认配置跑一次作对照。 **验收标准**：两次运行都能正常结束；对照组的 `output/logs/agent_<id>.log` 里有内容非空的 `Refl:` 行，消融组里这一行是空的（反思阶段没跑，`_refl_text` 从未被写，下游阶段对缺失数据是容错的，其余流程照常走完）；你能指出这次改动**不需要**在 `gaworld/core/comparability.py` 里加 Epoch（判断标准是"默认配置下跑出来的结果是否变化"，而你改的是一次运行的配置、不是代码默认值），并说明如果把这个消融设成**默认**，就需要加一个 Epoch。

ewpage

第 16 章　多用户部署：把平台交给一班学生

前十五章讲的都是「一个人怎么用 GAWorld」。这一章换一个视角：如果服务器上要同时坐着一个班的学生，每个人都有自己的城市、自己的运行、自己的结果，而且谁也看不见谁的东西——平台要做哪些改变？

答案是：加一个账号库，剩下的事情就都从这个开关长出来了。这一章把中间那档部署形态（多用户共享）讲透：怎么开账号库、权限分成哪几层、每个人的世界放在哪、结果怎么归属、配额怎么算，以及教师手里那块能管全班的面板。

16.1 三档部署形态，中间那档最难



图 16–1　三种部署形态 单机形态是默认：控制台绑在 `127.0.0.1:8766`，不需要登录，所有产物落在一个 `output/` 里。多用户共享部署在一台服务器上加了账号库与邀请码，学生各建各的世界。分布式世界再往前一步，把同一座城市的居民分到多台机器上跑，Dashboard 当枢纽。中间这一档最容易被低估——它既要隔离，又要让一班人同时看见同一件事。

三种形态不是三选一，而是逐级叠加的：单机 → 多用户 → 分布式世界（见第 17 章）。关键是，账号库是一个**开关**而不是一种部署。`gaworld/accounts/__init__.py` 里的 `enabled_store()` 只有在 `output/accounts/accounts.sqlite` 存在时才返回一个 store，否则返回 `None`；控制台的 `_guard` 拿到 `None` 就完全按老路走——不鉴权、所有人的操作都落在同一份 `output/` 上。

所以单机用户完全不需要做什么。仓库里那句注释说得很准：账号库不存在时，一切行为与今天逐字相同。

反过来，一旦你创建了账号库，控制台就**要求登录**：未登录访问任何页面会被重定向到 `/login`，未登录访问任何 API 会得到 401。`/login`、`/join`、`/reset` 三个页面是公开的，它们都由同一个 `site/auth/index.html` 按路径切换。

16.2 开启账号库：五个子命令

账号管理全部走 `python -m gaworld.accounts`，一共五个子命令。下面每条命令的参数都是照着 `python3 -m gaworld.accounts <子命令> --help` 的实际输出写的。

16.2.1 init：建库与第一个管理员

```
python -m gaworld.accounts init --admin 老师
```

它会交互式地要你输两遍密码，然后创建 `output/accounts/accounts.sqlite`（路径可用环境变量 `GAWORLD_ACCOUNTS_DB` 改）。这个管理员的 `role` 是 `admin`，并且天然带着**建城权限**。成功时打印一句「已创建账号库 …，管理员

「老师」。从现在起控制台需要登录。」

密码规则由 `store.check_password()` 守着：至少 8 位，最多 256 位。昵称最多 32 个字符，不能含控制字符。密码用 `hashlib.scrypt` 加盐存储，库里没有明文。

给脚本准备时用 `--password-stdin` 从标准输入读一次密码，省掉交互。

16.2.2 invite：一次性邀请码

```
python -m gaworld.accounts invite --count 30 --label 周三班
```

打印 30 个邀请码，一行一个。参数只有四个：`--count`（一次 1–200 个，默认 1）、`--label`（班级标签，会记进用户行）、`--expires-days`（默认 14 天）、`--can-create-city`（这批人注册后直接带建城权限）。

邀请码是**一次性**的：库里只存它的 SHA-256 摘要，用过即标记 `used_at`；过期或用过就是「无效、已使用或已过期」。CLI 打印之后就再也不会显示第二遍，所以生成后立刻复制走，或者在教师控制台上生成（那里还会直接给出 `/join?code=…` 的完整链接）。

学生拿到码以后打开 `http://<host>:8766/join?code=<码>`，填昵称、设一个至少 8 位的密码，就注册完成了。之后一律用「昵称 + 密码」在 `/login` 登录。

16.2.3 users / reset / city-permission

```
python -m gaworld.accounts users
python -m gaworld.accounts reset --nickname 小王
python -m gaworld.accounts city-permission --nickname 小王 --allow
```

`users` 列出每个人的编号、角色、昵称、班级标签、是否可建城、创建日期。`reset` 给某人打印一个一次性重置码（24 小时有效），学生打开 `/reset?code=<重置码>` 设新密码。`city-permission` 授予或撤销建城权限，`--allow` 与 `--deny` 二选一，必填。

忘记密码时的完整流程是：管理员发码 → 学生改密码 → 重置码作废。`reset` 每次都会先删掉该用户未用的旧重置码，所以不会攒出一串同时有效的码。

16.2.4 账号库里有什么

数据库是一个标准 SQLite 文件，路径 `output/accounts/accounts.sqlite`，表有八张：

表	装什么
`users`	昵称、密码摘要、`role`、`can_create_city`、班级标签
`sessions`	会话令牌摘要与过期时间（有效期 7 天）
`invites`	邀请码摘要、标签、有效期、是否可建城、谁用了
`resets`	一次性重置码摘要与有效期
`city_owners`	哪座城是谁建的
`worlds`	世界编号、名称、属主、城市、可见性
`settings`	运行限额与配额
`audit`	每个写请求的审计记录

这个库**不会**被当作静态文件提供，静态白名单里没有 `output/accounts/`。审计表值得教师偶尔翻一翻：谁在什么时候建了世界、改了配额、发了广播，都在里面。

16.3 权限模型：分三层看就够

多用户部署最容易被误解的地方，是以为「权限」是一个开关。实际上它是三层独立的东西，混在一起看就会得出错误结论。

16.3.1 第一层：成员能调哪些接口

这一层由 `gaworld/accounts/policy.py` 规定。核心函数 `required(method, path)` 把每个请求归到五档之一：

- `public`：`/api/health`、登录、注册、重置这三个接口，加 `/login`、`/join`、`/reset` 三个页面。
- `member`：登录成员就能调。采访、各类游戏、研究分析、复制实验（replication）、人物蒸馏、严肃游戏设计与开局、政策仿真、改自己的密码、保存自己的 API Key、创建和切换世界。
- `city`：需要 `can_create_city`。建城、给城加人口、改城里的居民、迁入、删城，都在这一档。拿到这个权限也不等于能改所有城市——控制台会查 `city_owners` 表，提示语是「只能修改自己建立的城市」。
- `world`：需要**当前世界的属主**。改世界配置、启停/定时仿真、编辑居民档案状态目标记忆财务关系大五、发干预，都在这一档。
- `admin`：其余全部。全局配置、模型定义、切换共享默认世界的城市、生命事件、绝大多数删除、`/api/worlds/settings`、`/api/worlds/broadcast`。

判定在 `allows(user, level, world)`：管理员直接放行；`member` 档只要登录了就放行；`city` 档看 `can_create_city`；`world` 档看 `world["owner_id"] == user["id"]`。

这一层的判定集中在这一个文件里，所以想查「某个接口谁能调」，先读它。不过少数接口还会在自己的 handler 里再补一道检查——世界的属主（`worlds_api._owned`）、城市的属主（`dashboard_server` 的建城校验）、成员在运行中能改哪些配置项（`policy.config_update_refusal`）——所以要确认一个接口的完整规则，还得顺手看一眼它自己的 handler。

16.3.2 第二层：角色与建城权限是两件事

`role` 只有 `admin` 和 `member` 两种——管理员（教师）与普通成员（学生）。而 `can_create_city` 是一个**独立**的权限位：管理员恒为真，成员默认是假，由教师逐人授予，或者随邀请码模板一次性授予。

为什么分开？因为它管的是完全不同的能力。角色决定「能不能动别人动不了的东西」（全局配置、别人的世界、删东西）；建城权限决定「能不能创造新的东西」（一座新城市、它的地图、它的人口）。一个学生可能很有创造力、值得给他建城权，但绝不该有删别人世界的权力。

16.3.3 第三层：谁能改某一个世界

这条规则一句话就能说清：

> 只有世界的创建者能改它；共享的默认世界只限管理员。

落到实现上，`world` 档要求请求的活动世界的 `owner_id` 等于当前用户的 `id`。共享的默认世界（也就是还没有进入任何个人世界时的那个 `output/`）没有属主，所以成员碰它一律 403，提示语是「没有权限：只有世界的创建者能修改它（共享的默认世界只限管理员）」。

可读性是另一回事，由 `dashboard_server.world_readable()` 决定：属主和管理员随时能看；别人只有当世界的可见性不是 `private` 时才能看。可见性有三档：`private`（默认，只有属主和管理员）、`class`（所有成员只读）、`open`（成员可以加入多人共玩）。

16.3.4 令牌管理员

`GAWORLD_DASHBOARD_TOKEN` 的持有者仍然是管理员——服务端把它映射成一个 `role = admin` 的虚拟用户（`TOKEN_ADMIN`），运维脚本照常用 `Authorization: Bearer $GAWORLD_DASHBOARD_TOKEN` 调接口。浏览器里打开一次 `http://<host>:8766/?token=<令牌>`，服务端会写一个 `HttpOnly; SameSite=Strict` 的 cookie 并把 URL 里的令牌去掉。

也就是说，开账号库之后你并没有失去令牌：令牌是**运维入口**，账号是**课堂入口**。两者可以同时存在。

16.4 每人一个世界

16.4.1 一个世界目录里有什么

创建世界时，学生选一座有居民的城市，平台把该城市的居民复制一份进这个人的世界。目录在 `output/worlds/<id>/`，编号形如 `w1a2b3c4`（`w` 加 8 位十六进制）：

```
output/worlds/<id>/
├── config.json       # 这个世界的配置叠加层
├── seed/             # 居民副本：agents.csv、profiles.md
│                     # 若该城市有的话，再复制大五人格表 agents_big5.csv
│                     # 与家庭 family_overrides.json
└── …                 # 全部运行产物（记忆、状态、记录、日志…）
```

`seed/` 是「复制进来再改」的机制：Agent Studio 改居民，改的是这个世界的副本，不是城市包里的原件。`config.json` 是这个世界的配置层，叠在全局 `dashboard_config.json` 之上——所以教师统一配的模型设置对所有人生效，学生改的仿真参数只影响他自己。

运行产物用的和城市运行同一套布局（`gaworld.city.config.RUN_PATHS`），只是根目录换成了 `output/worlds/<id>/`。仿真子进程启动时拿到一份 `GAWORLD_CONFIG_OVERRIDES`（世界配置 + 世界路径），这份覆盖块的优先级最高，仿真代码本身完全不知道「世界」这回事。

16.4.2 当前世界由一个 cookie 决定

「我现在在哪个世界」这件事由 `gaworld_world` cookie 决定，属性是 `HttpOnly; SameSite=Strict`。用 cookie 而不是请求头或 URL 前缀，是因为控制台里有几十个面板、iframe、SSE 事件流和静态 trace 文件，cookie 会自动带上，不要求每个面板各自改造。

cookie 只是一种**意愿**。`_guard` 在每个请求上重新查一次这个用户的可见性，查不到就退回共享的默认世界。也就是说，伪造 cookie 没用。

在这个 cookie 之下，每个 dashboard 解析出来的路径都经由 `gaworld/apps/world_paths.py` 解析。这个模块提供了一组按世界切换的函数：

函数	解析出什么
`effective_config()`	当前生效的完整配置（所有层叠加后的结果）
`state_csv_path()`	居民状态 CSV（世界里是 `seed/agents.csv`）
`profile_path()`	人物志 Markdown（世界里是 `seed/profiles.md`）
`big5_csv_path()`	大五人格表（世界里是自己的副本）
`records_dir()`	Recorder 事件流目录
`run_log_path()`	运行日志

函数	解析出什么
`economy_snapshot_path()`	财富快照
`run_root()` / `memory_file()`	运行根目录、某个居民的记忆文件

`effective_config()` 的叠加顺序值得记一下：代码默认值 → `dashboard_config.json` → 世界的 `config.json` → 环境变量覆盖 → 环境配置文件 → 城市包 → **世界的路径覆盖** → 环境变量覆盖（再一次）。世界的路径放在城市之后，所以一个世界永远赢过它所基于的那座城市；环境变量放最后，是为了让它能盖住一切。

16.4.3 一条硬规则

`world_paths.py` 的模块常量（`STATE_CSV_PATH`、`RECORDS_DIR`、`RUN_LOG_PATH` 等）**是共享默认世界**的值。因此有一条仓库级硬规则：

> 新 dashboard 代码不许通过模块常量读 `output/`。

在新代码里写 `os.path.join(REPO_ROOT, "output", "memory")` 这样的字面量，在多用户部署下会把所有学生的东西写到同一个地方去——这正是这套机制要避免的。正确写法是调用 `world_paths` 里的函数。这个模块还留了 `DASHBOARD_ALIASES` 一张迁移表，把旧名字指向新函数，方便把历史代码一段段挪过来。



图 16–2　组织面板：世界级产物的样子 这一屏展示的全是「本世界」的东西：左侧是可以创建的组织（社区资源分配组等模板，填负责人居民、初始资金、成员居民编号），中间是某个组织的账户、成员与实际执行记录，右侧是命令队列。之所以用它举例，是因为它把「世界的隔离」这件事具体化了——组织、身份、规则、账目和历史全部住在世界里，学生 A 在自己的世界里建的组织，学生 B 在自己的世界里根本看不到，也碰不到。

16.4.4 世界切换器

控制台顶栏有一个世界下拉框，列出当前用户能看到的所有世界；切换动作是 `POST /api/worlds/select`，服务端回一个设置 `gaworld_world` cookie 的响应头，页面随后重新加载。切换是**全浏览器生效**的——不只影响某一个面板。

「管理世界」按钮打开的对话框里可以改可见性、删除世界、设置多人共玩每 tick 等玩家的秒数、分配分布式节点。删除世界前必须先停掉它的仿真，否则会得到「先停止这个世界的仿真再删除」。

16.5 归属与可见性：谁做的东西归谁

世界隔离解决的是「运行产物」这一类东西。但还有一类东西不在世界里：采访会话、研究方案、蒸馏出的人物画像、各种游戏的作业记录。它们放在各自的目录或内存里，需要另一套机制。

这套机制在 `gaworld/accounts/ownership.py`，只有三个动作：

- `stamp()`：创建记录时调用，返回 `{"owner_id": …, "owner": 昵称}`，合并进这条记录。
- `visible(record)` / `owned(records)`：读和列之前过滤，只有创建者和管理员能看到。
- `sees_unowned()`：没有属主的记录（账号开启前留下的旧数据、运维令牌造的）只给管理员看。

落在哪些存储上是逐个接进去的：采访会话在 `output/interviews/<id>/session.json` 的属主字段里；研究方案在 `output/research/` 的方案 JSON 里；人物蒸馏在 `output/personas/<slug>/owner.json` 里（同名画像不能覆盖别人的）；游戏存档在 `output/games/<kind>/` 的存档 JSON 里。

16.5.1 后台任务必须通过 spawn 启动

这是归属机制里最容易被忽略、也最关键的一条。

Python 的新线程默认带一个**空**的上下文（`contextvars`）。所以后台任务如果直接 `threading.Thread(target=...)`，它就不知道自己是谁发起的、在哪个世界里——结果就是后台产出的东西没有属主，或者写进了错误的世界。

正确做法是 `ownership.spawn(target, *args, name=...)`：

```
from gaworld.accounts import ownership

ownership.spawn(job, payload, name="research-run")
```

`spawn` 内部用 `contextvars.copy_context().run` 启动线程，等于把发起请求时的上下文（当前用户 + 当前世界）复制一份带过去。仓库里有测试专门盯这条（`tests/test_worlds.py` 的 `test_api_jobs_start_through_ownership_spawn`），新增后台任务时必须走它。

16.6 配额与限额

16.6.1 模型调用按用户计数

每一次模型调用（**包括失败的**）都会记一笔账。钩子挂在 `gaworld/llm/providers.py` 的 `call_llm` 的 `finally` 块里，所以调用炸了也计数——否则一个写错 key 的学生就能无限次消耗别人的额度。

记录写在账号库旁边的 `output/accounts/usage.jsonl`，一行一个 JSON：`{"t": 时间戳, "user": 用户号, "world": 世界号, "task": 任务名}`。

计数单位是**调用次数**而不是 token。各家提供商的 token 回报不统一，而「一次采访大约 200 次调用」这种量级，才是老师做预算时能拿得住的数字——`gaworld/accounts/usage.py` 的模块注释里就是这么写的，它是仓库给出的量级参考，不是实测基线。

用户身份怎么传到各处：dashboard 内部的调用从请求上下文直接取；仿真进程、采访子进程这些子进程通过环境变量 `GAWORLD_USER_ID` 和 `GAWORLD_WORLD_ID` 继承（`usage.child_env()` 负责塞进去）。多进程写同一行不需要加锁，因为每行是一次短的 `O_APPEND` 写。

管理员在 `GET /api/auth/usage` 看到每人今日与累计的调用次数。

16.6.2 软配额

`daily_llm_calls_per_user` 是每天每人允许的模型调用次数，默认 `0` 表示不限。设了之后，用完额度的学生不能**发起新的模型工作**：启动或定时仿真、采访、研究分析与读论文、人物蒸馏、各类游戏、严肃游戏设计与开局、政策仿真、建城类的请求都会返回 429。

但正在跑的任务不中断，改配置、停止、删除也不受影响。判断哪些接口受管的是 `policy.QUOTA_GATED_EXACT` 与 `QUOTA_GATED_PREFIX` 两张清单，加上几个前缀规则。管理员永远不受限。

16.6.3 运行限额与排队

限额存在账号库的 `settings` 表里，`POST /api/worlds/settings` 改完即时生效，不用重启：

键	含义	默认
`max_concurrent_runs`	全服同时运行的仿真数	4
`max_runs_per_user`	每人同时运行的仿真数（管理员不受限）	1
`daily_llm_calls_per_user`	每人每日模型调用次数（0 = 不限）	0

超出限额时不会直接报错，而是进 FIFO 队列等位，面板上显示「排队中（第 k 位）」。队列有个细节：一条只因为**它自己属主的个人上限**而被挡住的条目，不会阻塞后面的人——所以一个学生开了三个世界，不代表他排在队首的人后面。

```
curl -X POST -H "Authorization: Bearer $GAWORLD_DASHBOARD_TOKEN" -H
        "Content-Type: application/json" \
     -d '{"max_concurrent_runs": 6, "max_runs_per_user": 1}' \
     http://127.0.0.1:8766/api/worlds/settings
```

16.7 教师控制台

控制台「系统」组里有一个「教师控制台」页签，页面在 `site/dashboard/admin.html`。页签默认隐藏，前端在拿到 `/api/auth/me` 后只对 `role == "admin"` 的人显示；即便显示出来，页面上用的每个接口在服务端还会再查一遍角色。

页面分六块。

班级上限：全服同时运行的仿真数、每人同时运行数、每人每日模型调用额度，三个数字改完点保存即生效。

邀请码：按数量、班级标签、有效天数、是否可建城批量生成，直接给出可点击的 `/join?code=…` 链接，只显示这一次。下面一张表列出所有邀请码的状态：未使用的可以作废，已经用掉的保留为「谁用它加入了」的记录。

名单：每个人的在线状态（两分钟内有请求就算在线，这个状态在内存里，重启清空）、今日与累计模型调用、建城权限开关、重置密码按钮。重置密码会给出一条 `/reset?code=…` 链接，24 小时内有效。

世界：所有世界的属主、城市、可见性、是否在运行或排队、今日调用。管理员可以**代为停止**任何一个世界的仿真，也可以导出该世界的多人共玩记录（Markdown 文档，含进出、行动、对话三段）。

广播事件：勾选若干世界，给其中每一位居民同一个生活事件。

审计：最近的写操作记录。

16.7.1 广播：一次事件，全班同时看到

广播是这块面板里对教学最有用的一个功能，接口是 `POST /api/worlds/broadcast`：

```
curl -X POST -H "Authorization: Bearer $GAWORLD_DASHBOARD_TOKEN" -H
        "Content-Type: application/json" \
     -d '{"title": "台风过境", "description": "全市停课停工一天，地铁停
        运。", "severity": 0.8, "world_ids": ["w1a2b3c4"]}' \
     http://127.0.0.1:8766/api/worlds/broadcast
```

它做的事很朴素：把这个事件写进每个选中世界**各自**的生活事件队列，模式是 `immediate`。运行中的世界在下一个 tick 触发；没在运行的世界，下次启动时触发。`world_ids` 里放一个空字符串表示共享的默认世界。

为什么说它在教学里非常有用？一堂比较实验课上，标准流程是「各组用不同参数跑同一个场景，然后比较结果」。但如果没有一个共同的外部冲击，各组跑的是不同的世界，比较起来只能比参数、没法比反应。现在教师在广播里发一次「台风过境」，同一个事件、同一个强度、同一时刻落进 N 个各自独立的世界——每组居民的反应就变得可比了。学生在自己世界里看到的是自己的居民怎么躲雨、怎么囤货、怎么给家人打电话，但所有人都清楚这场台风是同一场。

实现上它复用现成的 `gaworld.events.life`，不需要新的仿真机制。

16.8 手机入口：给学生的短链接

教师在「管理世界」对话框的「手机 / 其他设备进入」一栏里会拿到一条短链：

```
http(s)://<host>/play/<世界编号>
```

投到教室屏幕上，或者做成二维码，学生用手机扫了就进。服务端对 `/play/<world>` 的处理是：查这个世界存在且当前用户可见，然后发一个 303 重定向到 `/site/dashboard/play.html`，同时把 `gaworld_world` cookie 设成那个世界。所以手机登录后直接落在这个世界的「多人共玩」页上，不需要去桌面版的世界切换器里找。

窄屏下页面先显示「我扮演的居民」和它此刻在做什么；从后台切回来时立即续租。

多人共玩的基本规则（多人共玩本身在第 10 章展开，这里只讲部署相关的部分）：认领一位居民是一次 **2 分钟租约**，页面每 30 秒续一次；关闭页面时页面会用 `sendBeacon` 主动交还，即使没交还成功，两分钟租约一到居民也会自动回到模型手里。玩家的行动通过世界的干预队列（`player_claim` / `player_release` / `player_act` / `player_say`）送到运行中的仿真，在下一个 tick 生效。

前置条件是世界的可见性为 `open`——属主和管理员在自己的任何世界里都能玩，其他成员只能进开放的世界。

16.9 上线清单：照着做一遍

下面是一份可以直接照抄的顺序。每一步都给了命令。

第 1 步 · 准备仓库与依赖。 在服务器上放好代码，`pip install -r requirements.txt`。这一步和第 2 章一样，但多用户部署要确认服务器时钟是准的——邀请码有效期、密码重置码、每日配额都按本机时间算。

第 2 步 · 建账号库与第一个管理员。

```
python -m gaworld.accounts init --admin 老师
```

第 3 步 · 发邀请码。

```
python -m gaworld.accounts invite --count 30 --label 周三班
```

把每个码拼成 `http://<host>:8766/join?code=<码>` 发给学生。这一步要在建城权限之前，因为「随邀请码模板授予建城权」是 `invite --can-create-city` 干的。

第 4 步 · 给需要的人授建城权限。

```
python -m gaworld.accounts city-permission --nickname 小王 --allow
```

第 5 步 · 设管理员令牌与强制账号。

```
export GAWORLD_DASHBOARD_TOKEN="$(python3 -c 'import secrets; print
        (secrets.token_urlsafe(24))')"
export GAWORLD_REQUIRE_ACCOUNTS=1
```

`GAWORLD_REQUIRE_ACCOUNTS=1` 很关键：它让服务在**账号库丢失**时直接报 503 而不是悄悄退回单人模式。正式服务器应该一直开着。

第 6 步 · 起服务。

```
python generative_city_sim.py dashboard --host 0.0.0.0 --port 8766
```

用 `--host 0.0.0.0` 才会对外监听；只绑 `127.0.0.1` 的话学生连不上。

第 7 步 · 验证。

```
curl http://127.0.0.1:8766/api/health
curl -H "Authorization: Bearer $GAWORLD_DASHBOARD_TOKEN" http://12
        7.0.0.1:8766/api/auth/users
```

第一条应该返回 `ok: true` 且 `accounts: true`；第二条能列出用户（不带令牌则应该是 401）。

第 8 步 · 备份。 需要留档的是三样东西：账号库 `output/accounts/accounts.sqlite`、用量账本 `output/accounts/usage.jsonl`、以及每个学生的世界目录 `output/worlds/`。账号库丢了等于全班身份没了；世界目录丢了等于学生这几天的实验没了。这三样都在 `output/` 下，天然不会进版本库，但也意味着备份得自己安排——把它加进定时备份脚本。

16.10 安全：哪些事必须由部署方负责

平台侧把能堵的都堵了，剩下这些是部署方的责任。

密钥只走环境变量。 `OPENAI_API_KEY` 这类密钥写在环境里，不要写进 `dashboard_config.json`——那个文件是 `POST /api/config` 能改写的，写进去等于把它交给任何有写权限的成员。仓库的 `.gitignore` 已经排除了 `dashboard_config.json`（第 176 行），但你自己 fork 或自建部署时仍要确认它和 `output/accounts/`、`output/worlds/` 没被提交进版本库。

多人服务器还可以开启**个人密钥模式**：设 `GAWORLD_LLM_SECRETS_PATH` 指向仓库外的一个绝对路径，路径不能是符号链接，独立父目录权限 `0700`，密钥文件由服务创建为 `0600`。这样每位同学在「配置 → 模型」里填自己的 key，付费模型只使用当前账号的密钥，不使用服务器环境变量里的那份。缺用户身份时拒绝借用任何密钥；仿真启动前会先检查所选模型有没有 key，缺了就在重置数据和抓新闻之前报错，不会白跑一趟。



图 16-3　文档面板与密钥的放法 左边是文档目录（快速上手、教程 v2、功能与玩法……），中间是正文，顶部标着「共 30 篇文档」「直读仓库 docs/」——这个面板读的就是仓库里的文档本身，不是副本。正文里那段 `export OPENAI_API_KEY="your_key_here"` 就是本章要强调的放法：密钥进环境变量，不进文件。顺带一提，文档面板是少数几个普通成员也能读的静态前缀之一（`/docs/`）；源码、`dashboard_config.json`、`data/`、`output/memory/` 这些路径对成员一律 404，数据只能经 `/api/` 读。

静态文件是白名单，不是黑名单。 服务端只放行 `/site/`、`/docs/`、`/video/public/`、`/output/population/`、任意 `output/…/visualization/` 目录、文档面板列出的三份根目录文档，以及 `/`、`/console`、`/dashboard`、`/board` 这些页面入口。以点开头的路径（`/.env`、`/.git/…`）一律 404。往仓库根目录里新放一个带密钥的文件时，记得它默认是**不可见**的。

反向代理与 HTTPS 归你。 如果前面挂 Nginx，路径要转发对：`/api/`、`/site/`、`/docs/`、`/output/`、`/board`、`/dashboard`、`/video/`，再加上分布式节点和手机短链用的 `/api/cluster/`、`/play/`。节点上传记录单次最多 1 MB，所以 `client_max_body_size` 不要低于 `2m`。HTTPS 与证书是部署方的事，平台不替你做。

审计要看。 每个写请求都进 `audit` 表，教师控制台能看到最近 50 条。学生改了谁的世界的可见性、谁重置了谁的密码、谁把配额调成了 0，都在上面。

自己写代码时的两条硬规则，前面已经讲过，这里再强调一次：新 dashboard 代码不许通过模块常量读 `output/`，一律走 `gaworld/apps/world_paths.py`；后台任务一律用 `ownership.spawn()` 启动，不要直接开线程。

本章小结

- 账号库是一个开关：文件不存在就是单机单人模式，存在则控制台要求登录；`GAWORLD_REQUIRE_ACCOUNTS=1` 能防止它悄悄退回单人模式。
- 账号管理只有五个命令：`init`（建库 + 第一个管理员）、`invite`（一次性邀请码，默认 14 天有效）、`users`、`reset`（一次性重置码）、`city-permission`（建城权限）。
- 权限分三层：接口能不能调看 `gaworld/accounts/policy.py`；角色（admin / member）与建城权限（`can_create_city`）是分开的两件事；能不能改某个世界只看属主，共享默认世界只限管理员。

- 每个人的世界在 `output/worlds/<id>/`，含居民副本（`seed/`）、`config.json` 和全部运行产物；当前世界由 `gaworld_world` cookie 决定，所有路径经 `gaworld/apps/world_paths.py` 解析，新代码不许用模块常量读 `output/`。
- 归属靠三件事：`ownership.stamp()` 打标、`visible()` / `owned()` 过滤、`ownership.spawn()` 让后台任务带着发起人。
- 模型调用按用户计数记在 `output/accounts/usage.jsonl`，软配额只拦新的模型工作、不打断在跑的任务；运行限额存在账号库 `settings` 表，超出的排队而不是报错。

动手练习

1. **在本机开一个账号库，注册一个成员账号**（约 20 分钟）

   在仓库根目录执行：

   ```
   python -m gaworld.accounts init --admin 老师
   python -m gaworld.accounts invite --count 3 --label 测试班
   ```

   记下打印出来的三个邀请码，然后启动控制台 `python generative_city_sim.py dashboard --port 8766`，在浏览器打开 `http://127.0.0.1:8766/join?code=<其中一个码>`，用邀请码注册一个成员账号（昵称随意，密码至少 8 位）。

   验收标准：

   - 注册前访问 `http://127.0.0.1:8766/dashboard` 会被重定向到 `/login`（而不是直接看到控制台）；
   - 注册成功后能进入控制台，右上角显示当前昵称，退出后用「昵称 + 密码」能在 `/login` 登录回来；
   - `python -m gaworld.accounts users` 里同时出现 `admin` 和你这个 `member` 两行，且 member 行的「可建城」列为空；
   - `ls output/accounts/` 目录下有 `accounts.sqlite`（和 `usage.jsonl`，只要发生过模型调用）。

2. **两个账号各建一个世界，确认产物互不串**（约 25 分钟）

   用管理员账号和成员账号分别登录（或用两个浏览器/两个无痕窗口），在控制台顶栏「管理世界」里各建一个世界（选同一座有居民的城市即可），然后在两个世界里各跑一次 1 天、1–2 位居民的仿真。

   验收标准：

   - `ls output/worlds/` 下出现**两个**世界目录，编号形如 `w` 加 8 位十六进制，互不相同；
   - 每个目录下都有 `config.json`、`seed/agents.csv`、`seed/profiles.md`，并且两个世界的 `seed/agents.csv` 内容相同（因为来自同一座城市）但文件互不影响——在 A 的世界里改一位居民的档案，保存后到 B 的世界看同一位居民，档案没有跟着变；
   - 两个世界的运行产物各自落在自己的目录里（`run.log`、`records/`、`state/` 之类），没有互相覆盖；
   - 切换世界（顶栏下拉框）后，页面上的路径与数据跟着换回另一个世界，刷新浏览器后仍然停在同一个世界（说明是 cookie 在起作用）。做完这两步之后，把两个世界都删掉：`output/worlds/` 应当重新变空。

ewpage

第 17 章　分布式世界与多人共玩

前两章把平台交给了很多人用。这一章讲两件更「社交」的事：把**一座城拆到多台机器上**跑，以及让人**真的走进城里**扮演居民。

17.1 分布式世界：一座城可以分家

为什么需要

一座城 5 000 个居民跑 600 天，单机跑不完。分布式世界的思路很直接：**把居民分片，分给多台机器**，每台跑自己那份，Dashboard 当枢纽。

怎么搭

三步：

1. **世界拥有者在控制台分配居民到节点**（「管理世界 → 分布式节点」）：决定哪些居民归哪台机器。
2. **另一台机器加入**：

   ```
   python -m gaworld.cluster join <hub> --token <world>.<node>.<se
           cret>
   ```

   token 的形状是 `<world>.<node>.<secret>` 三段——世界 id、节点 id、密钥。
3. **节点跑自己那份居民**，并把产物回传枢纽。

Dashboard 就是枢纽（hub）。它提供 `/api/cluster/*` 这一组接口：

路径	作用
`/api/cluster/nodes`	节点名册与分配情况
`/api/cluster/node`	单个节点信息
`/api/cluster/node/heartbeat`	节点心跳
`/api/cluster/node/bundle`	节点取自己那份城市包
`/api/cluster/node/interventions`	**每节点干预发件箱**：枢纽下发的干预，节点取走后消费
`/api/cluster/node/records`	节点把运行记录**上传**回枢纽
`/api/cluster/node/sync`	**tick 同步**

tick 同步是这件事的关键

分布式仿真最难的不是把活分开，是让各机器**在同一天推进**。`ClusterSyncPlugin`（`gaworld/cluster/plugin.py`）在每个 tick 上设一道**同步屏障**：各节点推进到当天的某个时刻后，等其它节点也到达，再一起进入下一天。

没有这道屏障，A 机器跑到第 3 天中午、B 机器还在第 2 天晚上，两边的「事件表」就对不上了——而平行世界这类实验恰恰依赖「同一天、同一事件」。

玩家动作怎么路由

`play_api` 会把玩家对某位居民的动作**路由到真正跑着那位居民的机器**。也就是说：一个玩家在手机上跟「杭州的居民 42」对话，如果居民 42 被分到了 B 机器，动作就发往 B 机器执行，返回结果再传回来。

什么时候真的需要它

诚实地说：**大部分教学与研究场景不需要**。单机跑 50–200 个居民、几百天完全够用。分布式解决的是「规模到单机跑不动」的问题，而它带来的复杂度（同步、时钟一致性、产物回收、故障节点）不低。建议：先把单机的实验设计跑通，再考虑拆机器。

17.2 多人共玩：人走进城里

多人共玩（multiplayer）

在一个 `open` 的世界里，人们可以**扮演居民**（控制台「多人共玩」标签页）：

- **认领**：`player_claim(agent_id, player, until)` —— 某位居民在你手上，直到墙钟时间 `until`
- **行动**：`player_act(agent_id, text)` —— 你替这位居民做一次行动
- **说话**：`player_say` —— 居民在对话里说话
- **释放**：`player_release(agent_id)` —— 交还控制权

关键设计是**租约**：控制台自己决定谁玩谁，**每个认领只有两分钟**。租约过期，居民自动回到模拟里。这解决了多人场景里最现实的问题——一个人挂着不改。

租约是**可审计的干预**：这四个动作都注册成 `Controller` 的运行时干预，所以每一次玩家操作都会进审计记录，不会悄悄改变世界。

介入点

`MultiplayerPlugin` 挂在认知管线的几个位置上：

- `sections`：把玩家的对话插进居民可见的语境
- `override_action`：玩家的行动**覆盖**模型本来会选的动作
- `consume`：消费掉这次玩家输入，不让它进入常规的记忆累积
- `wait_for_players`：在需要玩家参与的时点等待

`consume` 这一条很关键：玩家说的话**不会**变成居民自己的记忆。这保持了「玩家在扮演，居民还是居民」的边界——也避免了玩家把居民变成自己的提线木偶。

手机端

学生用手机加入的短链接是 `/play/<world_id>`。控制台在「多人共玩」面板里生成这个链接给学生。

什么是「真人入场的游戏」

游戏场里有一类游戏专门为真人参与设计——**谁是真人**（`/api/games/whois/*`）：居民和真人同坐一个匿名群聊，真人凭座位链接入座。其它游戏是「人对模型」，这个是「人对模型 + 人」。

17.3 协作面板

多人场景还有一块是**多人协作**：多个玩家共同推进一件事（`site/dashboard/collaboration.html`）。面板里有成员名单、好友关系、轮次、发言人选择与完整对话记录。

17.4 部署形态全景

把三章合起来看，平台有四种部署形态：

形态	适合	复杂度
单机工作区	个人研究、教学演示	最低
多用户共享部署	一班学生、团队协作	中（第 16 章）
分布式世界	单机跑不动的规模	高（本章）

形态	适合	复杂度
多人共玩	课堂互动、公众参与	中（本章）

给教师的一条建议：课堂上一堂课最有冲击力的往往不是批量跑实验，而是**让学生亲自扮演一个居民**——用第 10 章的游戏场或本章的共玩，让他们撞上「说服不动他」「谣言传不出去」这种真实的挫败感，再回到第 12 章的平行世界去看可发表的对照结论。顺序反过来，学生会只记住「模型回答得好有意思」。

17.5 本章小结

- 分布式世界把居民分片给多台机器，Dashboard 当枢纽；关键是 **tick 同步屏障**，保证各机器在同一天推进。
- 枢纽提供 `/api/cluster/*`：节点名册、心跳、城市包、**每节点干预发件箱**、**记录上传**、**tick 同步**。
- 玩家动作会被路由到真正跑着那位居民的机器。
- 多人共玩用**两分钟租约**决定谁玩谁，过期自动交还；四个动作都是可审计干预。
- 玩家说的话**不进入**居民记忆（`consume` 环节消费掉），保持「玩家在扮演」的边界。
- 诚实地说：多数教学场景不需要分布式；课堂冲击力最高的往往是共玩。

17.6 动手练习

1. **在单机上演示共玩**（约 30 分钟） 起一个 dashboard，在「多人共玩」标签页认领一位居民，做一次 `player_act` 和一次 `player_say`，然后释放他。**验收标准**：能在审计记录里找到这几次干预；并去 `output/memory/` 里确认这次对话**没有**进入该居民的长期记忆。
2. **读懂分布式的一条链路**（约 30 分钟，不真搭集群） 读 `gaworld/cluster/plugin.py` 的 `wait_for_nodes`，再读 `gaworld/apps/cluster_api.py` 里 `/api/cluster/node/sync` 与 `/api/cluster/node/records` 的处理。**验收标准**：能用自己的话说出「一个 tick 在多台机器上是怎么对齐的」以及「节点跑完一天后产物是怎么回到枢纽的」，并指出哪一步断了会导致两个世界的数据对不上。

ewpage

第 18 章　测试与质量门

写插件、改行为、动默认配置——这三件事都会让「以前跑出来的结果」变得不可信。这一章讲这个仓库用什么办法拦住这种退化，以及你要为改动承担什么。

18.1 两套测试：冒烟与全量

```
pytest --suite core     # 冒烟套件，约 30 秒，跑主仿真路径
pytest --suite full     # 全量
pytest tests            # 等价于全量
```

`--suite` 由 `tests/conftest.py` 实现：core 套件只跑 `tests/suites/core.txt` 里列出的节点 id，其余被 deselect。

这里有个设计值得知道：**core.txt 里的陈旧项会让整次运行失败**。如果你重命名了一个测试却忘了更新 core.txt，`pytest --suite core` 会直接报 `UsageError: tests/suites/core.txt lists tests that do not exist`。这是故意的——它保证冒烟套件不会因为改名而悄悄少测一个。

常用标记：

标记	含义
`integration`	跨仿真组件的集成测试
`slow`	会起子进程或完整导入仿真器；`pytest -m "not slow"` 可以跳过它们求快

18.2 行数预算：主循环不许变胖

`generative_city_sim.py` 和 `run_simulation` 有**只减不增**的行数上限，写在 `tests/line_budget.json`：

```
{
  "generative_city_sim.py": {
    "lines": 4264,
    "functions": { "run_simulation": 1359 }
  }
}
```

`tests/test_line_budget.py` 会在超过时失败。规则是「上限只能往下走」：把代码搬出主文件会降低计数，这时要在同一次改动里把上限也调低；当你比上限低 50 行以上时，测试会提示你顺手再降一降。

这条预算的真正含义是**架构约束**：仿真主循环只能是一个骨架。新功能一律写成插件（第 15 章），不许往主文件里加内联逻辑。

18.3 Lint 棘轮：只许变好

仓库带了一批历史遗留的 ruff 发现和未格式化文件，`ruff check .` 每次都会红，但红得没有信息量。所以 CI 跑的是**棘轮**而不是全量检查：

```
python scripts/lint_ratchet.py --base origin/main   # CI：和目标分支比
python scripts/lint_ratchet.py                       # 本地：和 HEAD 比，检查未提交的改动
python scripts/lint_ratchet.py --update-baseline     # 清理之后把总数基线调低
```

它检查三件事：

1. **改过的 Python 文件不能有更多的 ruff 发现**——基线上有 3 个，改完还有 3 个可以，变成 4 个不行。新文件必须**零发现**。
2. **基线上格式化的文件改完仍要格式化**；新文件必须已格式化。基线上就没格式化的老文件，不会因为改了一行就被强行 `ruff format`。
3. **全仓库总数不得超过 `scripts/lint_baseline.json`**（只在基线记录的 ruff 版本下检查，新版 ruff 可能引入新规则）；当总数下降时用 `--update-baseline` 记录新基线。

其它工具：`ruff format --check .`（格式）、`black .`（行宽 110）、`mypy gaworld`（`gaworld/` 树严格类型检查，其它目录只是建议）。

18.4 Epoch：让「结果不可比」变成一件可声明的事

这是本仓库最特别的一个机制，也是做研究时最该懂的一个。

问题：如果你改了默认配置下的任何行为，那么以前跑出来的结果和以后跑出来的结果就**不能直接比较**了——但这一点无法从产物里看出来。

做法：`gaworld/core/comparability.py` 维护一条只增不重编号的 Epoch 记录。每一次「改变了默认配置下产出」的改动，都要追加一条：

```
Epoch(11, "2026-10-10",
      "真实工作（默认开）接单时在整个技能库里按任务搜索技能写进简报……",
      ("work", "skills", "memory", "cognition"),
      "gaworld/work/skill_lookup.py; tests/test_skill_search.py"),
```

每条记四个东西：**编号**、**生效日期**、**改了什么**、**影响了哪些量**、**写在哪**。

当前 Epoch 是 **11**。前几条能看出这类改动的性质：Epoch 1 是「经济子系统的 hook 注册在失效的模块名下——此前经济模块在仿真里根本没有运行」；Epoch 2 是货币系统改造；Epoch 7 是「加薪 / 裁员概率按月解释（此前按天抽）」。

什么时候必须加：

- 改了**默认配置**下跑出来的结果，让早先的运行不再可比 → 必须加
- 只是**新增了一个默认关闭的可选开关** → 不需要加

加完之后会发生什么：运行、compare-event 目录和平行世界实验都会被打上 Epoch 标记；**复现池化、研究结论判定和 bench gate 都拒绝跨 Epoch 合并**。也就是说，系统会主动拦住你把 Epoch 7 的结果和 Epoch 11 的结果放进同一个统计里。

18.5 测试环境是隔离的

`tests/conftest.py` 的第一件事就是**把测试套件和开发者本地的 dashboard 配置隔开**。它不会读你真实的 `dashboard_config.json`，也不会往你真实的 `output/` 里写东西。

这不是细节：否则跑一次测试就可能污染你正在分析的那次运行产物。测试用 manifest 记录自己写过什么，跑完清掉。

18.6 新代码的交付清单

改了代码之后，下面这几条是仓库自己要求的：

项	命令	判据
测试	`pytest --suite core`	冒烟套件全绿
全量测试	`pytest --suite full`	全绿

项	命令	判据
覆盖率	`pytest --cov=gaworld --cov-fail-under=80`	CI 用 80% 作为门槛
Lint 棘轮	`python scripts/lint_ratchet.py --base origin/<target>`	没有新增发现，仓库总数没上升
格式	`ruff format --check .` / `black .`	新文件必须已格式化
类型	`mypy gaworld`	`gaworld/` 树无新错误
行数预算	`pytest tests/test_line_budget.py`	没超；搬出代码要同时下调上限
可比性	判断是否需要追加 `Epoch`	改了默认产出就加
文档	改了路由就登记 `gaworld/apps/openapi.py`	`tests/test_openapi.py` 会对未登记路由失败

新代码必须带测试，这是硬要求，不在清单里商量。

18.7 写测试的取向

仓库自己定的规矩很实在：

- 测试放在 `tests/` 下，文件名 `test_*.py`
- **优先用轻量、可复现的测试**：mock 掉 `call_llm` 的模型调用，避免真实网络 IO
- 不要依赖真实模型返回的文本来断言——模型会改写措辞。断言**结构**、**数量**、**方向**，不断言具体句子
- 跑得慢的测试打上 `slow` 标记，别让冒烟套件超时

如果你在写插件，参考 `docs/PLUGIN_AUTHORING.md` 末尾的「可运行的参考实现」，以及 `gaworld/core/comparability.py` 里每条 Epoch 引用的那些测试文件——它们示范了「一个行为变更怎么被钉住」。

18.8 提交与协作

- 提交信息用简短的小写祈使句（历史如此，例如 `updated`、`sync`、`requirement`）
- PR 要写清：范围摘要、配置变更、避免或忽略了哪些运行时产物
- `output/` 的内容通常不进版本库

18.9 本章小结

- 两套测试：`pytest --suite core` 是约 30 秒的冒烟，`--suite full` 是全量；core.txt 里的陈旧项会直接让运行失败。
- 行数预算只减不增，是「主循环不许变胖」的架构约束，不是风格偏好。
- Lint 走棘轮：改过的文件不许变差，仓库总数不许上升。
- **Epoch** 是这个仓库最该懂的机制：改了默认产出就追加一条，系统会拒绝跨 Epoch 合并结果。
- 测试环境与本地配置隔离，跑测试不会污染你的运行产物。
- 交付清单：测试、覆盖率、棘轮、格式、类型、预算、Epoch、文档登记。

18.10 动手练习

1. **跑一次质量门**（约 15 分钟）

```
pytest --suite core
python scripts/lint_ratchet.py
python3 -c "from gaworld.core.comparability import CURRENT_EPOC
        H, EPOCHS; print(CURRENT_EPOCH); [print(e.number, e.aff
        ects, e.change[:40]) for e in EPOCHS]"
```

验收标准：说出当前 Epoch 是几，并挑一条 Epoch 记录，判断它是否属于「改了默认配置下产出」——这是你自己改动时也要回答的问题。

2. **让一次改动被正确记账**（约 30 分钟） 在一个副本里改一处**默认开启**的行为（比如某个插件的默认参数），跑一次仿真前后对比产物，确认这属于需要追加 Epoch 的改动；然后把 Epoch 记录写进 `gaworld/core/comparability.py`（**只在你自己的分支上做，别改主分支**）。 **验收标准**：能说清「改动前后的哪个量变了」，并给这条 Epoch 写出 affects 元组里该填哪几个量。

ewpage

第 19 章　排错手册

仿真系统出问题时，最费时间的往往不是修，而是判断「到底哪儿错了」。这一章给一条固定路径：先分诊，再按症状查表，最后学会怎么问出一个能被回答的问题。

19.1 先分诊：没跑起来 / 跑不动 / 跑完了但结果不对

绝大多数「GAWorld 坏了」的报告，其实是这三类之一。分错诊，后面所有努力都白费。

类别	判据	先看哪里
没跑起来	进程直接退出，或控制台报红	终端里的最后 20 行报错 + `output/logs/`
跑不动	进程活着，但慢到不可用，或卡住不动	`output/logs/run.log` 的最后 50 行
跑完了但结果不对	跑完了，产物也在，但内容不符合预期	产物本身：`output/state/`、`output/memory/`、`output/environment/timeline.jsonl`

三类的排查入口完全不同。**先确认属于哪一类**，再去对应的小节。

19.2 没跑起来

装不上 / 起不来

现象	原因	定位	修法
`SyntaxError` 指向新语法	Python 版本低于 3.11	`python3 -V`	换 3.11 或更高版本；部分测试用例也依赖 3.11+
`ModuleNotFoundError: gaworld`	装的是源码但没装成包，或装到别的环境	`pip show gaworld`、`which python3`	源码方式 `pip install -e ".[dev]"`；确认 `pip` 与 `python` 指向同一个解释器
pip 装完 `gaworld` 命令不存在	可执行文件目录不在 PATH	`which gaworld`	用 `python -m` 形式调用，或把脚本目录加进 PATH
端口被占	上一次的服务没退	`lsof -i :8766`	换端口 `--port 8767`，或杀掉旧进程
`gaworld init` 报目录非空	目标目录里已有文件	`ls <目标目录>`	换一个空目录；`init` 刻意不覆盖已有文件

模型调不通

这是最常见的一类。报错通常发生在第一次调用模型时。

现象	原因	定位	修法
提示 API key 缺失	环境变量没设，或 provider 的 `api_key_env` 指到了别的变量名	`cat dashboard_config.json`，看 `llm.providers.<name>.api_key_env` 是什么，就 export 那个变量	补上环境变量；或在面板里直接填 Key
401 / 认证失败	Key 无效、过期或余额不足	看 `output/logs/` 里的原始响应	换 Key，检查服务商账户余额
429 / 速率限制	并发太高或服务商限流	看日志里的状态码	调小并发、换服务商；`--fast` 模式调用量小得多
内容过滤被拒	触发了服务商的安全策略	看日志里是否标了 `new_sensitive` 之类	换 provider；或调整提问/档案措辞（研究用途下优先换模型而不是改研究设计）
模型看不到图片	路由到的 provider 不支持 vision	看 provider 配置里有没有 `"vision": true`	加这个字段，或换多模态模型；回答上会留下「材料降级」标记
一直转圈没输出	DNS 或网络不通	`curl -sI https://<服务商域名>`	检查网络与代理。注意工作区里默认不配置代理，代理软件开着时反而要确认它没接管
运行极慢但没报错	本地 Ollama 加载大模型，或云端首 token 慢	看日志里每次调用的耗时	换更快的模型；或用 `--fast`

城市造不出来

现象	原因	定位	修法
建城卡了很久	Overpass 镜像不通	日志里会打 `overpass error via <url>`	整体有 240 秒预算，超了会自动降级成程序化地图；急就直接 `--offline`
抓到的节点太少	真实路网稀疏（少于 8 个节点）	看城市包里的地图节点数	正常现象，程序化地图反而更像能住人的地方
中断后再建说已存在	上次留下了残骸目录	`ls data/cities/`	没有 `city.json` 的目录会被当作残

现象	原因	定位	修法
			骸直接覆盖；只有**完整**的城市才需要 `--force`
改了城市但仿真还是老地图	`use` 写的是配置，**下次运行生效**	`cat dashboard_config.json`	重新启动一次运行；已经跑着的进程不会中途换地图

19.3 跑不动

现象	原因	修法
跑得慢	默认是日内逐步，调用量最大	调小 `sim_days`、减少 `agent_ids`、用 `--fast-forward` 或 `--sim-months/--sim-years`
跑了半天还在第 1 天	每天秒数太大，日内推进过细	控制台的「每天秒数」调小；或直接开快进
内存涨得快	记忆与向量库随天数累积	缩短天数；`output/memory/vector_db.sqlite` 可以定期清
面板一直转圈	后台任务在跑，前端按 tick 轮询	看 `output/logs/`；长任务建议先用 CLI 跑，产物出来再打开面板
改了配置后行为还是老的	仿真有状态记忆	先 `reset` 再 `run`

成本上的经验：先用 `--sim-days 3` 验证配置和模型是通的，再放大到 60 / 600 天。反过来做，一个拼错的 provider 就能把一整晚的预算烧掉。

19.4 跑完了但结果不对

这是最需要耐心的部分。分四种情况。

改了配置但行为没变

先确认三件事：改的是**哪一层**配置（源码里的 `gaworld/settings/` 还是 `dashboard_config.json`）、改完有没有 `reset`、以及**这个特性是不是默认关的**。

几个默认关闭、需要显式打开的：

特性	打开方式	观察点
物理环境感知	`CONFIG["environment"]["local_physical"]["enabled"] = True`	感知里有没有「身边的物理环境」；P4 还需要顶层 `stateful=True` 才会落盘
Skill 注入	`CONFIG["skills"]["inject_into_*"] = True`	该 agent 是否真的持有 / 已挂载 Skill
真实工作	`CONFIG["real_work"]["enabled"] = True`	日志里有没有 `WorkerPool started`；接不到单多半是市场里没有匹配其 `job_label` 的活，或当日配额用完

特性	打开方式	观察点
自动提炼私有 Skill	`CONFIG["memory"]["skill_consolidation"]["enabled"] = True`	默认 OFF；且最近 episodes 不足 `min_episodes`（默认 4）会跳过
公共部门	默认关	配置面板里打开
离开本市（出差 / 探亲 / 旅行）	默认关	打开后看 `output/records/travel.*.jsonl`

记忆文件「看起来是乱码」

它不是乱码，是 JSON 没格式化：

```
cat output/memory/agent_31.json | python -m json.tool
```

某位居民的行为跟档案对不上

先确认这个人**有完整档案**。只有状态 CSV 那一行、没有 profile 段落的居民，看起来会像客服而不像居民——档案缺失是「角色扁平」最常见的原因。

想恢复成默认配置

面板的修改写在 `dashboard_config.json`，并在运行时覆盖源码配置。删掉这个文件即可回到 `gaworld/settings/` 里的原始值：

```
rm dashboard_config.json
```

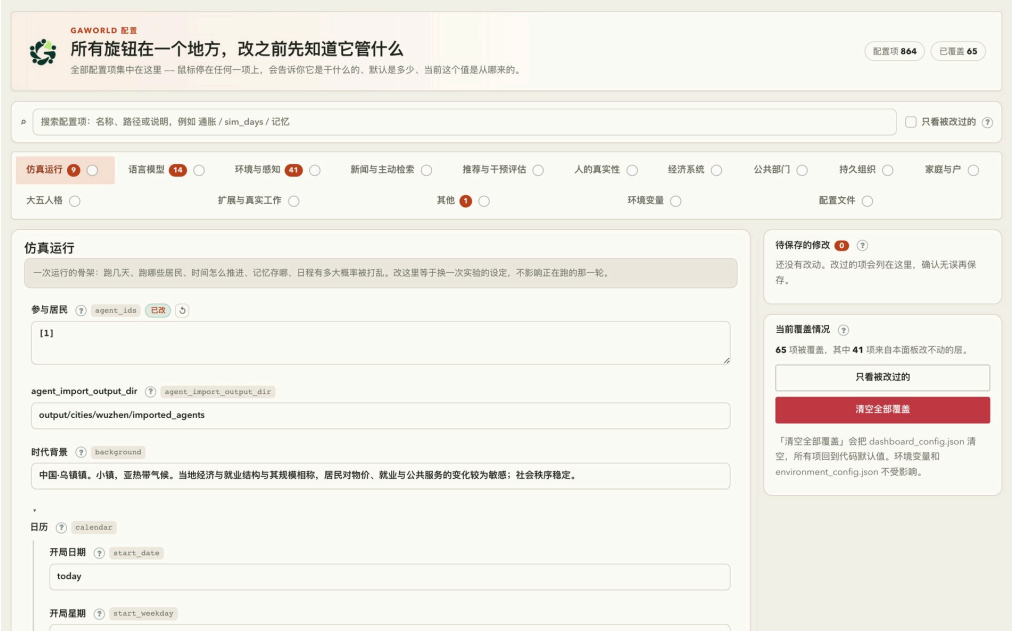


图 19–1　配置中心与「清空全部覆盖」

这一屏是全平台所有配置项的集合，按「仿真运行 / 语言模型 / 环境与感知 / 经济系统 / 家庭与户 / 大五人格」等分组展开，每个旋钮都带说明、默认值和当前值的来源。右上角的「已覆盖」数字是当前有多少项被 `dashboard_config.json` 改过；右侧红色按钮「清空全部覆盖」做的正是 19.4 说的那件事——把 `dashboard_config.json` 清空，所有项回到代码默认值（环境变量与 `environment_config.json` 不受影响）。

排查配置问题时，这一屏比读代码快：先看被覆盖了多少项，再按分组找到你改过的那一项。

19.5 面板显示空

现象	原因	修法
某面板没数据	该面板读的是特定产物目录，本次运行没产生	跑一次仿真生成对应产物；例如时间线要等 `output/environment/timeline.jsonl` 有内容
城市下拉是空的	没建过城市，或当前世界不对	`python -m gaworld.city list`；用 `use` 切到目标城市
居民下拉是空的	这座城没有居民	`python -m gaworld.city add-agents <city> --size 100`
选中的 Agent ID 越界	Agent ID 每座城市各自从 1 开始	切换城市后 ID 范围会变；用城市实际的人数范围
「最近对局」是空的	dashboard 重启过	对局只在内存里，不落盘；这是设计如此

19.6 读数看不出差别

跑完了，但对照组和处理组「看起来一样」。多数情况不是 bug，而是实验设计或材料的问题。

现象	真正的原因	修法
说服局「明明被说动了却判没说动」	问题太开放，复问答案里没有可比的结论	把问题改成有立场的形式重开一局
谣言局「0 条关系」	勾的人分散在不同小区，又没有同行 / 同龄的巧合	按小区分组整组勾，或点「随机两个小区」
传闻一跳就传遍	只勾了一个小区，同小区的人互相全认识	勾两个小区，看它能不能跨过去
所有人都选「私下求证」	传闻对这批人不痛不痒	换一条跟他们利害相关的传闻（钱 / 孩子 / 饭碗）
公投 14:0 通过	议案没人吃亏	改成有人得利、有人掏钱的版本
每一幕的行动分布几乎一样	灾难情境本身没升级	自定义灾难要让后面的幕**真的更糟**
平行世界「全程未明显分叉」	事件太弱，或落在了没被观测的那部分人身上	提高事件强度、扩大参与居民、延长天数

一个通用判断：如果两个世界的差异在噪声地板以内，那不是「没有效应」，是「这个实验现在没有能力检出效应」。对照着 `output/parallel_worlds/` 里的安慰剂世界看，噪声地板有多大，再决定是加剂量还是加样本。

19.7 多人部署的问题

现象	原因	修法
打开控制台要求登录	账号库已存在（`output/accounts/accounts.sqlite`）	用邀请码注册；管理员 `GAWORLD_DASHBOARD_TOKEN`
成员看不到某座城	城市是私有的，只有创建者和管理员能看	`python -m gaworld.accou city-permission --nickn <昵称> --allow`
成员调用某接口被拒	该接口不在 `gaworld/accounts/policy.py` 允许的范围内	查 policy；管理员令牌不受限
某个用户的模型调用被拒	触发每日软配额	查 `gaworld/accounts/usag` 换世界或等配额恢复
产物串到别人的世界去了	直接读了 `output/` 的模块常量	新代码必须 `gaworld/apps/world_paths`

19.8 怎么问出一个能被回答的问题

报问题时，一次性给出这五样，别人才接得住：

1. **你跑的命令**（完整，包括参数）
2. **报错原文**（不是「报错了」，是那几行 stack）
3. **你用的 Python 版本和 provider**（`python3 -V`，provider 名字）
4. **产物目录里有什么**（`ls output/`）
5. **你想达成什么**（一句话）

反过来说，**「我按文档做了但没效果」是最难接的问题**——因为「效果」无法验证。请把它换成一个可观察的东西：「跑 3 天后，`output/memory/agent_1.json` 里应该出现 X，实际只有 Y」。

19.9 一张症状速查表

症状	先看这一行
进程直接退出	终端最后 20 行
报 API key 缺失	`dashboard_config.json` → `llm.providers.*.api_key_env`
401 / 429	余额与限流；换 provider
建城卡住	`--offline`；日志里的 `overpass error via …`
跑到第 1 天很慢	「每天秒数」调小，或开快进
改了配置没变化	`reset`；确认该特性默认是否关闭
面板城市为空	`gaworld.city list` + `use`
居民档案很单薄	补 profile 段落
记忆文件像乱码	`python -m json.tool`
实验看不出差异	看安慰剂世界的噪声地板
想恢复默认配置	删掉 `dashboard_config.json`

19.10 本章小结

- 排错先分诊：没跑起来、跑不动、跑完了但结果不对，三类入口完全不同。
- 模型调不通是最常见的一类，九成是 Key、路由或网络问题，看 `output/logs/` 就能定位。
- 「改了配置没变化」先查三件事：改的是哪一层、reset 了没有、这个特性是不是默认关的。
- 面板改的配置写在 `dashboard_config.json`，删掉它就回到源码默认值。
- 实验读不出差别，多数不是 bug，而是效应落在噪声地板以内——先看安慰剂世界。
- 报问题时给出命令、报错原文、版本与 provider、产物目录、目标，五样齐全才接得住。

19.11 动手练习

1. **制造一次故障并修好它**（约 40 分钟） 故意把 `dashboard_config.json` 里的 provider 名字改错，跑一次仿真，观察报错长什么样，再改回来。 **验收标准**：能指出报错发生在哪个阶段（是 provider 解析、鉴权，还是请求发出后），并说出用哪条命令确认当前配置。

2. **做一张自己的症状表**（约 30 分钟） 在纸上画三列：现象 / 定位命令 / 修法。先填五条你今天真的遇到过的，再去 `docs/TUTORIAL.v2.md` 的常见问题里补三条你还没遇到的。 **验收标准**：五条里每条都有**你自己跑过**的定位命令，且命令的输出能区分「有这个问题」和「没这个问题」。

ewpage

第 20 章　速查附录

本章是查表用的：命令、配置、产物、术语。遇到「这个参数叫什么」「这个文件在哪儿」的时候来这里翻，不必读正文。

20.1 命令速查

仿真主命令

入口是 `generative_city_sim.py`，共 11 个子命令。

```
# 跑仿真
python generative_city_sim.py run                          # 默认步长
python generative_city_sim.py run --sim-days 600 --fast-forward
python generative_city_sim.py run --sim-months 24
python generative_city_sim.py run --sim-years 10
python generative_city_sim.py run --time-unit month --sim-years 10

# 重置（从 Day 1 重新开始）
python generative_city_sim.py reset

# 采访单个居民
python generative_city_sim.py interview --agent-id 31 --question
        "你这个月开销最大的一项是什么？"
python generative_city_sim.py interview --agent-id 31 --questions-f
        ile questions.txt
python generative_city_sim.py interview --agent-id 31 --question
        "..." --context "背景说明"

# 外部信息注入（RAG）
python generative_city_sim.py rag-add --agent-id 31 --text "..."
python generative_city_sim.py rag-import --agent-id 31 --file ./cor
        pus/

# 从社交媒体页面造一个居民
python generative_city_sim.py create-agent-from-social --url "http
        s://…"
python generative_city_sim.py create-agent-from-social --file page.
        html
python generative_city_sim.py create-agent-from-social --text "……"

# 实验
python generative_city_sim.py compare-event \
  --event-name "租金补贴" --event-description "每月补贴 300 元" --even
        t-day 3 --seed 42
python generative_city_sim.py parallel-worlds --spec worlds.json
python generative_city_sim.py parallel-worlds --sweep economy.shock
        s.layoff_base_prob=0.01,0.02,0.04 --placebo

# 服务
python generative_city_sim.py dashboard --host 127.0.0.1 --port 876
        6
python generative_city_sim.py serve-viz --host 127.0.0.1 --port 800
        0
python generative_city_sim.py serve-distributed --host 0.0.0.0 --po
        rt 8877
```

`run` 的完整参数：

参数	含义
`--sim-days N`	覆盖仿真天数
`--sim-months N`	跑 N 个日历月；自动开快进并按月为步长
`--sim-years N`	跑 N 个日历年；自动开快进并按年为步长
`--time-unit {day,month,year}`	快进步长单位；指定任意一个都会自动开快进
`--fast-forward`	长时段快进：每步压缩成每人一条简报，跳过日内时刻循环

`compare-event` 与 `parallel-worlds` 还支持 `--llm-provider`（强制所有分支用同一个 provider）、`--output-root`（产物根目录）、`--fast`（确定性认知、跳过日摘要与日记、3 人群体，调用更少但保真度下降）。

城市

```
python -m gaworld.city create "绍兴柯桥" --size 200
python -m gaworld.city create "柳溪村" --offline --scale tiny
python -m gaworld.city create "翡翠屿" --description "热带火山岛，靠渔
        业和潜水旅游为生"
python -m gaworld.city create "翡翠屿" --image ~/sketches/island.png

python -m gaworld.city list
python -m gaworld.city show 绍兴柯桥
python -m gaworld.city use 绍兴柯桥

python -m gaworld.city add-agents 绍兴柯桥 --size 200 --preset cn_co
        unty_town
python -m gaworld.city add-agent 绍兴柯桥 --name 林素 --age 34 --job
        "社区医生"
python -m gaworld.city migrate 绍兴柯桥 --agent-id 31

python -m gaworld.city knowledge 绍兴柯桥
python -m gaworld.city locale 绍兴柯桥
python -m gaworld.city news 绍兴柯桥
python -m gaworld.city delete 绍兴柯桥 --yes
```

`create` 的参数：

参数	可选值 / 含义
`--slug`	目录名，默认取地名的 slug
`--scale`	`large` / `medium` / `metro` / `small` / `tiny`，覆盖推断出的规模
`--offline`	跳过地理编码与 OSM，程序化生成
`--description`	按描述虚构一座城，而不是查地名
`--image`	按草图（PNG/JPEG）虚构，需要 vision 模型
`--size`	同时生成多少居民
`--preset`	`aging_community` / `cn_county_town` /

参数	可选值 / 含义
	`cn_tier1_district` / `college_town` / `custom` / `us_suburb`
`--seed`	程序化地图的随机种子
`--force`	覆盖同名城市

人口、群体与采访

```
# 参数化人口：预览 / 落盘
python -m gaworld.population --size 500 --seed 42 --check
python -m gaworld.population --size 500 --name my_town --out data/t
        own
python -m gaworld.population --spec spec.json --json

# 群体（cohort）模拟
python -m gaworld.group --size 500 --days 7 --no-llm
python -m gaworld.group --size 500 --days 7 --focal 7,42 --no-llm
python -m gaworld.group --size 500 --days 7 --network-coupling 0.7
        --no-llm
python -m gaworld.group --budget 20 --audit-fraction 0.1 --axes age
        _band,industry,hukou

# 群体模式的验证门（分水岭层不通过时退出码非零）
python -m gaworld.group.validate --size 100 --days 14 --network-cou
        pling 0.7

# 群体采访：一次问一群人
python -m gaworld.interview --spec round.json --out answers.json
```

账号与部署

```
python -m gaworld.accounts init --admin 老师
python -m gaworld.accounts invite --count 30 --label 某某班
python -m gaworld.accounts users
python -m gaworld.accounts reset --nickname student01
python -m gaworld.accounts city-permission --nickname student01 --a
        llow
```

pip 安装后的工作区入口：

```
gaworld init ./my-world      # 准备一个独立可写的工作区
cd my-world
gaworld dashboard              # 打开 http://127.0.0.1:8766/dashboar
        d
```

环境变量

变量	作用
`OPENAI_API_KEY` / `ANTHROPIC_API_KEY`	云端模型密钥；具体用哪个由 provider 的 `api_key_env` 决定
`MINIMAX_API_KEY`	MiniMax 兼容接口的密钥
`GAWORLD_DASHBOARD_TOKEN`	管理员令牌；带上的请求按管理员处理

变量	作用
`GAWORLD_CONFIG_OVERRIDES`	一段 JSON，运行时深合并覆盖配置，等价于改 `dashboard_config.json`
`GAWORLD_LOG_MODE`	`simple`（默认）或 `verbose`
`GAWORLD_LOG_LEVEL`	日志级别，如 `DEBUG`

20.2 配置速查

配置是分层的，底层是 `gaworld/settings/` 里的默认片段，上层是 `dashboard_config.json` 里的覆盖项。面板改配置写的是后者，删掉这个文件即可回到源码默认值。

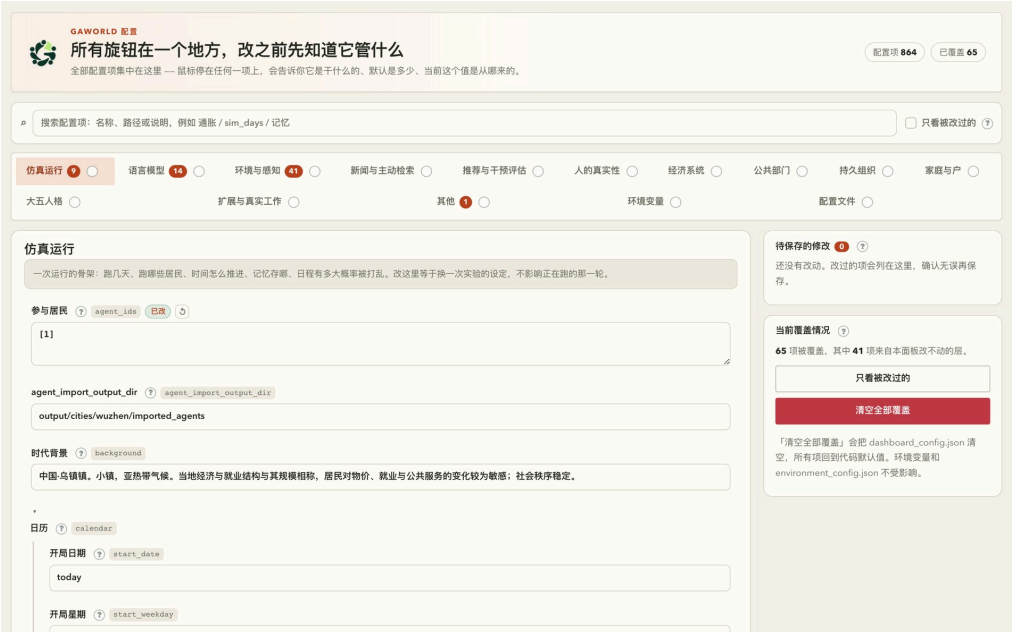


图 20–1　配置中心

全平台的配置项集中在这里，按子系统分组，每个旋钮都写了它是干什么的、默认值多少、当前值从哪来。查配置时先来这里，别去翻代码。

位置	负责什么
`gaworld/settings/llm.py`	provider 定义与路由（`llm.providers`、`llm.routing`）
`gaworld/settings/runtime.py`	运行期参数（天数、并发、日志、步长）
`gaworld/settings/behavior.py`	行为与动态行为开关
`gaworld/settings/economy.py`	经济参数与部门池
`gaworld/settings/environment.py`	环境、事件、局部物理感知
`gaworld/settings/family.py`	家庭与户
`gaworld/settings/personality.py`	大五人格三通道开关
`gaworld/settings/public_sector.py`	公共部门与政策
`gaworld/settings/organizations.py`	持久组织主体（默认关闭）

位置	负责什么
`gaworld/settings/overrides.py`	加载 `GAWORLD_CONFIG_OVERRIDES` 与 `dashboard_config.json`
`dashboard_config.json`	面板写入的覆盖项；含 `llm`、`sim_days`、`agent_ids`、`long_run`、`city` 等

几个最常被改的键：

```
CONFIG["pipeline"]["agent_step"]       # 12 阶段认知管线，消融/替换/插入阶段
CONFIG["llm"]["routing"]["default"]    # 默认模型
CONFIG["llm"]["routing"]["tasks"]      # 按任务分流（schedule 等）
CONFIG["long_run"]["randomness"]       # 快进期间的随机性
CONFIG["routine_change"]["randomness"] # 日程随机度
CONFIG["plugins"]                      # 第三方插件装配列表
```

`dashboard_config.json` 的实际结构（可对照本机文件）：

```
{
  "llm": {
    "routing": { "default": "…", "tasks": { "schedule": "…" } },
    "providers": { "…": { "type": "ollama|openai|anthropic", "url":
        "…", "model": "…", "api_key_env": "…" } }
  },
  "sim_days": 5,
  "seconds_per_day": 10,
  "agent_ids": [1],
  "simulate_realtime": true,
  "time_step_minutes": null,
  "long_run": { "enabled": false, "randomness": 1.0, "unit": "day"
      },
  "routine_change": { "randomness": 1.0 },
  "city": "wuzhen"
}
```

20.3 产物速查

```
output/
├── logs/              run.log（完整）、agent_<id>.log
├── memory/            agent_<id>.json、agent_<id>_episodes.jsonl
│                      agent_<id>_growth.json、growth_profiles.json
│                      agent_<id>_env_preferences.json、agent_<id>_sk
ills/*.md
│                      vector_db.sqlite
├── state/             状态历史（长表 CSV）
├── traits/            agent_traits.csv、calibration_audit.csv
├── economy/           daily_ledger.csv、wealth_snapshot.csv、macro_s
tate.json
│                      sectors.json、conservation_audit.csv、interven
tions.json
│                      agents/agent_<id>_ledger.csv、agent_<id>_snaps
hot.json
├── environment/       timeline.jsonl
├── network/           social_network.png
├── records/           family.*.jsonl、travel.*.jsonl 及其它插件事件流
├── visualization/     simulation_trace.json、latest_frame.json
├── work/              capabilities.json、queue.jsonl、market.jsonl
├── intervention/      intervention_metrics.csv
├── comparisons/       <事件名>/comparison_summary.md、comparison_metr
ics.csv
```

```
├── interviews/      <session_id>/session.json、report.md
├── parallel_worlds/ <实验名>/divergence_summary.md、divergence_metrics.csv
│                    report.json、experiment.json、worlds/<world_id>/
└── worlds/          多用户部署时每个世界一棵产物树
```

常用查看命令：

```
cat output/memory/agent_31.json | python -m json.tool        # 格式化 JSON
tail -f output/logs/run.log                                  # 跟运行日志
tail -n 20 output/environment/timeline.jsonl                 # 最近的事件时间线
ls -la output/state/                                         # 状态历史 CSV
```

20.4 术语表

术语	含义
智能体（Agent / 居民）	平台里的一个模拟居民，由状态、档案、人格、家庭关系与记忆构成
仿真日	仿真内部的「一天」，与真实时间无关；一天内可含多个时刻 tick
快进（fast-forward）	把一个仿真日压缩成每人一条简报，跳过日内时刻循环
步长单位	快进时一步代表多长：按天、按月、按年
阶段（pipeline stage）	认知管线的 12 个命名步骤之一，可消融、替换或插入
干预（intervention）	运行中改变世界的动作：改状态、改配置、移除居民、注入事件
分叉（divergence）	某个世界与基准世界从第几步开始不再重合
配对 ATE	同一位居民在两个世界里的差值再求平均，比组间比较更抗个体差异
剂量—反应	同一事件取不同强度，看效应量怎么随强度移动
安慰剂（placebo）	用一个不该有效应的事件当对照，测出噪声地板
群体（cohort）	一组同时携带均值与离散度的居民，每群每天只花 1 次模型调用
实体化（materialize）	把群体里的某个个体提升为完整保真度单独模拟
复制拟合度	让居民重做一项有真人对照的调查或实验，代码比对误差与效应对齐程度
预注册	在跑实验之前把假设、指标、条件与判定规则固定下来

术语	含义
Epoch	变更默认配置后追加的可比性版本号；跨 Epoch 的结果不可合并
城市包	`data/cities/<slug>/` 下一座城市的地图、环境、人口与知识库
钩子语义	插件订阅事件的两种约定：`observe` 只看、`collect` 汇总贡献
审计	每次运行时干预自动留下的记录，插件无法悄悄改世界

20.5 文档地图



图 20–2　控制台内置的文档面板

控制台里带了一个文档面板，项目自己的文档不用离开浏览器就能读、也能搜。下面的表是「离线去哪里找」的对照。

你要解决的问题	去哪里
这个功能到底有没有、怎么配	`docs/FEATURES.md`（能力目录）
某个面板的完整用法	`docs/RESEARCH_WORKBENCH_TUTORIAL.md` / `docs/PLAYGROUND_TUTORIAL.md` / `docs/CITY_TUTORIAL.md` / `docs/GROUP_INTERVIEW_TUTORIAL.md`
所有 HTTP 接口	`docs/API_REFERENCE.md`；`gaworld/client.py` 可按 operationId 直接调
插件怎么写	`docs/PLUGIN_AUTHORING.md`
怎么部署给一班学生用	`docs/SERVER_DEPLOYMENT.md`
方法论与经典案例	`textbook/`（22 章 + 4 附录，另有 pdf / epub）

你要解决的问题	去哪里
中英对照的长指南	`docs/PROJECT_GUIDE.zh-CN.md` / `docs/PROJECT_GUIDE.en.md`
效度怎么评	GAWorld Bench
这个仓库怎么改	`AGENTS.md`（项目结构与开发规则）、`CONTRIBUTING.md`

20.6 本章小结

- 命令分五组：仿真主命令、城市、人口与群体、账号与部署、环境变量；每条都能加 `--help` 查全。
- 配置分两层：源码里的 `gaworld/settings/` 与面板写入的 `dashboard_config.json`；删掉后者就回到默认值。
- 产物全部落在 `output/`，`output/worlds/<id>/` 是多用户部署时每个世界的那一棵。
- 术语表里的「分叉」「配对 ATE」「安慰剂」「Epoch」是后面做实验时反复出现的词，值得记住。
- 遇到任何「这个参数在哪」的疑问，先查本页；查不到再查 `docs/FEATURES.md`，最后才去读代码。

20.7 动手练习

1. **把速查表变成你自己的**（约 30 分钟） 照着 20.1 跑一遍 `run`、`reset`、`--help` 三类命令，在纸上记下你最常用的五条。 **验收标准**：不看手册，你能在终端里敲出「跑 10 年、按年一步」和「造一座离线虚构城市」这两条完整命令。
2. **核对产物树**（约 20 分钟） 跑一次 `python generative_city_sim.py run --sim-days 3`，然后按 20.3 的目录清单逐个检查 `output/` 下实际存在哪些文件。 **验收标准**：能指出至少 5 个目录里各有什么、哪个目录是记忆、哪个是状态历史、哪个是事件流。